# Orchestron User Documentation

Compiled from markdown documentation

Generated on 20 October 2026 at 13:37

# Table of Contents

**Orchestron User Documentation** . . . . 4
Browser-Clock Latency . . . . 7
**Instrument Design** . . . . 10
Patch Toolbar and Instrument Tabs . . . . 12
Runtime Panel and Compilation Workflow . . . . 16
MIDI Setup and Inputs . . . . 20
Live Status . . . . 22
Multitrack Arranger . . . . 24
Pattern Pads and Reusable Phrases . . . . 30
If, Switch and Drumset . . . . 32
Instrument Import / Export and CSD Export . . . . 35
Performance Import / Export . . . . 38
Audio mixer and routing . . . . 46
**Performance** . . . . 55
Sequencer timing . . . . 58
Instrument Rack and Engine Transport . . . . 59
Performance Controllers . . . . 62
Monophonic Legato . . . . 65
Melodic Sequencers and Step Editing . . . . 67
Drummer Sequencers . . . . 72
Controller Sequencers . . . . 76
Arpeggiators . . . . 80
Piano Rolls . . . . 84
MIDI Controllers . . . . 87
Opcode Catalog and Integrated Documentation . . . . 89
GEN Table Editor (GEN Meta-Opcode) . . . . 94
GUI Language and Integrated Help . . . . 101
Supported Opcodes . . . . 105
Physical Steel-String Guitar . . . . 116
Graph Editor . . . . 120
Input Formula Assistant . . . . 126
Standard Effects . . . . 130
Tube Overdrive . . . . 131
Stereo Delay . . . . 133
Stereo Flanger . . . . 135
Parametric Equalizer . . . . 137
Stereo Widener, Bit Crusher, and Harmonizer . . . . 140
Granular Capture: Stutter, Freeze and Grains . . . . 142
Stereo Moog Filter . . . . 146

**Configuration** . . . . . **148**
Audio Engine Settings (Config Page) . . . . . 149
Persistence and Defaults . . . . . 154
**Appendix** . . . . . **156**
Agent CLI . . . . . 157
Orchestron Patch Creator Skill . . . . . 158
Orchestron Performance Creator Skill . . . . . 159
Orchestron Composition Creator Skill . . . . . 161
Sung Melody Transcription Skill . . . . . 166

# Orchestron User Documentation

This manual documents the current Orchestron user-facing functionality as implemented in the repository (frontend, backend API behavior visible to users, and feature history from the git log) as of **2026-09-07**, including the DAW audio-routing and persistent Perform mixer changes in commit `b487abd`.

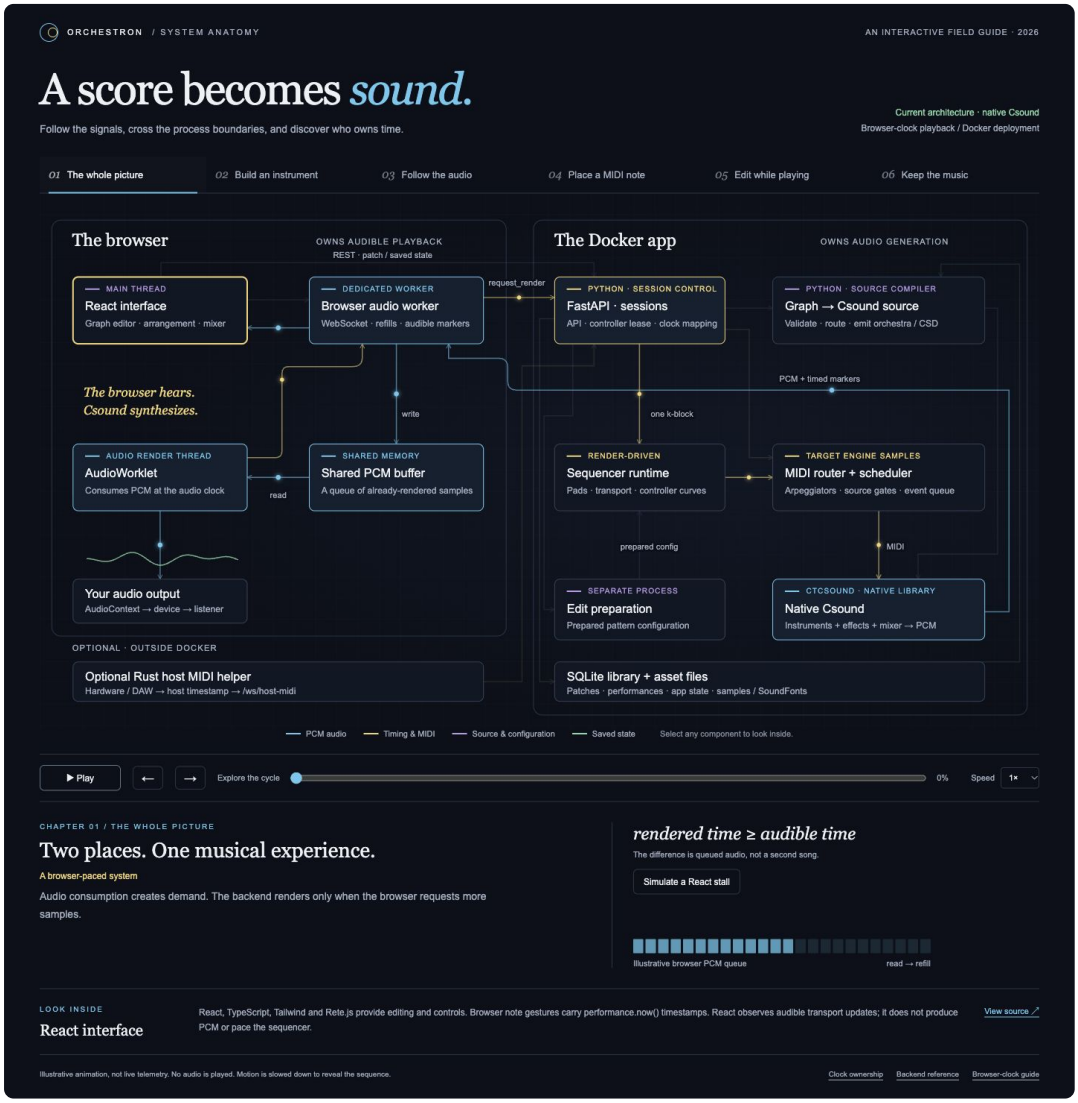
It is organized by chapter and subchapter so you can read it front-to-back or jump directly to a workflow.

When Orchestron is served locally from the backend, opening `http://localhost:8000/` redirects to the client application at `/client`.

## Interactive Architecture Guide

Open Orchestron: A score becomes sound in a browser to explore the current Docker architecture. The self-contained page works offline: download the HTML or open the repository file directly. Repository previews such as GitHub show its source instead of running it.

Six animated chapters explain the system map, graph compilation, PCM refills, MIDI timing, live sequencer edits and persistence. Select components for source-linked explanations; use Pause, the timeline and speed controls to inspect the flow. The experiments illustrate a React stall, MIDI notes, successful or failed preparation, and a backend restart. They do not control the application or play audio. Motion starts paused when the browser requests reduced motion, and narrow screens show a compact component view.




The animation describes the existing native Csound runtime. See clock ownership, the backend reference and browser-clock

latency for implementation details.

## Scope

- Visual instrument design (opcode graph editor, formulas, GEN tables, compile/runtime testing)
- Live performance workflow (instrument rack, audio mixer, Master, inserts, sends, sequencers, piano rolls, MIDI controllers, import/export)
- Configuration and runtime behavior (audio engine settings, MIDI setup, browser-clock audio, persistence)

## Reading Order

- Start with **Instrument Design** if you are creating patches.
- Continue with **Performance** to build and perform a multi-instrument setup.
- Use **Configuration** for audio engine tuning, MIDI setup, UI language/help, and browser-clock audio.

## Instrument Design

- Patch Toolbar and Instrument Tabs
- Opcode Catalog and Integrated Documentation
- Graph Editor
- Input Formula Assistant
- GEN Table Editor (GEN Meta-Opcode)
- Runtime Panel and Compilation Workflow
- Instrument Import / Export and CSD Export
- Supported Opcodes

## Performance

- Instrument Rack and Engine Transport
- Audio Mixer and Routing
- Melodic Sequencers and Step Editing
- Pattern Pads, Queued Switching, and Pad Looper
- Controller Sequencers
- Arpeggiators
- Piano Rolls
- MIDI Controllers
- Performance Import / Export
- Live Status and Safety Controls
- Standard Effects

## Configuration

- GUI Language and Integrated Help
- Audio Engine Settings (Config Page)
- MIDI Setup and Inputs
- Browser-Clock Latency
- Persistence and Defaults

## Appendix

- Agent CLI
- Orchestron Patch Creator Skill
- Orchestron Performance Creator Skill

- Orchestron Composition Creator Skill
- Sung Melody Transcription Skill

## Feature Coverage Map

| Feature | Where It Is Documented |
|---|---|
| Multi-language UI (EN/DE/FR/ES) | GUI Language and Integrated Help |
| Integrated help pages and opcode docs modal | GUI Language and Integrated Help, Opcode Catalog and Integrated Documentation |
| Visual graph editor, zoom, selection, deletion | Graph Editor |
| Stereo audio interfaces, built-in patch starters and draft audition | Patch Toolbar, Graph Editor, Audition |
| Persistent audio mixer, Master, inserts, sends and routing repair | Audio Mixer and Routing |
| Input Formula Assistant (multi-input formulas + functions) | Input Formula Assistant |
| GEN meta-opcode editor (GEN01/GENpadsynth/etc.) | GEN Table Editor (GEN Meta-Opcode) |
| Instrument import/export bundles + CSD export | Instrument Import / Export and CSD Export |
| Performance rack, multi-instrument assignments, engine transport | Instrument Rack and Engine Transport |
| Melodic sequencers, pattern pads, pad looper, transposition | Melodic Sequencers and Step Editing, Pattern Pads, Queued Switching, and Pad Looper |
| Controller sequencers and CC curve editor | Controller Sequencers |
| Backend arpeggiator virtual instruments and presets | Arpeggiators |
| Piano rolls, scale/mode following, mixed-mode highlighting | Piano Rolls |
| Manual MIDI controller panel (up to 6 knobs) | MIDI Controllers |
| Performance bundle import/export with conflict resolution | Performance Import / Export |
| Offline performance render export ( `.csd + .mid` ZIP or inline-score `.csd` ZIP) | Performance Import / Export |
| Audio engine settings ( `sr` , `control_rate` , `ksmps` , buffers) | Audio Engine Settings (Config Page) |
| Browser audio mode ( `browser_clock` PCM) | Browser-Clock Latency |
| Browser-clock latency tuning for unified browser audio | Browser-Clock Latency |
| App-state persistence and defaults | Persistence and Defaults |
| Supported opcode catalog | Supported Opcodes |

## File Format Quick Reference

- Instrument definition export: `.orch.instrument.json` or `.orch.instrument.zip`
- Performance export: `.orch.json` or `.orch.zip`
- Offline performance Csound export: `.csd.zip` ( `Export CSD (MIDI)` includes a `.mid` ; `Export CSD (SCORE)` embeds notes and controller sweeps in the score)
- Csound export from Instrument Design: `.csd`
- ZIP exports are used automatically when referenced GEN01 audio assets are included.

# Browser-Clock Latency

Orchestron now uses backend audio mode `browser_clock` on every supported platform. The browser becomes the playback clock and requests PCM render chunks from the backend, so the most important latency and stability controls live in the Config page's `Browser-Clock Latency` section.

## When This Section Appears

The `Browser-Clock Latency` section is shown when the backend is running in `browser_clock` mode, for example:

- `docker compose up --build`
- environment variable `VISUALCSOUND_AUDIO_OUTPUT_MODE=browser_clock`

This is the standard runtime path for macOS, Linux, Windows, and Docker. In practice, that means this section describes the normal live-audio path of the app, not a Docker-only special case.

## Why These Settings Matter

On the browser-clock path, audio is not pushed to a local DAC on the backend host. Instead:

- the backend renders PCM blocks on demand
- a dedicated browser audio worker keeps the target queue of decoded PCM and requests more audio
- the AudioWorklet consumes that queue independently of React rendering

That makes the browser-clock queue settings the main tradeoff between low live-play latency and glitch resistance.

The queue producer no longer runs on the React/main thread. Main-thread stalls can delay visual updates, but they do not stop the worker from requesting or buffering PCM. Playback starts only after the startup high-water target is reached. The Runtime panel reports current queued and pending-render milliseconds, underrun/overrun counts, and the latest backend render-time/audio-time ratio.

The backend suppresses Csound performance-time logging by default in headless browser-clock sessions. This prevents stdout or Docker logging I/O from delaying `performKsmps()`. To diagnose Csound's own runtime messages, start the backend with `VISUALCSOUND_CSOUND_PERFORMANCE_LOGGING=true`; this is intended as a temporary diagnostic setting rather than a latency control.

Lower values usually feel more immediate, but they also leave less headroom for:

- backend scheduling jitter
- network / websocket round-trip jitter between browser and backend
- browser tab throttling or temporary main-thread stalls
- heavier patches that take longer to render

Higher values add safety margin, but manual playing and note feedback feel less direct.

## Main Control Groups

### Queue Targets

- `Steady Low Water` / `Steady High Water` define the normal browser PCM queue window after startup has stabilized.
- `Startup Low Water` / `Startup High Water` define the larger queue window used while the stream is priming or recovering.

If you hear underruns after connect or after transport restarts, start by increasing the steady or startup queue targets slightly.

### Recovery Headroom

- `Underrun Recovery Boost` adds extra queue headroom after an underrun is detected.
- `Max Underrun Boost` limits how large that temporary recovery buffer may grow.

These controls help the browser recover cleanly from brief dropouts without forcing a permanently larger steady-state latency.

### Render Request Size And Parallelism

- `Max Blocks Per Request` limits how much audio the browser asks for in one render request.
- `Steady Parallel Requests`, `Startup Parallel Requests`, and `Recovery Parallel Requests` control how many render requests may stay in flight in each state.

Smaller requests can improve responsiveness, while larger or more parallel requests can make it easier to keep the queue full on slower systems.

### Live Note Responsiveness

- `Immediate Note Render Blocks` sends a small urgent render burst after a live note-on event.
- `Immediate Note Render Cooldown` limits how often those urgent note-triggered bursts may happen.

These are the most direct controls for making manual keyboard or MIDI playing feel snappier in browser-clock mode.

### Manual MIDI Safety Limits

Browser-clock manual MIDI is bounded on the backend even when events are routed through an arpeggiator. Deployments can tune the maximum accepted future timestamp with `VISUALCSOUND_BROWSER_CLOCK_MANUAL_MIDI_MAX_FUTURE_MS`, the accepted manual MIDI rate with `VISUALCSOUND_BROWSER_CLOCK_MANUAL_MIDI_RATE_PER_SECOND` and `VISUALCSOUND_BROWSER_CLOCK_MANUAL_MIDI_BURST`, and the per-session arpeggiator pending input cap with `VISUALCSOUND_ARPEGGIATOR_PENDING_INPUT_MAX_EVENTS`.

### Session Resource Limits

Runtime sessions allocate backend Csound worker resources, so session creation and live session-event observers are also bounded. `VISUALCSOUND_SESSION_MAX_ACTIVE` limits total active sessions, `VISUALCSOUND_SESSION_MAX_ACTIVE_PER_CLIENT` limits sessions owned by one client address, `VISUALCSOUND_SESSION_CREATE_RATE_PER_MINUTE` and `VISUALCSOUND_SESSION_CREATE_RATE_BURST` limit repeated creation attempts, and `VISUALCSOUND_SESSION_IDLE_TIMEOUT_SECONDS` deletes stopped idle sessions so capacity is returned automatically. Session event WebSocket observers are capped with `VISUALCSOUND_SESSION_EVENT_WS_MAX_SUBSCRIPTIONS_TOTAL` and `VISUALCSOUND_SESSION_EVENT_WS_MAX_SUBSCRIPTIONS_PER_SESSION`, while connection attempts are rate-limited with `VISUALCSOUND_SESSION_EVENT_WS_CONNECT_RATE_PER_MINUTE` and `VISUALCSOUND_SESSION_EVENT_WS_CONNECT_RATE_BURST`.

## Practical Tuning Order

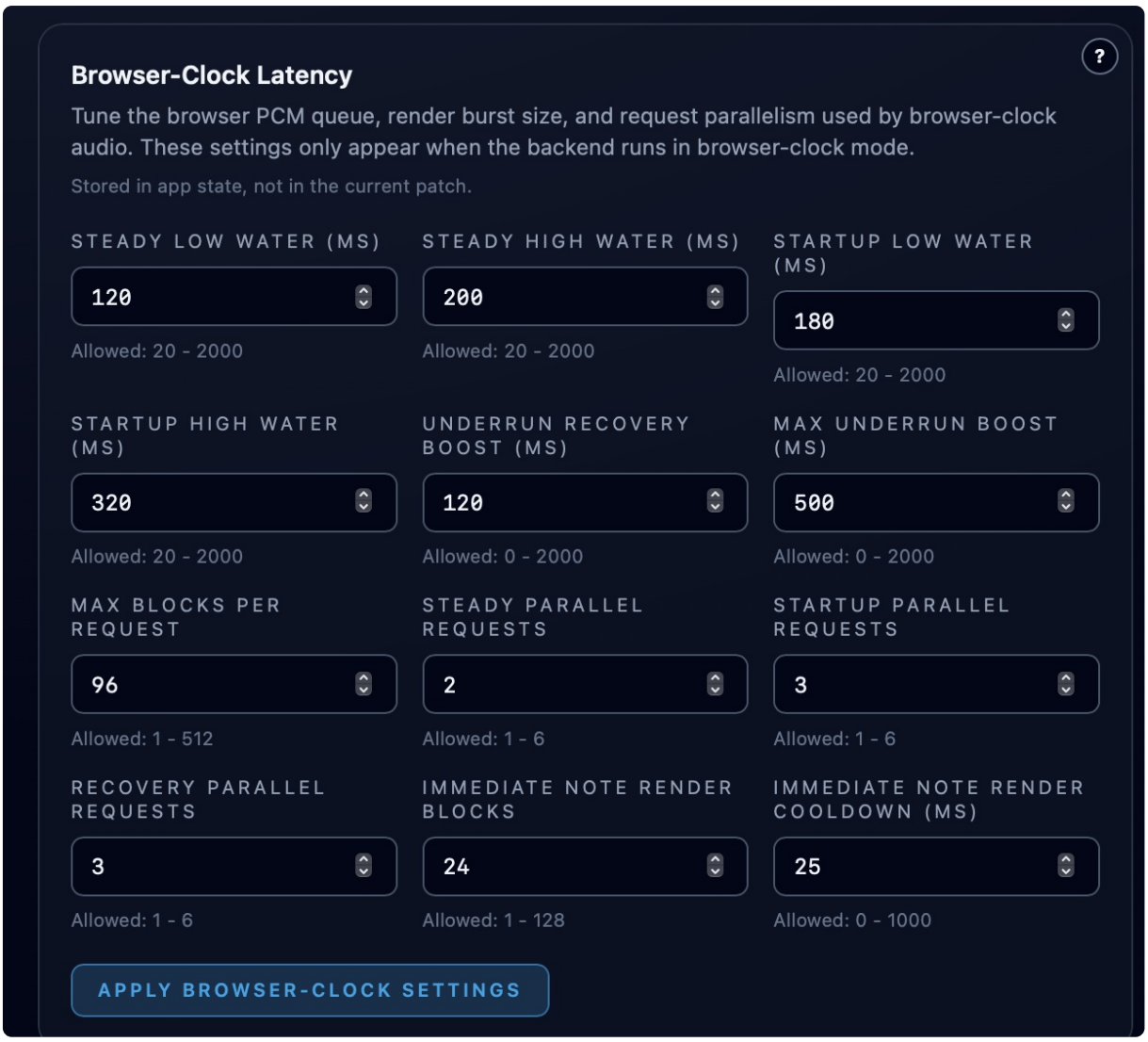
1. Start from the defaults shown by the Config page.
2. If the stream crackles or underruns, raise `Steady High Water` and `Startup High Water` first.
3. If latency still feels too high, lower the steady queue targets cautiously.
4. If live note response feels sluggish, reduce `Max Blocks Per Request` or adjust the immediate-note controls in small steps.
5. Apply the settings and test again with a realistic patch load.

Keep in mind that these settings are stored in app state for the current workspace. They are not saved into the patch itself.

They also work together with the engine settings from the patch:

- browser-clock settings tune queueing and request behavior
- patch engine settings such as `sr`, `ksmps`, `-b`, and `-B` tune the backend render block geometry

## Screenshot




*The Config page's Browser-Clock Latency section, used when Orchestron runs headless in `browser_clock` mode and the browser owns audio playback.*

# Instrument Design

This chapter covers the complete **Instrument Design** workflow on the `Instrument Design` page.

Patches have an editable Instrument Type. Load Patch browses collapsed type groups and searches names and descriptions after four characters and a 500 ms pause. Type is retained when saving and importing/exporting instruments.

## What You Can Do Here

- Build instruments visually from Csound opcodes using the graph editor.
- Maintain multiple instrument tabs (parallel drafts or different patches).
- Use localized integrated help and opcode-level documentation without leaving the app.
- Choose built-in instrument, effect and output starters and define grouped stereo audio interfaces.
- Compile the current graph, inspect generated ORC, and audition drafts in isolation or in a temporary performance snapshot.
- Define advanced input-combine formulas when multiple signals feed the same input.
- Configure function tables with the `GEN` meta-opcode (including `GEN01` audio-file tables and `GENpadsynth`).
- Export instruments as Orchestron bundle files and as `.csd`.

## Chapter Contents

- Patch Toolbar and Instrument Tabs
- Opcode Catalog and Integrated Documentation
- Graph Editor
- Input Formula Assistant
- GEN Table Editor (GEN Meta-Opcode)
- Runtime Panel and Compilation Workflow
- Instrument Import / Export and CSD Export
- Physical Steel-String Guitar
- Supported Opcodes

For the supplied stereo effects and their performance controls, see Standard Effects at the end of the Performance chapter.

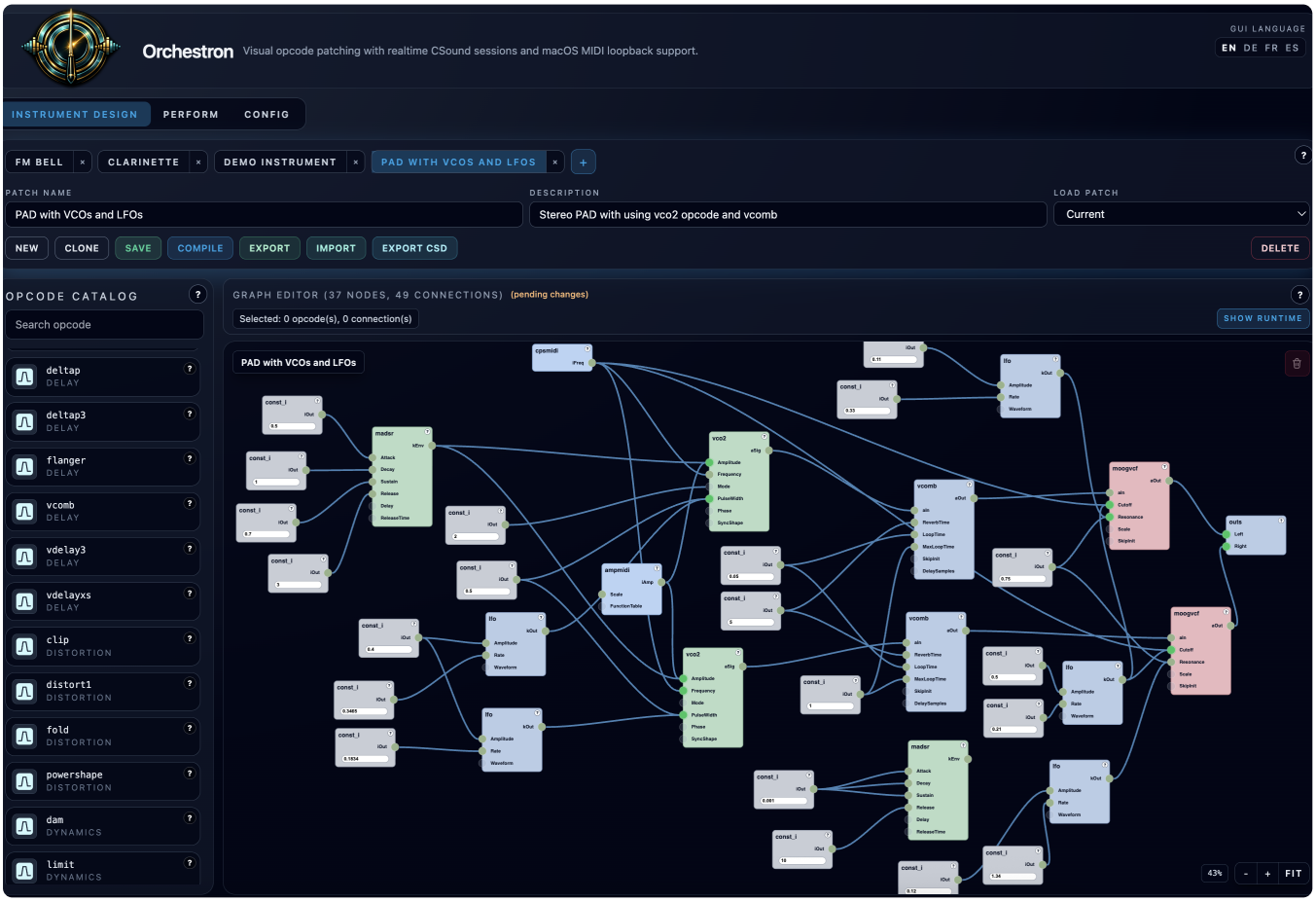
## Recommended Workflow

1. Create or load a patch in the patch toolbar.
2. Add opcodes from the catalog (click or drag-and-drop).
3. Connect signals and set constant values.
4. If needed, define an Input Formula on a socket with multiple inputs.
5. Compile and inspect the runtime panel output.
6. Save the patch.
7. Export a `.csd` or an Orchestron instrument bundle.

## Important Behavior

- The compile status badge is per active instrument tab/patch snapshot (`compiled`, `pending changes`, `errors`).
- Runtime starts collapsed to give more graph space; open it with `Show runtime`. Collapse `Patch controls` to hide metadata and file actions while keeping instrument tabs visible. Both choices survive view switches until browser reload.
- Saving a normal patch performs compile validation first; a failing compile prevents a bad save.
- Saving a patch marked `Template?` skips compile validation so incomplete starter graphs can be stored and reused. Template patches remain loadable in Instrument Design but are not selectable as Perform rack instruments.

# Screenshots




*Instrument Design page overview with catalog, graph editor, and runtime panel.*

# Patch Toolbar and Instrument Tabs

The patch toolbar is the main control surface for patch metadata, patch library operations, and instrument-level file actions.

## Collapse Patch Controls

Use Patch controls to collapse metadata and load/save/compile/import/export actions. Instrument tabs, including add and close, stay visible. Controls start expanded; your choice survives view switches until browser reload.

## Patch Toolbar Layout

The toolbar includes:

- Instrument tabs (multiple editable patch workspaces)
- Patch metadata fields ( `Patch Name` , `Description` )
- Instrument Type selector
- Grouped, searchable patch picker ( `Load Patch` )
- Action buttons ( `New` , `Clone` , `Delete` , `Save` , `Compile` , `Export` , `Import` , `Export CSD` )

## Instrument Tabs

- Use the `+` button to add a new instrument tab.
- Each tab stores its own editable patch snapshot (graph + metadata).
- Tabs can be switched without losing the current edit state.
- Tabs can be closed with the `x` button.
- Tabs are persisted in app state (see Configuration chapter), so drafts can be restored after reload.

## Patch Metadata

- `Patch Name` is the display name used in the patch library and performance rack dropdowns for normal patches.
- `Description` is a three-line patch note field and accepts up to 2048 characters. Hover over the field to see the full description in a tooltip.
- `Template?` marks the patch as a reusable starter graph. Template patches show a `TEMPLATE` token beside their name and are excluded from Perform rack instrument choices.
- `Instrument Type` offers Percussion, Melody, Bass, Effects / Noise, and Continuous (always-on). Choosing Continuous enables always-on scheduling; every other type uses MIDI activation. Continuous patches run when added to the Perform rack and started. They may generate audio without an inlet. Receiving audio requires an actual inlet, independently of activation. Musical role is separate interface metadata.
- Metadata updates affect the current tab immediately, but they are not stored in the patch library until you save.

## Patch Library Loading

- `Load Patch` opens collapsed type groups with counts. Expand a group to see patch names in alphabetical order; template patches retain their marker. Patches already open in instrument tabs are excluded.
- Search matches names and descriptions across all groups, ignoring case and surrounding spaces. Enter at least **four characters**; results update after **500 ms without typing**. Matching groups expand automatically. Shortening the search below four characters restores browsing immediately.
- Use Tab or Up/Down to move through controls and Enter to expand a group or load a patch. Enter in an active search loads its first result. Escape or clicking outside closes the picker; closing resets search and group expansion.
- Loading replaces the active tab with the selected saved patch.

## Actions

### New

- Opens the built-in creation chooser: Playable instrument, Drumset, Audio effect, Audio Output, or Empty patch.
- Creates an unsaved draft with the selected starter graph and default engine settings.
- This draft is not yet stored in the patch library until `Save` is used.

### New from Template

- Opens the same chooser with the five built-in starters plus any saved templates.
- Creates a new unsaved normal patch with the chosen template's graph and description.
- Built-in choices remain available when the library has no saved templates.

### Clone

- Creates a new saved patch by duplicating the current patch graph and metadata.
- Preserves the `Template?` flag and Instrument Type from the source patch.
- If the name already exists, Orchestron generates a `(... copy)` style name.
- The cloned patch is loaded automatically after creation.

### Delete

- Deletes the currently saved patch from the patch library.
- Requires confirmation.
- After delete, Orchestron loads another available patch or creates a new draft if none remain.

### Compile

- Compiles the current instrument graph into generated Csound ORC/CSD artifacts.
- Updates compile status badge state.
- Does not save the patch or start audio automatically. Use Audition in the graph header to hear an unsaved draft.

### Save

- Performs compile validation first for normal patches.
- If compile succeeds, persists the patch to the backend patch library.
- If compile fails, save is blocked and the patch remains unsaved.
- Template patches skip compile validation so incomplete starter graphs can be saved.
- Always-on patches still save as normal patch definitions; they run continuously only after they are added to a started Perform rack. A continuous source need not be an effect or have audio inlets.

### Export / Import

- `Export` writes an Orchestron instrument definition bundle (`.orch.instrument.json` or `.orch.instrument.zip`).
- `Import` loads an instrument definition bundle and supports conflict handling (overwrite / rename / skip).
- Details are in Instrument Import / Export and CSD Export.

### Export CSD

- Triggers compile and downloads the compiled `.csd` file.
- Useful for running the generated instrument outside Orchestron.

## Practical Notes

- `Compile` is the fast iteration action while designing.
- `Save` is the persistence action for the patch library.
- `Export` is the sharing/transfer action for Orchestron-specific instrument bundles.
- `Export CSD` is the interoperability action for raw Csound usage.
- The integrated `?` help for this toolbar now focuses on tab-local draft state versus saved library state, plus the

difference between `Compile`, `Save`, `Clone`, and export actions.

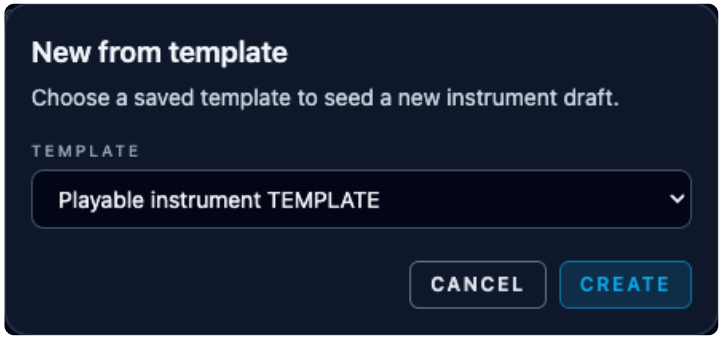
## Built-in creation choices

New and New from template offer Playable instrument (MIDI pitch/velocity, sine oscillator and madsr), Audio effect (stereo pass-through), Audio Output (stereo input to direct output), and Empty patch. These choices work without saved templates. Guided instruments connect to an fixed internal Master when added to a performance.

### Choose a starter

| Choice | Type, activation and audio interface | Next step |
|---|---|---|
| Playable instrument | Melody; MIDI notes; named stereo output; pitch, velocity, sine oscillator and envelope | Save, add to Perform, and play its MIDI channel. |
| Drumset | Percussion; MIDI notes; named stereo output; analog drum voices | Save, add to Perform, and sequence its drum notes. |
| Audio effect | Continuous; named stereo input and output, initially pass-through | Add processing between the input and output before using it as an effect. |
| Audio Output | Continuous; stereo input to Direct Audio Output ( `outs` ) | Save for use as a custom output processor. |
| Empty patch | Melody; empty graph for custom design | Build and compile the required signal path. |

Role describes the patch's audio interface; Instrument Type determines whether it runs continuously or on MIDI notes. Audio effect and Audio Output starters use Continuous; use Effects / Noise for MIDI-triggered sound effects. Every performance has a fixed internal Master. It has no editable patch or rack assignment; add processing through its mixer inserts.



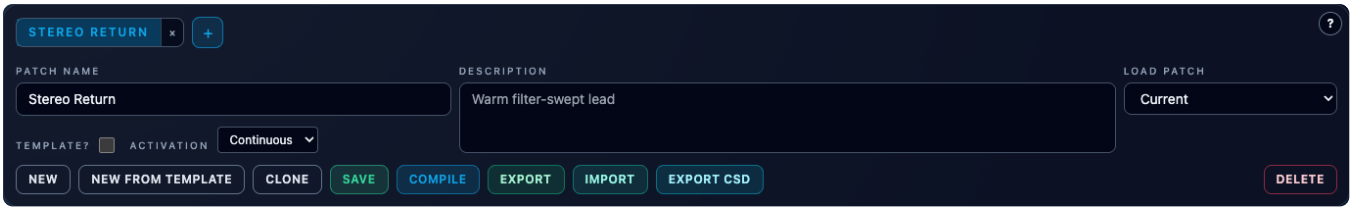

*New opens the starter chooser even when no saved templates exist.*

## Older patches and saved metadata

Existing always-on patches become Continuous without changing playback. Other older patches are classified from their names first, then descriptions: drum/percussion terms precede bass terms, followed by melodic and sound-effect terms. Unmatched patches default to Melody. Review the inferred Instrument Type and change it when needed; later name or description edits do not reclassify a patch.

Types are saved with the patch and retained in clones, templates, restored tabs, and instrument/performance JSON or ZIP bundles. New playable and empty patches default to Melody, drumsets to Percussion, and effect/output starters to Continuous.

## Screenshots




*Earlier toolbar with a continuous stereo-effect draft and library actions. Instrument Type now replaces Activation; an updated screenshot is pending.*

# Runtime Panel and Compilation Workflow

The Runtime panel supports live testing of the current patch and exposes compile/runtime diagnostics for the unified browser-clock runtime.

## Runtime Panel Sections

- `MIDI Input` selector
- `Compile Output` (generated ORC)
- `Browser Audio` status for browser-clock mode
- `Session Events` log (recent backend/runtime events)

## MIDI Input Binding

Use the `MIDI Input` dropdown to bind an external MIDI input to the active runtime session.

- This is how external hardware/DAW MIDI reaches the running instrument session.
- `internal:loopback` is always available as the built-in app-only loopback.
- Internal sequencers, piano rolls, and manual controller lanes always go straight to the session engine and do not depend on this binding.
- The selected input is session-specific and also reflected on the Performance page status footer.

See MIDI Setup and Inputs for OS-level MIDI setup guidance.

## Compile Output (Generated ORC)

After compiling a patch, the panel shows the generated ORC text.

This is useful for:

- Verifying the compiled signal chain
- Understanding how formulas/GEN/meta-opcodes are rendered
- Debugging compile issues with concrete generated code

A compilable patch must contain at least one audio output sink: either `outs` for direct stereo output or `outleta` for named audio routing into another instrument. This allows source instruments to feed always-on effects without also writing directly to the final stereo output.

## Session Events

The event list shows recent session events (most recent first, limited window in the UI).

Typical uses:

- Confirm that start/stop events happened
- Inspect runtime payloads and state transitions
- Diagnose issues during live testing

## Browser Audio

The runtime now always uses `browser_clock` audio mode, so the Runtime panel can show:

- Browser PCM queue/runtime status (`connecting`, `live`, `error`)
- Error message when the controller socket or AudioWorklet path fails
- No `<audio>` player, because playback is owned directly by the browser `AudioContext`

See Browser-Clock Latency for the browser-clock mode guide and tuning workflow.

## Runtime Panel Collapse / Show

Runtime starts collapsed. Open it with Show runtime in the graph header and close it with Hide. Your choice survives view switches until browser reload; compilation and playback continue while it is hidden.

To maximize graph editor space:

- Click `Hide` in the Runtime panel to collapse it
- Use `Show runtime` in the graph header to bring it back

This is especially useful on smaller displays or complex patches.

## Compile Status Badge (Graph Header)

The instrument page header above the graph displays a compile-state badge for the current patch/tab snapshot:

- `compiled` - the current graph snapshot matches the last successful compile
- `pending changes` - the graph changed since the last successful compile
- `errors` - the last compile for this exact snapshot failed

This helps you avoid assuming a graph is live-valid after edits.

## Recommended Test Loop

1. Edit the graph
2. Compile
3. Read compile output / diagnostics
4. Open Audition, choose a preview context, then Prepare audition
5. Use Stop and audition to hear the draft; close the preview when finished
6. Save only after compile is clean

The integrated `?` help for this panel now focuses on session-specific external MIDI binding, generated ORC inspection, runtime event feedback, and browser-clock PCM playback.

## Audition a draft

Compile validates the graph without starting audio. Audition offers Isolated and In current performance. An isolated effect receives the built-in test instrument or a selected saved source. Current-performance audition replaces one selected rack instance in a temporary snapshot; choose the instance explicitly when a patch appears more than once.

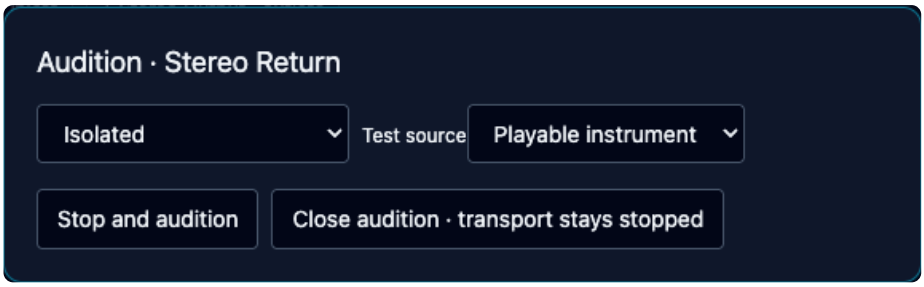
Prepare compiles the temporary preview before interrupting the performance. Stop and audition explicitly stops the original performance and starts the preview. Closing deletes the temporary session and restores the original performance state and playhead with transport stopped. Draft patches, test sources and preview routes are never saved into the library.

### Preview step by step

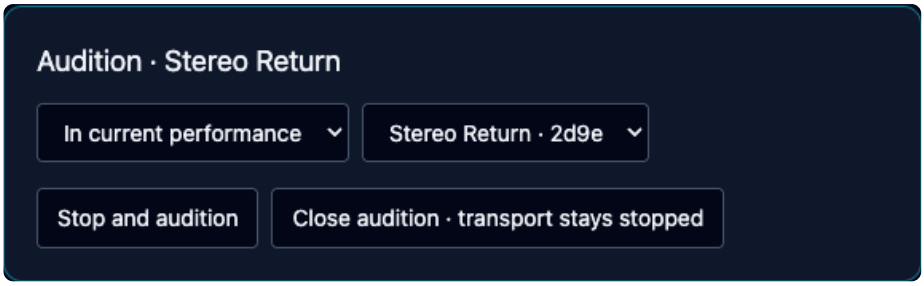
1. Click **Audition** in the graph header with the draft you want to hear.
2. Choose **Isolated** to test only this patch. For an effect, select **Test source**: the built-in Playable instrument or a saved source with suitable outputs. Isolated preview needs a stereo main output; source outputs must match the effect input count.
3. Alternatively choose **In current performance** and select the rack instance in **Source**. The short ID distinguishes instances of the same patch, including a return and a separate insert. The draft replaces only that selected instance in the temporary performance snapshot.
4. Click **Prepare audition**. Compilation errors appear in the dialog; fix the draft and prepare again. Preparation does not interrupt the original performance.

5. Click **Stop and audition** to stop the original engine and start the compiled preview. Note-triggered preview sources receive a test note (MIDI 60, velocity 96); current-performance mode also uses the performance sequencer configuration.
6. Click **Close audition · transport stays stopped**. The preview session is deleted and the original performance state/playhead is restored with transport stopped. Restart the performance explicitly when ready.

If you edit the draft after preparing, prepare again before listening. Preview is temporary: use Save separately to update the library patch.




*Isolated effect preview after successful preparation, ready for Stop and audition.*




*Current-performance preview targets one explicitly selected rack instance.*

## Screenshots

*Runtime panel showing MIDI input selection, generated ORC, active browser-owned PCM runtime, queue diagnostics and recent session events.*

## Audition notes for branching instruments

The **Test MIDI note** field chooses which note audition sends. A Switch driven by `notnum` starts with its first MIDI-number case (36 for Drumset); other patches start at 60. Change it to 38 or 42 to test the other drums, or to an unmatched note to check Default Silence. Branching guide.

# MIDI Setup and Inputs

This page covers practical MIDI setup and how external MIDI binding works in Orchestron's unified browser-clock runtime.

## External MIDI Input Binding In Orchestron

External MIDI input is bound in the **Instrument Design Runtime panel** (not on the Config page).

Workflow:

1. Open Instrument Design.
2. Use the Runtime panel `MIDI Input` dropdown.
3. Select the desired input:

- `internal:loopback` for the built-in app-only loopback and default fallback binding
- a `host_bridge` input published by `host-midi-helper` for external hardware or DAW MIDI

1. Start the instrument session and play from your hardware/DAW.

The selected MIDI input is reflected in the Perform page footer status line.

## Internal Performance Controls Do Not Require External MIDI

These features generate MIDI/control events internally and do not require an external MIDI input device or host bridge:

- melodic sequencers
- controller sequencers
- arpeggiators
- piano rolls (on-screen keyboard)
- manual MIDI controller knob lanes

External MIDI input is only needed when you want to play Orchestron from outside the app. Internal MIDI is always delivered directly into the session engine and ignores the external input binding.

That means:

- switching the session input does not disable sequencers, arpeggiators, piano rolls, or manual controller lanes
- binding a helper-provided external input adds another source into the same engine scheduler

## External MIDI Bridge

The backend no longer relies on direct OS MIDI enumeration for session playback. External MIDI arrives through the optional Rust host bridge in `host-midi-helper/`.

General setup pattern:

1. Start the backend with `VISUALCSOUND_HOST_MIDI_TOKEN` set.
2. Start `host-midi-helper` on the same host OS with the same token.
3. Bind the helper-published input in the Runtime panel.

If no helper is running, `internal:loopback` remains available and internal performance tools still work.

The backend still performs backend-native MIDI enumeration where available, but live session playback now uses the helper-backed `host_bridge` path for external devices.

## macOS Loopback Setup (IAC Driver)

For DAW/app -> Orchestron MIDI routing on macOS:

1. Open **Audio MIDI Setup**.
2. Open **MIDI Studio**.
3. Open **IAC Driver**.
4. Enable **Device is online**.
5. Add/select an IAC bus (for example `IAC Driver Bus 1`).
6. Route your DAW/app MIDI output to that IAC bus.
7. Select that input in Orchestron's Runtime panel.

# Troubleshooting MIDI Inputs

## No External MIDI Inputs Listed

- Ensure your OS/device/virtual bus is enabled (for example IAC Driver on macOS, ALSA/JACK route on Linux, loopMIDI on Windows).
- Confirm `host-midi-helper` is running and uses the same token as the backend.
- Restart the helper after changing system MIDI configuration.

## Perform Page Footer Shows `midi input: internal:loopback`

- Internal app MIDI is working.
- External hardware/DAW MIDI will not reach the session until a helper-provided input is bound.

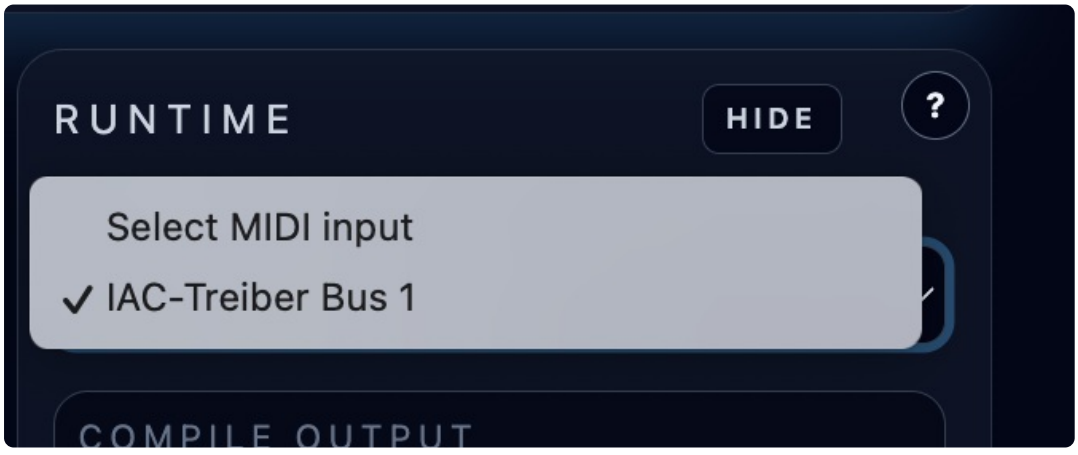
## Sequencer/Piano Roll Works But Hardware MIDI Does Not

- This usually means internal controls are working, but no external helper-provided MIDI input is bound.

# Related Pages

- Runtime Panel and Compilation Workflow
- Live Status and Safety Controls

# Screenshots




*Runtime panel MIDI input dropdown expanded with available MIDI inputs.*

# Live Status

The Perform page includes several status for live operation.

## Browser Audio Health

The Runtime panel shows queued PCM and pending-render milliseconds, underrun and overrun counters, and the latest backend render-time/audio-time percentage while a browser-clock session is active.

Audio refills and PCM writes run in a Dedicated Worker. Sequencer step and pad updates are carried with the rendered PCM and applied only when their target frame becomes audible. If the Perform page is hidden, visual animation pauses and resumes from the current audible snapshot instead of replaying missed steps.

Performances are limited to 128 flattened backend note tracks. Each drummer row becomes one backend note track, so the total includes melodic sequencers plus all drummer rows.

## Footer Status Bar

At the bottom of the Perform page, Orchestron shows live status values such as:

- `playhead` position (`current step / step count`)
- `cycle` count
- active `midi input` name (for example `internal:loopback` or a helper-provided device)

This footer is useful during performance because it gives quick feedback without opening the Instrument Design runtime panel.

## Track State Labels (Queued Changes)

A track's state badge can reflect queued actions that apply at the next cycle boundary, for example:

- starting @ step 1
- stopping @ step 1

This helps you understand why a track button press did not take effect immediately while transport is running.

## Error Banner (Transport / Runtime Errors)

The Perform page displays an error banner for problems such as:

- no active instrument session when trying to start sequencers
- trying to play piano roll notes before the instrument engine is ready
- pad queue request failures
- controller send failures
- sequencer config sync failures

## Editing During Playback

Notes, controller curves, tempo and arrangement edits keep the transport at its current position. Rapid edits are combined after an 80 ms pause; the previous configuration keeps playing while the update is prepared. The completed edit takes effect at the next engine block. Editing a displayed pad changes that pad, including when the arranger has selected it automatically.

Automatic pad changes, playhead movement and status updates do not resubmit the performance configuration. Queued pad launches still wait for their usual pad boundary. Stop cancels pending configuration application. Shortening a finite arrangement past the current position stops playback without rewinding.

If preparation fails, the error banner reports it and the previous configuration continues playing. Your edit remains in the editor. Make another edit or explicitly restart to try again; the app does not continuously retry a failed update. Saving and exporting retain the existing file formats.

## MIDI Input Reference

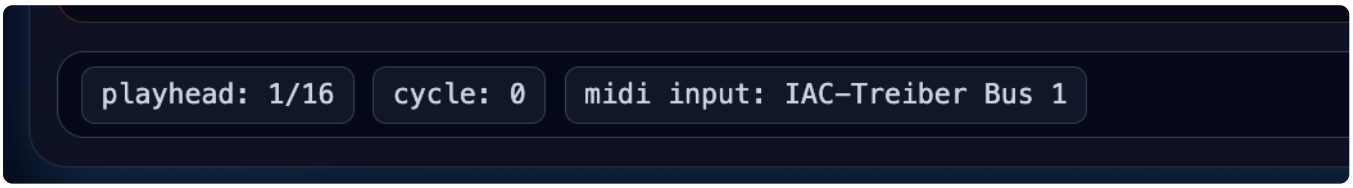
The footer's MIDI input display reflects the session's bound external MIDI input (selected in the Instrument Design Runtime panel). If it shows `internal:loopback`, internal app MIDI is still active, but external hardware/DAW MIDI will not reach the session until a helper-provided input is bound.

See Runtime Panel and Compilation Workflow and MIDI Setup and Inputs.

## Live Performance Safety Checklist

1. Confirm instrument session is running.
2. Confirm expected MIDI input is bound.
3. Check track/controller/piano-roll enable states.
4. Save the performance after major routing changes.

## Screenshots




*Footer status bar with playhead/cycle/MIDI input.*

## Definition audition

Audition is a session-only track override on the shared transport. The sequencer’s existing Audition control switches a running track at its next pad/rest boundary after preparation succeeds; failures preserve current playback and edited drafts. Other tracks continue. Return to arrangement locates the authored sequence at the then-current song position, including pauses anc finite endings. Stop audition clears the override and stops only that track.

Arranger Play clears all auditions before applying normal playback-source behavior. Arranger Stop clears/stops auditions while preserving independently started manual tracks. Seeks and loop wraps restart active auditions from their first token at the destination. Removing devices, replacing the performance or engine session, and engine shutdown clear transient audition state. Runtime status never becomes authored configuration.

## Temporary lane output controls

Arranger Mute/Solo gates a lane's future MIDI output without disabling its sequencer or changing its playhead, audition, repeat or playback source. Existing notes release normally. Drummer rows change together; multiple solos are supported and explicit mute wins. These controls are separate from mixer-strip controls, so lanes sharing one instrument remain independently controllable. See lane Mute/Solo for arpeggiator dependencies, controller values and reset behavior.

Arranger speaker previews use a 250 ms hold delay, then switch at the next lane cycle boundary during arranger playback. While stopped, they start at the next available engine block after preparation without moving the arranger cursor or changing its highlighted Stop state. Play is highlighted only during song playback. Releasing before the queued boundary cancels the preview without disturbing the current notes. Release restores the previous arrangement/manual/audition mode at the current clock position; stopped lanes stay stopped. Gesture identities and revisions reject stale commands, including release-before-start. Pending attacks and controllers from replaced material are discarded; note releases retain source ownership so other lanes sharing a channel remain intact. Preview restoration state is temporary and never saved.

# Multitrack Arranger

The arranger is the only editor of song order. Each lane places references to its eight pads, reusable groups and supergroups, and rests. Sequencers edit pad contents; melodic and drummer devices assemble reusable phrases in their upper-right pattern workspace. In the arranger, edit phrases by ungrouping, rearranging, and regrouping timeline material. Editing a shared definition changes every occurrence; changing its duration shifts following material on that lane.

Pads are labelled **1–8** in arranger lanes and palettes, and **#1–#8** in sequencers and phrase editors. Pads are green, groups are red, and supergroups are violet throughout the sequencers, phrase libraries, palettes and arranger lanes. Rests keep their dark styling; the number in **Rest 1/2/4/8/16/32** is its duration in local beats, while a pad number identifies a reusable slot. Right-click a musical item or palette entry to Set colour or Reset colour. Custom colours apply to every occurrence of that definition within its lane and are saved with the performance. The picker offers swatches and a custom colour; text contrast adjusts automatically. Sequencer colours remain the defaults for now. Selection outlines and Playing/Queued indicators remain separate from these colours.

## Compact track headers

Each lane starts collapsed. Click the triangle or track title (Enter/Space when focused) to open or close its editing tools. Click anywhere in an occurrence box to select it and open the lane; the triangle can close it again while retaining the editing context. Clicking empty timeline space clears occurrence highlights and chooses the insertion position without changing panel expansion. Double-click a pad or choose Edit pattern to open its sequencer editor. Open in arranger opens the lane and focuses the definition in its palette.

The expanded lane shows **Position (master beats)** followed by the wrapping palette. There is no separate Patterns and phrases panel in the arranger; melodic and drummer sequencers provide their own pattern workspace. Choose Playback source in the sequencer; the arranger transport provides the Loop song toggle for repeating or stopping at the song end. Disclosure choices survive view switches; New/Load/Import resets them. Closing panels preserves drafts, selections and scroll, cancels active gestures and momentary previews, and leaves normal playback running.

## Temporary lane Mute and Solo

The small **Mute** and **Solo** buttons control this sequencer lane, independently of the instrument mixer. Several lanes can drive the same instrument and still be controlled separately. A drummer's rows act together. Multiple Solo buttons may be active; explicitly muted lanes stay muted. A dashed amber Mute button indicates suppression by another lane's Solo.

Lanes keep advancing through pads, rests, loops and auditions while suppressed. Changes apply at the next available engine block; already buffered audio still plays. New attacks are suppressed, while sounding notes complete their scheduled releases and effects decay normally. Unmuting resumes future events at the current position without retriggering held notes. Controller lanes stop sending CC values while muted; the last applied value remains until an active source updates it.

Solo preserves the upstream inputs and downstream arpeggiator processing needed by the selected lane. Explicit mute overrides these dependencies. External MIDI and independent Live arpeggiators remain outside arranger solo selection. Instrument mixer mute/solo still applies independently.

Lane controls survive transport stop/start and engine restart within this performance. They reset on New/Load/Import and browser reload; deleted lanes lose their controls. Save, autosave, native bundles and both CSD exports exclude this temporary state.

## Place and move material

Expand a lane to show its palette of populated pattern pads, groups, supergroups and rests on the Position row. Melodic, drummer and arpeggiator pads appear only when they contain a non-rest step; their original numbers are preserved. Empty pads remain editable in the sequencer, and existing timeline occurrences remain intact. Controller curves remain available, including zero-valued curves. Drag a palette item to add an occurrence. Clicking an entry only focuses it. Empty phrase drafts cannot be placed or previewed. There are no placement toolbar buttons or number-key placement shortcuts; copy, paste and duplicate are available for timeline ranges through the context menu and keyboard.

Drop at an element edge to insert and shift later material on this lane. Drop musical material inside a rest when it fits to replace that portion of silence. Drop after the final element to fill the gap with supported pause tokens and append. Dropping inside occupied musical material is rejected with a red preview. Rest palette items insert time at boundaries or extend the lane after its end.

The ruler numbers master bars and beats. Placement snaps to whole local beats using the lane's beat ratio; previews show the exact master-beat position and duration. Drag an occurrence body to move it, or move a contiguous selection together. Moves leave equivalent silence at the source; dropping at the original position changes nothing. Moving left or right into a rest can overlap the vacated source position. Available silence is split before and after the moved item; other musical items keep their positions, and overlapping them is rejected. Placement stays within the originating lane. Invalid or over-limit edits leave the whole arrangement unchanged.

Right-click to open lane actions. Right-clicking a selected occurrence preserves the selection; right-clicking another selects it. Cmd/Ctrl-click toggles additional selections. Grouping requires at least two adjacent selected elements. **Group** accepts pads/rests; **Supergroup** also accepts groups. **Ungroup** expands one level and retains the reusable definition in the palette.

After ungrouping, rearrange the material, then select and group it again. Choose **Create new definition** (the default) to leave other occurrences unchanged, or explicitly select **Update existing definition** to update all its references. The dialog lists uses and explains that duration changes shift following material. Groups retain letter IDs and supergroups Roman numeral IDs.

**Create variation** copies a pad into an unused slot, or copies a group/supergroup under a new ID, and retargets only the selected occurrence. It inherits the source colour. The eight-pad limit remains. Palette context menus allow deleting unused definitions; saved references list their uses and block deletion. Temporary workspace references expand into the saved contents, including retained drafts in closed panels; the deleted definition’s own draft is removed.

Use Enter/Space to select a focused occurrence, Cmd/Ctrl-click for multiple selection, and Shift+F10 or the Context Menu key for its menu. Arrow keys navigate occurrences or menu actions; Escape closes menus and cancels gestures. Menus return focus to their originating item.

## Copy and insert song sections

Drag the **top beat ruler** to select a section across every arranger lane, including collapsed and muted lanes. To select some lanes, drag a rectangle from empty timeline background, or **Shift-drag** over existing elements. The cyan highlight includes silence and displays its duration in master beats and the number of selected lanes. Selection expands outwards to preserve whole pads, groups and supergroups across all included lanes; rests can split at whole local beats. Lanes with different beat ratios may therefore expand the range farther than expected: check the preview before releasing.

The top ruler’s cyan **edit position** is separate from playback. Click it to choose a paste position. The bottom **Loop selected range** ruler retains its seek and playback-loop behavior. Selecting or copying material does not seek or change the loop.

For two 16-beat sections **A–B**, select the first 16 beats and choose **Duplicate — insert time, all lanes**, or press **Cmd/Ctrl+Shift+D**. The result is **A–A–B**. The new copy stays selected, so the same command can repeat it again.

| Action | Shortcut | Result |
|---|---|---|
| Copy range | Cmd/Ctrl+C | Copies selected lanes and the entire range duration, including silence. |
| Paste — overwrite | Cmd/Ctrl+V | Replaces that interval on copied lanes without shifting later material. |
| Paste — insert time, all lanes | Cmd/Ctrl+Shift+V | Inserts the copied duration across all lanes, shifting later material together. Unselected lanes receive silence where needed. |
| Paste — insert, copied lanes only | Context menu | Shifts later material only on the copied lanes. |
| Duplicate — insert time, all lanes | Cmd/Ctrl+Shift+D | Inserts a copy immediately after the selection and shifts every lane’s later material. |
| Duplicate — insert, selected lanes only | Context menu | Inserts the copy after the selection and shifts only its lanes. |

| Action | Shortcut | Result |
| --- | --- | --- |
| Undo / Redo | Cmd/Ctrl+Z / Cmd/Ctrl+Shift+Z | Reverses or restores one complete arranger action, including multi-lane range edits. Ctrl+Y also redoes. |

Right-click inside the highlighted range to keep the selection and open its commands. Right-click elsewhere in a timeline or on the ruler to choose a destination; occurrence menus also contain paste actions. **Shift+arrows** extends a selection in time or across lanes. **Shift+F10** opens its menu. Shortcuts apply only inside the arranger, outside text fields and controls.

**Alt/Option-drag a selected occurrence** to copy the entire range with overwrite behavior. Hold **Shift** as well to insert time across all lanes. The destination preview states Overwrite or Insert, its position and the affected lanes; red indicates an invalid destination and explains why. Escape, pointer cancellation or loss of focus cancels the gesture. Ordinary occurrence dragging exits range selection and retains its lane-local move behavior. Delete commands remain occurrence actions; select individual occurrences before deleting them.

Paste keeps the copied lane assignments. It does not remap instruments or copy between performances. Copies reuse their original pads, groups, supergroups and colours. **Create variation** separates an occurrence's definition; for independent note or controller edits, create a variation of its pad too. The eight-pad limit still applies.

Insertion requires a musical boundary on every shifted lane; overwrite requires valid start and end boundaries. Durations and rests must be whole local beats. Invalid boundaries, missing lanes, changed definitions/durations or sequence limits reject the whole edit. Empty space after a lane ends copies as silence. Insertion leaves already-ended lanes unchanged unless receiving copied material. Automatic repetitions are not expanded.

After the first placement selects Arrangement, range edits preserve Playback source, Repeat, the transport cursor and loop coordinates, including dormant arrangements on Manual-pad lanes. Independently playing Manual pads continue. During playback, the whole edit follows the normal live-preparation workflow; preparation failure retains the previous playing configuration and the edited draft.

Selection and clipboard survive collapse and view switches within the session. New/Load/Import resets them; save/export excludes them. After definition or duration changes, copy again.

## Undo and redo

The two curved-arrow buttons immediately to the right of the device summary undo or redo one arranger action. Their tooltips describe the next action; unavailable directions are disabled. The same history is available through context menus and **Cmd/Ctrl+Z**, **Cmd/Ctrl+Shift+Z**, or **Ctrl+Y** inside the arranger. Text fields retain their normal editing shortcuts.

Hold either button for **500 ms** to open its history without taking a step. Undo lists past actions, newest first; choosing one undoes that action and everything after it. Redo lists future actions in execution order; choosing one restores through that action. The result applies as one update. Arrow Down on a focused button opens the menu; arrows navigate, Enter selects, and Escape closes it. Menus also close on outside clicks, collapse, view changes and performance replacement.

History retains **25 actions total across Undo and Redo**, including placement, movement, deletion, rest resizing, range edits, grouping/ungrouping, definition updates/deletion, variations, colours and the automatic source change on first placement. Older saved source/repeat history entries remain supported. A completed drag, multi-lane edit or pad variation is one action. Failed, cancelled and unchanged edits do not count. After undoing, making a new arranger edit discards the future actions and starts a new branch.

Both past and future are saved with the performance, autosave and native exports. Load/Import restores the incoming history; New and older files start empty. History survives collapse and view switches. Invalid or incompatible saved history is discarded with a notice while the musical performance still loads.

Changes outside the arranger are not recorded. Changes to related arrangement fields, shared definitions, pad lengths, beat ratios, removed lanes or a pad slot restored by a variation clear the entire history with a brief notice, even while the arranger is closed. Unrelated notes, mixer controls, Mute/Solo, transport, auditions and editor navigation preserve it. Undo does not restore the playhead, loop coordinates or editor selections, and does not overwrite unrelated musical edits. During playback, musical restoration follows the normal live-edit preparation rules; colour-only restoration does not prepare audio. Independently running devices keep their playback, with source changes applied only to the affected devices.

## Delete and resize rests

- **Delete/Backspace** and **Remove, leave gap** replace selected musical occurrences with equal-duration silence. Other occurrences keep their positions.
- **Remove and close gap** (also Shift+Delete) removes selected musical occurrences or rest spans and closes their time on this lane. Multiple selections form one edit.
- Consecutive pause tokens appear as one rest span. Drag its right edge or choose Duration from its context menu to resize it in whole local beats, shifting later items.
- Leading, internal, trailing and rest-only arrangements are preserved. Trailing silence contributes to Fit, playback duration, repetition and exports.

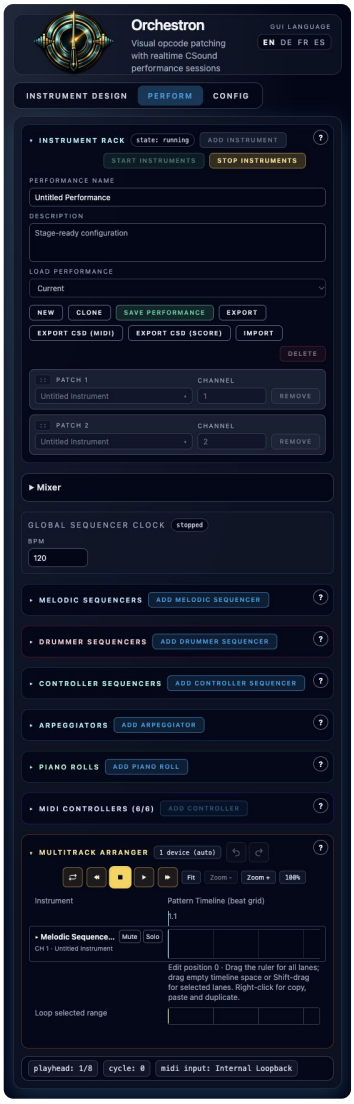
## Playback and audition

**Playback source** is a two-option toggle in each sequencer: **Arrangement | Manual pads**. New devices use Manual pads until the first item is placed on their arranger lane, which selects Arrangement automatically. Subsequent source choices are saved with the performance and survive later edits and reloads. Closing all time returns an empty lane to Manual pads; ordinary deletion leaves rests and retains the selected source.

The arranger controls provide **Loop song** to repeat or stop at the song end. Imported per-lane repeat settings remain intact. **Loop selected range** is the separate shared transport loop. Click its ruler to seek and clear the loop, or drag a range to loop it. Rewind/Fast forward move one master beat. Fit shows the whole longest lane, including trailing rests.

**Play** runs from the current cursor to the end of the longest lane, including trailing rests. **Loop song** is the repeat-arrows toggle immediately left of Rewind: when enabled, playback returns to the beginning at the song end; when disabled, it stops at the end. Changing it during playback keeps the current position, and changing it while stopped does not start playback. The toggle is saved with the performance and defaults to off for new and older performances. It is independent of arranger Undo/Redo. With a marked loop range, playback repeats only that range regardless of the Loop song toggle; clearing the range restores the selected whole-song behavior. **Repeat track sequence** can repeat a shorter lane within these bounds; it never extends song playback beyond the end. The loop ruler and lane playheads share the same horizontal origin when zooming and scrolling.

**Play** ends temporary auditions and workspace playback, starts Arrangement lanes, and preserves independently playing Manual pads. It starts the instrument engine when needed. **Stop** clears temporary playback and stops Arrangement lanes while preserving independently started manual sequencers. If Play started the instruments, Stop also stops them. If the instruments were already running before Play, Stop leaves them running until you stop them manually. Double-click Stop resets the arranger cursor to the loop start or song start without resetting a running manual pad.




Each device's **Play** works with the arranger stopped. With **Arrangement**, the first device starts at the selected loop's beginning, or at the stopped arranger cursor when no loop is selected. A cursor at song end restarts from song start. Further Arrangement devices join the current song position without restarting the others. Authored rests and each lane's Stop/Repeat setting still apply. Independent playback leaves the arranger's cursor and global Play indicator stopped.

With **Manual pads**, device Play starts the pad selected for editing from its beginning and repeats it. While it runs, clicking another pad queues it for the end of the current pattern: orange marks the queued pad, then blue-green marks it once playing. While stopped or using Arrangement, clicks only select the editing pad. Manual pads keep their phase through arrangement seeks, loops and endings. Device Stop affects only that device; changing Playback source while playing changes only that device. This lets you develop one pattern while other devices play backing parts from the arrangement. Controller sequencers and Arranger-mode arpeggiators follow the same rules; arpeggiators still need input notes. Live arpeggiators keep their existing behavior.

Every musical occurrence and available pad/group/supergroup has a small speaker icon near its right edge. Hold it for**250 ms** to begin a momentary preview from its first token. Shorter clicks do nothing. While the arranger is running, prepared previews start at the lane’s next pad/rest cycle boundary. While stopped, they start at the next available audio block without moving the arranger playhead; existing audio buffering still contributes latency. It repeats while held. Selecting content never launches it.

Release the speaker to end preview and restore the lane's previous playback at the current shared-clock position. An Arrangement lane can return into a rest or a finite track that has ended. Previously stopped lanes stay stopped. Independently playing manual pads and existing sequencer auditions are restored. Other lanes continue and lane Mute/Solo remains effective. Releases preserve other lanes sharing an instrument/channel.

Hold Space/Enter on a focused speaker for the same behaviour. Releasing outside it, cancelling the gesture, losing focus, collapsing, or leaving the view ends the preview. Preparation failures retain current playback; releasing before preparation

finishes prevents a late launch. Transport Play/Stop cancels previews, and a late release cannot restart playback.

With the arranger stopped, preview starts only that lane through the normal instrument startup flow. The song cursor stays fixed and the Stop button stays highlighted. Play resumes the song from that cursor. During song playback, Play is highlighted instead. Seeks and loop wraps restart held previews at their first token on the shared clock. During song playback, song range/end remain authoritative. Arranger arpeggiators support previews; independent Live arpeggiators remain unchanged.

Melodic and drummer sequencers use a pattern workspace with a Play/Stop toggle and the same momentary speakers. A speaker held over workspace playback restores the workspace on release; stopping the workspace restores the prior lane state. Controller sequencers and arpeggiators retain their existing Audition/Return/Stop controls.

## Saved format

Performances store root sequences, pads, groups, supergroups and supported pause tokens. Optional `sequencer.arrangerSongLoopEnabled` stores the whole-song loop toggle and defaults to `false` when absent. Save/load, autosave, and native bundles preserve it; CSD exports still render the complete finite song. Optional `arrangerHistory` metadata has its own version 1 and stores up to 25 transactions with a current cursor; it is excluded from audio configuration. Current saves use performance version 18 and support imports of versions 1–18; app state uses version 3 and the native bundle envelope remains version 1. Old enabled arrangements that resolve to no tokens are imported as an explicit occurrence of their former active pad, preserving their repeat setting and sound.

Optional `definitionColors` metadata maps typed definition references (`pad:0`, `group:A`, `super:I`) to hexadecimal colours. It does not change sequence compilation or trigger audio preparation. Missing/invalid colours use type defaults. Save/load, autosave, and native bundles retain valid colours. CSD audio ignores them.

Audition/preview commands and restoration state, selection, position, sounding pads and temporary enablement are session state. Save, autosave, native bundles and both CSD exports use authored data. Musical edits made while auditioning save normally.

# Pattern Pads and Reusable Phrases

Each sequencer has eight pattern slots, #1–#8. Click a pad to select it for editing; on a running Manual pads device, this also queues it for playback. Playback and queued launches never change this editing selection. The Playing and Queued indicators show what is sounding separately.

Pads are labelled **1–8** in arranger lanes and palettes, and **#1–#8** in sequencers and phrase editors. In melodic and drummer sequencers, pads are green only when their playable steps contain at least one note or active hit with velocity above zero; empty, hold-only and zero-velocity pads stay dark. Selection and queued outlines still appear on empty pads. Groups are red and supergroups violet. Controller pads with only two zero-valued endpoints stay dark; extra points or nonzero values mark content. Zero remains valid CC output. Arpeggiator pads turn green for a note-producing step with positive velocity and probability; rest-only, tie-only and suppressed pads stay dark, regardless of held notes. Rests keep their dark styling; the number in **Rest 1/2/4/8/16/32** is its duration in local beats, while a pad number identifies a reusable slot. Selection outlines and Playing/Queued indicators remain separate from these type colours.

## Pad contents

Melodic pads store notes, chords, holds, timing offsets and theory settings. Drummer pads store hits and velocities for each row. Controller pads store automation curves. Arpeggiator pads store complete musical variations and rhythm steps.

Pad lengths are 1–16 local beats for melodic/drummer devices and 1–32 for controller devices, subject to 128 steps. Arpeggiators retain 1–8 or 16 master beats. Arpeggiator durations use master beats. Meter, subdivision and playback speed belong to the owning sequencer; tempo is shared.

Drag one pad onto another to copy it explicitly. Melodic pad edge buttons transpose by a scale degree on a short press, or change the diatonic key on a long press. Drummer and controller pads do not use these transpose controls.

## Pattern workspace

Melodic, drummer and controller sequencers and arpeggiators show an always-visible **Pattern workspace** in the upper-right header area. On narrow screens it stacks below the musical controls. The eight pads and step grid remain below the header.

Drag pads or saved groups/supergroups into **Free workspace** to assemble a phrase. A drop inserts at the visible marker; dragging an existing item or selection moves it. Dragging one of the eight pads onto another still copies that pad. Workspace references stay within their owning sequencer.

Click an item to select it. Cmd/Ctrl-click toggles extra items; Shift-click selects a range. **Group** gathers at least two selected pads/rests in left-to-right order, including separated selections, and places the new group at the first selected position. **Supergroup** also accepts groups. Unselected items retain their relative order. These actions create reusable definitions immediately; groups use letter IDs and supergroups Roman numeral IDs. Groups cannot contain groups; supergroups cannot contain supergroups.

Right-click an item, or use Shift+F10, for **Edit**, **Group**, **Supergroup**, **Ungroup**, and **Remove**. Ungroup expands one level locally and keeps the saved definition. Delete/Backspace removes selected workspace items and closes the gap. Use **Add rest…** for rests of 1, 2, 4, 8, 16 or 32 local beats. Long assemblies scroll horizontally; the saved phrase palette has a bounded height.

Choose a saved phrase or double-click a workspace group/supergroup to edit its draft. **Apply** updates every saved use; **Save as new** creates an independent definition. **Discard changes** restores its saved contents. The editor lists arrangement/phrase uses and warns that changing duration shifts later song material. Workspace construction never edits song order; place saved phrases in the arranger. **Open in arranger** focuses the selected definition there.

Each definition has a separate draft, and the free workspace is retained when switching between definitions. Drafts, selection and scroll survive panel collapse and view switches. New/Load/Import and browser reload clear temporary drafts. Loose workspace items and unapplied edits are excluded from save, autosave and export; Group/Supergroup creation and Apply save through the existing performance workflow.

**Delete definition** is available in saved phrase and arranger palettes. Saved arrangement or phrase references block deletion.

Temporary references expand into the saved contents, preserving order and selection even in closed workspaces; the definition’s own draft is removed. Empty definitions can remain drafts but cannot be placed or auditioned. Referenced definitions cannot be applied empty. Containers and expanded sequences are limited to 256 tokens; invalid edits leave drafts unchanged.

All four device types use this workspace, including Live arpeggiators. Live mode hides arrangement source and navigation controls while retaining phrase creation and editing.

## Playback source and audition

The **Playback source** toggle in each sequencer switches between **Arrangement** and **Manual pads**. New devices default to Manual pads; placing their first item on an arranger lane selects Arrangement. Later choices are saved with the performance and retained through edits and reloads.

**Arrangement** follows the song timeline. Device Play works while the arranger is stopped: the first device starts at the selected loop's beginning, or at the stopped song cursor without a loop; a cursor at song end restarts from the beginning. Other Arrangement devices join that position. The arranger’s **Loop song** toggle chooses whether song playback repeats or stops at the end.

With **Manual pads**, device Play repeats the pad selected for editing from its beginning. Click another pad to select it for editing and queue it for the end of the current pattern. The queued pad has an orange outline; the playing pad stays blue-green until the switch, when the new pad turns blue-green. Clicking the playing pad cancels the queue; clicking another pad replaces the queued choice. Launch pad also queues the selected pad. While stopped or using Arrangement, pad clicks only select for editing. Manual pads continue through arrangement seeks, loops and endings. Arranger Play adds all arrangement backing parts while preserving playing manual pads; Arranger Stop removes the backing parts and temporary playback, preserving independent manual pads. Device Stop affects only that device. Groups and supergroups play through speakers or workspace Play, never implicitly through device Play.

The play button beside the workspace loops the **entire displayed assembly**, regardless of selection. An amber outline follows the currently playing pad, group, supergroup or rest, independently of the cyan editing selection. It follows audible playback, including repeated pads and loop wrap; a momentary speaker preview hides the workspace outline until release. Click it again to restore the track's previous arrangement, manual pad or stopped state. It works before saving a phrase. Workspace edits enter at the next cycle boundary after the normal 80 ms coalescing delay. Failed preparation retains the previous sound and the edited draft; another edit or explicit restart retries.

Every pad, group and supergroup has a separate speaker button. Hold for **250 ms** to begin a momentary preview; shorter clicks do nothing. Preview repeats until release. With the arranger running it enters at the next lane cycle boundary; with the arranger stopped it starts at an audio block without moving the song cursor. Space/Enter supports the same hold/release gesture. The speaker never selects, transposes or drags its item.

A speaker preview temporarily overrides workspace playback, then returns to that workspace at the current clock position. Stopping the workspace restores the original track state. Releasing before preparation or a queued boundary cancels the preview. Pointer cancellation, focus loss and leaving the view end speaker previews; collapse/navigation also end workspace playback. Arranger Play/Stop cancels both layers, and late replies/releases cannot restart them. Other tracks continue and lane Mute/Solo stays effective.

Live arpeggiator workspace Play and held speakers start at the configured next rhythm cycle or master bar when a Live clock is established; otherwise they start at an audio block. Pad duration determines phrase lengths. Incoming or held notes are required; no test chord is supplied. Hold and Active/Bypass/Mute remain effective. Chord changes during previews are retained, and release restores the underlying Live pulse. **Launch pad** explicitly launches the editing pad in Live/manual mode, separately from temporary previews. Saved definitions retain the same performance format.

# If, Switch and Drumset

## Choose a Synthesis Case at Note Start

**If** and **Switch**, in the **Control flow** catalog category, are editor constructs that generate Csound `if/elseif/else/endif`. Each synthesis voice chooses one case when it starts. Only that case initializes and performs; selection remains fixed until the voice ends. Different notes, including repetitions of the same note, have independent voices.

1. Add If or Switch. New blocks produce stereo audio. If compares two init-rate inputs using `==`, `!=`, `<`, `<=`, `>` or `>=`. Switch compares its init-rate selector with unique numeric case values. For drums, connect `notnum.inote` to `selector`. Input connections and the existing arithmetic Input Formula Assistant take precedence over literal values in **Configure branches**.
2. Expand the cases on the canvas and drag opcodes from the catalog into a case. A valid drop into**Silence** changes it to **Synthesis** and exposes its result inputs. Ordinary canvas drops create main-graph nodes. Dropping onto a collapsed block expands it; drop again into a specific case.
3. Connect each synthesis case to its managed **Case Result** audio inputs. Several sources can sum at an input, or use an input formula. Connect the block's common output to shared processing or a named Stereo Output. Every non-silent case needs a value for every result channel, even if that case is rarely selected.
4. False and Default initially use **Silence**. Silence assigns zero to every output and contains no synthesis nodes. Changing an existing case to Silence previews the nodes, connections and formulas that will be removed. Switch cases can be renamed, reordered, added and deleted; Default remains last and cannot be deleted. If keeps its two ordered True/False cases.
5. Collapse to see the condition, case summary, shared inputs and common output. Node IDs, selection, positions and the viewport survive collapse, save and reload. Captured input formulas are retained; expand to edit them. Format changes show an impact preview: mono preserves `left` and removes `right` wiring only after confirmation; switching back to stereo leaves `right` unconnected. Block/case deletion removes owned content and associated formulas and configuration together.

### Arrange and transfer nodes

The block stays above and left-aligned with the first case, with room for its complete body and buttons. Case frames measure every visible parameter row at any zoom. They automatically grow to the right and bottom and retain their size when nodes move inward, are removed, or become smaller; **Case Result** stays in a separate row at the bottom-right. Later cases move down or up without changing their internal arrangement. The title band is 40 graph units, padding is 24, and cases have a 32-unit gap. Empty cases keep a 240 × 160 content area.

Drag the **Case Result title or unused body** to resize a branch in both dimensions, including shrinking it. This also works for the Silence marker. The pointer displacement sets the requested size; complete node bounds, padding and the reserved result row limit how small it can become. Other selected nodes stay still and selection is preserved. Sockets, formula editing and help buttons keep their usual actions. Release commits the size and managed positions together; **Escape**, cancellation or focus loss restores the full prior layout. Ordinary dragging retains the largest frame size reached during the gesture. Bright purple borders stay about two screen pixels wide at every zoom. Sizes survive collapse, block movement, reordering, save/reload, copies and export/import.

- Drag main-graph nodes into an expanded case to transfer them. Dragging existing case members normally only rearranges them within that case. Their top and left movement limits keep them clear of the header; selected groups keep their relative spacing. Moving the block moves all contents together.
- Hold **Alt/Option before starting the drag** to transfer case members to another case or the main graph. Keep the desired group selected. **Move selected nodes** in Configure branches is the alternative, with identical validation.
- A transfer requires every connection and formula binding touching a moved node to end at another moved node or a node already in the destination scope. A cyan border marks an allowed target; a red border and diagnostic list refused connections and unselected endpoints. Nothing is automatically selected, disconnected or deleted. An isolated ordinary node can always transfer.
- This restriction applies to **membership transfers**. Existing shared main-graph inputs remain valid; you can deliberately connect them after placing the nodes. Case-local values still leave through Case Result only. Main-graph-only constructs, nested blocks and independent Case Result transfers are refused.

- Release to commit positions and membership together. A refused drop, **Escape**, pointer cancellation or loss of focus restores the entire group and clears highlighting. A refused drop leaves Silence unchanged.

### Play the Drumset template

Choose **New from template → Drumset** for 17 analog percussion voices: **35/36** kicks, **37** side stick, **38** snare, **39** clap, **40** electric snare, **41/43/45/47/48/50** toms, **42/46** closed/open hi-hat, **49** crash, **51** ride and **54** tambourine. Other notes select Default Silence. Velocity scales finite envelopes; branch-local `xtratim` preserves tails after short note-offs. Shared BD snappiness, shared hi-hat tuning and snare tuning default to **1**, preserving the previous sounds. Each added voice has one characteristic control. Save, assign a MIDI channel in Perform and route Stereo Output through the mixer. Controls affect newly started notes.

Processing after the block runs **once per voice**. Put effects in the performance mixer when they should process the **sum of the whole kit**. The velocity-dependent If example uses `ampmidi` and If to choose sine or saw timbre by velocity; it requires no separate layering subsystem.

Continuous switching, nested blocks, numeric results, raw jumps, crossfades, MIDI learn, ranges and choke groups are outside this release. Keep `outs`, `inleta`, `outleta`, `sfload`, `maxalloc` and `midi_legato` in the main graph. GEN nodes and duration controls may belong to cases. Cycles and illegal scope crossings are compile errors, and all cases are validated.

## Patch/API format

Existing patches remain at `schema_version: 1`. Adding a block promotes the patch to **2**; removing the last block does not downgrade it. Create, update and compile use the existing APIs. Unsupported versions and malformed structural records are rejected. Native instrument/performance exports, application state, patch copies, audition and both CSD export modes retain the configuration.

The graph remains flat. Add a typed `graph.control_flow` map keyed by the structural node's ID; each key must reference an `If` or `Switch` node. The example below describes a mono Switch whose ordinary flat node array also contains `kit`, `kick`, `kick_result` and `default_result`:

```
{
  "kit": {
    "kind": "switch",
    "output_format": "mono",
    "operator": "==",
    "cases": [
      {"id": "kick_case", "name": "Kick", "value": 36,
       "node_ids": ["kick", "kick_result"],
       "result_node_id": "kick_result", "silence": false},
      {"id": "default_case", "name": "Default", "value": null,
       "node_ids": ["default_result"],
       "result_node_id": "default_result", "silence": true}
    ]
  }
}
```

- `kind`: `if` or `switch`; `output_format`: `mono` or `stereo`.
- `operator`: one of the six comparison operators; only If uses it.
- `cases`: ordered records with stable `id`, editable `name`, numeric `value` for Switch cases, and `null` for Default or both If cases.
- `node_ids`: exclusive membership, including the managed `CaseResult` node identified by `result_node_id`. Nesting and overlapping membership are invalid.
- `silence`: explicit boolean. A silent case contains only its result marker, with no input wiring, parameters or formulas.
- Block inputs: `lhs` / `rhs` for If, `selector` for Switch, all init-rate. Result and block output ports: `left`, plus `right` for stereo. CaseResult has no output ports.
- `graph.ui_layout.editor_state`: optional presentation with `selection` (`nodeIds` and canonical `connections`)

and `viewport` (`x`, `y`, positive zoom `k`).

- `graph.ui_layout.control_flow_blocks[blockId]`: `true` means collapsed.
- `graph.ui_layout.control_flow_case_sizes[blockId][caseId]`: optional `{ "width": 1200, "height": 900 }` dimensions in graph units. Missing or malformed entries are initialized after all visible bodies are measured. Case/block deletion removes corresponding entries. Positions stay on ordinary node records; measured bounds and drag previews are transient. Input formulas retain their existing `ui_layout.input_formulas["nodeId::portId"]` format. Presentation never defines execution scope.

The compiler orders the main graph with each block as one operation, orders each case separately, emits shared dependencies first and downstream consumers afterwards, and assigns the common audio variables on every path. Score export retains the `notnum` to score-note conversion. Use the full drumset fixture as a complete authoring reference.

# Instrument Import / Export and CSD Export

This page documents all user-facing import/export options available from the Instrument Design page.

## Instrument Export Types

### 1. Orchestron Instrument Bundle (`Export` button)

The `Export` button creates an Orchestron instrument definition bundle containing the patch graph and metadata.

File extensions:

- `.orch.instrument.json` (JSON bundle)
- `.orch.instrument.zip` (ZIP bundle)

### JSON vs ZIP Export Selection

Orchestron chooses the format automatically:

- JSON export when no referenced uploaded GEN01 or `sfload` assets are present
- ZIP export when the instrument references uploaded GEN01 audio assets or `sfload` SoundFont assets (so audio dependencies can be included)

### 2. Csound CSD Export (`Export CSD` button)

The `Export CSD` button compiles the patch and downloads a `.csd` file.

Use this when you want to run the generated instrument outside Orchestron in a Csound workflow.

## What Is In An Instrument Bundle

An instrument bundle contains:

- patch name
- patch description
- template flag
- instrument type (Percussion, Melody, Bass, Effects / Noise, or Continuous)
- synchronized always-on flag for compatibility
- schema version
- full patch graph (`nodes`, `connections`, `ui_layout`, `engine_config`)

If a ZIP bundle is required, it also contains referenced uploaded audio/SoundFont assets under an `audio/` folder in the archive.

## Instrument Import (`Import` button)

The import button accepts:

- standalone instrument bundle JSON
- standalone instrument bundle ZIP
- performance bundle JSON/ZIP (to extract included patch definitions)

This means you can import patches directly from a performance export without manually splitting files. Instrument Type is preserved for both new and overwritten patches, including imports renamed during conflict resolution. Older bundles without a type receive an inferred type; always-on patches become Continuous and unmatched MIDI patches default to Melody.

## Conflict Resolution (Patch Name Already Exists)

When importing one or more patch definitions and a patch name already exists, Orchestron opens a conflict dialog.

Per patch, you can choose:

- `Overwrite` (update existing patch)
- `Rename` (disable overwrite and provide a new target name)
- `Skip` (patch imports only)

The dialog validates name collisions before proceeding. Cancelling leaves the library and audio storage unchanged. After confirmation, the selected patches and their new audio assets are saved together; a failed import rolls back the whole batch. Invalid definitions and duplicate source IDs are rejected before anything is stored.

## Audio Dependency Handling

For ZIP imports that include referenced GEN01 audio or `sfload` SoundFont assets:

- Orchestron validates the archive structure
- Imports the selected patches' audio files into backend storage after confirmation
- Restores the patch graph with the asset references intact
- Rejects archives that exceed backend bundle limits before unpacking large members

## Advanced Format Notes (for power users)

- Type metadata is exported as `instrumentType`; imports also accept `instrument_type`. Supported values are `percussion`, `melody`, `bass`, `effects_noise`, and `continuous`. An explicit type determines activation even if an older `alwaysOn` / `always_on` flag conflicts. Existing graph schema and bundle version numbers are unchanged.
- Instrument ZIP exports contain exactly one root JSON file plus optional `audio/...` entries
- Missing referenced audio assets in a ZIP import cause import failure (prevents broken GEN01 references)
- Default import limits are 256 MiB compressed request body, 8 MiB root JSON, 512 ZIP entries, and 256 MiB total uncompressed ZIP content
- Deployments can tune these with `VISUALCSOUND_BUNDLE_IMPORT_MAX_BYTES`, `VISUALCSOUND_BUNDLE_IMPORT_JSON_MAX_BYTES`, `VISUALCSOUND_BUNDLE_IMPORT_ZIP_MAX_MEMBERS`, and `VISUALCSOUND_BUNDLE_IMPORT_ZIP_MAX_UNCOMPRESSED_BYTES`
- Individual imported audio/SoundFont assets are still limited by `VISUALCSOUND_GEN_AUDIO_ASSET_MAX_BYTES`

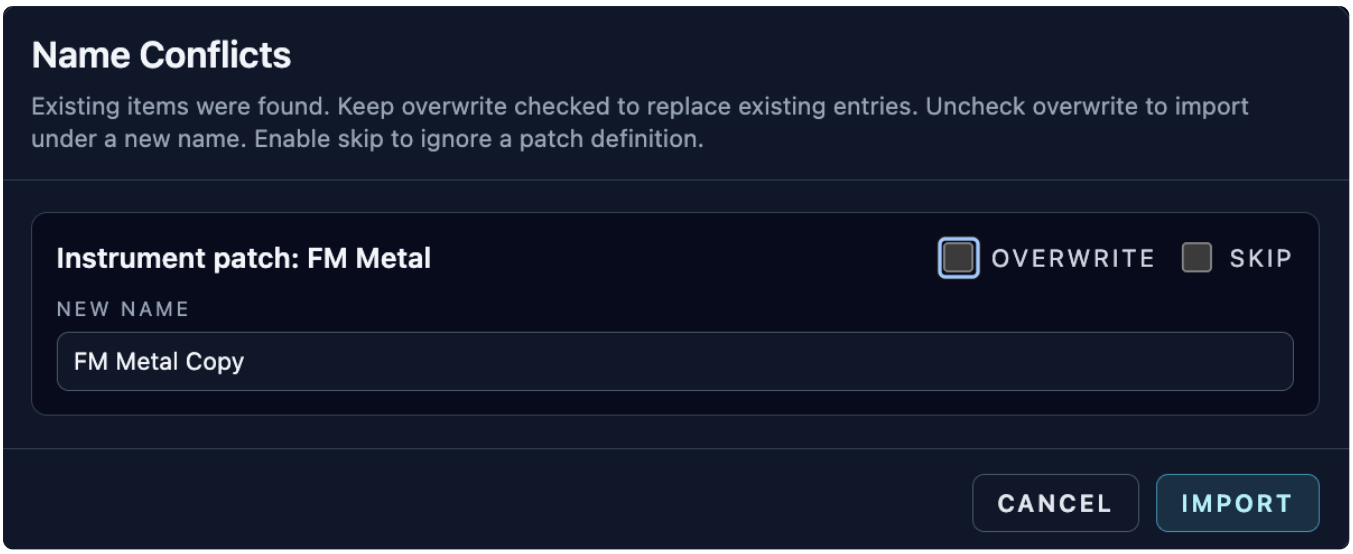
## Instrument Bundle vs Performance Bundle

Use the right export for the right task:

- Instrument bundle: share one patch design
- Performance bundle: share a full performance setup (instrument rack + sequencers + controllers + piano rolls + optional patch definitions)

See Performance Import / Export for performance bundles.

## Screenshots




*Patch import conflict dialog with overwrite/rename/skip decisions.*

# Performance Import / Export

Performance import/export lets you move complete live setups between machines or save versioned backups.

Save/load, native import/export and autosave recovery preserve up to **16 melodic, 16 drummer and 16 controller sequencers** independently, including their MIDI channels, pattern pads and arrangements. Each sequencer still has eight pattern pads.

## Performance Export (`Export`)

The Performance page `Export` button creates an Orchestron performance bundle from the current performance workspace.

Use this export when you want to:

- Re-import the performance into Orchestron later
- Move a full performance setup to another machine
- Keep versioned backups of a performance together with its patch references

File extensions:

- `.orch.json`
- `.orch.zip`

## What Gets Exported

A performance export includes:

- Performance metadata ( `name` , `description` )
- Up to 25 arranger Undo/Redo actions and their current history position, including redo actions after an undo. New performances and older files without history start empty; incompatible history is discarded with a notice.
- Custom names for all six performance device types. Existing names are preserved on import; the GUI applies its name validation only when renaming a device.
- Sequencer/drummer-sequencer/arpeggiator/piano-roll/controller/controller-sequencer configuration snapshot
- Instrument assignments with stable instance IDs
- Explicit audio routes, internal Master settings, insert ownership and bypass states, strip gain/balance/mute/solo, and send level/pre-post settings
- Referenced patch definitions for the instruments currently assigned in the performance rack
- Patch names (included in the snapshot for easier remapping on import)

If any referenced patch contains uploaded GEN01 audio assets or `sfload` SoundFont assets, export is automatically produced as a ZIP and includes those assets.

## Offline Render Export (`Export CSD (MIDI)` / `Export CSD (SCORE)`)

The Performance page provides two offline Csound render exports:

- `Export CSD (MIDI)` creates the traditional ZIP with a compiled CSD and a separate MIDI file.
- `Export CSD (SCORE)` creates a ZIP with the performance notes and controller sweeps embedded as Csound score events.
- Both CSD export modes seed enabled manual MIDI Controller lane values at time 0 on their selected MIDI channels (all 16 by default).

Both modes select instruments by device assignment and mixer routing:

- Include rack instances whose MIDI channel is assigned to a melodic sequencer, drummer sequencer, piano-roll keyboard or arpeggiator output. An arpeggiator input alone is not an instrument assignment.
- Device assignments count even when the device is stopped, disabled or has an empty pattern. Selection does not

depend on whether a note happens during the exported arrangement.
- Include continuous ( `always on` ) instruments referenced by mixer routes, internal Master settings or insert ownership. Continuous generators with implicit direct audio output are also included.
- Bypassed continuous inserts contribute no effect synthesis or initialization in either CSD mode. Their inputs connect to downstream consumers, preserving active effects and required mixer controls. Assets exclusive to bypassed effects are omitted; shared assets remain when an active instance needs them. Native bundles retain the full patches and bypass state for later reactivation.
- Omit other rack instances, routes and mixer controls belonging to those omitted instances, and assets used exclusively by omitted patches. Native `Export` still preserves the complete rack.

Each CSD starts with a comment header containing the performance title, description and creation timestamp, followed by an Orchestron attribution, a short summary of Orchestron's visual instrument design, performance devices, audio routing and portable Csound export features, plus the project GitHub link. Each exported instrument also has CSD comments containing its name, description, rack instance ID and Csound reference. Multiline descriptions remain comments on every line. Routing comments identify source/destination instruments and ports, raw/strip stages, insert ownership, Master processing, direct paths that bypass Master, pre/post-fader taps, and initial gain, balance, mute and solo settings. Mixer instruments are emitted in signal-flow order.

This export is different from the normal `Export` bundle:

- `Export` is for Orchestron import/export workflows
- `Export CSD (MIDI)` and `Export CSD (SCORE)` are for rendering outside Orchestron with standard Csound tools

The MIDI ZIP contains:

- A compiled `.csd` with the selected non-template device instruments and routed continuous instruments
- Offline render settings forced to `sr = 48000` and `ksmps = 1`
- WAV output written as 32-bit float ( `-f` ) so exported files preserve headroom instead of baking clipping into 16-bit samples
- Always-on effect instruments started with Csound `alwayson`
- A finite `f 0 ...` score duration sized for the exported arranger playback plus a release-tail buffer
- The arranger playback rendered as a `.mid` file from beginning to arrangement end
- Enabled manual MIDI Controller lane values written into the MIDI file at tick 0 on their selected MIDI channels (all 16 by default)
- Uploaded bundled sample audio / SoundFont files used by the exported instruments
- A `README.txt` with the exact Csound command line needed to render the package

The SCORE ZIP contains the same compiled instruments and bundled assets, but omits the `.mid` file. Instead, it embeds note events as score `i` statements, embeds controller sequencer sweeps and enabled manual MIDI Controller lane values as score-controlled CC setter events, and rewrites supported MIDI opcodes such as `cpsmidi` , `ampmidi` , `midi_note` , `notnum` , and `midictrl` for score playback. If export-time approximations are needed, such as best-effort `ampmidi` function-table mapping, the ZIP includes `WARNINGS.txt` and the README lists the warnings.

Score statements are ordered by start time and grouped under comments such as `; Measure 14 | 00:26` , with a blank line between measure blocks. Measure numbers follow the arranger's master-bar ruler, independently of each device's local meter or playback speed. The timestamp is the measure's start time in the exported audio, shown as elapsed minutes and whole seconds ( `MM:SS` , rounded down). An export starting partway through the arrangement retains its original measure numbers; its first partial measure starts at `00:00` . Only measures containing note or controller starts have blocks, so silent gaps retain their measure numbers. Sustained notes remain in their starting measure with their full duration.

Only assets stored through Orchestron's upload/import flow are bundled. GEN01 and `sfload` nodes must reference uploaded assets; compile, session start, and offline performance CSD export reject raw filesystem `samplePath` values instead of passing them to Csound.

Offline performance CSD export is bounded before synthesis starts: looping playback is rejected, both the range length and its absolute end position are limited to 65,536 transport steps, a single step can carry at most 16 notes, and the estimated MIDI event budget is limited to 200,000 events. MIDI generation also stops if it exceeds the event budget or takes more than 5 seconds. The CSD export request is limited to the configured bundle JSON size (8 MiB by default).

ZIP layout:

- The ZIP contains a single top-level directory
- That directory has the same basename as the exported performance `.csd`
- Inside the MIDI export directory you will find the `.csd`, `.mid`, `README.txt`, and `assets/` subdirectory
- Inside the SCORE export directory you will find the `.csd`, `README.txt`, optional `WARNINGS.txt`, and `assets/` subdirectory

Typical extracted structure:

```
Offline_Export/
  Offline_Export.csd
  Offline_Export.mid
  README.txt
  assets/
```

Typical SCORE structure:

```
Offline_Export/
  Offline_Export.csd
  README.txt
  WARNINGS.txt
  assets/
```

Typical use:

- Share a performance as a portable offline render package
- Render the arrangement outside Orchestron with stock Csound
- Archive a self-contained `.csd` + `.mid` + assets bundle, or a single inline-score `.csd` + assets bundle, for later mastering or batch rendering

Rendering workflow:

1. Click `Export CSD (MIDI)` or `Export CSD (SCORE)` on the Performance page.
2. Extract the downloaded ZIP.
3. Change into the bundled directory inside the extracted archive.
4. Run the command from `README.txt`, for example the MIDI export command:

```
csound -d -W -f -o Offline_Export.wav -F Offline_Export.mid Offline_Export.csd
```

For SCORE exports the command omits `-F`:

```
csound -d -W -f -o Offline_Export.wav Offline_Export.csd
```

The exported `.csd` already includes matching `CsOptions`, so `csound Offline_Export.csd` also works after you change into that directory.

## Performance Import (`Import`)

The Performance page `Import` button supports two categories of files:

Bundle imports are bounded by backend request and ZIP limits before large archive members are decompressed. Defaults are 256 MiB compressed request body, 8 MiB import JSON, 512 ZIP entries, and 256 MiB total uncompressed ZIP content. Tune these with `VISUALCSOUND_BUNDLE_IMPORT_MAX_BYTES`, `VISUALCSOUND_BUNDLE_IMPORT_JSON_MAX_BYTES`, `VISUALCSOUND_BUNDLE_IMPORT_ZIP_MAX_MEMBERS`, and `VISUALCSOUND_BUNDLE_IMPORT_ZIP_MAX_UNCOMPRESSED_BYTES`; individual bundled audio/SoundFont assets still use `VISUALCSOUND_GEN_AUDIO_ASSET_MAX_BYTES`. Persistent generated-asset

storage is capped by `VISUALCSOUND_GEN_AUDIO_ASSETS_MAX_TOTAL_BYTES` and `VISUALCSOUND_GEN_AUDIO_ASSETS_MAX_COUNT`; unreferenced generated assets older than `VISUALCSOUND_GEN_AUDIO_ASSET_GC_MIN_AGE_SECONDS` are eligible for garbage collection.

### 1. Full Orchestron Performance Export (recommended)

If the file matches the performance export format, Orchestron opens an import options dialog.

### 2. Raw/legacy sequencer snapshot JSON

If the file is not a full performance export bundle, Orchestron attempts to apply it directly as a sequencer configuration snapshot.

This is useful for advanced workflows and backward compatibility.

## Import Options Dialog (Performance Bundle)

When importing a full performance bundle, you can choose whether to import:

- `performance`
- `patch definitions` (if present in the bundle)

You may import either or both. The preview stores no audio assets. Cancelling either dialog leaves the library and audio storage unchanged. After confirmation, all selected instruments, the performance, and new audio assets are saved together. A failure rolls back that batch. Malformed instrument definitions and duplicate source IDs are rejected before storage.

Typical uses:

- Import both (full environment restore)
- Import only patch definitions (extract instruments from a performance file)
- Import only performance (if the destination machine already has matching patches)

## Conflict Resolution Dialog

If names already exist, Orchestron opens a conflict dialog.

### For Patches

Per patch definition you can choose:

- Overwrite existing patch
- Rename and create a new patch
- Skip importing that patch

### For Performance

For the performance itself you can choose:

- Overwrite existing performance
- Rename and create a new performance

(Performance entries do not use `Skip` in the conflict dialog; skipping is handled via the import options step.)

## Patch ID Remapping During Performance Import

Performance bundles store instrument patch references. On import, Orchestron remaps patch IDs using:

1. Imported patch definitions (source ID -> newly created/updated destination patch ID)
2. Existing patch name matches (when patch definitions are not imported but patch names match)

This makes performance imports more portable across machines and patch databases.

## Import Validation (Resolvable Instruments Required)

A performance import fails if no instrument assignments can be resolved to available patches.

In that case:

- import patch definitions from the bundle, or
- create/match patches locally by name first

## Related Workflows

- Instrument bundle import/export is documented in Instrument Import / Export and CSD Export.
- You can also import patch definitions from a performance bundle on the Instrument Design page.

## Mixer snapshot

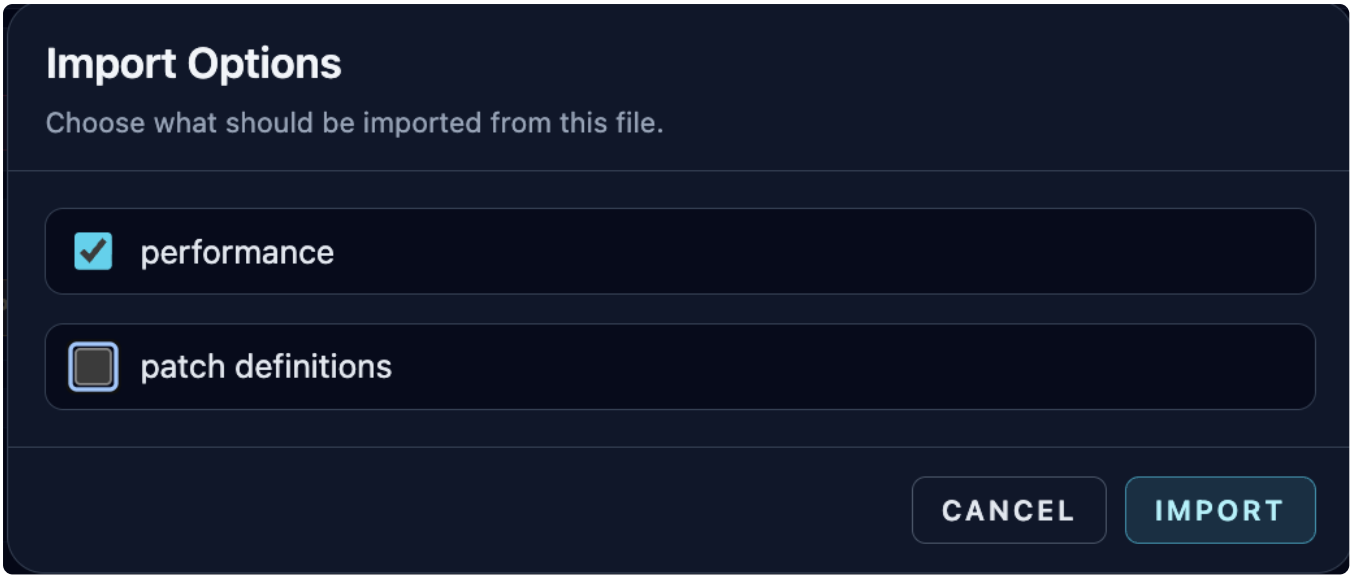
Native bundles and both performance CSD modes capture version 11 routing and mixer settings: gain, pan/balance, mute/solo, sends including pre/post, Master and inserts. Export waits for outstanding live mixer acknowledgments and reports synchronization failures. Mixer values are initialized inside each CSD and require no control client. MIDI and SCORE notes retain their authored velocities; SCORE instrument references come from the compiler manifest.

Offline rendering remains 48 kHz with ksmps=1, float WAV output, packaged assets and release tails. Equivalent routing and settings are guaranteed, not sample-identical live/offline waveforms. Finite exports without MIDI notes are supported when a routed continuous source can produce audio. Standalone patch exports remain independent of Perform settings. See Audio mixer and routing for migration and signal flow.

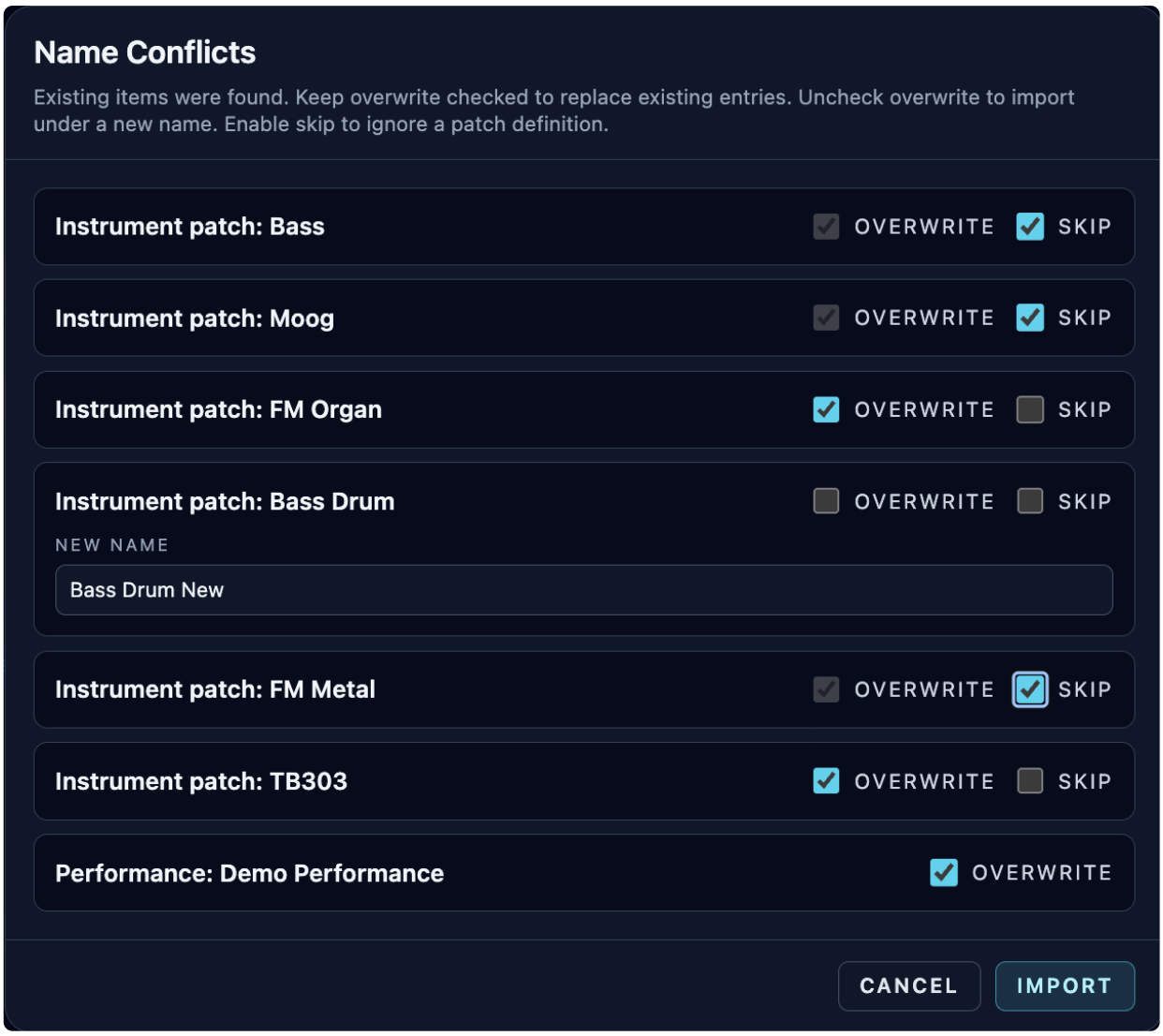
Both performance CSD modes use an offline compilation profile. Saved mixer settings become fixed coefficients, so rendering avoids interactive mixer smoothing and unused peak/RMS meters. These exported coefficients cannot be changed through live mixer control channels. Musical MIDI/controller curves, per-instance settings, synthesis, routing and effect tails retain their existing behavior. Previously exported CSD files are unchanged.

Csound's default logging is preserved: inlet/outlet instance creation and removal messages may still appear as notes play. For optional quiet rendering, run `csound -m0 Offline_Export.csd` from the extracted package directory. This also suppresses warning-level messages; omit `-m0` when diagnosing a render.

## Screenshots




*Performance import options dialog for importing performance and/or patch definitions.*




*Conflict dialog for imported patch/performance name collisions.*

## Performance controller settings (version 12)

Native JSON/ZIP bundles and both CSD export modes preserve each rack instance's `performanceControllerValues`, keyed by the `perf_controller` node ID. Unchanged controls use patch defaults; reset removes the override. Version 12 adds these values while preserving version 11 routing and mixer data; versions 1–11 still load. Patch-ID remapping during import preserves node identities and their settings.

CSD (MIDI) and CSD (SCORE) initialize values in the Csound orchestra before notes and continuous instruments start. No additional MIDI CC messages or external control client are required. Export waits for outstanding live controller updates and reports synchronization failures. Standalone patch exports use defaults. See Performance controllers.

## MIDI controller channel selections (version 13)

Version 13 stores each MIDI Controller and Controller Sequencer's `targetChannels` selection as a sorted list of MIDI channels 1–16. At least one channel is required. Selections survive Save/Load Performance, browser reload, and native JSON/ZIP export/import. Versions 1–12 still load; missing channel selections default to all 16 channels (OMNI).

Both CSD modes route manual initial values and automated curves to the selected channels. Manual lanes sharing a CC number keep independent values on different channels; at render start, the later enabled lane wins only where both channel and CC number overlap. Channel selection does not change which instruments are included in the CSD. The native bundle envelope remains version 1, and browser app state remains version 2.

## Internal Master (version 14)

The fixed Master is stored as `$master` in `audioGraph.masterId`, route endpoints, mixer strips and insert ownership. It has no instrument assignment or patch definition. Native imports normalize designated neutral legacy Masters before importing patches, preventing duplicate library entries. Both CSD modes generate the internal output and initialize its saved controls and inserts. Direct Audio Output paths retain their own controls and continue to bypass Master.

Versions 1–13 remain readable. App state remains version 2 and native bundle envelopes remain version 1. See Master migration and repair for backups, customized processors and missing references.

## Arpeggiator Patterns (version 15)

Version 15 introduced arpeggiator patterns. App-state remains version 2; the native envelope remains version 1. Arpeggiators store eight musical pads, rhythm steps, seeds, processing/hold/playback settings, and Pad Looper arrangements. User presets store musical pad settings only.

On load/import, old musical settings migrate to #1 and accent cycles become step velocities. The remaining pads use defaults. Old performances adopt Arranger synchronization, Hold Off, Continue, corrected swing and full-range ordering; there is no legacy playback engine. Migration stays in memory until normal Save/export. See Arpeggiator migration.

MIDI and SCORE CSD exports use the same arpeggiator scheduler as live audio, including source timestamps, pad boundaries, rests, ties, ratchets, overlap and seeded variation. Export event estimates account for chord expansion, Hold and ratchets; the existing hard event and time limits still apply. Export contains sequenced inputs; live keyboard gestures must be represented as sequence data to appear offline.

## Per-note timing (version 16)

Current saves and exports write performance config **20** and accept versions **1–20**. Melodic steps and drummer cells store `timingOffsetPercent`, an integer from –50 to +50; missing values mean zero. Browser app state is version 3 (reads 1–3) and the native bundle envelope remains version 1. MIDI and SCORE exports use the same shifted attacks and releases as live playback, within each format's timing resolution.

## Arrangement compatibility and temporary audition

The arrangement uses performance format v20, with imports of v1–20. Root sequences, group/supergroup references and pause tokens retain their meaning. Unused and empty draft definitions survive round trips. An imported enabled empty arrangement becomes an explicit occurrence of its previous active pad, retaining its repeat behavior. New empty lanes use Manual pads.

Save, autosave, native bundles and both CSD modes read authored configuration. Temporary audition selection, queued commands, playback position and active-pad/enablement overrides are excluded; edits to musical content still save normally.

Optional `padLoopPattern.definitionColors` stores per-definition display colours in performance v16. Keys are `pad:0` – `pad:7`, `group:<ID>` and `super:<ID>`; values are six-digit hexadecimal colours. Invalid/missing entries use defaults. Unused definitions retain their colours. Save/load, autosave, and native bundles preserve valid metadata; compiled sequences and both CSD modes ignore it. Momentary speaker previews and their restoration state remain excluded from every persistence path.

## Meter timing migration (version 17)

Before normalization, legacy /8 melodic/drum/controller tracks double local beat lengths and halve steps per beat. Notes, holds, velocities, offsets, curve positions and speed ratios are retained. Rests, nested phrases, workspace drafts and arranger history (including its basis and before/after pads) convert together. New lengths support 1–16 melodic/drum beats and 1–32 controller beats; rests include 32. Derived counts and compiled sequences are rebuilt. The 128-step capacity remains enforced. Migration is idempotent and preserves on-grid schedules and arrangement positions; timing-offset rounding stays within the previous clock precision.

Performance config v17 introduced the new meter-beat meaning, retained in v20; app state uses v3. The native JSON/ZIP envelope remains v1. New session requests explicitly use `beat_unit: "meter"` ; omission preserves quarter-beat interpretation for older API clients. See Sequencer timing.

## Drummer ratchets (v18)

Drummer cells also preserve `ratchets` (1–8, default 1) and `ratchetEndVelocity` (0–127 or null for constant velocity). They survive inactive cells, hidden steps, pad copies, saves and native bundles. MIDI and SCORE exports render the same rolls and velocity ramps as live playback, within each format's timing resolution. Repeated hits count toward the export event limit. Older performances default to one hit without a ramp. App-state v3 and native envelope v1 are unchanged; importing v17 does not repeat the meter conversion. See ratchet editing.

## Melodic strumming (v19)

Melodic steps preserve `strumDirection` ( `off` , `up` , `down` , default `off` ) and `strumSpreadPercent` (integer 0–100, default 0), including inactive settings on rests or steps without a chord. All current writers use v20 and readers accept v1–20. Existing performances play unchanged. App-state v3 and native envelope v1 remain unchanged; the meter migration still applies only to performances older than v17.

Both CSD modes use the live sequencer's strum scheduling, including pitch order, timing offsets, overlapping note durations and same-pitch retriggers, within each export format's timing resolution. Finite exports cancel unfinished strums at the end. Individual attacks and downstream arpeggiator activity count toward export limits. The standalone CLI preserves and validates these fields and converts them to `strum_direction / strum_spread_percent` for runtime requests. Use `edit strum list|set|reset` to inspect or author them, or use `step_strum` lists and inline event/progression fields in YAML/JSON scores. See strum editing.

## Controller steps (v20)

Controller sequencers save `controllerMode` ( `curve` or `step` , default `curve` ), `resetValue` (0–127, default 0), and 128 nullable integer `steps` per pad. Curves and step data are both retained, including hidden trailing cells. Legacy versions load in Curve Mode. Save/Load, autosave and native JSON/ZIP preserve these fields; MIDI and SCORE exports emit populated steps, repeated values, and start/stop resets using the live scheduler. App-state v3 and native envelope v1 are unchanged. See Controller Sequencers.

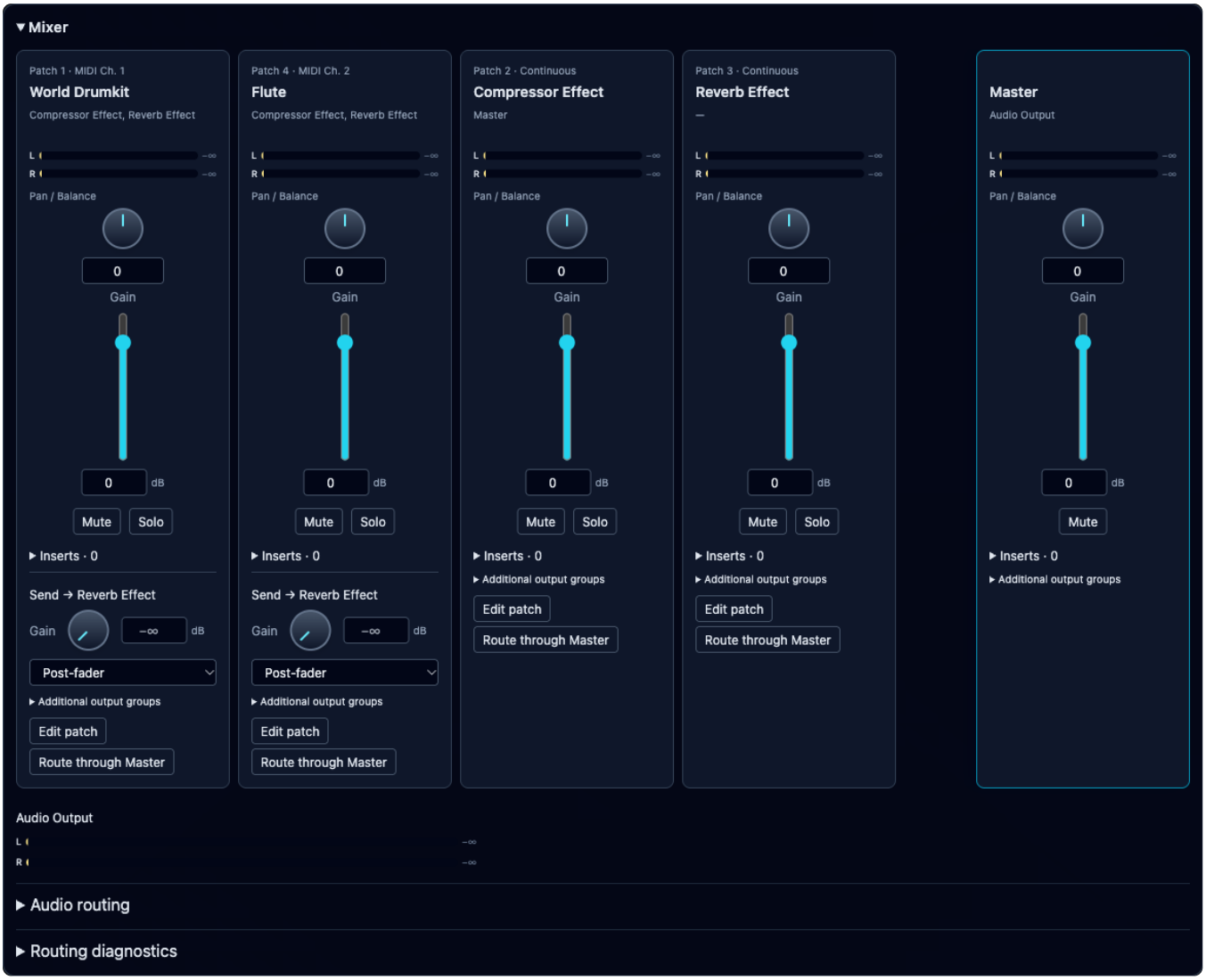
# Audio mixer and routing

The collapsible mixer sits below the instrument rack. Select a strip to edit its outputs and sends. The pinned Master controls audio routed into it. A **Direct Audio Output** badge identifies paths that bypass Master; these still respond to their own strip's fader, balance, mute and solo.

Collapsing Mixer suspends its visual contents and meter subscriptions while audio controls and meter collection keep running. Nested sections suspend independently and remember their expansion state when their parent closes. Reopening restores routing selections, editor feedback and strip scroll position, and meters show the latest snapshot. See Collapsible Panels for state lifetime and gesture cleanup.

## Build your first mix

1. In Instrument Design, choose **New → Playable instrument**, name the draft and **Save** it. Create and save an **Audio effect** when you need a processor. The built-in effect is a stereo pass-through starter; add your processing inside its graph.
2. In Perform, use **Add Instrument** and select the saved instrument. Set its MIDI channel. Guided instruments connect to the fixed internal Master automatically. For existing patches, inspect their destination labels and use **Route through Master** if needed.
3. Expand **Mixer** below the rack. Above each instrument name, a small label shows its rack patch number and MIDI channel, for example **Patch 2 · MIDI Ch. 3**. Continuous instruments and effects show **Patch 4 · Continuous**. Patch numbers match the rack even when insert processors have no separate mixer strip. Each strip header selects its source for the routing editor. The Master strip stays pinned at the right; scroll horizontally when the rack has many strips. Strips follow the instrument rack's order, with note-triggered instruments before continuous returns. Stop the instruments and drag the rack's `::` handles to reorder within either group. This changes display order only; insert processors remain in their owner's insert chain.
4. Start with 0 dB gain and centered Pan / Balance. Build routes and insert chains while stopped, then open **Routing diagnostics → Check routing**.
5. Click **Start Instruments** and play via a piano roll, sequencer or MIDI input. Change gain, balance, mute/solo and existing send settings while listening. Watch strip meters and the final Audio Output meter.
6. Use **Save Performance** to retain this mix. Stop the engine before adding/removing routes, replacing assignments or reordering inserts.




*Live “Ratchet drums demo” mixer, stopped: the strips show their rack patch numbers and MIDI channels or Continuous above their names, with Master pinned at the right. World Drumkit and Flute have post-fader send controls for Reverb Effect; both sends are set to silence (−∞ dB). This performance has no insert processors.*

## Signal flow

| Path | Processing order |
|---|---|
| Instrument | Summed voices → Perform inserts → pan/balance → fader → destination |
| Pre-fader send | After inserts → send amount → destination |
| Post-fader send | After pan/balance and fader → send amount → destination |
| Return | Summed inputs → effect patch → inserts → balance → return fader |
| Master | Incoming routes → internal stereo bus → inserts → balance → Master fader → Audio Output |

Effects drawn inside an instrument graph run per voice. Perform inserts receive summed voices. Each inserted effect is a dedicated rack instance. Reordering an insert changes explicit connections; if custom connections no longer form a simple chain, **Custom routing** preserves those connections.

Additional output groups retain their own port names and mappings. They share the owning strip's gain and mute; inserts process the designated main output. Custom routes can express other processing arrangements.

## Inserts versus shared returns

An **insert** processes one strip in series after its voices are summed. Select that strip, choose a saved continuous patch with audio inputs and outputs in the **Inserts…** selector below the routing controls, then click the adjacent**Add**. Open **Inserts** on the strip to inspect its chain. Click a processor name to edit its patch, ↑ to move it earlier, or × to remove that insert instance. Reusing one saved effect creates separate processing instances; an insert does not appear as a separate mixer strip.

Removing an instrument from the rack also removes its dedicated inserts, including any inserts on those processors, plus their connections and mixer settings. Removing a single insert through the rack or its × button reconnects the remaining simple chain. Custom connections between surviving instances are preserved for inspection and repair; unrelated returns, sends and Master controls remain unchanged.

A **return** is a continuous rack instance that can receive sends from several sources and has its own mixer strip. Add it with **Add Instrument**, connect its output to Master, then add sends from the source strips. For a parallel reverb/delay arrangement, design the return patch to output the desired wet signal; the starter pass-through would merely add another copy of the dry signal.

## Compare inserts with Bypass

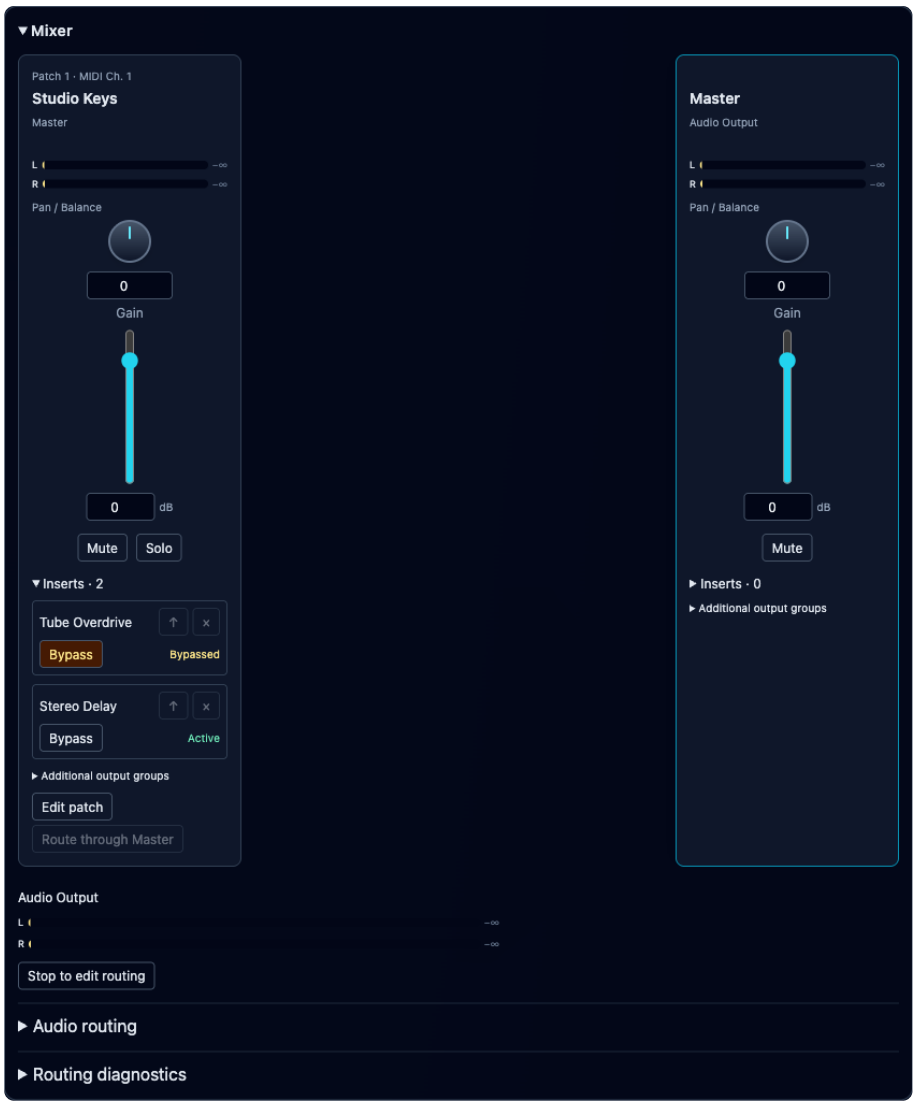
Open **Inserts** on the owning mixer strip. Each continuous insert has a **Bypass** button and an **Active** or **Bypassed** label. You can switch while instruments are running. Reordering and removing inserts still require stopping playback. The same controls appear for nested inserts and for inserts with custom routing.

Bypass passes that effect's main input to its main output, using the declared channel order. A 20 ms crossfade suppresses abrupt transitions. The selected effect's output, including its tails, fades out; downstream effects keep processing and can retain their own tails. The bypassed processor keeps receiving audio and running internally, so reactivation retains its current state. Live bypass does not reduce CPU use.

For example, bypassing Distortion in `Instrument → EQ → Distortion → Reverb` lets Reverb receive the EQ output directly. Other inserts, including inserts nested inside the bypassed processor, retain their own states. Strip gain/balance and mute/solo still apply. Additional effect outputs have no dry substitute and fade to silence. Bypass requires matching main input/output channel counts; an incompatible mapping disables the button. Bypass does not infer or compensate a patch's internal processing latency.

The control is available only on continuous instances assigned as inserts, not on standalone generators, shared returns or note instruments. New inserts default to Active. Each instance's state survives Save/Load, Clone, autosave and native JSON/ZIP bundles without changing its library patch. Match listening levels when comparing effects that change volume.

Both **CSD (MIDI)** and **CSD (SCORE)** omit bypassed effect synthesis, initialization, performance-controller channels and assets used only by those effects. Incoming audio connects directly to downstream consumers; necessary strip and route controls remain. Exported CSDs contain no live bypass crossfade stage. Native exports retain all effect definitions so they can be reactivated after import.




*Each insert has its own Bypass control and explicit state. The remaining chain stays active.*

### Add a stereo send

1. Stop the engine. Add the saved effect as a rack instance and use its **Route through Master** button to connect its main output.
2. In **Audio routing**, choose the instrument under **Source** and the return under **Destination**.
3. Leave **Stereo** checked and verify **Exact channel mapping**, for example `left / right → left / right`. Choose **Send** in the Route selector and click the routing row's **Add**.
4. On the source strip, find **Send → [return name]**. New sends start at silence; enter a value such as -12 dB to hear the return.
5. Choose **Post-fader** when the send should follow the source's fader/balance, or **Pre-fader** for an independent send amount. The source's Mute still silences either kind of send.

Adding a send does not replace the main output route. Avoid adding duplicate connections: parallel paths sum and can increase output level. Up to 1,024 individual channel connections and 64 rack instances (including processors, excluding internal Master) are supported; a stereo pair uses two connections.

## Controls

- Faders span –60 to +12 dB, with an exact-silence position below –60. Clearing the numeric value selects silence. Double-click resets to 0 dB.
- Sends span silence to +6 dB. New sends start silent and post-fader. Pre/post can change during playback.
- Rotary controls support vertical dragging, arrow keys, Page Up/Down, Home/End, exact numeric editing and double-

click reset. Shift-drag provides finer adjustment.

- Stereo balance preserves both channels at unity in the center and attenuates the opposite channel toward either side. Explicit mono-to-stereo routing uses equal-power pan. Custom mono mappings stay unchanged.
- Mute blocks all outgoing paths, including pre-fader sends. Processors keep running, allowing downstream tails to decay.
- Multiple solos are allowed. Solo preserves required upstream inputs and downstream processors. Soloing a return excludes its sources' unrelated dry branches. Mute takes precedence. Master has no Solo control.
- Gain, balance, send and mute/solo changes use finite 20 ms ramps. Playback starts at the saved values, without an initial fade.

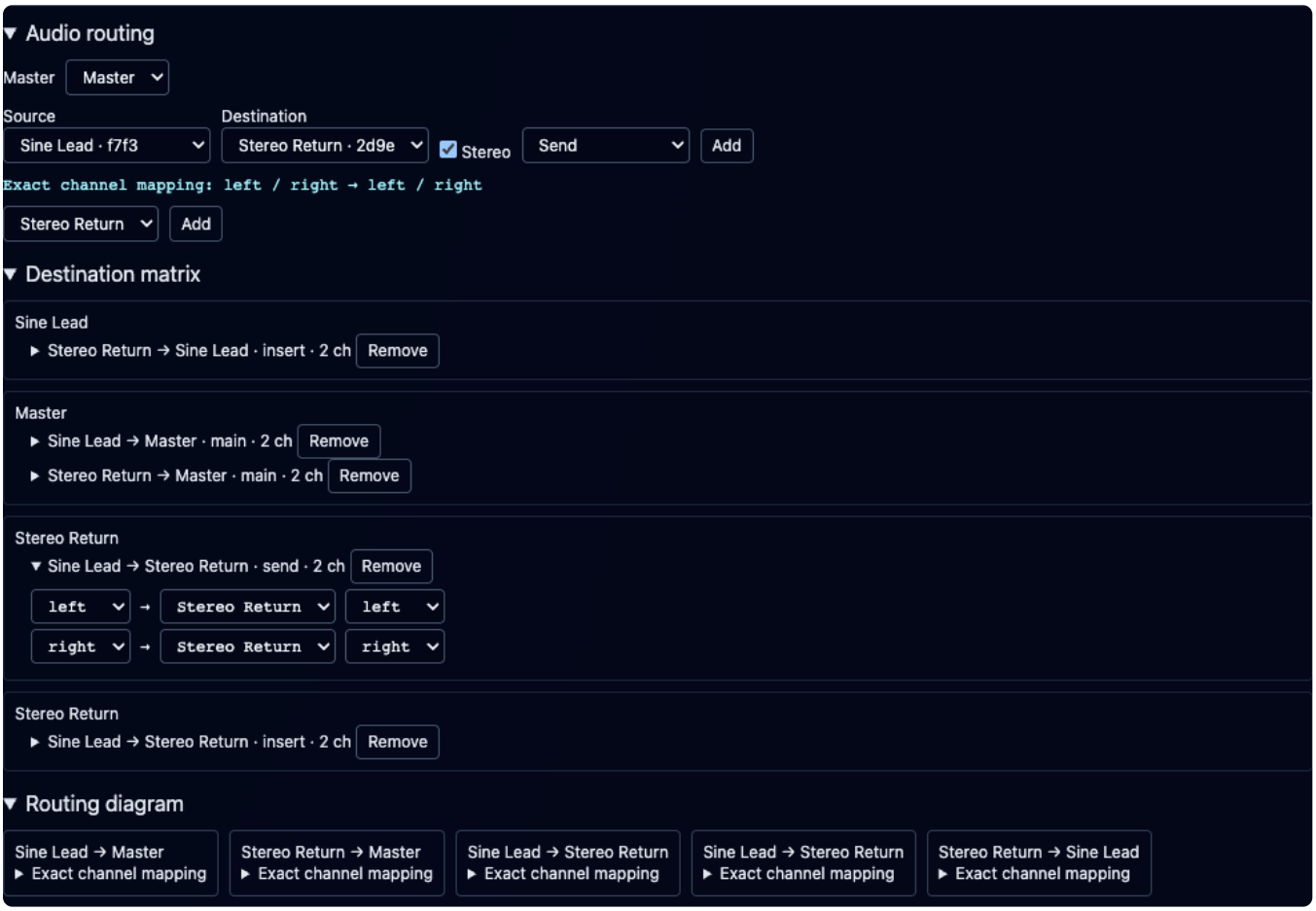
Meters show stereo peak and RMS. CLIP indicates a peak at or above 0 dBFS; no automatic limiting, normalization or clipping is applied. The final **Audio Output** meter includes Master and direct paths. Connectivity diagnostics describe structure; meters measure actual audio activity.

## Routing and repair

Choose a source, destination and explicit channel mapping before adding a main route, send or custom connection. Stereo groups expand into their exact member ports. Uncheck **Stereo** to select one exact source port and destination inlet at a time, including custom or mono mappings. Group display names are not inlet names; legacy label suggestions never rename ports.

Open **Destination matrix** to inspect incoming routes grouped by destination. Expand a route row to change its source port, destination instance or destination port; **Remove** deletes that grouped connection. This is also where broken references can be repaired instead of discarding the performance.

Open **Routing diagram** to follow the same connections as source → destination cards. Expand **Exact channel mapping** on a card to inspect port names and processing stages. Click source/destination names to select routing endpoints; use the strip's **Edit patch** action to open its instrument graph.




*The send's left and right mappings expanded in Destination matrix. Matrix and diagram show the same routes, including the dedicated insert's input and return-to-strip connections.*

**Route through Master** replaces the selected strip's main routes with stereo connections to the designated Master in this performance. Sends and custom routes are retained. Master is a fixed internal stereo output with volume, balance, mute, meters and effect inserts. It is always available, has no editable patch and occupies no library entry or rack slot. Its controls and inserts belong to the performance. **Audio Output** remains available as an editable starter for custom output processors.

New guided instruments connect to Master automatically. Loading an older performance preserves its authored direct/custom paths. Master gain or mute affects only paths routed into Master: a direct strip can still be audible. Check the **Direct Audio Output** badge and the final Audio Output meter when tracking down sound that remains after lowering Master.

Topology changes require stopping the engine. **Stop to edit routing** makes that action explicit. Existing mixer controls remain live. Invalid references and genuine feedback cycles block playback and CSD export. Broken routes remain available for repair. Unused ports, incomplete stereo pairs and parallel direct/processed paths are warnings.

## Troubleshoot routing and silence

Expand **Routing diagnostics** and click **Check routing** after editing connections. Diagnostics describe connectivity, while meters report measured sound. Clicking an instance-related diagnostic can open its patch for inspection. A **Muted or silent path** entry selects the corresponding strip.

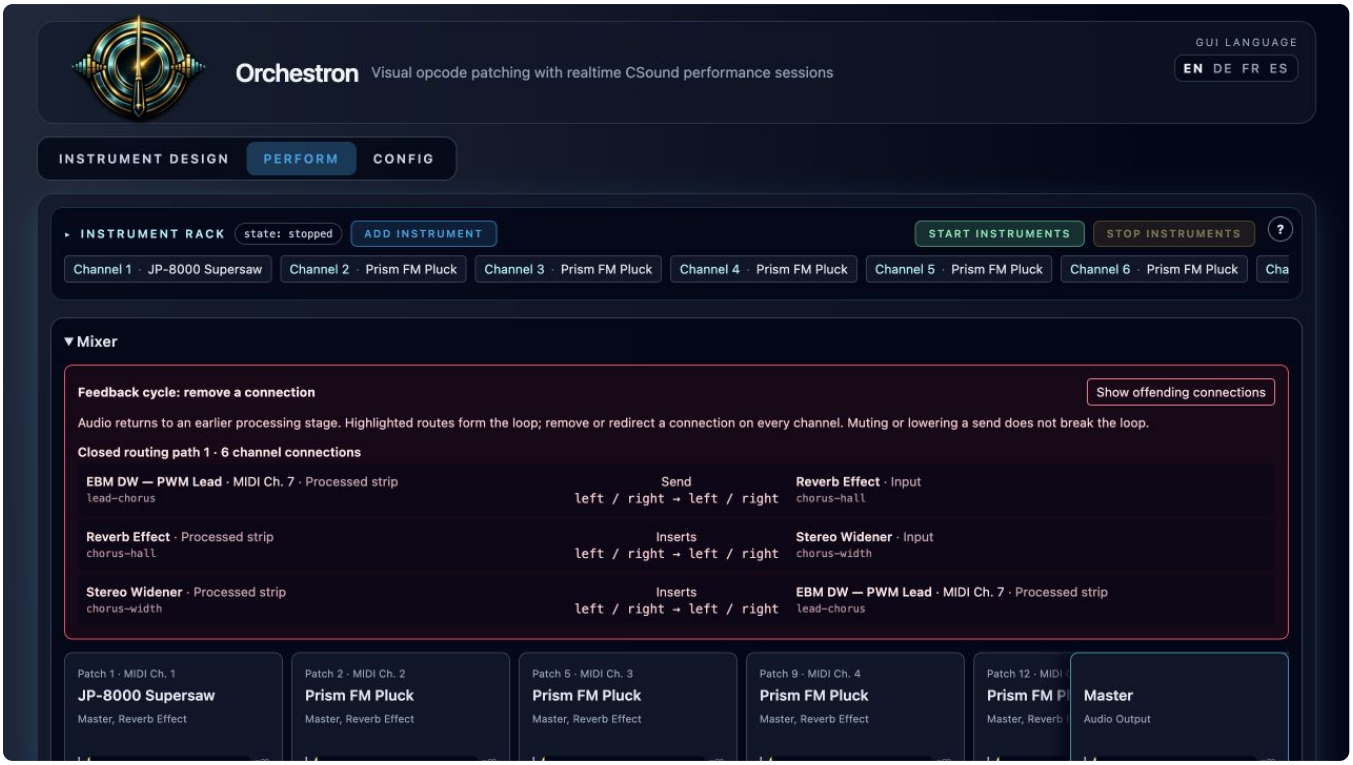
Invalid insert assignments appear in red, including an insert whose processor or owning strip is missing, or which refers to itself. These errors are detected even when all visible connections to the deleted instance have already been removed. Backend validation failures appear directly beside **Check routing**. Previous check results are discarded after changes to the rack, patch definitions, routing or mixer settings; check again after editing.

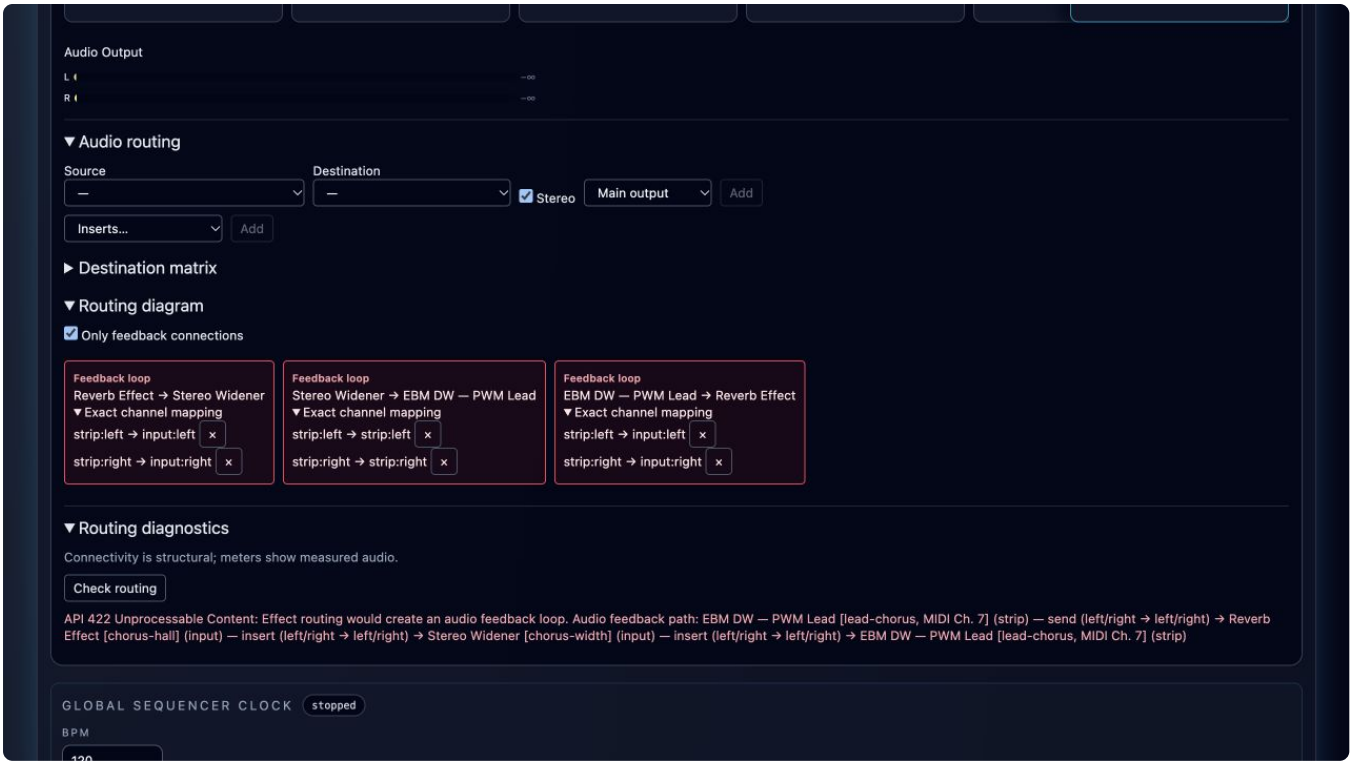
Feedback loops appear above the mixer strips as soon as their connections are present. Each path lists instrument and effect names, instance IDs (to distinguish repeated uses of a patch), MIDI channels, exact audio ports and processing stages. **Before inserts** is the instrument's raw output; **Processed strip** is its output after insert returns; **Input** feeds the effect's processing. Ordinary insert chains returning to their owner's strip are valid.

Choose **Show offending connections** to open the routing diagram and enable **Only feedback connections**. Red borders and **Feedback loop** badges identify the affected strips, inserts, sends and route groups; the Destination matrix highlights the same connections. Expand **Exact channel mapping** to inspect or remove individual channel connections. Turn off the filter to inspect surrounding routes. Highlights update immediately after a routing edit, and all connections become visible again once the loop is gone.

The display shows one closed path for each connected group of feedback loops and highlights every cyclic connection in that group. A group may contain additional overlapping loops. Unrelated incoming routes and downstream output routes are not highlighted. Break every remaining loop, including both stereo channels; muting, bypassing or lowering a send does not remove its processing dependency.

For example, sending a lead's processed strip into its own Reverb insert closes **Lead → Reverb → Stereo Widener → Lead**. Use a separate shared Reverb instance with its own output return if that is the intended effect. The screenshot below uses a temporary copy of Effectomania's earlier routing.



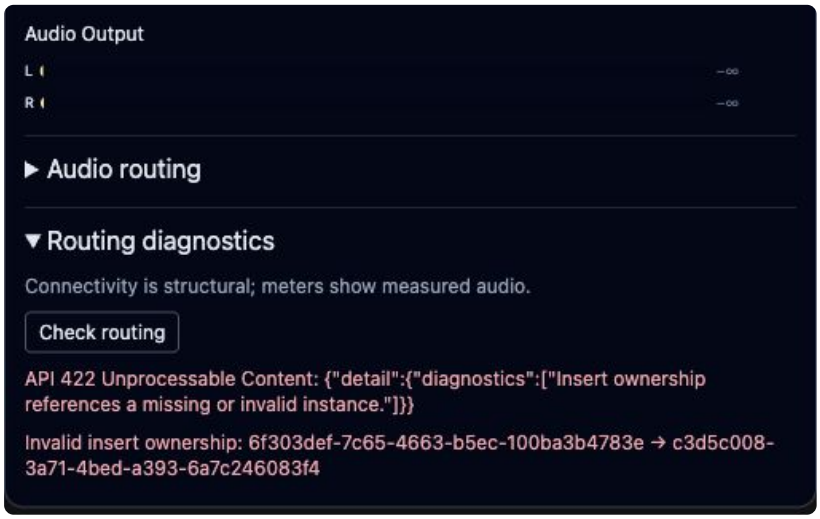




| Symptom or diagnostic | What to check |
| --- | --- |
| Repair route / Missing reference | Restore the referenced patch or repair the destination and exact port names in Destination matrix. |
| Invalid insert ownership | Repair the insert assignment that refers to a missing instance or to itself. Removing audio connections alone does not repair an invalid insert assignment. |
| Feedback cycle | Follow the named path and choose **Show offending connections**. Remove or redirect a connection on every affected channel. Playback and CSD export remain blocked until all cycles are resolved. |

| Symptom or diagnostic | What to check |
|---|---|
| Connect an audio output / Select an audio source | Follow the source to an output sink; route audio into effects that require inputs. |
| Complete stereo mapping / Unused ports | Check both channel mappings and the declared audio groups. These structural warnings do not prove a patch is silent. |
| Review parallel output paths | Check whether both direct and processed copies intentionally reach the output. Serial insert chains, including Master inserts, do not trigger this warning. |
| Muted or silent path | Clear Mute or raise a fader/send from silence; also check active solos and whether the source is producing notes/audio. |
| CLIP | Reduce gain along the contributing paths. No limiter is applied automatically. |

The same routing diagnostics appear in CSD exports. A serial insert feeds back into its owner's strip before reaching Audio Output. A parallel-output warning means the other branch can reach Audio Output through a separate exit, such as a shared effect return or an additional output from an insert processor.

*A valid route can still be silent: here the Sine Lead strip is muted.*




*A diagnostic preview of the damaged Sunlit Strings performance: the missing insert assignment is flagged locally and the backend failure appears in the same section. The saved performance was repaired separately.*

## Persistence and compatibility

Current performance saves use configuration version 20 and read versions 1–20. Optional per-strip `bypass` defaults to false for older saves. They retain the internal Master introduced in version 14, with stable instance IDs, explicit routes, mixer strips, sends and insert ownership. App state version 3 reads versions 1–3 and saves this same model. Save/Load, Clone and native JSON/ZIP bundles preserve the current settings. Mixer automation recording is not included.

Versions 1–10 convert old Level values using `20 * log10(level / 10)`, including continuous effects. Missing Level means 0 dB. Legacy inlet selection is resolved once and saved explicitly. Level no longer scales MIDI velocity; velocity-sensitive patches can therefore change timbre after migration. Authored note velocities remain unchanged.

Both performance CSD modes initialize the same routing, gain, balance, mute, solo and send settings without a browser. Offline rendering retains 48 kHz, `ksmps=1`, float WAV output and release tails. Live/offline settings match; different engine rates need not produce identical waveforms. A routed continuous source can render a finite range with no MIDI notes.

Standalone instrument exports contain patch design and interface metadata, without Perform mixer settings.

## Control API

Session creation and validation accept `audio_graph` and `mixer`. `GET /api/sessions/{id}/mixer` returns desired values and revision; `PUT` accepts batched partial scalar updates and an optional expected revision. Stale revisions return 409. The active browser-clock controller can send the equivalent `mixer_update` message and receive `mixer_ack` / `mixer_error` replies. Updates are queued and applied at render-block boundaries. Meter frames carry engine sample timestamps and are delivered at most 15 times per second.

Feedback failures return HTTP 422 with the existing `detail.diagnostics` string list plus `detail.audioDiagnostics`. Each structured feedback diagnostic contains `code: "feedback"`, all cyclic `routeIds` and `instanceIds`, and an ordered `routingPath`. Each path step contains `kind`, its grouped `routeIds`, and `source` / `target` endpoints with `id`, `name`, `midiChannel` (0 for continuous), `stage`, `ports`, and optional `insertOwnerId`. Stereo routes with the same endpoints, stages and kind are grouped. General validation errors keep the existing response format. Legacy routes receive generated diagnostic IDs; explicit mixer routes retain their authored IDs.

`POST /api/sessions/preview` accepts a session plus inline draft patch definitions. It creates a transient runtime without saving drafts into the patch library. Normal stop/delete session operations clean it up.

## Upgrading existing Masters

On backend startup, the SQLite library is migrated before it is served. The migration creates a sibling `.before-internal-master-<timestamp>.bak` database backup before changing data. Repeated starts make no further changes after migration.

Known neutral Master graphs are replaced by the internal output while preserving each performance's routes, gain, balance, mute and inserts. Customized Masters retain their processing as ordinary rack instruments feeding the internal Master. Unreferenced neutral Master copies are removed from the library only after references in saved performances and app state have been migrated. Modified patches, edited drafts and patches still used elsewhere are retained.

A genuinely missing legacy patch stays visible for repair. While stopped, **Replace missing Master with internal output** transfers compatible stereo connections and saved Master controls; save the performance afterward. Unknown port mappings must first be repaired in the routing matrix.

To preview the migration manually with the backend stopped, run `uv run python -m backend.tools.migrate_internal_master --database backend/data/visualcsound.db`. Add `--apply` to create a backup and apply it. Use the actual database path for Docker or custom installations. To roll back, stop the backend and restore the backup together with the previous application version.

# Performance

This chapter covers the `Perform` page (labeled `Perform / Performance` depending language), where you build a playable multi-instrument setup and perform it live.

The performance description field displays three lines with automatic text wrapping and preserves manual line breaks. Longer descriptions scroll vertically within the field. Hover over the description to see its full text in a tooltip.

## Sequencer timing

Use Meter, Pattern length and Subdivision for melodic, drum and controller sequencers. A one-bar 4/4 pad with three steps per beat displays twelve steps in four labeled beat groups. Playback speed is under Advanced timing. See Sequencer timing for the full guide.

## Device Names

Melodic sequencers, drummer sequencers, controller sequencers, arpeggiators, piano rolls, and MIDI controllers all support custom names. Click the pen immediately to the right of a device name to edit it. The input receives focus with its current name selected.

- Save or Enter applies the name. Cancel or Escape discards the draft. Clicking elsewhere does not save; switching performances or removing the device discards the draft.
- Names must be nonempty after trimming surrounding whitespace and contain at most 65 Unicode characters. HTML and angle brackets ( `<` and `>` ) are rejected.
- Names must be unique across all six device types within the performance, ignoring case and surrounding whitespace. Validation messages appear while typing, and Save is disabled until the name is valid.
- Names retain their capitalization. Melodic synchronization choices and arranger track titles show the stored names. New devices receive an unused numbered default name.
- Renaming preserves device IDs, synchronization, routing, patterns, and playback state. Names survive performance save/load, browser persistence, raw snapshot import, and native JSON/ZIP export/import.
- Existing/imported names remain intact under the existing loading rules, even if they violate the new editing rules. They display as plain text and need a valid replacement only when renamed. API acceptance is unchanged.

## Collapsible Panels

Click a panel title or focus it and press Enter or Space to collapse or expand its contents. Header actions, status and help remain available. Panels collapse independently, start expanded and remember your choices across view switches until browser reload. Adding a device opens its group. Collapsing preserves edits and does not stop playback. The shared tempo stays outside the sequencer groups.

Instrument Rack, Melodic Sequencers, Drummer Sequencers, Controller Sequencers, Arpeggiators, Piano Rolls (keyboards), MIDI Controllers, and Multitrack Arranger each have their own toggle. Empty drummer/controller groups keep their Add buttons. The arranger appears when at least one melodic, drummer, controller sequencer, or arpeggiator exists. These layout choices are not saved into performances or exports.

Collapsed visual bodies are suspended: hidden grids and timelines are not built, animation loops stop, and hidden meters unsubscribe. This includes the Mixer and each nested routing, matrix, diagram, channel-mapping, diagnostic and insert section. Playback, pad switching, MIDI processing, audio controls and meter collection continue. Visible headers, rack summary, shared tempo and footer transport status keep updating. Expanding displays the current playback position and latest meters immediately, without replaying missed animation frames.

Draft names and controller values, validation feedback, arpeggiator preset drafts, pad-loop selections, open containers, arranger selections, range clipboard and undo history, zoom and scroll positions survive collapse/expand and switching away from Perform. Nested Mixer sections remember their open state when a parent collapses. New, Load and Import clear retained editor state after successfully replacing the workspace, even when reloading the same performance. Saving does not clear it. Removed devices and invalid selections are discarded; restored scroll positions are limited to the current content. This

temporary editor state is not stored in performance files or browser storage and does not survive a page reload.

Collapsing closes temporary menus and pickers, cancels active drag previews and releases manually held piano notes. Already-applied edits remain, and existing input save/commit rules still apply. Sequenced notes continue playing. Offscreen panels that remain expanded still render normally. Collapsing reduces visual work; any reduction in audio dropouts depends on the workload and audio system.

The collapsed rack shows channel and patch name in one horizontally scrolling row. See Compact Rack for continuous instruments, long names, and keyboard scrolling.

## What You Can Do Here

- Assemble an instrument rack from saved patches and assign MIDI channels
- Mix audio with dB faders, pan/balance, mute/solo, meters, inserts and pre/post sends
- Route instruments and returns through Master or explicit direct paths
- Start/stop the instrument engine session
- Create multiple melodic sequencers, drummer sequencers, and controller sequencers
- Add backend-run arpeggiators that turn held notes into routed arpeggiated instrument output
- Edit reusable pads and phrases, assemble song order in the arranger, and audition phrases without changing the song
- Arrange multiple track timelines in the multitrack arranger with shared transport and loop-range playback
- Copy song sections across all or selected lanes, overwrite destinations, or insert time and shift later material together
- Perform live with piano rolls and manual MIDI controller knobs
- Save/load/clone/delete performances
- Import/export performance bundles (with optional patch definitions)
- Export offline Csound render ZIPs either with `.csd + .mid` playback or with inline Csound score playback

## Chapter Contents

- Instrument Rack and Engine Transport
- Audio Mixer and Routing
- Melodic Sequencers and Step Editing
- Drummer Sequencers
- Pattern Pads and Reusable Phrases
- Multitrack Arranger
- Controller Sequencers
- Arpeggiators
- Piano Rolls
- MIDI Controllers
- Performance Import / Export
- Live Status and Safety Controls
- Standard Effects

## Important Concept: Patch vs Performance

- **Patch (Instrument Design page):** one instrument graph
- **Performance (Perform page):** a full live setup that references one or more patches and stores sequencer/piano-roll/controller state plus explicit audio routes and mixer settings

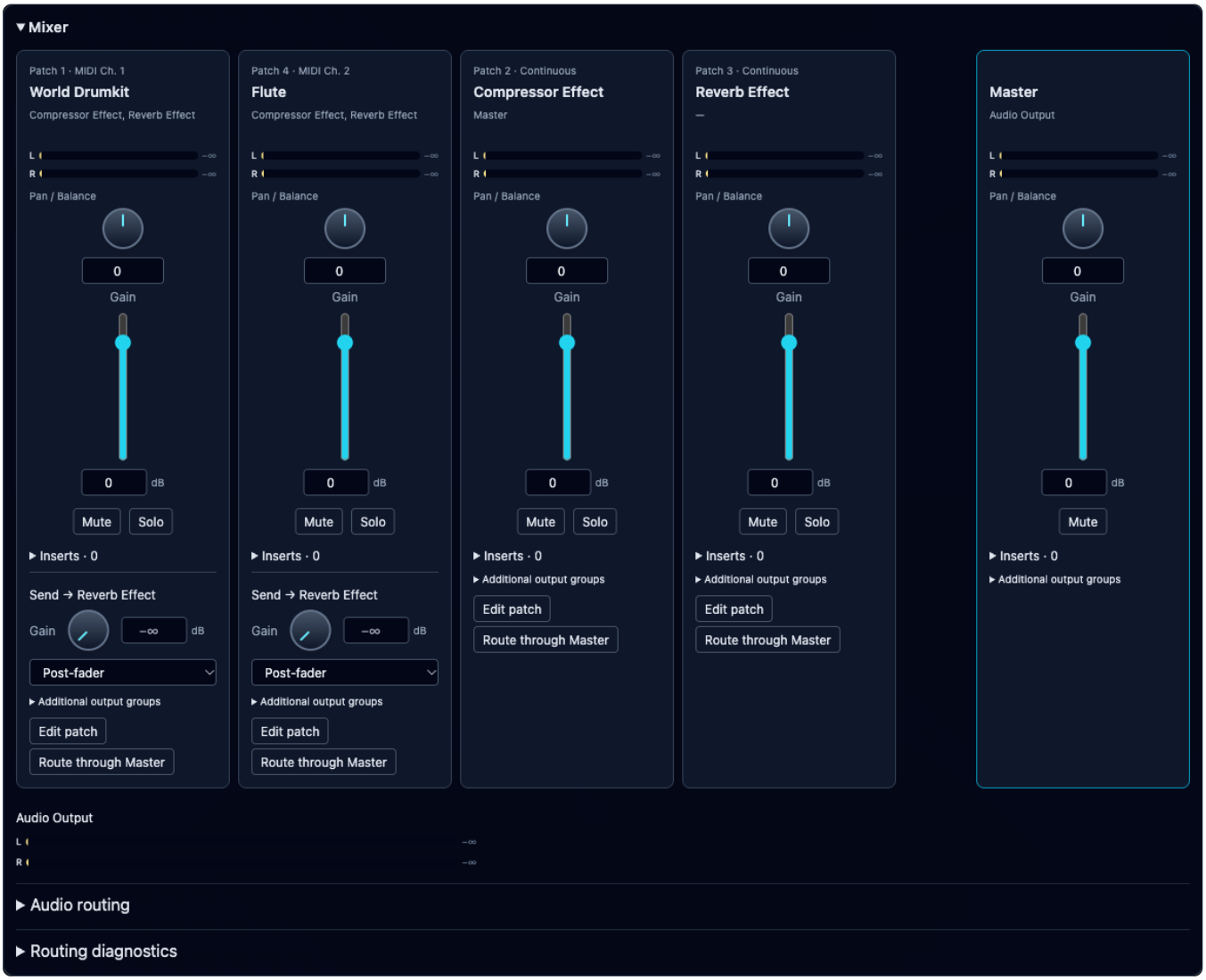
## Practical Workflow

1. Save at least one patch in Instrument Design.
2. Add instruments to the performance rack and assign channels.
3. Open Mixer, check the destination of each strip, and connect effects/returns. Guided instruments get a neutral

Master automatically; existing direct routes stay direct.

4. Use Routing diagnostics → Check routing, then Start Instruments. Adjust existing mixer controls during playback.
5. Build melodic sequencers / drummer sequencers / controller sequencers / piano rolls.
6. Save the performance to retain the rack, routes and current mix.
7. Export the performance bundle for backup/sharing, or use `Export CSD (MIDI) / Export CSD (SCORE)` when you need an offline Csound render package.

## Screenshots




*Live “Ratchet drums demo” mixer with World Drumkit and Flute before the continuous effect strips, followed by pinned Master. Playback is stopped. See the following chapters for rack and sequencer controls.*

Arranger lane palettes start collapsed. Each sequencer provides an Arrangement / Manual pads toggle; Loop song in the arranger controls repeating or stopping at the song end. Melodic and drummer sequencers have an always-visible upper-right pattern workspace for assembling reusable phrases; controller and arpeggiator phrase panels remain collapsible. The arranger uses context menus for grouping and ungrouping. Lane Mute/Solo is temporary and independent of the instrument mixer; see compact tracks and lane controls.

# Sequencer timing

Performance

## Meter, pattern length and subdivision

For a one-bar triplet bass alongside ordinary drums, choose **Meter 4/4**, **Pattern length 1 bar · 4 beats**, and **Subdivision Eighth-note triplets · 3 per beat**. The bass shows **12 steps in four labeled beat groups**. Leave Playback speed at 1×. At 120 BPM both one-bar patterns last two seconds.

Pattern length applies to the selected pad: any integer from 1–16 local beats for melodic/drum pads, or 1–32 for controller pads. Whole bars show both bars and beats; partial bars show beats. Meter, subdivision and playback speed apply to all pads of that sequencer. Subdivision offers 1, 2, 3, 4, 6 or 8 steps per meter beat. Musical names follow the denominator: three steps per beat means eighth-note triplets in /4 and sixteenth-note triplets in /8.

Global BPM counts quarter notes. A local beat is a quarter note in /4 and an eighth note in /8: one bar of 6/8 lasts three song beats (1.5 seconds at 120 BPM). Changing the meter numerator changes grouping without resizing pads. Changing the denominator retains the beat count and steps, but changes elapsed duration. Compound-meter accent grouping is not applied.

Changing subdivision keeps numbered step positions, notes, holds, velocities and timing offsets. It changes the number of visible steps within the same pattern duration; it does not redistribute the music. Hidden steps remain stored and return when you expand the pattern. Controller keypoints keep their normalized positions. Each pad has a 128-step limit; choices that would exceed it on any affected pad are disabled with an explanation.

**Advanced timing → Playback speed** contains the existing ratios, with multipliers: 1:1 = 1×, 3:2 = 1.5×, 2:1 = 2×. A speed other than 1× stays visible when Advanced timing is closed. The summary shows step count, subdivision and elapsed quarter-note song beats, separate from the pad's local length. Bar boundaries are stronger than beat boundaries, including incomplete final bars; no silent cells are added. Controller guides use bar.beat labels.

Live edits use the usual coalesced preparation path without restarting transport. A preparation failure retains the playing configuration and the edited draft. Performances saved before format v17 and app state saved before v3 use the older quarter-note beat convention. Loading their /8 tracks doubles local beat lengths and halves subdivision, preserving step arrays, ratios, rests and valid undo history. For example, an old six-beat /8 pad becomes twelve eighth-note beats.

# Instrument Rack and Engine Transport

The Instrument Rack is the top section of the Perform page and controls the live session, instrument assignments, and performance metadata.

## Compact Rack

When collapsed, the rack keeps Add Instrument and Start/Stop available. A single horizontally scrolling row shows MIDI channel and patch name for every instance, including duplicates; continuous instruments show Continuous instead of a channel. Long names have full-name tooltips. Note-triggered instruments appear first, followed by continuous instruments, with a thin vertical divider when both groups are present. The row follows the saved rack order; expand the rack to reorder instruments. Focus the row and use the arrow keys to scroll. Metadata, file actions and detailed controls return when expanded.

The header contains rack status, Add Instrument, Start/Stop, and help in both views. Collapse choices survive view switches until browser reload.

## Performance Metadata and Library Actions

The rack includes fields and actions for the current performance:

- `Performance Name`
- `Description`
- `Load Performance` dropdown
- `Save Performance`
- `Clone`
- `Delete`
- `Export`
- `Export CSD (MIDI)`
- `Export CSD (SCORE)`
- `Import`

These actions operate on the performance configuration (instrument rack + audio routing/mixer + sequencers + controllers + piano rolls), not on individual patch definitions.

- `Export` writes an Orchestron `.orch.json` / `.orch.zip` performance bundle for backup, sharing, and re-import.
- `Export CSD (MIDI)` writes an offline-render ZIP with a compiled `.csd`, the arranger performance as `.mid`, bundled uploaded sample/SF assets, and a `README.txt` with the render command. `Export CSD (SCORE)` embeds notes and controller sweeps directly in the Csound score, omits the `.mid`, rewrites supported MIDI opcodes for score playback, and writes a matching no- `-F` render command. Both modes seed enabled manual MIDI Controller lane values at time 0 on each assigned instrument channel and use 32-bit float WAV output ( `-f` ) to preserve headroom. GEN01 and `sfload` sample files must be uploaded/imported assets; raw local `samplePath` values are rejected before compilation.

## Instrument Assignments (Rack Slots)

Each rack entry selects a saved patch and a MIDI channel (1–16), or displays Continuous activation. Up to 64 user patch instances are supported, including processors. Generated mixer stages do not occupy rack slots or MIDI channels. Use distinct MIDI channels for note-triggered instances.

### Instrument Order

Note-triggered instruments appear above continuous (always-on) instruments. A thin horizontal line separates the groups when both are present.

With the instrument engine stopped, drag a card's `::` handle onto another card in the same group. An insertion line shows

the destination: the upper half inserts before the target, and the lower half inserts after it. You can also focus the handle and use ↑/↓ to move one position within its group. Instruments cannot move across the divider. Press **Escape** to cancel a drag; collapsing the rack or starting the engine also cancels it.

The compact rack and mixer strips follow this order. Master stays pinned at the right of the mixer, and insert processors remain inside their owner's insert chain. Moving a rack card changes display order without changing MIDI channels, routing, insert processing order, or controller and mixer values. Autosave, Save/Load, Clone, and native export/import preserve the instrument order.

### Instrument Controls

Patches containing `perf_controller` show five compact tuning knobs per standard rack-card row; extras wrap below. They remain editable while playing. New notes use updated values; continuous instruments adopt them after restarting the rack. Save Performance stores explicit overrides.

The separate audio mixer below the rack provides audio faders, pan/balance knobs, mute/solo, pre/post sends, inserts and meters. The old numeric Level field has been removed. Mixer gain affects held notes and external MIDI audio without changing note velocity.

While instruments run, adding/removing assignments, changing patch/channel assignments, and changing routing topology are locked. Existing mixer controls stay live. Use **Stop to edit routing** before changing connections or insert order.

### Add Instrument

- Use `Add Instrument` to create another rack slot.
- Open the slot's patch picker to browse instruments in collapsed type groups: Percussion, Melody, Bass, Effects / Noise, and Continuous (always-on). Expand a group to see its instruments alphabetically.
- Search names and descriptions across all types by entering at least four characters; results appear after a 500 ms typing pause. Clear the search to return to browsing. Template patches are excluded from the rack picker.
- This enables multi-instrument performances driven by different MIDI channels.
- Continuous effects run as explicit rack instances. Adding an insert creates a dedicated instance automatically; simply saving an effect in the library does not start it.
- The button is unavailable while the engine is running; stop instruments before changing rack assignments.

## Rack Transport (Instrument Engine Control)

The rack transport controls the instrument runtime session itself.

Buttons:

- `Start Instruments`
- `Stop Instruments`

#### `Start Instruments / Stop Instruments`

These start/stop the underlying instrument engine session.

Global arrangement transport lives in the multitrack arranger section. Its `Rewind`, `Stop`, `Play`, and `Fast forward` buttons control Arrangement-source devices and move the song cursor in `1–beat` blocks. Arranger `Play` starts all Arrangement-source devices and preserves independently playing Manual pads. Arranger `Stop` stops Arrangement devices and clears temporary audition/workspace playback while preserving independent Manual pads and the instrument engine. `Stop Instruments` stops the engine. Double-clicking arranger `Stop` resets the song cursor to the selected loop start or step `0`, preserving manual pad phase. Device Play can start its chosen source and the engine while the arranger stays stopped; see Playback source and audition.

## Session State Badge

The rack shows current session state (localized label), for example:

- running
- stopped / idle

This is the state of the instrument engine session, not just the sequencer transport.

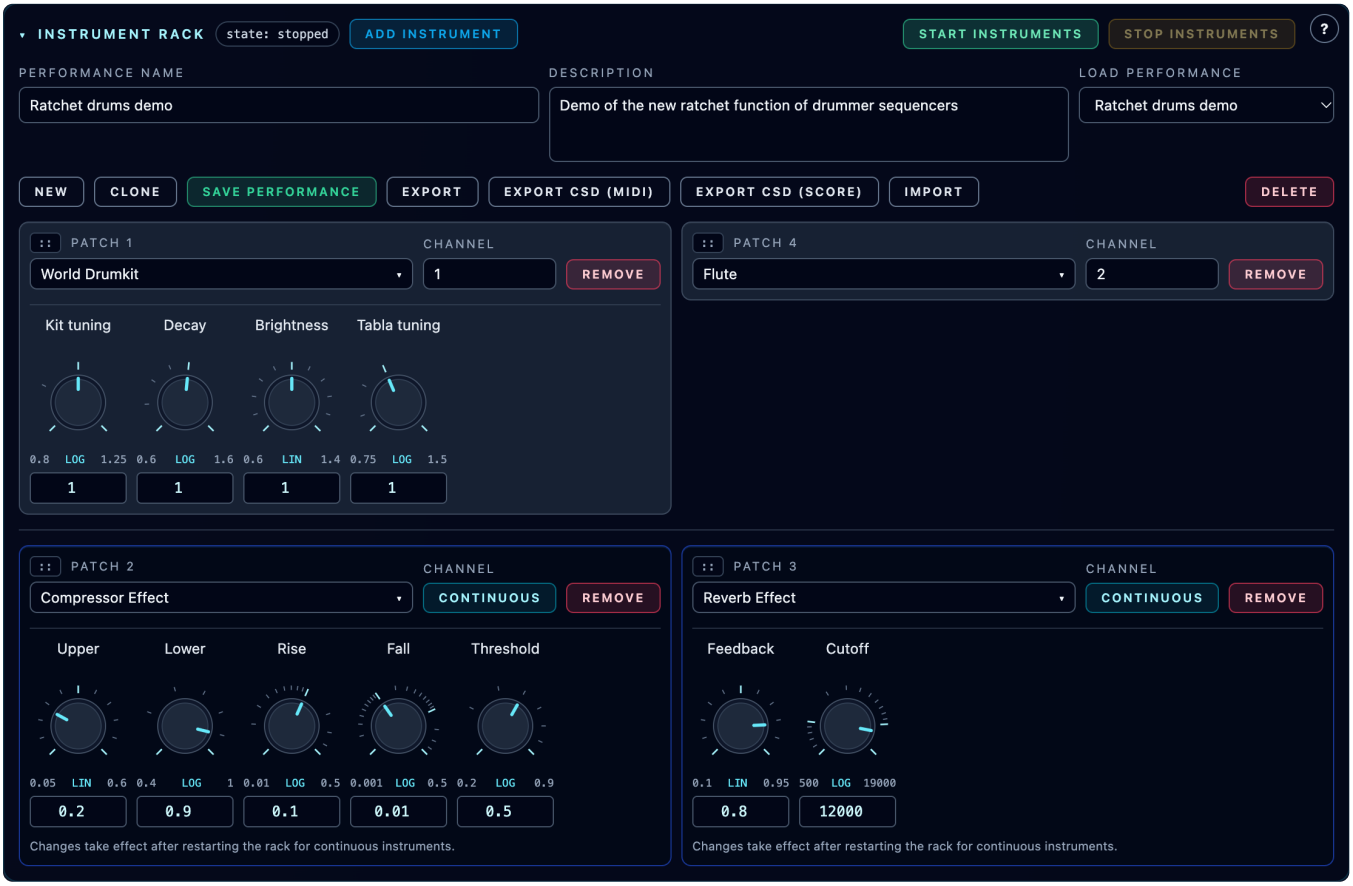
## Error Banner

If engine start/stop or transport actions fail, the Perform page shows an error banner below the rack transport section.

## Tips

- Save the performance after significant changes (rack assignments, sequencers, piano rolls, controller mappings).
- Use distinct MIDI channels for note-triggered rack instances.
- Use `sname` labels on source `outleta` nodes to name the available matrix channels. The label can be stored directly on the node or supplied by a direct `const_s` connection.

## Screenshots




*Live “Ratchet drums demo” rack: World Drumkit and Flute above the thin divider, with continuous compressor and reverb below. The :: handles reorder instruments within each group while the engine is stopped.*

# Performance Controllers

Use `perf_controller` to adapt a patch to a particular performance: envelope times, sustain, distortion amount, or another parameter accepting I-rate. Each rack instance owns its settings, even when several instances use the same patch.

## Define a Controller

Add `perf_controller` from Constants in Instrument Design. Click its gear button and configure:

| Field | Initial value | Meaning |
|---|---|---|
| Minimum | 0 | Lowest setting |
| Maximum | 1 | Highest setting |
| Default | 0.5 | Setting before a rack override |
| Scale | Linear | Linear or logarithmic knob movement |
| Label | Parameter | Caption displayed in the rack |

The fields are fixed values, without input sockets or formulas. Numbers must be finite, minimum smaller than maximum, and default within the range. Logarithmic minimum must be positive. Labels must contain non-whitespace text and be no more than 128 characters long. For a logarithmic attack-time control, try minimum `0.001`, maximum `5`, default `0.01`, and label `Attack (s)`. Connect `iout` to the envelope's I-rate attack input, then save the patch.

## Tune a Rack Instrument

Controllers appear in patch-node order beneath patch and channel. Controls wrap onto additional rows to fit the available card width. Each shows a caption, scale dashes, endpoints, LIN/LOG indicator, and current value. Linear ticks are equally spaced; logarithmic ticks follow ratios and mark decade boundaries.

- Drag vertically, or use arrow keys. Hold Shift for fine adjustment.
- Enter an exact number below the knob; Enter or leaving the field applies it. Escape cancels.
- Home and End choose the endpoints. Double-click, Delete, or Backspace on the focused knob resets to the patch default.
- Changes affect newly started notes. Sounding notes keep their initialized value. Continuous instruments require a rack restart.

For an instrument using `midi_legato`, settings are sampled at the start of each **phrase**. Touching and overlapping notes keep the same voice and settings; the next note after a gap reads current values.

For example, the steel-string guitars expose Finger → Plectrum, Brightness, Body resonance, String sustain, and Release. The legato version also exposes Slide half-time. Finger → Plectrum blends attack and tone continuously, but its setting is still read at the next phrase or note; moving the knob does not morph a string that is already sounding.

The Tube Overdrive example exposes Drive (dB), Tone (Hz), and Output (dB) for warm stereo saturation. Each insert instance retains its own settings; restart the rack after changing these continuous-effect controls.

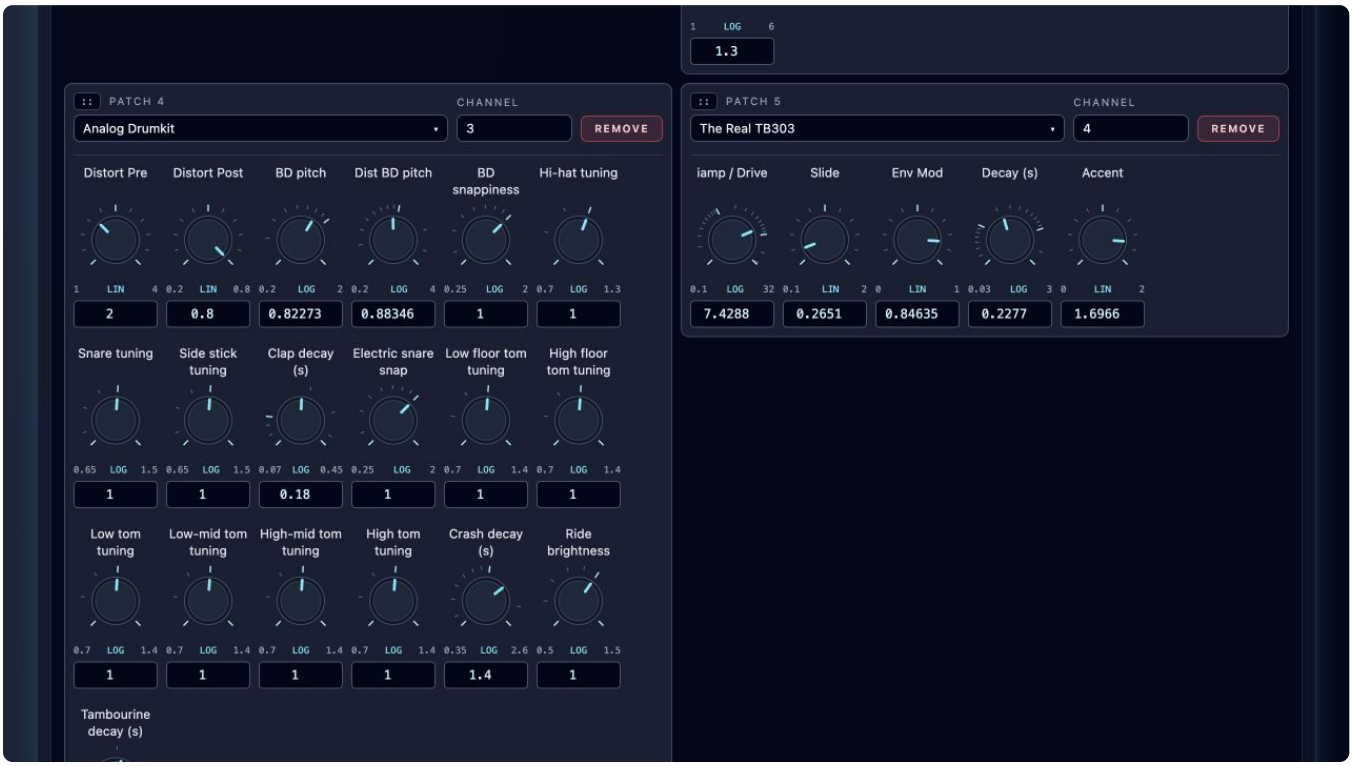
Stereo Delay exposes Tempo (BPM), Beats, Feedback and Stereo. At its defaults, echoes repeat every two seconds. Set Tempo manually and restart the rack to apply changed settings.

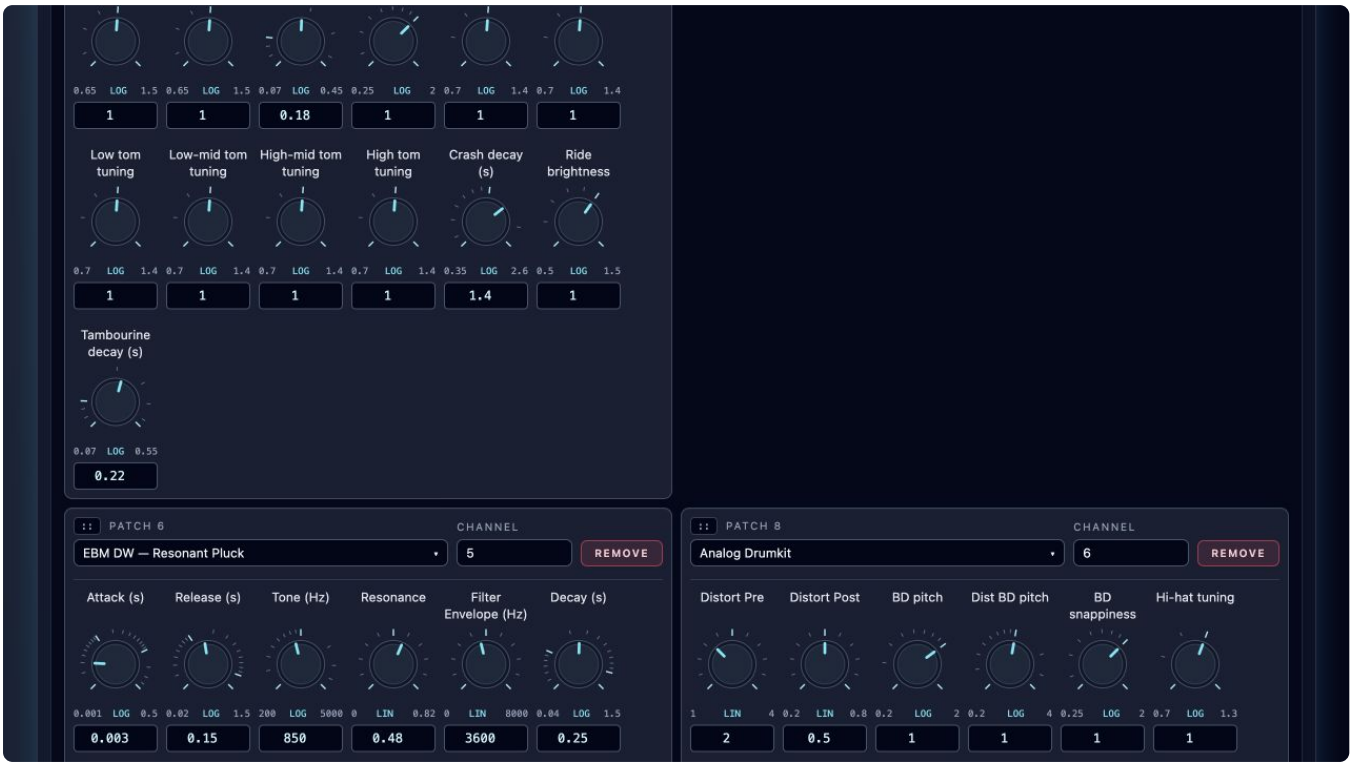
Stereo Flanger provides Rate, Delay, Depth, Feedback, Stereo Phase and Mix. It preserves separate left/right paths, supports positive and negative feedback, and can provide a wet-only send return. Restart the rack after changing its six per-instance settings.

Parametric Equalizer provides seven gain controls at 63 Hz, 160 Hz, 400 Hz, 1 kHz, 2.5 kHz, 6.3 kHz and 16 kHz. All use a linear –20 to +20 dB range with 0 dB defaults. The graph converts dB to amplitude for each filter; restart the rack after changing settings.

The Analog Drumkit has shared **BD snappiness** for notes 35/36, shared **Hi-hat tuning** for 42/46, and **Snare tuning** for 38.

Each defaults to **1**, preserving the original sound. Side stick 37, clap 39, electric snare 40, toms 41/43/45/47/48/50, crash 49, ride 51 and tambourine 54 each have one individual characteristic control. The original four library controls and their saved overrides remain valid. New notes adopt changes.




## Stereo Chorus Depth

Download Stereo Chorus and add it as a mixer insert. **Amount** controls all twelve LFO amplitudes from `0.001` to `0.1` seconds (default `0.02`); **LFO Frequency Multiplier** ranges from `0.1` to `10` (default `1`). Both controls use logarithmic scales and require a rack restart. Existing controller IDs and performance overrides are preserved.

Each tap uses `base delay + Amount + LFO` , with a 250 ms buffer. Since the LFO swings between negative and positive Amount, taps remain between 14 and 230.5 ms at the supplied 48 kHz / 32-sample settings. Larger Amount values increase both modulation depth and wet delay, producing a more pronounced modulated-delay sound. The dry stereo mix and feedback remain unchanged.

Earlier copies that use only `base delay + LFO` with a 50 ms buffer are unsafe with large Amount values. Update the instrument and restart the rack to clear invalid delay-buffer contents. Merely enlarging the buffer does not prevent negative tap times.

## Save, Reload, and Export

Untouched controls follow the patch default. After a value changes, an explicit instance override remains until reset, including when it is subsequently set to the current default. App state retains edits immediately through the existing persistence workflow; Save Performance is required to update a saved performance. Native JSON/ZIP bundles and both CSD (MIDI) and CSD (SCORE) exports preserve instance values. The CSD contains its initial settings and runs without a controller client. These controls do not send MIDI CC and do not become MIDI automation.

Node IDs identify controls; duplicate labels are allowed. Changing the selected patch clears its overrides. Editing a patch range clamps saved values; deleting a controller removes its override, with a notice in the rack. Standalone patch export and isolated audition use defaults; audition within a performance uses the selected instance’s values.

`perf_controller` is an Orchestron editor construct lowered to I-rate chnget and channel initialization. It is not a native Csound opcode.

The Stereo Widener, Bit Crusher, and Harmonizer examples expose four, five, and six performance controls respectively. These continuous effects read their settings when the rack starts. Restart the rack after editing them. Harmonizer aligns its dry signal with its measured 32 ms processing delay at 48 kHz.

# Monophonic Legato

## Phrase Behavior

Add one **midi_legato** node from **MIDI** to the main graph of a MIDI instrument. Connect **Frequency (Hz)** to control-rate oscillator frequencies and **Velocity** to amplitude and blowing-pressure formulas. At least one **madsr** phrase envelope is required; compilation fails without it. Keep **cpsmidi** and **ampmidi** for initial values needed by init-rate inputs.

- Touching and overlapping notes share one synthesis voice. Pitch changes immediately without glide. Envelopes, oscillator phase, resonances and vibrato continue.
- The last note played has priority. Releasing it returns to the most recently held note, including its velocity.
- Velocity is MIDI velocity divided by 128, matching `ampmidi 1`. Its first value is immediate; changes have a **10 ms half-time**.
- A gap starts a new phrase, even during a release tail. Initialization contours and performance-controller values refresh then. Connected notes retain phrase-initial settings and do not repeat attack bursts or pressure overshoots.
- All events within the existing Csound control boundary settle together. There is **no extra legato grace period**. Live resolution follows `ksmps`; CSD exports use `ksmps=1`.
- `madsr` retains delay, attack, decay, sustain and release semantics, including explicit `ireltim`. A new phrase applies a 2 ms amplitude declick to interrupt an old release; this is not pitch glide.
- `maxalloc` limits synthesis voices; internal note collectors remain available. Each rack instance has separate state. Live MIDI, sequencers and MIDI/SCORE CSD exports use the same phrase logic. Stopping the instrument engine clears its phrase state; MIDI panic releases the phrase. Stopping a sequencer releases its own notes without stopping other devices or the rack engine.

This is an **Orchestron virtual opcode**, not a native Csound opcode. Use it once, outside If/Switch cases, on a MIDI-activated instrument. Native release/turnoff opcodes other than `madsr` are not supported in a legato graph. Patches without this node retain their normal voice behavior. Recompile/restart a running instrument after adding it.

## Steel-string guitar example

The acoustic steel-string guitar pair includes a legato patch that rounds its pitch slide to equal-tempered semitones. Connected notes preserve the decaying string without another pluck; the phrase's initial velocity sets its pluck strength. A gap starts a new pluck and reads the current performance controls. The Slide setting is a smoothing half-time, so larger intervals take longer to reach their final fret. The matching polyphonic patch gives each note its own pluck for chords and manually timed strums. Both guitars use `wgpluck2` with velocity-sensitive pluck position and adjustable damping. Since that model's pitch is fixed at initialization, a period-synchronous delay readout transposes the ringing string. The guitar rounds the target pitch before this readout, with no pitch smoothing afterward and no new excitation at fret changes. Its default Slide half-time is 0.12 seconds so intermediate frets have more distinct dwell times. Reset an existing rack override to adopt this default.

## Expressive tenor saxophone example

Tenor Saxophone — Expressive Legato uses one continuous STK reed model with velocity-sensitive breath, reed response, filtering and loudness. Play concert A♭2–E5 (MIDI 44–76); there is no automatic tenor transposition. Route **Stereo Output → Master** in Perform.

The balanced acoustic revision retunes the reed's blow position, aperture, stiffness and pressure together, then adds body at 850 Hz and presence at 2,400 Hz. Normal velocities have a richer upper-harmonic spectrum, with restrained breath noise and output gain compensated for the stronger excitation. The example guide provides level-matched before/after previews; the four controls keep their IDs, ranges and defaults so existing performance overrides remain valid.

Velocities 1–95 cover normal playing, 96–114 progressively add growl, and 115–127 blend in a sustained overblown octave squeal. Hard connected notes can provoke the squeal; softer notes and held-note fallback restore their own velocity. Gaps restart a 10–35 ms tongue contour followed by 80 ms settling; release is 100 ms. Tone, Growl, Squeal and Vibrato are 0–1 performance controls, with defaults 0.5, 0.35, 0.65 and 0.2. Zero Growl/Squeal disables its added effect. Knob changes apply at the next phrase, while note velocity remains live throughout legato.

The native model fixes pitch at initialization, so period-synchronous delay readouts transpose the same reed signal for legato and overblowing. The readout can add up to two initial note periods of delay, and wide leaps can color the timbre. The graph stays editable using existing opcodes. Csound STK and its rawwave data are required and included in Orchestron's Docker image. The example guide includes native import instructions, control translations and WAV previews.

## Implementation references

Csound release, madsr, reinit, portk.

# Melodic Sequencers and Step Editing

Melodic sequencers are step-based pattern sequencers used for note playback.

This page covers the **melodic sequencer** editor. Drum-machine style programming is documented in Drummer Sequencers.

Use the pen beside the device name to rename it. SeeDevice Names for editing controls, validation, and import/export behavior.

See Sequencer timing for triplets, beat grouping, editing behavior and legacy compatibility.

## Collapse the Panel

Melodic, drummer, and controller groups collapse independently. The shared tempo remains above the groups. Each group has its own Add button, which expands it. Panels start expanded and remember your choice across view switches until browser reload. Collapsing does not stop playback or discard edits.

## Global Sequencer Clock

The sequencer section contains a global clock with:

- Running/stopped state badge
- `BPM` field (range `30..300`)

The backend runs the step timing (native runtime clock), which reduces browser timing jitter during playback.

## Adding / Removing Melodic Sequencers

- `Add Melodic Sequencer` creates a new melodic sequencer card
- `Add Drummer Sequencer` creates a drummer sequencer card (documented separately in Drummer Sequencers)
- A performance can contain up to 16 melodic sequencers
- Each melodic sequencer has its own `Remove` button

## Per-Sequencer Controls

Each melodic sequencer card provides:

- sequencer state badge (running/stopped or queued start/stop state)
- `Start / Stop` (sequencer enable state; can be used manually while the multitrack arranger is stopped)
- `Remove`
- `Clear Steps`
- `MIDI Channel (1..16)`
- `Scale` (root + scale type)
- `Mode` (Ionian, Dorian, Phrygian, Lydian, Mixolydian, Aeolian, Locrian)
- `Meter (2..7 over 4 or 8)`
- `Subdivision` (1, 2, 3, 4, 6 or 8 steps per local meter beat)
- `Advanced timing → Playback speed (1:1, 2:1, 3:2, 4:3, 3:4, 5:4, 4:5, 7:4)`
- `Pattern length` (every integer from 1–16 local beats; whole bars show both units)

The step editor width is derived from the sequencer's own timing and length:

- steps per beat = the sequencer's selected `Subdivision`
- pad steps = `beats * steps per beat`
- default timing (`4/4`, grid `4`) gives `16` steps for a `4`-beat pad
- odd meters can still use matching one-bar pad lengths such as `3` beats in `3/4` or `5` beats in `5/4`

`Advanced timing → Playback speed` changes how quickly the sequencer advances relative to the shared transport beat without changing its stored pad length, meter, or grid:

- `1:1` keeps normal speed
- ratios above `1:1` make the melodic sequencer run faster
- ratios below `1:1` make the melodic sequencer run slower

This is separate from `Meter` and `Subdivision`, so you can combine polymeter and true polyrhythm on the same performance page.

### Queued Start/Stop State Labels

When changes are queued to take effect at the next cycle boundary, the sequencer state label can show queued states such as:

- starting at step 1
- stopping at step 1

## Scale, Scale Type, and Mode

Orchestron separates:

- Scale root (for example `C`, `Db`, `F#`)
- Scale type (`major`, `neutral`, `minor`)
- Mode (7 diatonic modes)

This supports guided note entry and better readability in the sequencer step editor and piano roll highlighting.

## Pattern Workspace

The **Playback source** toggle selects **Arrangement** or **Manual pads**. New sequencers use Manual pads; placing their first item on an arranger lane selects Arrangement automatically. Later source choices are saved with the performance. The arranger's **Loop song** toggle controls repeating or stopping at the song end.

The upper-right pattern workspace assembles pads and reusable phrases with drag-and-drop. Cmd/Ctrl-click selects items to group; right-click splits groups. Apply commits shared phrase edits explicitly. Play loops the whole workspace; hold a speaker for 250 ms to preview an individual pad or phrase until release. Only pads with a note above zero velocity are green; rests, holds without notes and zero-velocity patterns stay dark. Selection outlines remain separate. With Manual pads running, click another pattern pad to queue it for the end of the current pattern: its outline is orange while queued and blue-green once playing. While stopped or using Arrangement, clicks only select the editing pad.

## Step Editor (Per-Step Note Programming)

Each step cell supports:

- Rest or note selection
- Chord selection, including the standard `5` power chord (root + perfect fifth)
- Hold toggle (`HOLD`)
- Octave field (separate from pitch class)
- In-scale/out-of-scale status display
- Playhead highlighting during playback

### Step Note Entry Workflow

Each step has a pitch-class dropdown grouped into:

- `Rest`
- In-scale notes (for the current sequencer scale/mode)

- Out-of-scale notes

The octave is edited separately via the `OCT` field.

This split note/octave design makes pattern entry faster than selecting full MIDI note numbers.

### Theory Aids In Steps

- In-scale notes are highlighted
- In-scale notes show degree labels ( `1..7` )
- Out-of-scale notes remain selectable (for chromatic writing)
- Enharmonic note labels are available where relevant (for example sharps/flats)
- Chord choices are grouped as none, diatonic, or chromatic for the selected step root and mode. A `5` power chord is diatonic only when both the root and perfect fifth are in the current mode.

### Hold Toggle

Each step includes a `HOLD` toggle.

Use `HOLD` to sustain behavior across steps according to the sequencer runtime logic and the note/gate pattern.

### Step Cell Visual States

Step cells visually indicate:

- Active playhead step (when the melodic sequencer + transport are running)
- In-scale programmed note
- Out-of-scale programmed note
- Rest / hold combinations

## Early and Late Notes

Timing moves an individual attack by **–50% to +50% of one local step**, in 1% increments. Zero means **On grid**. The millisecond readout follows the current tempo, grid and beat ratio: at 120 BPM with 4/4, Subdivision 4 and speed 1×, –20% is 25 ms early. Timing belongs to each pad and does not change its length, meter or playback speed.

Moving a note also moves its release, preserving its length and any HOLD extension. A following attack can shorten an ordinary preceding note to avoid overlap. Timing moves the entire chord, including its strum. Strummed notes retain their individual durations and can overlap later steps. For coincident ordinary attacks, the later logical step wins; for strummed notes this applies only to identical pitches.

An early first step plays before the boundary when the same pad repeats. On a fresh start or a different-pad launch it plays at the boundary instead. Queued stops, pauses and finite arrangement ends suppress repeat anticipation. A command received after an anticipated attack has already sounded cannot undo that attack.

Copying steps or pads preserves timing. Clear Steps resets it. Inactive drum cells retain their timing for reactivation. Old performances load on grid; Save/Load, browser restoration, native bundles and both CSD export modes preserve offsets. Timing edits during playback use the normal coalesced live-edit workflow.

Right-click a melodic note or chord, press **Shift+F10** while focusing a step, or click its ⋮ button to open **Step properties** above the grid. Use the **Timing** slider or signed percentage field; the reset arrow restores zero. Timing remains available for rests. Nonzero timing appears as a clickable badge in the step. The slider supports arrow keys; the note/chord copy handle also copies timing.

## Strummed Chords

In **Step properties**, select **Strum** → **Low** → **High** or **High** → **Low**. Notes play in pitch order, including the chord's octave and voicing. **Off** plays the chord together. **Spread** sets the time from the first to the last note as an integer **0–100% of one local grid step**; the slider, percentage field and reset share the same value. The millisecond readout follows tempo, meter,

subdivision and playback speed. Spread uses the next grid boundary, even if that step is empty.

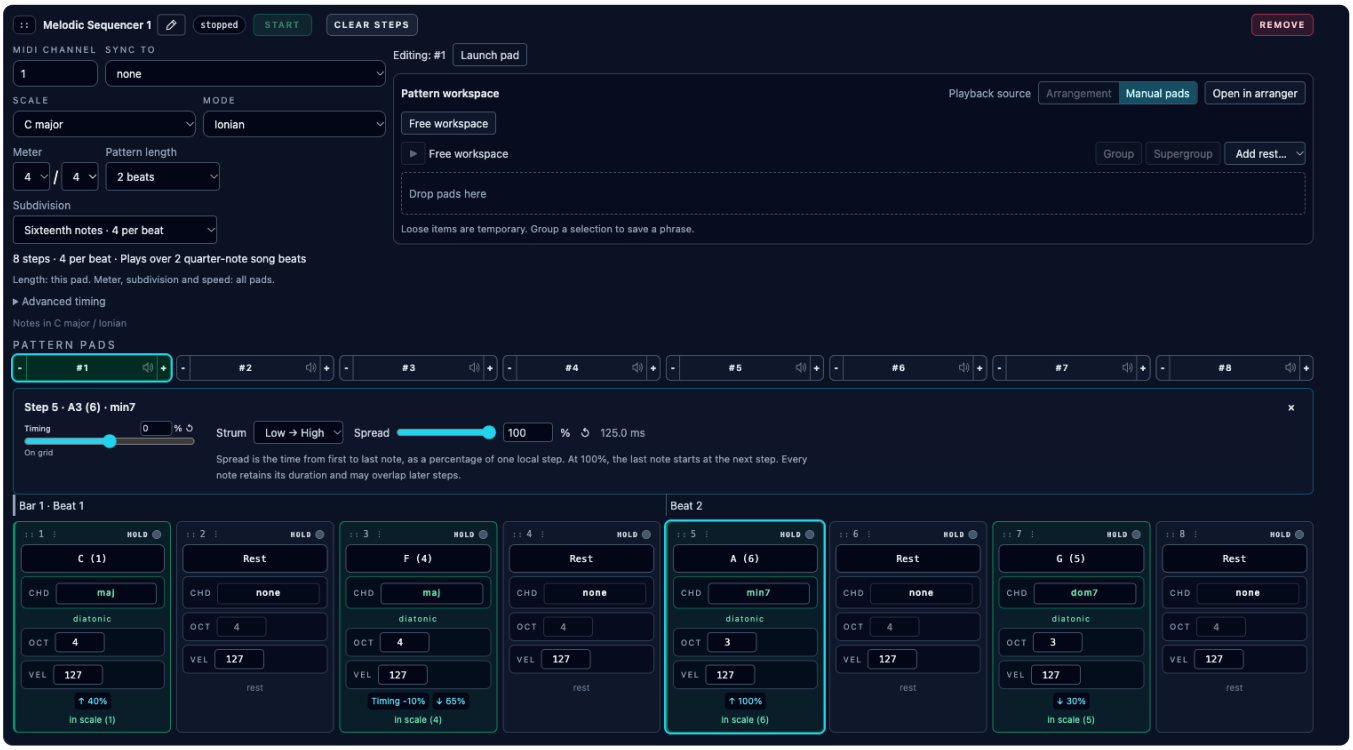
For a three-note chord at 120 BPM, 4/4, Subdivision 4 and speed 1×, a step lasts 125 ms:

| Spread | First note | Second note | Last note |
|---|---|---|---|
| 0% | 0 ms | 0 ms | 0 ms |
| 40% | 0 ms | 25 ms | 50 ms |
| 100% | 0 ms | 62.5 ms | 125 ms |

Timing shifts all these attacks together. At 100%, the last note lands at the next step relative to the shifted start. Each note keeps the original chord duration, including HOLD extensions, so later notes may overlap following steps. A repeated pitch retriggers: the newer note replaces that pitch's old release. Different pitches sounding together at a step boundary are retained. This is one strum per chord occurrence; melodic steps have no ratchet controls.

Select a note and chord to enable Strum. Removing the chord or changing the note to a rest disables strumming but retains the settings for reuse. **Off** retains Spread; **0%** plays simultaneously. A compact ↑ **40%** or ↓ **40%** badge opens the same panel. Step/pad copies, hidden steps, saves, browser restoration and native bundles preserve the settings; **Clear Steps** resets them. MIDI and SCORE exports use the same attacks and durations.

The panel edits the displayed pad and closes on pad changes, collapse, removal, New/Load/Import, or Escape. Direction, spread and timing edits to a strum already in progress apply on its next occurrence; live HOLD edits still update its release. Same-pad repeats can carry delayed notes across their boundary. Stops, seeks, a different pad, arrangement rests and finite endings cancel unfinished strums and release their notes.



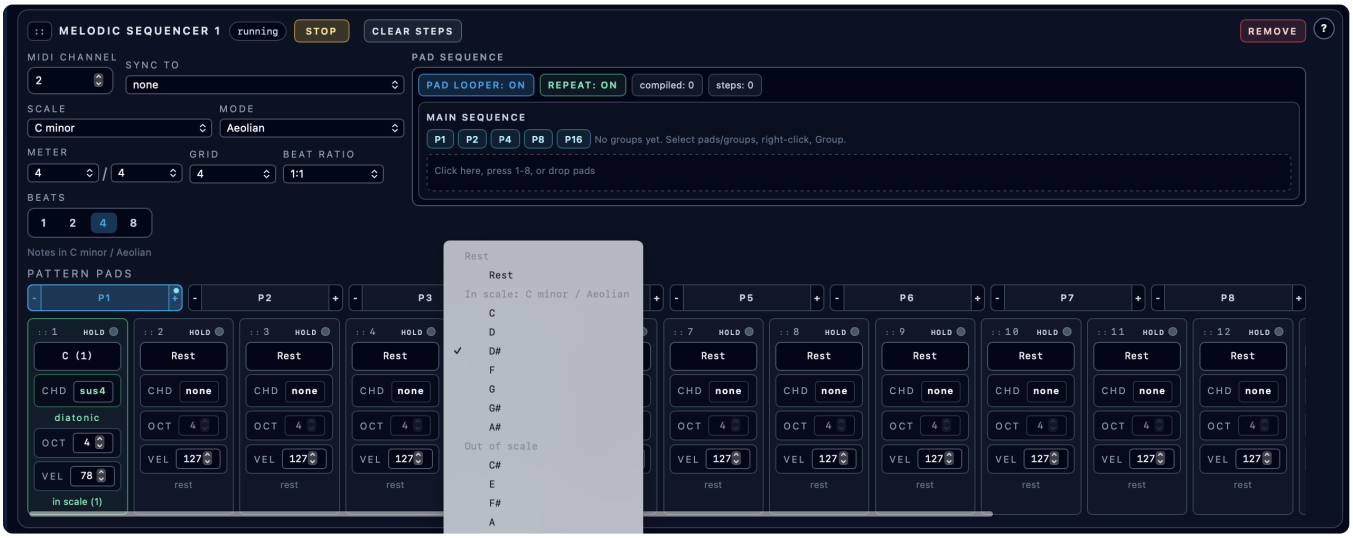

## Clear Steps

`Clear Steps` resets the current melodic sequencer step contents (for the active pad pattern context) so you can quickly reprogram it.

## Related Features

- Pattern pads (#1..#8), queued pad switching, pad copying, transposition, and phrase library and arranger controls are documented in Pattern Pads and Reusable Phrases.

# Screenshots




*Melodic sequencer detail showing step editing, note selection, octave, and hold controls.*

# Drummer Sequencers

Drummer sequencers are drum-machine style step sequencers for fixed MIDI drum keys.

They are optimized for per-step drum hit programming instead of melodic note/chord entry.

Use the pen beside the device name to rename it. See Device Names for editing controls, validation, and import/export behavior.

See Sequencer timing for triplets, beat grouping, editing behavior and legacy compatibility.

## Collapse the Panel

The drummer group collapses independently of melodic and controller sequencers. Its header and Add button remain available even when empty; adding a drummer sequencer expands the group. Panels start expanded and remember your choice across view switches until browser reload. Collapsing does not stop playback or discard edits.

## What Makes Them Different

Compared to melodic sequencers, drummer sequencers:

- use MIDI key numbers ( `0..127` ) per row (drum instrument selection)
- do not use scale / mode / chord controls
- do not use pad transpose edge buttons
- let you program multiple drum rows per step
- store velocity per active drum hit (per row + step)

## Adding / Removing Drummer Sequencers

- Use the `Add Drummer Sequencer` button in the sequencer section header to add a drummer sequencer card
- A performance can contain up to 16 drummer sequencers
- Each drummer sequencer card has its own `Remove` button

## Per-Drummer-Sequencer Controls

Each drummer sequencer card provides:

- sequencer state badge (running/stopped or queued start/stop state)
- `Start / Stop` (sequencer enable state; can be used manually while the multitrack arranger is stopped)
- `Remove`
- `Clear Steps`
- `+ Key` (add another drum row)
- `MIDI Channel (1..16)`
- `Meter (2..7 over 4 or 8)`
- `Subdivision` (1, 2, 3, 4, 6 or 8 steps per local meter beat)
- `Advanced timing → Playback speed (1:1, 2:1, 3:2, 4:3, 3:4, 5:4, 4:5, 7:4)`
- `Pattern length` (every integer from 1–16 local beats; whole bars show both units)
- Playback source and upper-right pattern workspace (same as melodic sequencers)

## Drum Rows (`Keys`)

The left side of the drummer grid contains vertically stacked drum rows.

Each row has:

- row index label
- MIDI key input ( `0..127` )
- row remove button ( `x` )

### Auditioning Drum Keys While Editing

When the instrument engine session is running, changing a row key sends a short one-shot MIDI note preview on the drummer sequencer's MIDI channel.

This helps identify which drum sound is mapped to a given MIDI key number.

## Drum Step Grid (LED Matrix)

The grid is row-based:

- rows = selected drum keys
- columns = `beats * steps per beat`

This keeps the `Keys` column horizontally aligned with the LED rows.

### LED States

- inactive step: dark LED with visible border
- active step: red LED
- current playing column: highlighted across every drum row
- active step in the current playing column: flashing green LED

### Velocity (Per Hit)

Velocity is shown by LED color saturation (stronger color = higher velocity).

The LED border color stays visible even at low velocities, so low-velocity active hits remain distinguishable from inactive steps.

While dragging vertically on a hit, the UI also shows a live numeric `velocity: xx` readout next to the edited step.

### Editing Workflow

- Click an inactive LED to activate a hit
- Click an active LED (without dragging) to deactivate it
- Click and drag vertically on an LED to change velocity (`0..127` )
- While dragging, watch the live `velocity` readout for the exact value
- Keyboard:
- `Enter` / `Space` toggles the hit
- `Arrow Up` / `Arrow Down` adjusts velocity

## Early and Late Notes

Timing moves an individual attack by **–50% to +50% of one local step**, in 1% increments. Zero means **On grid**. The millisecond readout follows the current tempo, grid and beat ratio: at 120 BPM with 4/4, Subdivision 4 and speed 1×, –20% is 25 ms early. Timing belongs to each pad and does not change its length, meter or playback speed.

Moving a hit also moves its release. A following attack in the same drum row can shorten the preceding hit to avoid overlap. If neighboring attacks in that row land at exactly the same instant, the later logical step wins. For repeated hits within a step, see Ratchets and Velocity Ramps.

An early first step plays before the boundary when the same pad repeats. On a fresh start or a different-pad launch it plays at the boundary instead. Queued stops, pauses and finite arrangement ends suppress repeat anticipation. A command received

after an anticipated attack has already sounded cannot undo that attack.

Copying steps or pads preserves timing. Clear Steps resets it. Inactive drum cells retain their timing for reactivation. Old performances load on grid; Save/Load, browser restoration, native bundles and both CSD export modes preserve offsets. Timing edits during playback use the normal coalesced live-edit workflow.

Drag a hit **left/right** to adjust timing, or **up/down** to adjust velocity. The first drag direction locks the property until release; clicking without dragging still toggles the hit. Changed hits show a signed percentage and a displaced indicator.

Use **Left/Right** arrow keys for timing and **Up/Down** for velocity. Right-click a hit, or press **Shift+F10**, to open **Step properties**, which includes Timing controls with a slider, numeric field and reset. The controls affect that hit in the displayed pad. Closing or collapsing the editor ends the gesture.

## Pattern Pads and Reusable Phrases

Drummer sequencers support the same `#1..#8` pattern-pad workflow as melodic sequencers.

While a drummer sequencer is running in Manual pads mode, clicking another pad selects it for editing and queues playback at the next pattern boundary. While stopped or using Arrangement, clicking a pad only selects it for editing.

Drummer pad length is stored in beats, while meter, grid, and beat ratio are configured per drummer sequencer.

This keeps drummer pads beat-based while still allowing one-bar patterns in odd meters such as `3/4`, `5/4`, or `7/8` by matching the pad beat count to the meter, while `Advanced timing → Playback speed` changes how fast the row pattern cycles against the shared transport without changing the stored beat length.

Edit pad contents here and assemble reusable groups/supergroups in the upper-right pattern workspace. Drop and reorder pads, Cmd/Ctrl-click to group selections, and right-click to ungroup. Apply commits edits to existing phrases; loose workspace items stay temporary. Play loops the assembly, and holding a speaker for 250 ms previews one item until release. Place saved phrases in the multitrack arranger. The Playback source toggle chooses Arrangement or Manual pads. New devices use Manual pads; the first item placed on their arranger lane selects Arrangement, and later source choices are saved. Loop song in the arranger controls repetition at the song end. Click selects a pad. While Manual pads is running, clicking another pad also queues all rows for the end of the current pattern. The queued pad has an orange outline, then turns blue-green when playing. While stopped or using Arrangement, clicks only select for editing. Launch pad remains available.

Differences vs melodic sequencer pads:

- no transpose edge buttons ( – / + )
- pad content is drum-row hit data + per-hit velocities (not note/chord/theory state)

## Screenshots

*Drummer sequencer showing multiple drum rows, per-step hits, and velocity editing on the LED grid.*

## Typical Uses

- Kick / snare / hat step programming
- Percussion layers with multiple rows
- Switching between groove variations using pattern pads
- Automating drum pattern changes with the arranger while performing on piano roll/controllers

## Ratchets and Velocity Ramps

Right-click a drum cell, or focus it and press **Shift+F10**, to open **Step properties**. Set **Ratchets** to 1–8: 1 is the normal single hit, and higher counts divide one local step into evenly spaced hits. An **×N** badge identifies the roll in the grid. At 120 BPM with Subdivision 4 in 4/4, four hits occur at 0, 31.25, 62.5 and 93.75 ms within a 125 ms step.

Enable **Velocity ramp** and choose **Final velocity** (0–127). The first hit uses the cell's velocity and the remaining hits interpolate to the final value. For four hits, 100 → 40 produces 100, 80, 60, 40. Zero-velocity hits are silent. Disable the ramp for constant velocity. At one hit the ramp controls are disabled, but their values are retained.

Timing shifts the whole roll while preserving its spacing. The next active cell in the same drum row cuts off unfinished repeats; an equal-time collision belongs to the later step. On a fresh start or a different-pad launch an early first roll starts at the boundary. On a confirmed same-pad repeat it can begin early, and a late final roll can finish across that boundary. Stopping or seeking that device, launching a different pad, entering an arrangement rest or reaching its finite end cancels unfinished rolls. Independently running Manual pads retain their rolls through arranger Start, Stop and seeks. A roll already in progress keeps its count, ramp and timing when those settings are edited; the next occurrence uses the prepared changes.

Editing these properties does not activate an inactive cell. Disabling and re-enabling a cell preserves them; copying pads and saving/exporting preserves them, including hidden steps. Clear Steps restores one hit without a ramp. The panel edits the displayed pad and closes when changing pads, collapsing the drummer section or removing the target. Escape also closes it. Existing click toggles, horizontal timing drags, vertical velocity drags and arrow-key editing remain available.

# Controller Sequencers

Controller Sequencers automate MIDI CC values using editable curves or discrete steps.

Use the pen beside the device name to rename it. See Device Names for editing controls, validation, and import/export behavior.

See Sequencer timing for triplets, beat grouping, editing behavior and legacy compatibility.

## Collapse the Panel

The controller group collapses independently of melodic and drummer sequencers. Its header and Add button remain available even when empty; adding a controller sequencer expands the group. Panels start expanded and remember your choice across view switches until browser reload. Collapsing does not stop playback or discard edits.

## What A Controller Sequencer Does

A controller sequencer sends timed MIDI Control Change messages on its selected channels. **Curve Mode** (the default) samples an editable curve. **Step Mode** sends an integer at each populated grid cell. The mode applies to all eight pads; switching modes preserves both datasets on every pad.

Typical uses:

- filter sweeps
- morph controls
- modulation depth animation
- macro movement synchronized to transport

## Adding / Removing Controller Sequencers

- `Add Controller Sequencer` creates a new controller sequencer card
- A performance can contain up to 16 controller sequencers
- Each controller sequencer has a `Remove` button

## Per-Controller-Sequencer Controls

Each controller sequencer provides:

- Running/stopped state badge
- `Start / Stop` enable toggle; can be used manually while the multitrack arranger is stopped
- `Controller # (0..127)`
- **MIDI Channels** checkboxes, 1–16 in one row beside `Clear Steps`
- `Meter (2..7 over 4 or 8)`
- `Subdivision` (1, 2, 3, 4, 6 or 8 steps per local meter beat)
- `Advanced timing → Playback speed (1:1, 2:1, 3:2, 4:3, 3:4, 5:4, 4:5, 7:4)`
- `Pattern length` (every integer from 1–32 local beats, subject to 128 steps)
- CC label preview (`CC N`)
- **Curve Mode / Step Mode** toggle
- Curve editor or step grid
- **Reset Value** (`0..127`, default `0`) in Step Mode

### MIDI Channels

Check the channels that receive the sequencer's CC messages. All 16 are checked by default (OMNI); at least one must remain checked. The row scrolls horizontally on narrow screens. This is a device setting shared by all its pads.

Channel edits use the same 80 ms live-edit synchronization as curve edits and take effect at the next engine block without restarting transport or losing queued pads. In Curve Mode, the current curve value is sent to the new selection; removed channels retain their last received value. In Step Mode, departing CC/channel pairs receive Reset Value, newly targeted pairs receive Reset Value, and continuing pairs hold their state until the next populated step.

Selections survive Save/Load Performance, browser reload, native JSON/ZIP export/import, and both CSD export modes. Older performances without the setting load with all 16 channels checked.

### Pattern Length (1–32 local beats)

This defines the pattern repeat length relative to the controller sequencer's own timing. Step Mode shows `length × subdivision` cells, up to 128; shortening hides trailing cells without deleting them. Keypoints stay normalized across the full pad duration, so the same curve shape stretches automatically when you choose a longer beat length.

`Advanced timing → Playback speed` changes how quickly the controller sequencer moves through that curve relative to the shared transport. Faster ratios create repeating automation polyrhythms without changing the stored keypoint positions.

## Pattern Workspace

The **Playback source** toggle chooses **Arrangement** or **Manual pads**. New controller sequencers use Manual pads until their first item is placed on an arranger lane, which selects Arrangement. Later choices are saved with the performance. **Loop song** in the arranger controls repeating or stopping at the song end.

The always-visible workspace sits beside the musical controls, or below them on narrow screens. Drag pads into it, select several with Cmd/Ctrl-click, then Group or Supergroup. Reorder by dragging, split through the context menu, and Apply shared phrase changes explicitly. See Pattern Workspace for editing, deletion and retained drafts.

Play loops the displayed assembly and outlines its current pad, phrase or rest. Hold a speaker for 250 ms to temporarily override it; release resumes the workspace at its current position. Stop restores prior playback. Both modes and channel routing remain effective. In Curve Mode, two zero-valued endpoints alone look empty; additional points or nonzero values highlight the pad. In Step Mode, any populated visible cell highlights it, including zero. Copying a pad copies both datasets. **Clear Steps** clears only the displayed pad in the selected mode.

## Step Editor and Reset Value

Enter a whole number from **0 to 127** in a cell. Press Enter or leave the cell to apply; Escape cancels. Delete the text to leave a cell empty. Empty cells send nothing and the receiver holds its previous value. Zero sends a real CC message, and repeated identical values each send again at their own step. Beat/bar groups and the highlighted playing step follow the device's own meter, subdivision and playback speed.

**Reset Value** is sent for the chosen CC on the selected channels on start and stop, including finite endings. It is sent before the first step and notes at the start position. Starting in an empty part of the pattern waits for the next populated step. Choose a value appropriate to the controlled parameter: zero is the default, not a universal neutral setting for every CC.

A seek during playback sends Reset Value, then continues from the new position without replaying earlier cells. A stopped seek sends nothing. Normal pad changes, loops and arrangement rests hold the last received value without resetting. Workspace/speaker previews initialize on entry and reset when restoring prior playback at its existing phase. Arranger controls leave independently running Manual pads alone.

Entering Step Mode while running initializes Reset Value. Leaving it resets the old step output before the curve takes over. Editing Reset Value takes effect on the next reset. Reset messages obey lane mute/solo, and only address the selected CC/channel pairs; no automatic “Reset All Controllers” message is sent. Sources sharing a CC/channel still use last-message-wins behavior.

See Granular Capture for a complete four-lane stutter, freeze, capture and granular step pattern.

## Curve Editor Interactions

The curve editor is an interactive graph view (spline-based curve display and sampling).

When all keypoints have the same value, the curve appears as a horizontal line across the editor. This also applies to the minimum ( `0` ) and maximum ( `127` ) CC values.

Supported interactions:

- Click background to add a point (interior points only)
- Drag a point to change position/value
- Double-click an interior point to remove it
- Endpoints are boundary anchors (positions remain at start/end of the curve)

### Visual Playback Indicators (When Running)

When the main transport is running and the controller sequencer is enabled, the editor shows:

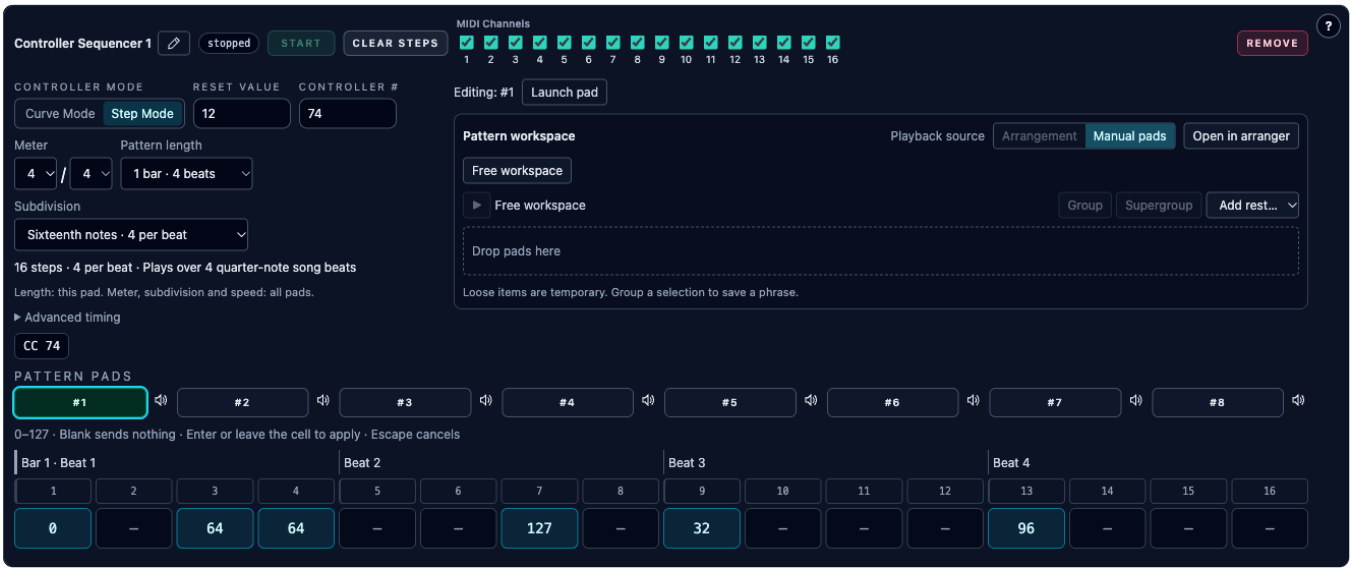
- a vertical playback position line
- a marker showing the currently sampled CC value on the curve

This makes it easy to understand exactly what value is being sent at each transport moment.

## Live Use Notes

- Controller sequencers run alongside melodic sequencers on the same backend transport clock.
- Curve and step edits are combined after an 80 ms editing pause and prepared while the previous configuration keeps playing. They take effect at the next engine block without resetting transport. The editor modifies the displayed pad even when the arranger selected it automatically.
- See Editing During Playback for failure handling and transport behavior.
- Pad clicks select the editing pad. While Manual pads is running, clicking another pad also queues it at the end of the current pattern: orange marks it while queued, then blue-green marks it when playing. Launch pad also queues the selected pad; the separate speaker previews it after a 250 ms hold.
- You can combine automated controller sequencers with manual MIDI controller knob lanes on the same performance page.
- If you automate the same CC number from multiple sources, the last-sent value wins at the MIDI receiver side (plan mappings accordingly).

## Screenshots




*Step Mode sends each populated cell, including zero and repeated values; empty cells hold the last value.*

*Controller sequencer with curve editor, CC configuration, and playback indicator.*

# Arpeggiators

An arpeggiator processes MIDI notes and drives an existing rack instrument. Send chords from a melodic sequencer, an on-screen keyboard, or external MIDI to its **Input Channel**. Choose the named instrument in **Target Channel**. Both controls are always visible in the top row, with **Input Channel → Target Channel** showing the MIDI direction. Inputs must be unique; an arpeggiator cannot target another arpeggiator input.

## Start and Synchronize

The device uses the shared green **Start** and orange **Stop** button styles.

**Arranger** is the default playback mode. Device Play starts the chosen playback source independently: Arrangement starts at the selected song loop or stopped arranger cursor, joining any Arrangement devices already playing; Manual pads starts the selected editing pad from its beginning. Arranger Play starts all Arrangement-source devices. Arranger Stop stops those devices and clears previews/workspace playback while preserving independently playing Manual pads and Live devices. Incoming chords join the next subdivision of the shared beat. Chord changes preserve the phrase position. Seeks and selected loop wraps reconstruct Arrangement phrases at the destination; finite arrangements stop at their end. Manual pads retain their phase through these song operations. Both sources still need input notes to make sound.

**Live** runs independently while the instrument engine is active. Its Start button can start the engine. The first incoming note establishes the Live clock; subsequent chords retain its pulse. Both modes use global BPM, including tempo changes.

**Restart** controls phrase position without moving the subdivision clock:

- **Continue:** keep advancing through chord changes (default).
- **First held note:** restart the phrase on a fresh key gesture.
- **Every beat / Every bar:** restart at the next subdivision of each master beat/bar.

Straight, triplet and dotted rates use musical beat positions. Swing alternates long and short intervals with an unchanged total duration for each pair.

## Hold and Processing

**Hold Off** is the default. Releasing all input notes silences the output, preserving rests in a source sequence. **Hold and replace** remembers the last chord after release; the next fresh gesture replaces that chord. **Add/remove notes** toggles pitches explicitly. **Clear held notes** releases the arpeggiator's notes and clears pending input and launches.

**Active** generates the pattern; **Bypass** forwards incoming notes to the target; **Mute** consumes notes silently. Stop, removal, routing changes and processing/playback-mode changes release notes owned by this arpeggiator.

## Eight Musical Pads

Each pad stores a complete musical variation: order, rate, octave range, transpose, expression, harmony, rhythm, rotation and random seed. Click **#1–#8** to choose the editing pad. In Arranger mode with Manual pads running, this also queues the pad at the next launch boundary (rhythm cycle by default). The queued pad has an orange outline, then turns blue-green when playing. Clicking the playing pad cancels the queue. Hold its separate speaker for 250 ms to preview it. Use Launch pad for an explicit launch in Live/manual mode. The playing pad has a dot; a queued pad has a clock mark. Automatic pad changes leave your editing pad and unfinished preset name in place.

Each pad has two independent lengths:

- **Rhythm steps:** 1–32 steps (default 16 sixteenths).
- **Pad duration:** 1–8 or 16 master beats (default four beats), used by the arranger.

For example, a five-step rhythm continues across successive four-beat occurrences of #1. Changing to #2 restarts the musical cursors at that launch boundary. Arrangement pauses silence output while still tracking incoming pitches.

Drag one pad onto another to copy its musical variation. The shared phrase library defines groups and supergroups; song

placement and repeat controls live in the multitrack arranger. New arpeggiators start with Manual pads and an empty arrangement. The Playback source toggle selects Arrangement or Manual pads; placing the first item on the lane selects Arrangement, and subsequent source choices are saved. Their named arranger lanes contribute to Fit, arrangement bounds, seeking and both CSD exports, including performances containing only arpeggiators.

In Live mode, pad launches use **Next cycle** by default, or **Next master bar**. Stopped launches select immediately and Cancel launch removes a queued launch. In Arranger mode, held speakers start at the next pad/rest boundary while the arranger runs, or at an audio block while stopped. Releasing restores the previous playback.

## Pattern Workspace in Arranger and Live

The upper-right workspace offers the same drag assembly, Group/Supergroup, local Ungroup and explicit Apply/Save as new/Discard controls as the other sequencers. It stacks beneath routing controls on narrow screens; pads and the rhythm grid remain full-width. See Pattern Workspace for draft retention and definition deletion.

Play loops the entire displayed assembly, highlighting its current pad, phrase or rest. Its phrase timing uses pad duration, independently of rhythm length. A speaker held for 250 ms overrides the workspace until release; the workspace resumes at its current clock position. Stop restores the prior arrangement, manual or stopped state. Edits enter at the next workspace cycle boundary.

Live starts follow Next cycle or Next master bar when a Live clock is established, otherwise an audio block. Live hides arrangement-only controls but keeps phrase editing and audition available. Input or held notes are required; no test chord is generated. Hold and Active/Bypass/Mute still apply. Chord changes during audition remain current when the underlying Live pulse resumes. A temporarily auditioned stopped device accepts notes without changing its saved Start/Stop setting.

Green pads contain a note-producing step with positive velocity and probability. Rest-only, tie-only, zero-velocity and fully suppressed pads remain dark, regardless of held notes. Selection and queued outlines remain separate. Zero-velocity steps are silent.

## Edit the Rhythm Grid

Select a step, then choose its action:

| Action | Result |
|---|---|
| Next note | Play the next pitch in the selected order |
| Chord-note position | Play a relative position in the ascending, octave-expanded input pool; oversized positions wrap |
| Rest | Advance time silently |
| Tie | Extend the preceding note/chord without retriggering; silent without a preceding note |
| Chord | Play the entire expanded input pool |

Velocity bars scale incoming accents in percent. Each step also has probability, a gate override and **1–4 ratchets**. Gate supports **5–200%**; values above 100% overlap different pitches. **Use pad gate** clears the step override. Rests and probability misses keep the automatic note cursor stationary unless **Advance through rests** is enabled. **Note repeats** repeats an automatic pitch before moving on. **Rotation** changes the rhythm's starting step.

Keyboard editing: Left/Right selects adjacent steps; Space toggles a rest; Delete/Backspace inserts a rest; T inserts a tie; C plays the chord; N restores Next note. Double-click also toggles a rest. Controls have accessible text labels.

Note previews use large 18 px labels. Narrow grids scroll horizontally; long chord labels stay on one line with an ellipsis. Hover over a step for the complete note list and scale degrees, also available to screen readers.

With **Follow source** or **Custom** scale handling, step borders use the piano keyboard's degree colours: **1 red, 2 orange, 3 yellow, 4 green, 5 teal, 6 blue, 7 violet**. Degrees describe the generated pitches after transposition and quantization. Follow source uses each note's incoming scale context, with the pad's scale as fallback. Octave repetitions keep the same colour. Chords show equal, distinct border segments for their degrees, ordered 1–7. **Scale Off**, missing previews, rests and ties keep

neutral borders. Previews and colours appear only when editing the playing pad. A cyan outer outline marks the selected step independently of its degree colour; the playback background and keyboard focus remain separate.

## Expression and Harmony

Input velocity preserves accents; Fixed and Random velocity remain available. Timing and velocity humanization change individual strikes. Timing variation stays inside each strike's scheduling window and never moves the next clock position.

**Repeat variation** repeats seeded random choices; **Evolve** varies them by cycle. **New variation** changes the saved seed. Each pad has its own seed, independent of other devices; identical input and transport positions produce reproducible events in playback and exports.

Octaves expand the input pool before applying note order. Down descends through the complete range. Up/Down reverses without duplicating endpoints. **Octave by octave** retains an alternative traversal in Advanced timing.

Scale handling defaults to **Off**. **Follow source** uses the source's root/mode and falls back to the pad's configured scale when absent. **Custom** always uses the pad's scale. The effective scale and resolved pitch preview help explain the output. Playback highlights follow audible PCM status.

## Presets, Live Edits and Collapse

Presets apply only to the editing pad. **Modified** marks changes from its preset. **Update preset** overwrites a user preset; **Save as preset** creates a new one. Built-in presets remain available. Routing, playback settings and other pads are preserved.

Live edits coalesce for 80 ms and apply at audio render boundaries, preserving transport and pending launches. Invalid edits retain the working configuration. Status updates never resubmit musical settings. Collapse hides visual work while preserving playback, pad selection, details and drafts.

*Arpeggiator showing its input and target routing, eight musical pads, per-step rhythm controls, custom harmony settings, and pattern workspace sequence.*

# Migration to Performance Version 15

Versions 1–14 load in memory into the new model. Existing settings become #1; the other seven pads use defaults. Identities, routes, rate, order, octaves, expression and user presets are retained; old accent cycles become grid velocities.

Existing performances adopt Arranger mode, Hold Off, Continue, corrected swing and full-range ordering, so they can sound different. Save/export writes version 16; app-state version 2 and native bundle-envelope version 1 stay unchanged. Runtime positions and held notes are not saved.

# Piano Rolls

Piano Rolls provide manual performance input with scale-aware keyboard highlighting.

Use the pen beside the device name to rename it. See Device Names for editing controls, validation, and import/export behavior.

## Collapse the Panel

Collapse the Piano Rolls header to hide the keyboards and their settings. Add and help remain available; adding a keyboard expands the group. Panels start expanded and remember your choice across view switches until browser reload. Collapsing does not stop playback or discard edits.

## Purpose

Piano Rolls are for live/manual playing on an on-screen keyboard while the instrument engine is running.

They coexist with melodic sequencers and controller sequencers, which means you can jam manually on top of sequenced playback.

Starting a piano roll is independent from the multitrack arranger transport. If the instrument engine is stopped, `Start` brings up the engine so the keyboard can play without starting arrangement playback.

## Add / Remove / Enable

Each piano roll supports:

- `Add Piano Roll`
- `Remove`
- `Start / Stop` (enable/disable that piano roll)

A piano roll only sends notes when:

- that piano roll is enabled
- instruments are running

## Piano Roll Controls

Each piano roll has:

- `MIDI Channel (1..16)`
- `Velocity (0..127)`
- `Scale`
- `Mode`
- `Autochord` (off by default)
- `Chord notes (3, 4, or 5, default 3; available when Autochord is on)`

MIDI Channel and Velocity are grouped together, with a small gap before Scale, Mode, Autochord, and Chord notes. All labels appear above their controls; the Autochord checkbox matches the height of the adjacent selectors.

Each MIDI-driven rack instrument requires a unique MIDI channel. Set the piano roll to the channel of the instrument you want to play.

## Autochord

Enable **Autochord** to extend a highlighted key into an ascending chord. **Chord notes** counts the pressed note, so in C major

pressing C4 plays:

| Chord notes | Sounding notes |
|---|---|
| 3 | C4–E4–G4 |
| 4 | C4–E4–G4–B4 |
| 5 | C4–E4–G4–B4–D5 |

The chord starts on the key you press and stacks every other highlighted note, continuing into higher octaves. It follows the same scale/mode as the keyboard. In **mixed** mode, it uses only notes common to the active scales; the resulting intervals may differ from thirds. For example, C major and G major share C, D, E, G, A, B, so pressing C4 with three notes plays C4–E4–A4. Sparse intersections can produce octave-spaced voicings. Keys outside the highlighted set still play single notes.

All chord notes use the keyboard's MIDI channel and velocity, and visible sounding keys light up together. Release ends the chord; notes shared with another held chord stay on until their last held key releases. Changing scale, mode, channel, size, or the toggle affects the next press without altering held chords. Upper voices may extend beyond the visible keyboard, but notes above MIDI 127 are omitted.

Each piano roll retains its own toggle and size through Save/Load, Clone, autosave, and native JSON/ZIP export/import. Turning Autochord off retains the chosen size. Older performances default to off/3. These controls apply to the on-screen keyboard; live gestures are not recorded into sequencers or offline CSD exports.

# Scale/Mode Following Behavior During Live Playback

When the instrument engine is running and a piano roll is enabled, the piano roll can follow the currently running melodic sequencers for harmonic guidance.

## Shared Scale/Mode Case

If running melodic sequencers agree on the same scale+mode:

- the piano roll shows that shared scale/mode
- keys are highlighted according to that theory

## Mixed Scale/Mode Case

If running melodic sequencers disagree:

- piano roll controls show `mixed`
- highlighting is computed from the intersection of the active theories
- only notes common to the active scales/modes are highlighted

This is especially helpful when multiple tracks create potentially conflicting harmonic contexts.

# Keyboard Range and Layout

The piano keyboard spans:

- `C0 .. B7` (96 notes)

UI behavior:

- Full-width horizontal keyboard section
- Scroll arrows appear when the keyboard overflows the viewport
- Smooth horizontal scrolling controls

# Key Highlighting and Degree Labels

Highlighted keys show:

- in-scale status
- scale degree labels ( `1..7` )
- rainbow degree coloring for quick visual recognition

In mixed-theory situations, a key can show combined degree labels (for example `2/5` ) when it belongs to multiple active track theories.

## Playing Notes (Pointer Interaction)

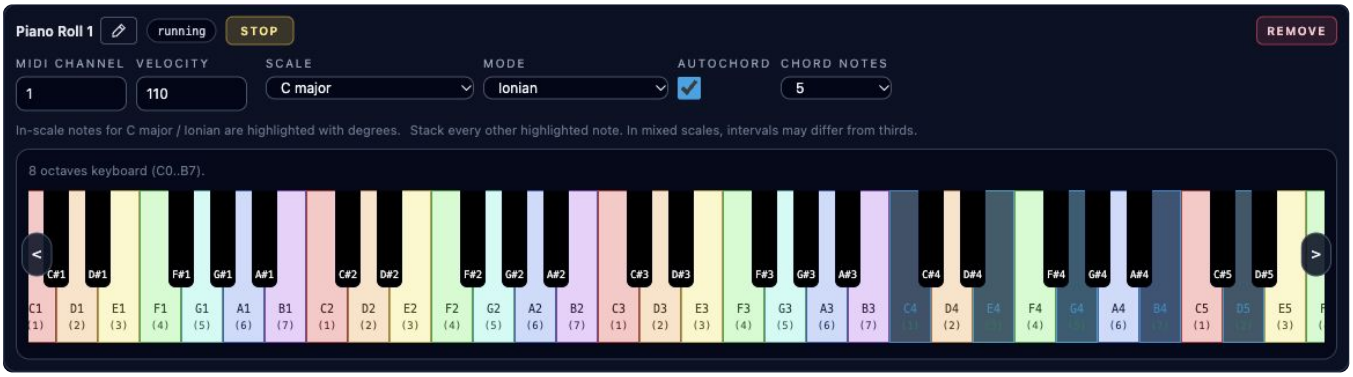
- Press/hold on a key sends `note_on`
- Release sends `note_off`
- Works for both white and black keys

The UI tracks held notes and releases them on pointer cancellation, lost capture, panel collapse, keyboard removal, or when the piano roll or instrument engine stops.

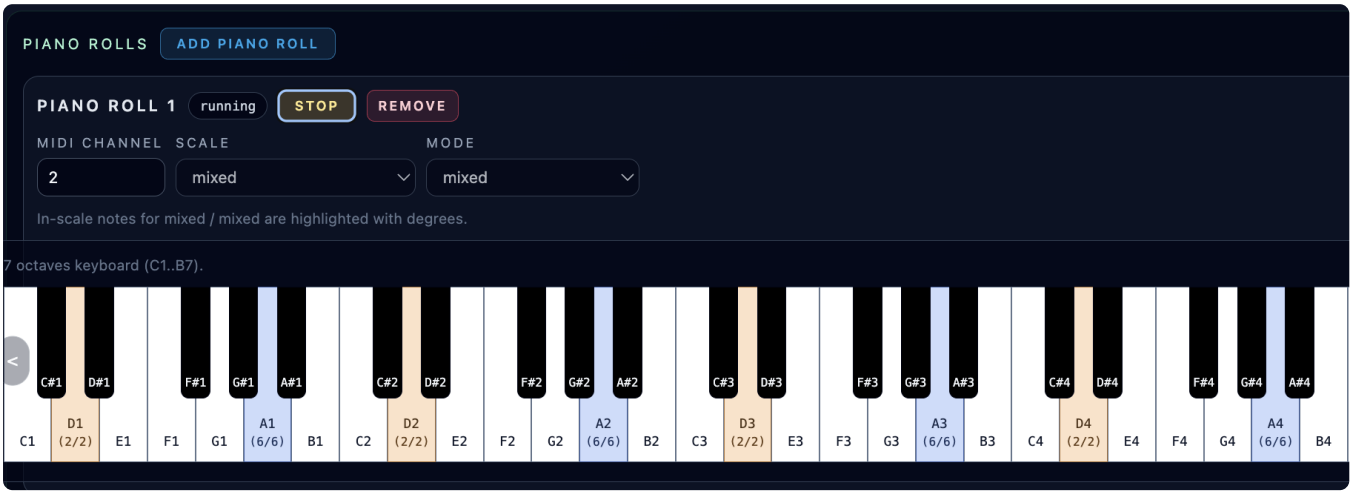
## Practical Use Cases

- Live melody or bass improvisation on top of sequencer patterns
- Testing an instrument patch in musical context without external MIDI hardware
- Performing harmonically guided lines while melodic sequencers define the key center

## Screenshots




*Autochord with five notes selected and the generated chord highlighted.*




*Piano roll with scale-aware key highlighting and mixed-theory display behavior.*

# MIDI Controllers

The MIDI Controllers panel provides manual CC control lanes with interactive knobs.

Use the pen beside the device name to rename it. SeeDevice Names for editing controls, validation, and import/export behavior.

## Collapse the Panel

Collapse the MIDI Controllers header to hide the knobs and their settings. The count, Add button and help remain available; adding a controller expands the group. Panels start expanded and remember your choice across view switches until browser reload. Collapsing does not stop playback or discard edits.

## Overview

This panel is for manual MIDI Control Change performance (hands-on CC control), separate from controller sequencer automation.

## Capacity

- Up to **6** controller lanes can be added

The `Add Controller` button is disabled after 6 lanes are present.

## Per-Controller Lane Controls

Each controller lane includes:

- Controller name label ( `Controller N` if unnamed)
- Running/stopped state badge
- `Controller #` field ( `0..127` )
- **MIDI Channels** checkboxes at the top right: 1–8 above 9–16
- `Start / Stop` enable toggle
- `Remove` button
- Knob control (value `0..127` )
- Numeric value display

## MIDI Channels

Check the channels that should receive this lane's CC messages. All 16 are checked by default (OMNI); at least one must remain checked. The last checked box cannot be unchecked until another channel is selected.

When the lane and instrument session are running, changing channels immediately sends the current value to the new selection. Unchecked channels retain their last received value. Starting a lane or session, changing the CC number, and adjusting the knob all use the selected channels.

The selection survives Save/Load Performance, browser reload, native JSON/ZIP export/import, and both CSD exports. Older performances without a selection load with all 16 channels checked. Controllers using the same CC on different channels can have independent values; overlapping mappings receive the latest transmitted value.

## Knob Interaction

The knob uses pointer drag interaction:

- Click and drag vertically to change the value

- Up increases the CC value
- Down decreases the CC value

The UI also shows a numeric value readout next to the knob for exact control.

## Live Sending Behavior

When a controller lane is enabled and the instrument session is running:

- changing the knob sends MIDI `control_change` messages immediately

This is ideal for expressive live performance adjustments.

## Manual Controllers vs Controller Sequencers

Use manual controller lanes when you want:

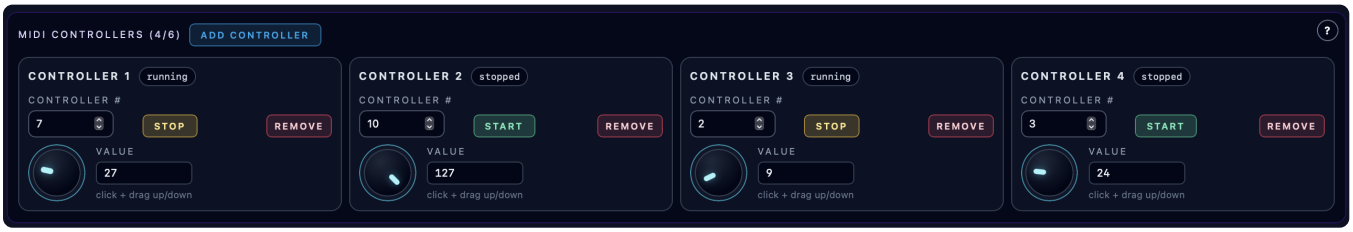
- direct, improvised control
- tactile-feeling UI knobs
- on-the-fly sweeps and tweaks

Use controller sequencers when you want:

- repeatable automation
- transport-synced movement
- curve-based modulation patterns

Both can exist in the same performance.

## Screenshots




*Manual MIDI controller lanes with knobs, CC numbers, and values.*

# Opcode Catalog and Integrated Documentation

The opcode catalog is the source list of all supported building blocks you can place in the graph editor.

## What The Catalog Supports

- Search by opcode name
- Search by category
- Search by description text
- Search by tags
- Click to add an opcode to the graph
- Drag-and-drop an opcode into the graph canvas at a chosen position

## Search Behavior

The search field filters on these opcode fields:

- `name`
- `category`
- `description`
- `tags`

This makes it easy to find opcodes by function (for example `filter`, `midi`, `reverb`, `soundfont`) even if you do not remember the exact Csound opcode name.

## Adding Opcodes

### Click To Add

- Clicking an opcode row inserts the node into the graph using the app's add-node behavior.
- This is fastest when exact placement is not important yet.

### Drag-and-Drop To Place

- Drag an opcode from the catalog and drop it into the graph canvas.
- The graph editor computes the drop location and places the node there.
- While dragging over the graph, the canvas highlights to confirm drop targeting.

## Visual Information In The Catalog

Each catalog entry shows:

- Opcode icon (backend-served icon asset)
- Opcode name
- Category label

These same categories influence node coloring in the graph editor for quick visual grouping.

## Integrated Opcode Documentation (`?` on node)

After placing an opcode in the graph, use the node's `?` button to open the opcode documentation modal.

The opcode documentation modal provides:

- Localized description (English/German/French/Spanish)
- Category and syntax/template summary

- Input/output port descriptions (including optional/default/accepted types)
- Tags
- Direct link to the Csound reference page (`Open Csound Reference`)

## Monophonic Legato

Use `midi_legato` for touching or overlapping notes that share a voice. Its live pitch and velocity outputs drive the sound while `madsr` follows the phrase. Gaps restore articulation. The node is opt-in and belongs in the main graph.

## Atone High-Pass Filters

Search for `atone` or `highpass` in the `filter` category:

- atone filters an audio signal (`asig`) and produces audio (`aout`). Its `khp` cutoff accepts control-rate or init-rate values and starts at 200 Hz.
- atonek filters a control signal (`ksig`) and produces control output (`kout`) for modulation. Its `khp` cutoff starts at 10 Hz. Both inputs accept control-rate or init-rate values.
- atonex cascades multiple `atone` stages for a sharper cutoff. Its audio input/output are `asig` / `aout`; `xhp` accepts audio-rate, control-rate, or init-rate cutoff values and starts at 200 Hz. The optional init-rate `inumlayer` sets the stage count and defaults to 4.

Each filter exposes optional init-rate `iskip`: 0 clears its internal state at initialization, while a nonzero value retains previous state. The default is 0. Setting only `iskip` on `atonex` retains the default four layers. Cutoff defaults are editor starting values; Csound requires a cutoff argument.

Connect the audio outputs of `atone` and `atonex` to downstream audio nodes or both `outs` inputs. Connect `atonek` to a control input such as oscillator amplitude or frequency. The node's `?` help documents every port in English, German, French, and Spanish.

## ZDF and Custom IIR Filters

Search for `zdf` in the `filter` category. All five nodes accept audio input `ain`; `xcf` accepts init, control, or audio rate and starts at 1200 Hz. The two-pole and ladder filters also expose `xq` (0.5–25, default 1) at the same rates.

- `zdf_1pole`: 6 dB/oct, with `kmode` 0 = low-pass, 1 = high-pass, 2 = allpass.
- `zdf_1pole_mode`: simultaneous `alp`, `ahp` outputs, in that order.
- `zdf_2pole`: 12 dB/oct, with `kmode` 0 = low-pass, 1 = high-pass, 2 = band-pass, 3 = unity-gain band-pass, 4 = notch, 5 = allpass, 6 = peak.
- `zdf_2pole_mode`: simultaneous `alp`, `abp`, `ahp` outputs, in that order.
- `zdf_ladder`: four-pole (24 dB/oct) Moog ladder low-pass; Q = 25 reaches self-oscillation.

`kmode` accepts control or init rate and defaults to 0. Every ZDF node has optional init-rate `istor`: 0 clears filter memory, nonzero retains it. Setting `istor` alone preserves other defaults. Cutoff and Q defaults are editor starting values.

`zfilter2` uses custom IIR coefficients with control/init-rate `kdamp` and `kfreq`. Positive damping adjustments lengthen ringing and negative ones shorten it; positive warp shifts pole frequencies upward and negative warp shifts them downward. Both start at 0 (unchanged poles); use values strictly between -1 and 1.

Edit its inline **Coefficients** field as a comma-separated list: all numerator coefficients starting at b0, followed by denominator coefficients starting at a1. The leading denominator coefficient a0 is implicitly 1. **NumeratorCount** (`im`) counts numerator coefficients including b0; **DenominatorCount** (`in`) counts denominator coefficients. Connect `const_i` nodes to change these counts. Defaults are 3 and 2, with `0.06745527, 0.13491055, 0.06745527, -1.1429805, 0.4128016`, a stable second-order low-pass filter.

You can instead connect init-rate sources to **Coefficients** in coefficient order. Each connection supplies one coefficient, replacing the entire inline list; these inputs are not summed, and the list does not accept a combine formula. Connection order survives save/export. Use separate upstream init-rate calculations for individual coefficients.

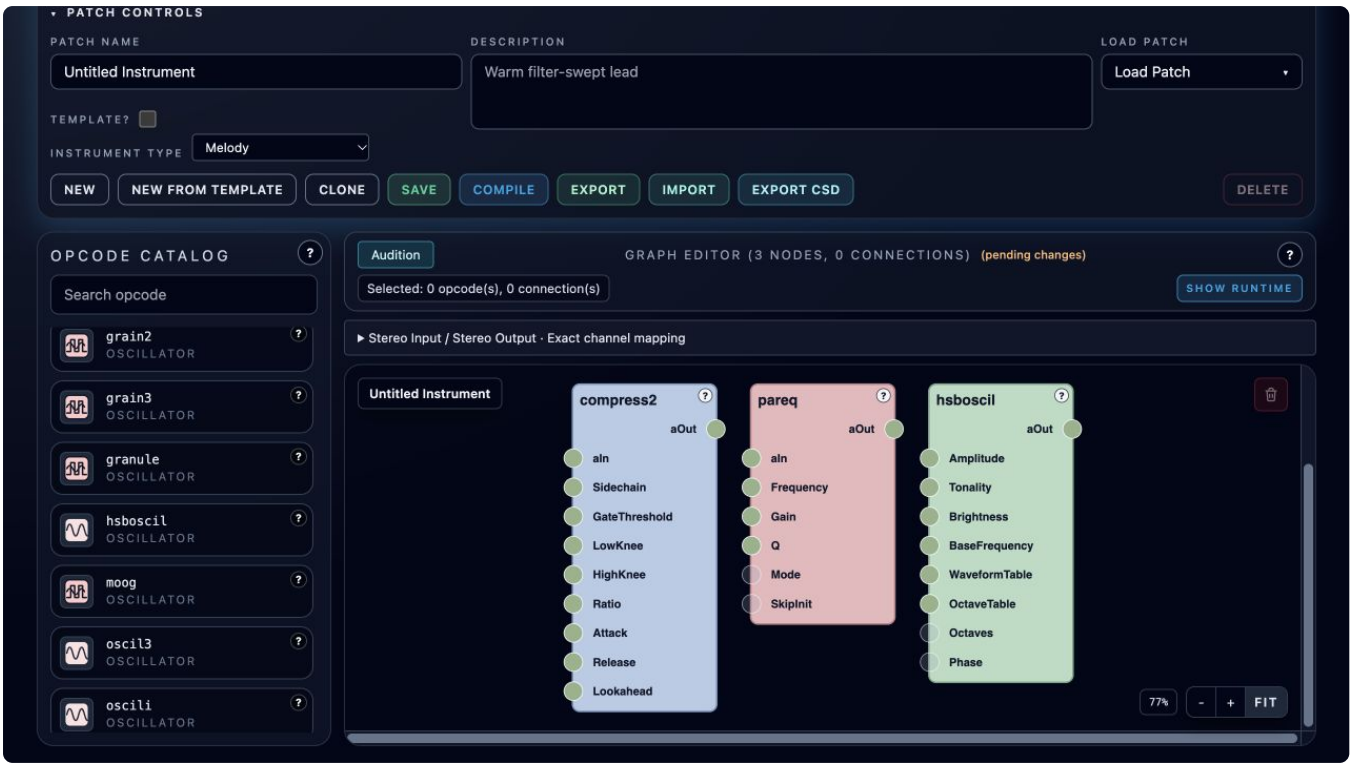
Supply exactly `im + in` coefficients. The editor supports 1–51 numerator coefficients and 1–49 denominator coefficients for Csound 6 compatibility: zero poles and 50 poles encounter native bugs in Csound 6.18. Invalid literal counts fail compilation; invalid connected counts stop the affected voice before the native filter initializes. Coefficients remain the responsibility of the filter design: count checks do not establish stability.

## Compression, Equalization and Cyclic Tonality

- `compress2`, in **dynamics**, processes `aasig` using peak detection on the separate `acsig` sidechain. Connect the same source to both for ordinary compression, or a different source for ducking. Gate and knee levels use dBFS. Editor defaults are -90 dB gate, -52/-30 dB knees, ratio 3, attack 0.01 s, release 0.1 s and lookahead 0.05 s. Lookahead adds audio latency. The detector and ratio controls accept control or init rate; lookahead is init rate.
- `pareq`, in **filter**, offers **Mode** 0 = peaking, 1 = low shelf, 2 = high shelf. **Gain** is linear: 1 is neutral, less than 1 cuts, greater than 1 boosts. Frequency, gain and Q accept control or init rate; their editor defaults are 1000 Hz, 1 and approximately 0.7071. Optional init-rate **SkipInit** defaults to 0; setting it alone retains peaking mode.
- `hsboscil`, in **oscillator**, separates cyclic **Tonality** from **Brightness**, both measured in octaves relative to **BaseFrequency** (initially 220 Hz). Ramp tonality with fixed brightness for a Risset glissando. Amplitude, tonality and brightness accept control or init rate. **Octaves** is an optional init-rate count from 2 to 10, default 3; **Phase** defaults to 0, with -1 skipping initialization.

`hsboscil` needs two function tables. **WaveformTable** defaults to the built-in sine table 1. **OctaveTable** must be supplied: add a **GEN** node with routine **19**, normalization disabled, table size **1024**, and a GEN19 partial row with **Partial number 1**, **Strength 0.5**, **Phase 270°**, and **DC offset 0.5**; connect its `ift` output to **OctaveTable**. Connect a second GEN node to **WaveformTable** for a custom waveform. Table numbers use init-rate ports, matching GEN's `ift` output. They are distinct from the `f` signals used by spectral nodes.

Each node produces mono audio; connect its output to both `outs` inputs for centered stereo. The `?` help describes all ports in English, German, French and Spanish. Prefilled required parameters are editor starting values; optional parameters retain Csound defaults.




## STK Instruments

Search for `stk` to find the 27 Synthesis Toolkit instruments in the `physical_modeling`, `fm`, and `oscillator` categories. They include strings, winds, percussion, voices, organs, and electric pianos. Each node produces one mono audio signal; connect it to both `outs` inputs for centered stereo output.

- `ifrequency` and `iamplitude` are init-rate inputs: frequency in Hz and note amplitude from 0 to 1. Use `cpsmidi` for note pitch; apply envelopes or gain after the audio output for continuous level changes.
- `STKDrummer` uses the frequency to select a drum sample. Connect `cpsmidi` and play MIDI drum notes. `STKDrummer`, `STKPlucked`, and `STKSitar` have no documented controller inputs.
- The other nodes expose the manual's controller number/value pairs in order. The controller number is prefilled with the documented number. Set or connect its paired `kv` value to enable that control. Both ports accept init-rate or control-rate sources, so controller values can change during a note.
- Unset controller values omit the entire pair and retain STK's instrument defaults. You can enable a later pair without filling earlier pairs. For example, set only `kv7` to `3` on `STKBandedWG` to send instrument preset controller `16` with value `3`.
- Most values range from 0 to 127, but preset selectors use model-specific ranges. `STKShakers` uses controller **1071**, not a MIDI CC in the 0–127 range, for its instrument selection. Check the node's `?` help for each controller's meaning.

Runtime requires the Csound STK plugin (`stkopd`) and its rawwave data on the machine running the Csound backend. Set `RAWWAVE_PATH` to the installed rawwaves directory before starting the backend; some Csound builds do not register any STK opcodes without it. For example, an Apple Silicon Homebrew installation of STK uses `RAWWAVE_PATH=/opt/homebrew/opt/stk/share/stk/rawwaves/`. Catalog availability and successful graph compilation do not install this runtime dependency.

## Context Help (`?` in page sections)

In addition to opcode-level docs, the app provides context help buttons for page sections (toolbar, catalog, graph, runtime, sequencer panels, config panels). These open integrated markdown help content in the currently selected GUI language.

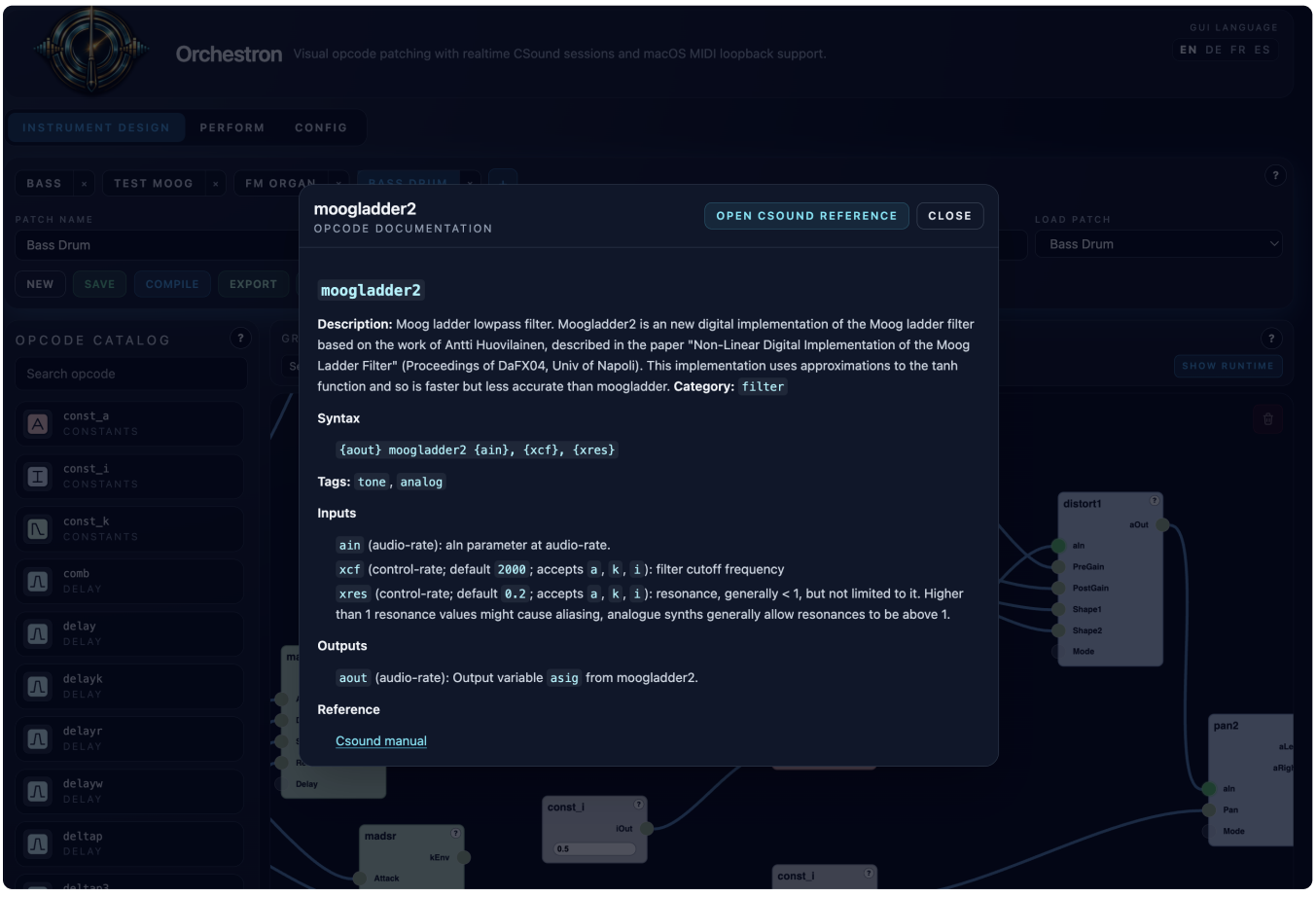
For the opcode catalog specifically, the integrated help now focuses on search/add workflow and on the distinction between catalog help and the node-level opcode documentation modal.

See also:

- GUI Language and Integrated Help
- Supported Opcodes for the full opcode index table

## Screenshots

*Opcode catalog filtered by search text.*




*Integrated opcode documentation modal opened from a graph node.*

# GEN Table Editor (GEN Meta-Opcode)

The `GEN` meta-opcode is a specialized table generator node that opens a dedicated editor for building `ftgen` / `ftgenonce` lines visually.

## Why The GEN Meta-Opcode Exists

Csound function-table generation is powerful but argument-heavy. The GEN editor provides a structured UI for common GEN routines and a preview of the generated `ftgen` / `ftgenonce` line.

## Opening The GEN Editor

- Add the `GEN` opcode to the graph.
- Click the `GEN` button on the node card.

## Editor Overview

The editor has two main areas:

- Left side: generation mode, routine selection, parameters
- Right side: preview ( `Effective GEN` , flattened args, rendered line) and notes

## Table Generation Mode (Top Section)

### Opcode Mode

Choose which Csound table opcode style to generate:

- `ftgen`
- `ftgenonce`

### Routine Selection

You can choose from structured editors for common routines and a named routine option:

- `GEN01` - Audio File
- `GEN02` - Value List
- `GEN07` - Segments
- `GEN08` - Cubic Spline Segments
- `GEN10` - Harmonic Sine Partials
- `GEN11` - Harmonic Cosine Partials
- `GEN17` - Step Table From x/y Pairs
- `GEN19` - Sinusoidal Partials with DC Offset
- `GEN20` - Window / Distribution Function
- `GENpadsynth` (named routine)
- Custom routine number / raw arguments

### Core Table Fields

- `Routine Number`
- `Table Number`
- `Table Size`
- `Start Time` (disabled for `ftgenonce` UI mode)
- `Normalize table` (positive GEN number vs negative GEN number behavior)

The backend validates GEN table settings before storage and again before Csound orchestra emission. `Table Size` can be `0` only for `GEN01`; other routines require a positive size. The maximum table size is `4194304` samples, and raw or structured GEN argument lists are capped at 512 entries. Raw argument text is limited to 8192 characters, with each token limited to 512 characters.

## Supported Routine Editors (Structured UI)

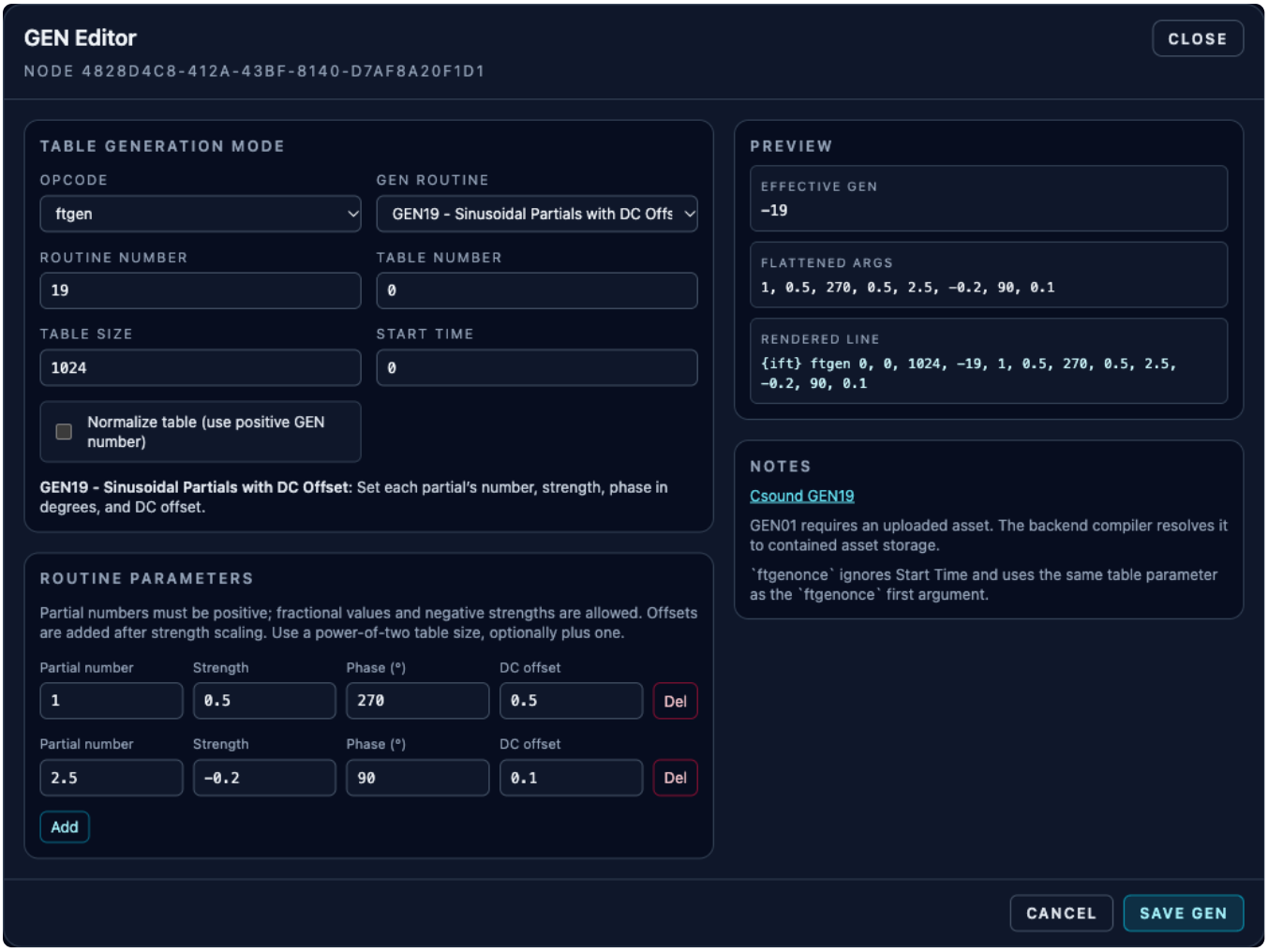
| Routine | Purpose | Main Parameters In Editor |
|---|---|---|
| `GEN10` | Harmonic sine partial amplitudes | Harmonic amplitude list |
| `GEN11` | Harmonic cosine partials | harmonic count, lowest harmonic, multiplier |
| `GEN02` | Literal value list | Numeric value list |
| `GEN07` | Segments | Start value + repeated length/value rows |
| `GEN08` | Cubic spline segments | Start value + repeated length/value rows |
| `GEN17` | Step table from x/y pairs | x/y pairs (editable list) |
| `GEN19` | Sinusoidal partials with independent phase and DC offset | repeated partial number / strength / phase / offset rows |
| `GEN20` | Window / distribution | window type, max, optional parameter (for selected windows) |
| `GEN01` | Load audio file into table | uploaded asset, skip time, format, channel |

## GEN19: Sinusoidal Partials with DC Offset

Choose **GEN19** and add a row for each sinusoid. Each row emits four arguments in order: **Partial number**, **Strength**, **Phase (°)**, **DC offset**. Partial numbers are positive frequency ratios, including fractions; strengths may be negative. The offset is added after strength scaling. Phase uses degrees.

The initial row is `1, 1, 0, 0`. For an `hsboscil` octave window, use `1, 0.5, 270, 0.5`, table size `1024`, and disable normalization. Use a power-of-two table size or that size plus one. Up to 128 partials fit the 512-argument limit. Both `ftgen` and `ftgenonce` support these rows; **Normalize table** selects `19` or `-19`.

Existing GEN19 patches with raw arguments keep their raw editor and values on save/reload. Structured rows take precedence if a configuration also contains old raw arguments. See the Csound GEN19 reference.




## `GENpadsynth` (Named Routine)

Orchestron supports a named GEN routine for padsynth (`GENpadsynth`).

- Select the `GENpadsynth` routine option.
- Enter padsynth parameters and partial amplitude/frequency pairs in the raw arguments editor.
- Argument order follows the Csound `GENpadsynth` documentation.

## Custom / Raw Routine Arguments

For routines without a dedicated structured editor:

- Use the raw arguments editor.
- Separate tokens by commas or new lines.
- Quote string literals manually (for example `"file.wav"`).
- Prefix an argument with `expr:` to force raw Csound expression rendering without quotes.
- Keep raw argument lists within the backend validation limits: 512 tokens total, 512 characters per token, and 8192 characters for the raw text field.

Examples:

```
1, 0.5, expr:1024*2, "file.wav"
```

## GEN01 Audio File Workflow (Important)

The `GEN01` editor loads sound files through uploaded backend assets:

### Uploaded Asset

- Click `Upload Audio File`
- Select an audio file (UI accepts common formats such as WAV/AIFF/FLAC/MP3/OGG and `audio/*`)
- Orchestron stores the uploaded asset on the backend and references it in the GEN node config

Benefits:

- Instrument/performance exports can include the audio file automatically
- Imports can restore the asset automatically when using ZIP bundles
- Runtime compilation resolves the asset inside the configured backend asset directory
- Raw absolute paths and parent-traversal paths are rejected before Csound starts

### Additional GEN01 Parameters

- `Skip Time`
- `Format`
- `Channel`

### Upload Limits and Notes

- Default backend upload limit for GEN audio assets is 64 MiB per file (`VISUALCSOUND_GEN_AUDIO_ASSET_MAX_BYTES`)
- Persistent generated-asset storage is also capped by total bytes (`VISUALCSOUND_GEN_AUDIO_ASSETS_MAX_TOTAL_BYTES`, 1 GiB by default) and asset count (`VISUALCSOUND_GEN_AUDIO_ASSETS_MAX_COUNT`, 1024 by default)
- Garbage collection removes unreferenced generated assets older than `VISUALCSOUND_GEN_AUDIO_ASSET_GC_MIN_AGE_SECONDS` (24 hours by default)
- The backend streams GEN audio uploads and rejects requests once they exceed the configured cap
- ZIP bundle imports also enforce request, member-count, JSON, and total-uncompressed-size caps before importing bundled assets
- Deployments should set a matching maximum request size at the ASGI server, reverse proxy, or load balancer
- The preview/note panel explains that uploaded assets are required for GEN01 sample loading

## Preview Panel

The preview section shows:

- `Effective GEN` (including normalization sign / named routine behavior)
- Flattened argument list
- Rendered `ftgen` / `ftgenonce` line preview

Use this to verify the exact Csound line before compile.

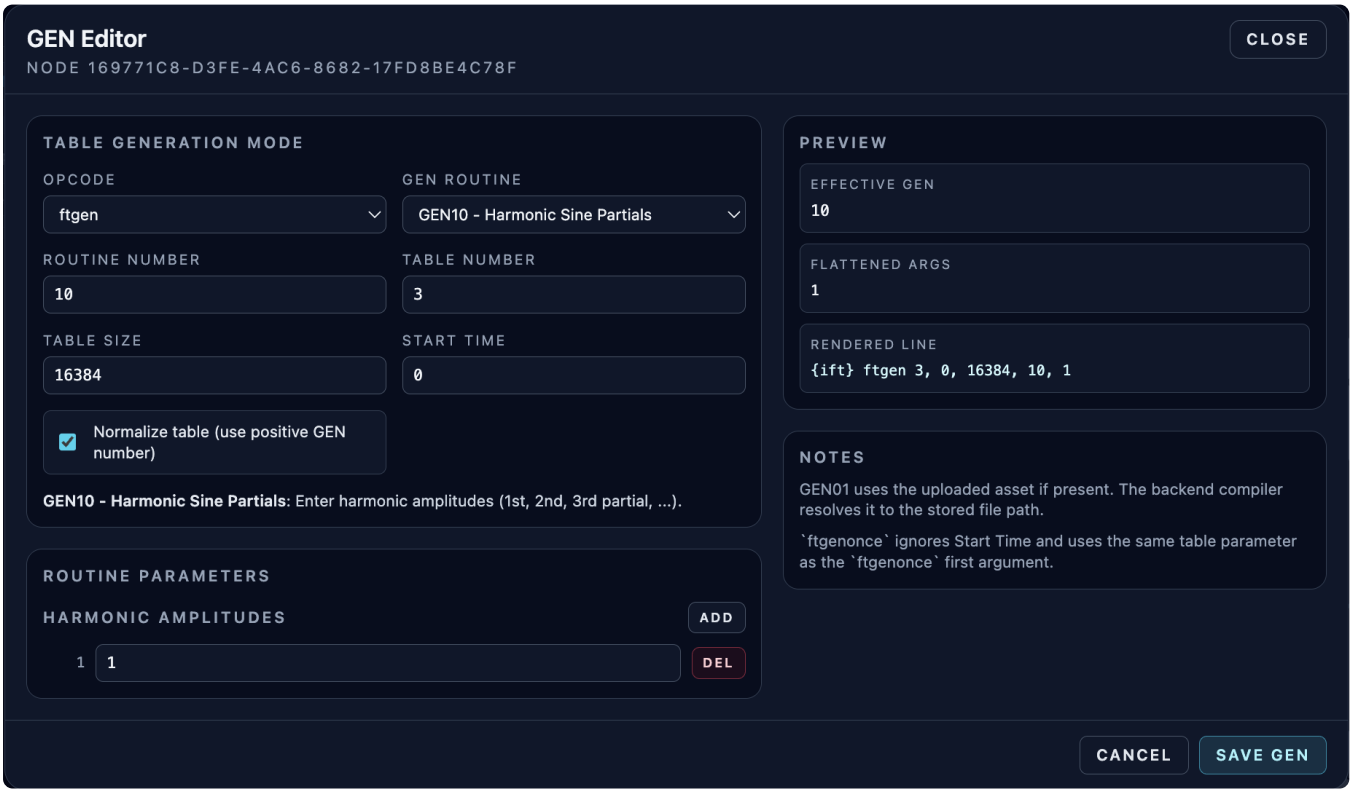
## Editor Notes Worth Knowing

- `ftgenonce` behavior differs from `ftgen` and the editor shows notes about the differences.
- `GEN01` asset-backed generation is resolved by the backend compiler to contained stored asset paths.
- `GEN01` asset dependencies are automatically bundled in ZIP exports when referenced.

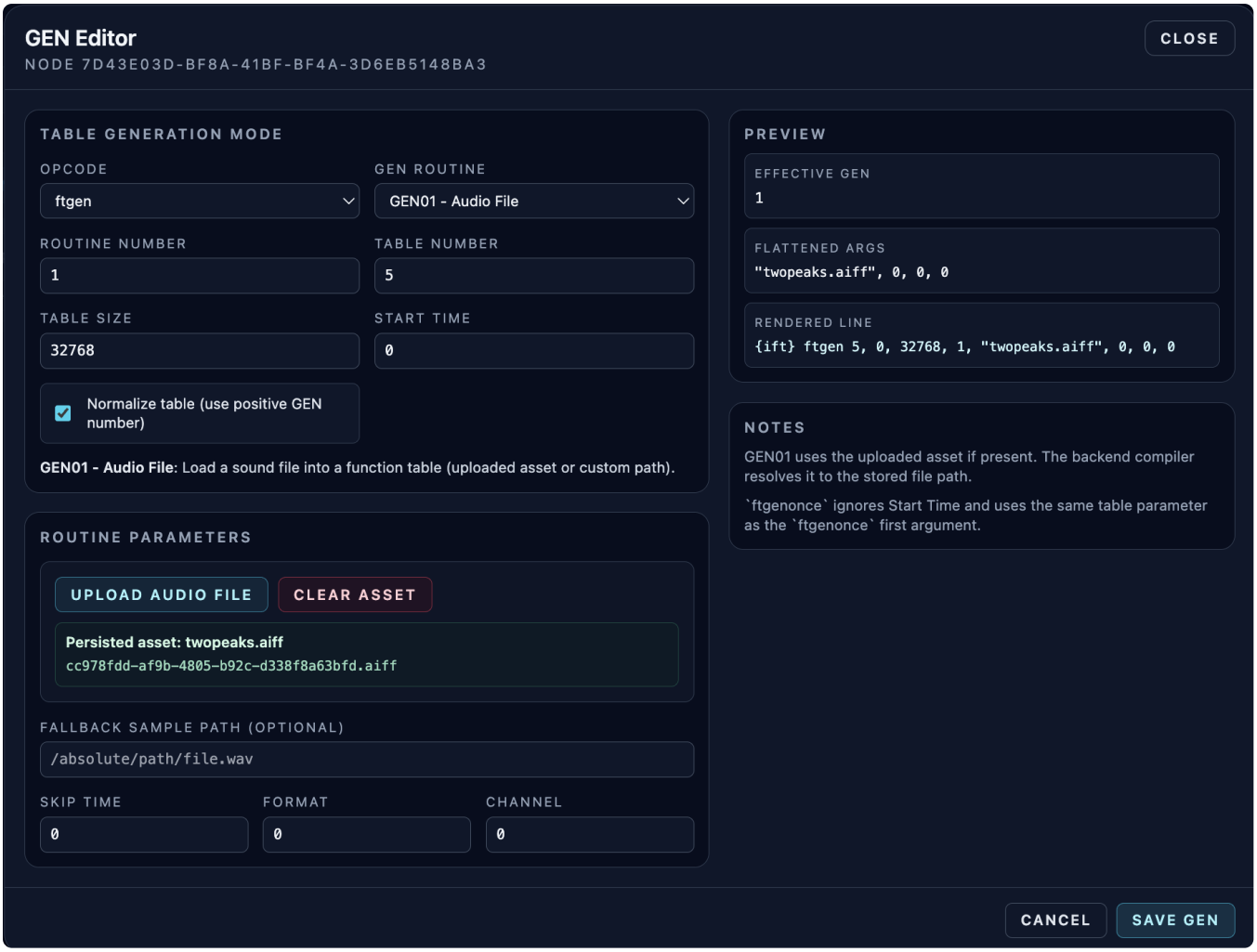
## Save Behavior

- `Save GEN` writes the GEN node configuration into patch UI metadata (`graph.ui_layout`).
- Closing without saving leaves the previous GEN node settings unchanged.

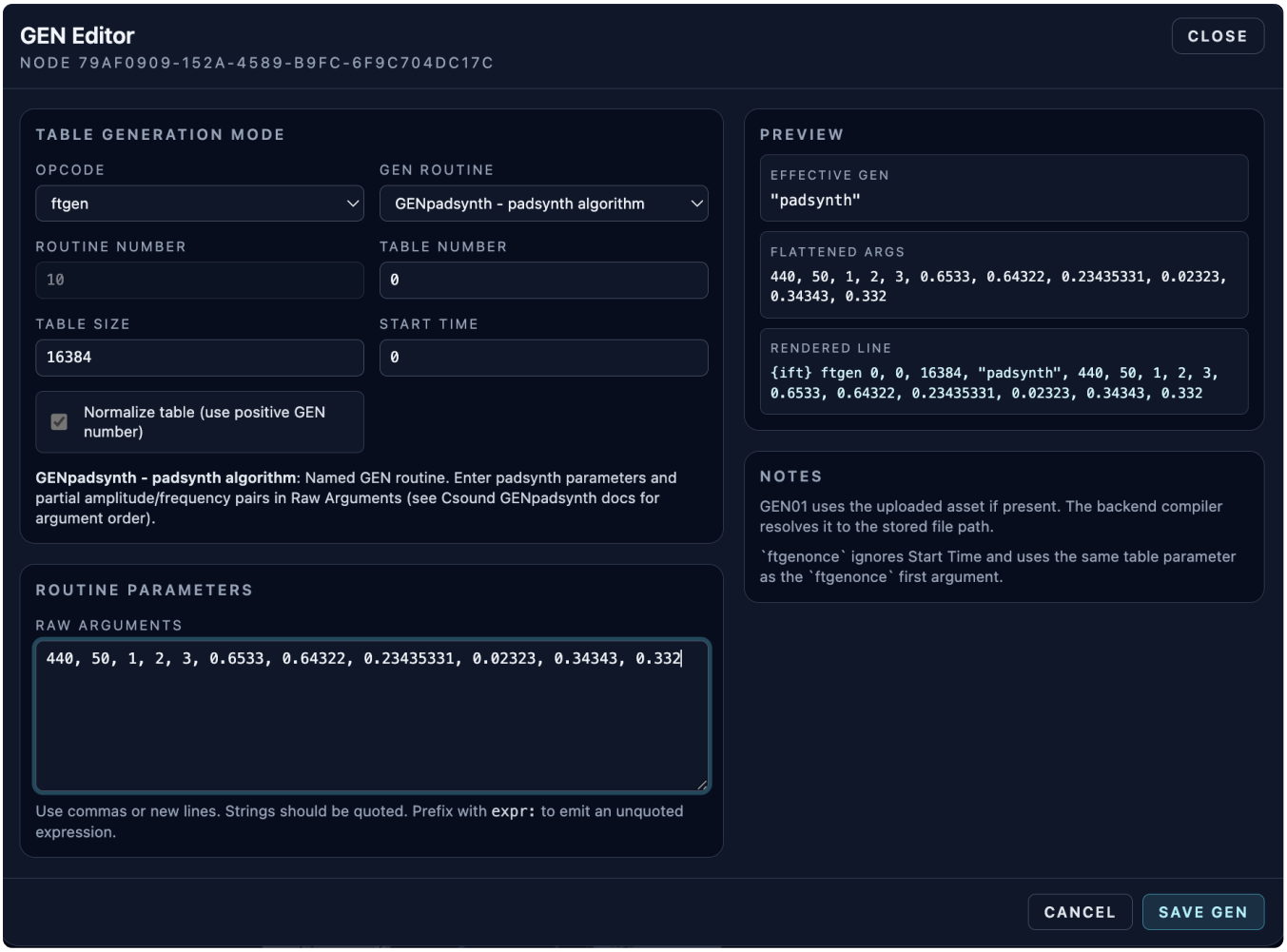
# Screenshots




*GEN editor in structured routine mode with live preview panel.*




*GEN01 table setup using an uploaded backend audio asset.*




*GENpadsynth named routine using raw arguments input.*

# GUI Language and Integrated Help

Orchestron includes multilingual UI labels and integrated documentation so you can stay inside the app while learning and working.

## GUI Language Selector

The language selector is in the top-right area of the main app header.

Supported languages:

- English
- German
- French
- Spanish

The selector is presented as quick language links (short labels) for fast switching.

## What Changes When You Switch Language

Language switching affects user-facing text such as:

- Page labels (`Instrument Design`, `Perform`, `Config`)
- Toolbar/button labels
- Context help pages (integrated help modals)
- Opcode documentation modal content (localized generated markdown)
- Many labels inside advanced editors (including the GEN editor)

## Integrated Help (`?` Buttons)

Orchestron provides context help buttons (`?`) across the UI.

Examples include:

- Instrument patch toolbar
- Opcode catalog
- Graph editor header/area
- Runtime panel
- Sequencer sections (rack, tracks, multitrack arranger, piano rolls, MIDI controllers)
- Config page sections

Clicking a context `?` opens a markdown help modal tailored to that UI area.

On the Instrument Design, Perform, and Config pages, these help modals focus on the current tool, device, transport section, or engine setting panel instead of repeating generic UI conventions.

## Opcode Documentation (`?` On Opcode Node)

Placed opcode nodes provide a node-level `?` button that opens opcode documentation.

The modal includes:

- Localized description
- Inputs and outputs (with detailed port descriptions)
- Syntax/template summary
- Tags

- `Open Csound Reference` link to the official Csound documentation page for that opcode

## Modal Behavior

Both help modals and opcode documentation modals support:

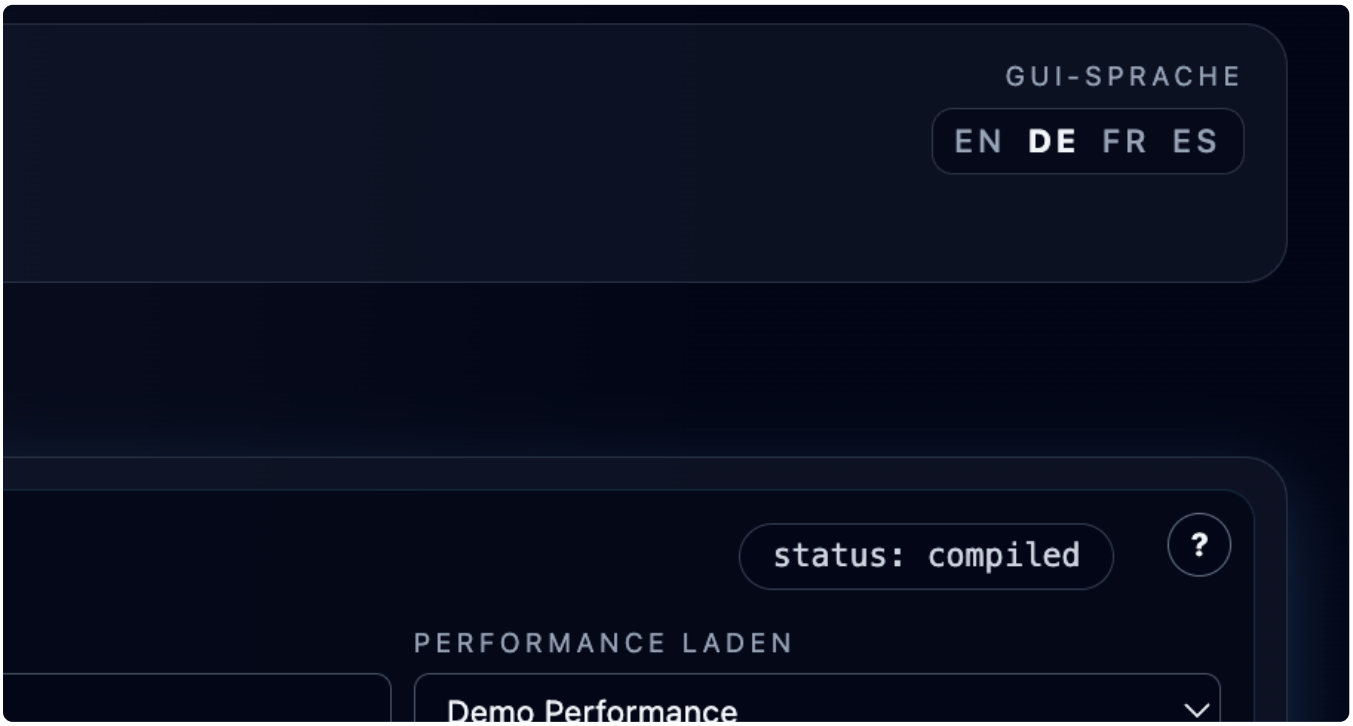
- Close button
- Click outside modal to close
- `Esc` key to close

## Best Practice

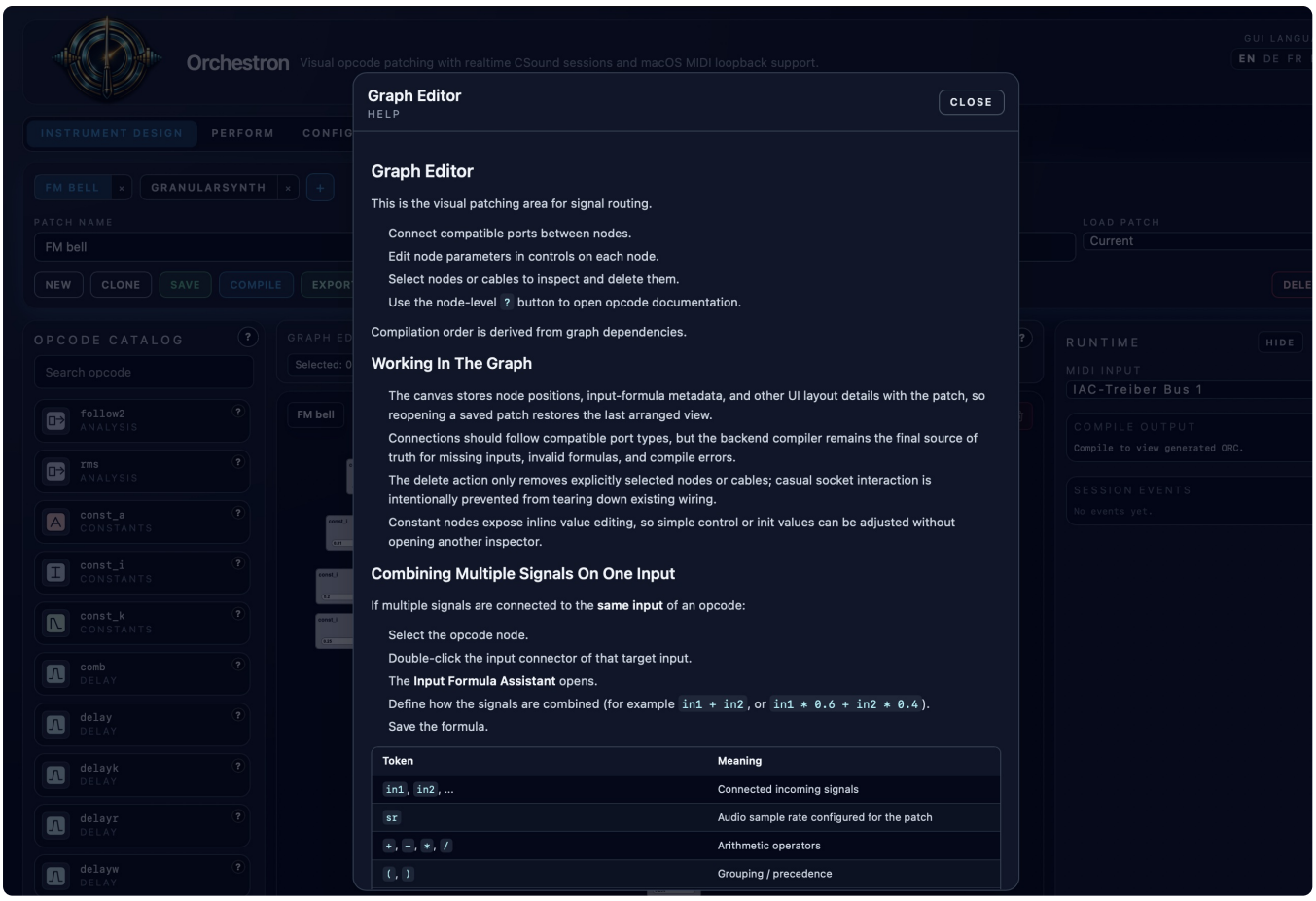
Use context help for workflow guidance and opcode docs for technical/operator details.

This combination is faster than switching repeatedly between the app and external documentation while designing patches.

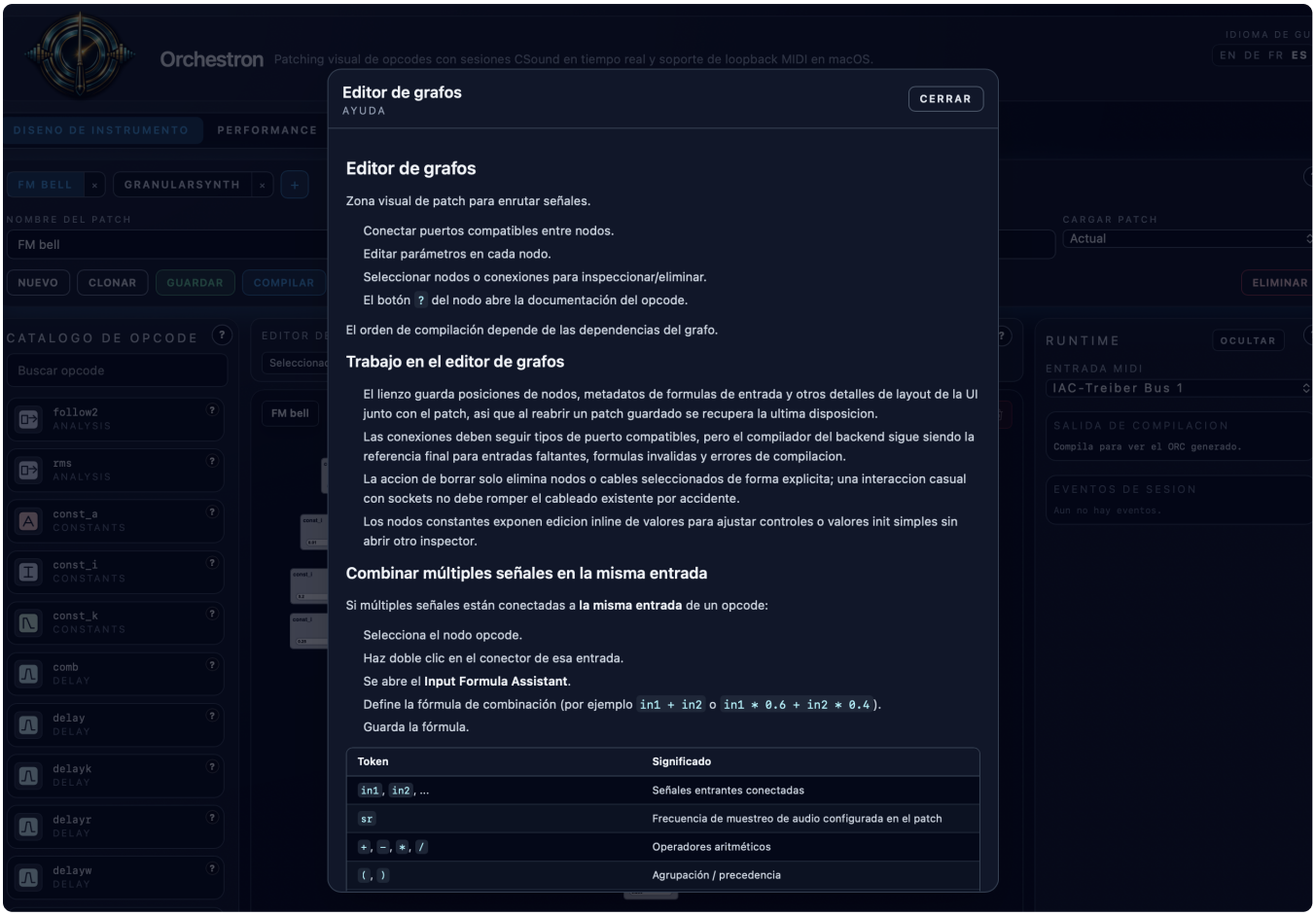
## Screenshots




*Top-right header language selector links.*




*Context help modal integrated into the workflow.*




*Integrated help and UI localization in multiple supported languages.*

# Supported Opcodes

This appendix is generated from `backend/app/data/opcodes.json` and currently lists **193** user-selectable opcodes and editor constructs in the Instrument Design opcode catalog.

## How To Use This Appendix

- Use the table for quick discovery (category, signal I/O, one-line purpose).
- In the app, click the `?` on an opcode node to open the integrated, localized documentation with port-level details and a Csound reference link.
- The `Inputs / Outputs` columns show port count and signal-rate shorthand.
- Signal legend: `a` = audio-rate, `k` = control-rate, `i` = init-rate, `S` = string, `f` = phase-vocoder frame signal.

## Category Counts

| Category | Count |
|---|---|
| analysis | 2 |
| constants | 5 |
| control_flow | 2 |
| delay | 12 |
| distortion | 4 |
| dynamics | 3 |
| envelope | 12 |
| filter | 27 |
| fm | 17 |
| math | 3 |
| midi | 10 |
| mixer | 3 |
| modulation | 10 |
| noise | 3 |
| oscillator | 20 |
| output | 1 |
| physical_modeling | 30 |
| reverb | 4 |
| routing | 4 |
| soundfont | 3 |
| spectral | 9 |
| tables | 4 |
| utility | 5 |

## Opcode Index

### analysis

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| follow2 | 3 (a, k, k) | 1 (a) | Envelope follower with independent attack and release controls. |
| rms | 3 (a, i, i) | 1 (k) | RMS envelope follower for an audio signal. |

## constants

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| const_a | - | 1 (a) | Audio-rate constant value. |
| const_i | - | 1 (i) | Init-rate constant value. |
| const_k | - | 1 (k) | Control-rate constant value. |
| const_s | - | 1 (S) | String constant value. |
| perf_controller | - | 1 (i) | Virtual per-instance performance setting with a labeled rack knob. |

## control_flow — editor constructs

These nodes generate structured Csound conditionals; they are not ordinary opcode calls. If, Switch and Drumset documents note-start selection and the managed Case Result boundary.

| Construct | Inputs | Outputs | Short Description |
|---|---|---|---|
| If | 2 (i, i) | 1 or 2 (a) | Select True/False by an init-rate comparison for each voice. |
| Switch | 1 (i) | 1 or 2 (a) | Select a named numeric case or mandatory Default for each voice. |

## delay

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| comb | 4 (a, k, i, i) | 1 (a) | Comb filter / feedback delay. |
| delay | 3 (a, i, i) | 1 (a) | Simple non-interpolating audio delay line. |
| delayk | 3 (k, i, i) | 1 (k) | Control-rate delay line. |
| delayr | 2 (i, i) | 1 (a) | Read tap from a classic delay-line memory buffer. |
| delayw | 1 (a) | - | Write into the delay-line memory buffer. |
| deltap | 1 (k) | 1 (a) | Read a delay tap using linear interpolation. |
| deltap3 | 1 (k) | 1 (a) | Read a delay tap using cubic interpolation. |
| flanger | 4 (a, k, k, i) | 1 (a) | Flanger effect with delay modulation and feedback. |
| stereo_capture | 13 (a, a, k, k, k, k, i, i, i, i, i, i, i) | 2 (a, a) | Orchestron stereo stutter, freeze and granular processor. |
| vcomb | 6 (a, k, k, i, i, i) | 1 (a) | Variable-time comb reverb with controllable loop time. |

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| vdelay3 | 4 (a, k, i, i) | 1 (a) | Variable delay line with cubic interpolation. |
| vdelayxs | 6 (a, a, a, i, i, i) | 2 (a, a) | Stereo variable delay with high-quality sinc interpolation. |

## distortion

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| clip | 4 (a, i, i, i) | 1 (a) | Signal clipper with selectable transfer curves. |
| distort1 | 6 (a, k, k, k, k, i) | 1 (a) | Waveshaping distortion with configurable transfer curve. |
| fold | 2 (a, k) | 1 (a) | Artificial foldover effect for audio signals. |
| powershape | 3 (a, k, i) | 1 (a) | Power-law waveshaper for controllable nonlinear distortion. |

## dynamics

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| compress2 | 9 (a, a, k, k, k, k, k, k, i) | 1 (a) | Compressor, limiter, expander and gate with an audio sidechain and dBFS thresholds. |
| dam | 6 (a, k, i, i, i, i) | 1 (a) | Dynamic amplitude processor (downward compressor/noise suppressor). |
| limit | 3 (a, k, k) | 1 (a) | Hard clamp limiter. |

## envelope

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| adsr | 4 (i, i, i, i) | 1 (k) | Control-rate ADSR envelope. |
| envlpxr | 8 (k, i, i, i, i, i, i, i) | 1 (k) | Control-rate envelope with optional release scaling controls. |
| expon | 3 (i, i, i) | 1 (k) | Control-rate exponential segment between two points. |
| expseg | 9 (i, i, i, i, i, i, i, i, i) | 1 (k) | Control-rate exponential breakpoint envelope generator. |
| expsega | 9 (i, i, i, i, i, i, i, i, i) | 1 (a) | Audio-rate exponential breakpoint envelope generator. |
| expsegr | 5 (i, i, i, i, i) | 1 (k) | Control-rate exponential envelope with release segment. |
| linenr | 4 (k, i, i, i) | 1 (k) | Control-rate envelope with note-release stage. |
| linseg | 7 (i, i, i, i, i, i, i) | 1 (k) | Control-rate linear breakpoint envelope generator. |
| linsegr | 5 (i, i, i, i, i) | 1 (k) | Control-rate linear breakpoint envelope with release segment. |
| madsr | 6 (i, i, i, i, i, i) | 1 (k) | MIDI release-sensitive ADSR envelope. |

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| mxadsr | 6 (i, i, i, i, i, i) | 1 (k) | Extended MIDI release-sensitive ADSR envelope. |
| transegr | 7 (i, i, i, i, i, i, i) | 1 (k) | Control-rate transition envelope with release-triggered final segment. |

## filter

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| atone | 3 (a, k, i) | 1 (a) | Audio-rate high-pass filter complementary to tone. |
| atonek | 3 (k, k, i) | 1 (k) | Control-rate high-pass filter complementary to tonek. |
| atonex | 4 (a, k, i, i) | 1 (a) | Cascaded atone high-pass filters with a sharper cutoff. |
| butterbp | 4 (a, k, k, i) | 1 (a) | Second-order Butterworth band-pass filter. |
| butterbr | 4 (a, k, k, i) | 1 (a) | Second-order Butterworth band-reject filter. |
| butterhp | 3 (a, k, i) | 1 (a) | Second-order Butterworth high-pass filter. |
| butterlp | 3 (a, k, i) | 1 (a) | Second-order Butterworth low-pass filter. |
| diode_ladder | 6 (a, k, k, i, i, i) | 1 (a) | Diode ladder low-pass filter model. |
| exciter | 5 (a, k, k, k, k) | 1 (a) | Harmonic exciter that adds controlled upper partials. |
| fofilter | 5 (a, k, k, k, i) | 1 (a) | Formant filter. |
| mode | 4 (a, k, k, i) | 1 (a) | Mass-spring-damper resonator for acoustic body modes. |
| moogladder | 3 (a, k, k) | 1 (a) | Moog ladder low-pass filter. |
| moogladder2 | 3 (a, k, k) | 1 (a) | Nonlinear Moog-style ladder filter with audio-rate modulation support. |
| moogvcf | 5 (a, k, k, i, i) | 1 (a) | Moog ladder voltage-controlled filter emulation. |
| pareq | 6 (a, k, k, k, i, i) | 1 (a) | Parametric equalizer with peaking, low-shelf and high-shelf modes. |
| resonx | 6 (a, k, k, i, i, i) | 1 (a) | Multi-layer resonant band-pass filter. |
| rezzy | 5 (a, k, k, i, i) | 1 (a) | Resonant low-pass or high-pass filter. |
| skf | 5 (a, k, k, i, i) | 1 (a) | Sallen-Key low-pass or high-pass filter. |
| statevar | 5 (a, k, k, i, i) | 4 (a, a, a, a) | State-variable filter with simultaneous HP/LP/BP/BR outputs. |

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| tbvcf | 6 (a, k, k, k, k, i) | 1 (a) | TB-303 style voltage-controlled filter model. |
| vclpf | 4 (a, k, k, i) | 1 (a) | Virtual-analog low-pass filter. |
| zdf_1pole | 4 (a, k, k, i) | 1 (a) | One-pole zero-delay feedback filter with selectable low-pass, high-pass, or allpass output. |
| zdf_1pole_mode | 3 (a, k, i) | 2 (a, a) | One-pole zero-delay feedback filter with simultaneous low-pass and high-pass outputs. |
| zdf_2pole | 5 (a, k, k, k, i) | 1 (a) | Two-pole zero-delay feedback filter with seven selectable response modes. |
| zdf_2pole_mode | 4 (a, k, k, i) | 3 (a, a, a) | Two-pole zero-delay feedback filter with simultaneous low-pass, band-pass, and high-pass outputs. |
| zdf_ladder | 4 (a, k, k, i) | 1 (a) | Four-pole zero-delay feedback Moog ladder low-pass filter. |
| zfilter2 | 6 (a, k, k, i, i, i) | 1 (a) | Custom IIR filter with control-rate pole damping and frequency warping. |

## fm

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| STKBeeThree | 12 (i, i, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK four-operator FM organ inspired by Hammond organs. |
| STKFMVoices | 12 (i, i, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK FM singing-voice instrument with vowel and spectral-tilt controls. |
| STKHevyMetl | 12 (i, i, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK four-operator FM instrument for metallic timbres. |
| STKPercFlut | 12 (i, i, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK four-operator FM percussive flute instrument. |
| STKRhodey | 12 (i, i, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK four-operator FM electric piano inspired by Rhodes pianos. |
| STKTubeBell | 12 (i, i, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK four-operator FM tubular-bell instrument. |
| STKWurley | 12 (i, i, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK four-operator FM electric piano inspired by Wurlitzer pianos. |
| crossfmi | 9 (k, k, k, k, k, i, i, i, i) | 2 (a, a) | Interpolating crossed frequency-modulation oscillator pair. |
| crossfmpmi | 9 (k, k, k, k, k, i, i, i, i) | 2 (a, a) | Interpolating crossed frequency/phase-modulation oscillator pair. |
| crosspmi | 9 (k, k, k, k, k, i, i, i, i) | 2 (a, a) | Interpolating crossed phase-modulation oscillator pair. |
| fmb3 | 10 (k, k, k, k, k, i, i, i, i, i) | 1 (a) | B3 organ FM model. |
| fmbell | 12 (k, k, k, k, k, k, i, i, i, i, i, i) | 1 (a) | Bell FM model. |

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| fmmetal | 11 (k, k, k, k, k, k, i, i, i, i, i) | 1 (a) | Metallic FM model. |
| fmpercfl | 11 (k, k, k, k, k, k, i, i, i, i, i) | 1 (a) | Percussive flute FM model. |
| fmrhode | 11 (k, k, k, k, k, k, i, i, i, i, i) | 1 (a) | Rhodes electric piano FM model. |
| fmvoice | 11 (k, k, k, k, k, k, i, i, i, i, i) | 1 (a) | Voice-like FM model. |
| fmwurlie | 11 (k, k, k, k, k, k, i, i, i, i, i) | 1 (a) | Wurlitzer electric piano FM model. |

## math

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| a_mul | 2 (a, a) | 1 (a) | Multiply two audio signals. |
| k_mul | 2 (k, k) | 1 (k) | Multiply two control signals. |
| tanh | 1 (a) | 1 (a) | Hyperbolic tangent function for audio waveshaping and limiting. |

## midi

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| ctrl7 | 5 (i, i, k, k, i) | 1 (k) | Explicit-channel MIDI CC reader, including continuous instruments. |
| initc7 | 3 (i, i, i) | - | Initialize a shared MIDI CC once at instrument startup with a normalized value. |
| ampmidi | 2 (i, i) | 1 (i) | Generate init-rate amplitude from MIDI note velocity. |
| ampmidicurve | 3 (k, k, k) | 1 (k) | Map MIDI velocity to gain using dynamic range and curve exponent. |
| ampmidid | 2 (k, i) | 1 (k) | Map MIDI velocity to amplitude using a decibel range. |
| cpsmidi | - | 1 (i) | Read active MIDI note pitch as cycles-per-second. |
| midi_legato | - | 2 (k, k) | Opt-in monophonic phrases, immediate pitch and smoothed velocity. |
| midi_note | 1 (i) | 2 (k, k) | Extract MIDI note frequency and velocity amplitude. |
| midictrl | 3 (i, i, i) | 1 (k) | Read a MIDI controller value with optional scaling. |
| notnum | - | 1 (i) | Read the MIDI note number for the current note event. |

## mixer

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| mix2 | 2 (a, a) | 1 (a) | Mix two audio signals. |
| ntrpol | 3 (a, a, k) | 1 (a) | Linear crossfade between two audio signals. |

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| pan2 | 3 (a, k, i) | 2 (a, a) | Stereo panner. |

## modulation

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| jitter | 3 (k, k, k) | 1 (k) | Control-rate segmented random line generator. |
| lfo | 3 (k, k, i) | 1 (k) | Low-frequency oscillator for control-rate modulation. |
| portk | 3 (k, k, i) | 1 (k) | Control-rate portamento/slew limiter with optional init value. |
| random | 2 (k, k) | 1 (k) | Control-rate random value generator between a minimum and maximum. |
| randomh | 5 (k, k, k, i, i) | 1 (k) | Control-rate random sample-and-hold generator with update frequency control. |
| randomi | 5 (k, k, k, i, i) | 1 (k) | Control-rate interpolated random generator with update frequency control. |
| release | - | 1 (k) | Return 1 during note release phase and 0 otherwise. |
| samphold | 4 (k, k, i, i) | 1 (k) | Control-rate sample-and-hold processor. |
| vibr | 4 (k, k, i, i) | 1 (k) | Simple vibrato control oscillator with table lookup. |
| vibrato | 9 (k, k, k, k, k, k, k, k, i) | 1 (k) | Randomized vibrato generator. |

## noise

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| noise | 4 (k, k, i, i) | 1 (a) | Variable-color random audio noise. |
| pinker | - | 1 (a) | Pink noise generator. |
| pinkish | 2 (a, i) | 1 (a) | Pinkening filter for shaping white noise toward a pink spectrum. |

## oscillator

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| STKMoog | 12 (i, i, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK sampled-wave synthesizer with swept formant filters. |
| STKSimple | 10 (i, i, k, k, k, k, k, k, k, k) | 1 (a) | STK wavetable and noise synthesizer with filters and an ADSR envelope. |
| fof | 15 (k, k, k, k, k, k, k, k, i, i, i, i, i, i, i) | 1 (a) | Formant/granular source using sinusoid bursts. |
| fof2 | 15 (k, k, k, k, k, k, k, k, i, i, i, i, k, k, i) | 1 (a) | FOF source with per-grain phase indexing and glissando. |

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| foscili | 7 (k, k, k, k, k, i, i) | 1 (a) | Audio-rate FM oscillator with harmonic ratios. |
| gbuzz | 7 (k, k, k, k, k, i, i) | 1 (a) | Generalized buzz oscillator with controllable harmonics. |
| grain | 10 (k, k, k, k, k, k, i, i, i, i) | 1 (a) | Classic granular synthesis oscillator with table-based grains. |
| grain2 | 9 (k, k, k, i, k, i, i, i, i) | 1 (a) | Easy-to-use granular synthesis texture generator. |
| grain3 | 13 (k, k, k, k, k, k, i, k, i, k, k, i, i) | 1 (a) | Granular oscillator with independent frequency and phase modulation controls. |
| granule | 22 (k, i, i, i, i, i, i, i, i, i, k, i, k, i, i, i, i, i, i, i, i, i) | 1 (a) | Multi-voice granular processor with independent gap and grain-size controls. |
| hsboscil | 8 (k, k, k, i, i, i, i, i) | 1 (a) | Oscillator with independent cyclic tonality and octave brightness controls. |
| moog | 9 (k, k, k, k, k, k, i, i, i) | 1 (a) | Mini-Moog style synthesizer model source. |
| oscil3 | 4 (k, k, i, i) | 1 (a) | Cubic-interpolating oscillator with low distortion. |
| oscili | 3 (k, k, i) | 1 (a) | Classic interpolating oscillator. |
| poscil3 | 4 (k, k, i, i) | 1 (a) | High-precision cubic interpolating oscillator. |
| syncphasor | 3 (k, a, i) | 2 (a, a) | Audio-rate normalized phase generator with sync trigger I/O. |
| vco | 5 (k, k, i, k, i) | 1 (a) | Band-limited voltage-controlled oscillator. |
| vco2 | 6 (k, k, i, k, k, i) | 1 (a) | Improved anti-aliased analog-style oscillator. |
| voice | 8 (k, k, k, k, k, k, i, i) | 1 (a) | Formant-based vocal source synthesizer. |
| vosim | 8 (k, k, k, k, k, k, i, i) | 1 (a) | Vocal formant synthesis using overlapping sinusoidal bursts. |

## output

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| outs | 2 (a, a) | - | Stereo output sink. |

## physical_modeling

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| STKBandedWG | 16 (i, i, k, k, k, k, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK banded waveguide model of bowed bars, glasses, and bowls. |
| STKBlowBotl | 10 (i, i, k, k, k, k, k, k, k, k) | 1 (a) | STK blown-bottle model with a Helmholtz resonator and jet excitation. |
| STKBlowHole | 12 (i, i, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK clarinet model with register and tone holes. |

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| STKBowed | 12 (i, i, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK bowed-string waveguide instrument. |
| STKBrass | 12 (i, i, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK brass waveguide instrument with lip and slide controls. |
| STKClarinet | 12 (i, i, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK single-reed clarinet waveguide instrument. |
| STKDrummer | 2 (i, i) | 1 (a) | STK drum synthesizer using sampled raw waves and one-pole filters. |
| STKFlute | 12 (i, i, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK flute waveguide instrument with jet and breath controls. |
| STKMandolin | 12 (i, i, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK mandolin model with paired strings and sampled body responses. |
| STKModalBar | 16 (i, i, k, k, k, k, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK modal percussion instrument with bar and bell presets. |
| STKPlucked | 2 (i, i) | 1 (a) | STK Karplus-Strong plucked-string instrument. |
| STKResonate | 12 (i, i, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK noise-driven resonator with pole and zero controls. |
| STKSaxofony | 16 (i, i, k, k, k, k, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK saxophone-like reed waveguide instrument. |
| STKShakers | 14 (i, i, k, k, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK stochastic percussion and environmental-sound models. |
| STKSitar | 2 (i, i) | 1 (a) | STK plucked-string sitar model. |
| STKStifKarp | 8 (i, i, k, k, k, k, k, k) | 1 (a) | STK stiff-string model with pickup, sustain, and stretch controls. |
| STKVoicForm | 12 (i, i, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK four-formant voice synthesizer with voiced and unvoiced excitation. |
| STKWhistle | 12 (i, i, k, k, k, k, k, k, k, k, k, k) | 1 (a) | STK whistle model with a pea and fipple modulation. |
| dripwater | 8 (k, i, i, i, i, i, i, i) | 1 (a) | Stochastic dripping-water physical model source. |
| marimba | 11 (k, k, i, i, i, k, k, i, i, i, i) | 1 (a) | Physical model of a marimba bar and resonator. |
| pluck | 6 (k, k, i, i, i, i) | 1 (a) | Karplus-Strong plucked-string model. |
| sekere | 5 (i, i, i, i, i) | 1 (a) | PhISEM sekere shaker model. |
| sleighbells | 8 (k, i, i, i, i, i, i, i) | 1 (a) | PhISEM sleigh bells model. |
| wgbow | 8 (k, k, k, k, k, k, i, i) | 1 (a) | Waveguide bowed-string model. |
| wgbowedbar | 9 (k, k, k, k, k, i, i, i, i) | 1 (a) | Waveguide bowed-bar model. |
| wgclar | 10 (k, k, k, i, i, k, k, k, i, i) | 1 (a) | Waveguide clarinet model. |
| wgflute | 10 (k, k, k, i, i, k, k, k, i, i) | 1 (a) | Waveguide flute model. |

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| waveguide_string | 10 (a, i, i, i, i, i, i, i, i, k) | 2 (a, a) | Virtual dual-polarization string with local sample feedback and compensated dispersion. |
| wgpluck2 | 5 (i, k, i, k, k) | 1 (a) | Waveguide plucked string model with pick and reflection controls. |
| wguide2 | 4 (a, k, k, k) | 1 (a) | Two-point waveguide resonator. |

## reverb

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| freeverb | 6 (a, a, k, k, i, i) | 2 (a, a) | Stereo Freeverb processor with room size and high-frequency damping controls. |
| platerev | 9 (i, i, k, i, i, i, i, a, a) | 2 (a, a) | Physical-model plate reverb with stereo output taps. |
| reverb2 | 4 (a, k, k, i) | 1 (a) | Schroeder reverb processor. |
| reverbsc | 7 (a, a, k, k, i, i, i) | 2 (a, a) | 8-delay-line stereo FDN reverb based on Sean Costello's design. |

## routing

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| inleta | 1 (S) | 1 (a) | Receive an audio-rate signal from a named instrument inlet port. |
| inletk | 1 (S) | 1 (k) | Receive a control-rate signal from a named instrument inlet port. |
| outleta | 2 (S, a) | - | Send an audio-rate signal to a named instrument outlet port. |
| outletk | 2 (S, k) | - | Send a control-rate signal to a named instrument outlet port. |

## soundfont

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| sfinstr3 | 8 (i, i, k, k, i, i, i, i) | 2 (a, a) | Play a SoundFont instrument as stereo audio with cubic interpolation. |
| sfload | - | 1 (i) | Load a SoundFont2 file and return a file handle. |
| sfplay3 | 8 (i, i, k, k, i, i, i, i) | 2 (a, a) | Play a SoundFont preset as stereo audio with cubic interpolation. |

## spectral

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| pvsanal | 7 (a, i, i, i, i, i, i) | 1 (f) | Phase-vocoder analysis from an audio input to an fsig stream. |
| pvscale | 5 (f, k, k, k, k) | 1 (f) | Transpose spectral audio by multiplying its frequencies by a ratio. |

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| pvshift | 6 (f, k, k, k, k, k) | 1 (f) | Shift fsig partial frequencies by a fixed amount in Hz. |
| pvsmooth | 3 (f, k, k) | 1 (f) | Smooth fsig amplitude and frequency trajectories with lowpass filtering. |
| pvsmorph | 4 (f, f, k, k) | 1 (f) | Morph between two fsig streams by amplitude and frequency interpolation. |
| pvsosc | 8 (k, k, k, i, i, i, i, i) | 1 (f) | Generate oscillator spectra directly as an fsig stream. |
| pvsvoc | 5 (f, f, k, k, k) | 1 (f) | Cross-synthesize fsig amplitudes and excitation frequencies. |
| pvswarp | 7 (f, k, k, k, k, k, k) | 1 (f) | Warp and shift the spectral envelope of an fsig stream. |
| pvsynth | 2 (f, i) | 1 (a) | Resynthesize audio from an fsig stream via overlap-add. |

## tables

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| GEN | - | 1 (i) | Routine-aware function table generator (meta-opcode) that renders ftgen/ftgenonce from a specialized editor, including GEN19 sinusoidal partials with phase and DC offset. |
| ftgen | 12 (i, i, i, i, i, i, i, i, i, i, i, i) | 1 (i) | Create a function table at init time using a GEN routine. |
| ftgenonce | 12 (i, i, i, i, i, i, i, i, i, i, i, i) | 1 (i) | Generate a function table once and reuse it across instances. |
| vco2init | 6 (i, i, i, i, i, i) | 1 (i) | Precompute band-limited wavetable sets for vco2 oscillators. |

## utility

| Opcode | Inputs | Outputs | Short Description |
|---|---|---|---|
| downsamp | 2 (a, i) | 1 (k) | Convert an audio signal to control-rate by downsampling. |
| k_to_a | 1 (k) | 1 (a) | Interpolate control signal to audio-rate. |
| maxalloc | 1 (i) | - | Limit simultaneous instrument allocations for the compiled instrument. |
| upsamp | 1 (k) | 1 (a) | Convert a control signal to audio-rate by repeating k-values. |
| xtratim | 1 (i) | - | Extend current note duration by an additional init-time amount. |

See Physical steel-string guitar for the editable excitation, string and body example.

# Physical Steel-String Guitar

**Steel String Guitar — Physical Model** is a new polyphonic Melody instrument. Each note has its own pluck and string state. Overlapping notes and manually timed strums ring independently. The two existing steel-string guitars remain available.

Import `Steel_String_Guitar_Physical_Model.orch.instrument.json` from the example folder, add it to Perform, and route its Stereo Output to Master. The output is centered and dry. Play E2–E6 for the intended register; use note lengths to control damping. There is no legato, sample playback, convolution, reverb, or pitch-transposition stage.

## Playing controls

These are per-instance performance settings. Changes apply to new notes; held notes retain their settings.

| Control | Range | Default | Effect |
|---|---|---|---|
| Finger → Plectrum | 0–1 | 0.35 | Sharper contact and more attack brightness toward Plectrum. |
| Brightness | 0–1 | 0.55 | Contact softness and high-frequency retention inside the string. |
| Body resonance | 0–1 | 0.45 | Amount of fixed wooden-body resonances. |
| String sustain | 1–8×, logarithmic | 3 | Nominal string decay multiplier. |
| Release | 0.05–0.8 s, logarithmic | 0.18 s | Damping and final body fade after note-off. |
| Pluck position | 0.08–0.35 | 0.18 | Contact location as a fraction of string length. |
| Palm mute | 0–1 | 0 | Shorter, darker decay. |
| Natural variation | 0–1 | 0.2 | Small changes to contact, position and energy. |

Velocity changes excitation energy, contact duration and brightness. Natural variation does not move note timing or wobble pitch. Chords and strums use the individual note timing you author.

## Editable synthesis

The graph exposes a displacement pulse, filtered noise, the string model, twelve parallel body resonators, and final output shaping as separate blocks. The body frequencies stay fixed from 95 Hz to 6.1 kHz. Mode gains compensate frequency, sample rate and Q before radiation weighting; Q alone does not normalize their resonant gain. A direct-string component preserves the fundamental, with a restrained body blend and a release envelope after both paths.

The reference-informed revision preserves bass fundamentals with a 5 Hz DC blocker. Two contact low-pass stages track note frequency, velocity, Finger → Plectrum and Brightness to soften displacement excitation. A separate brief filtered contact-noise path reaches the body without circulating in the string. The observation position and excitation level across the register are also retuned. The second polarization loses upper partials faster, while both retain the same compensated fundamental. These changes keep all eight controller IDs, ranges and defaults, and preserve the two older guitars.

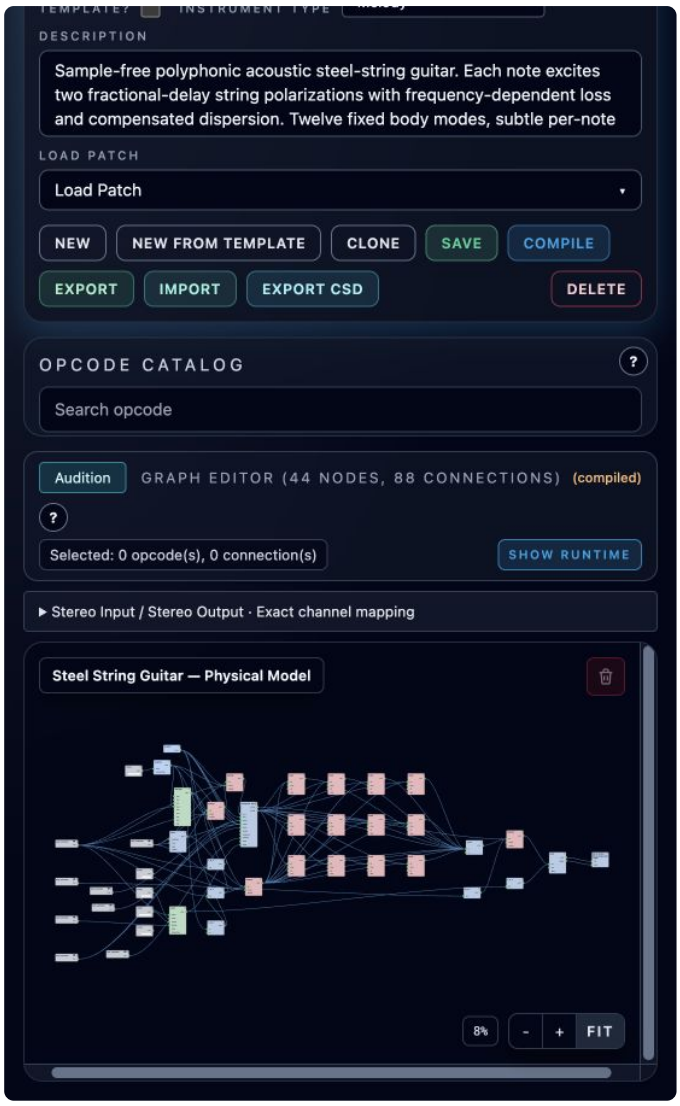
The virtual `waveguide_string` node embeds a Csound UDO in the compiled orchestra. Its two polarization loops use `delayr`, `deltap3`, and `delayw` with local `setksmps 1`. Each loop has frequency-dependent damping, two short all-pass sections, and conservative saturation. Delay length compensates the filters' phase at the fundamental; no subsequent pitch correction is applied. The second polarization has different decay and loss, without detuning or chorus.

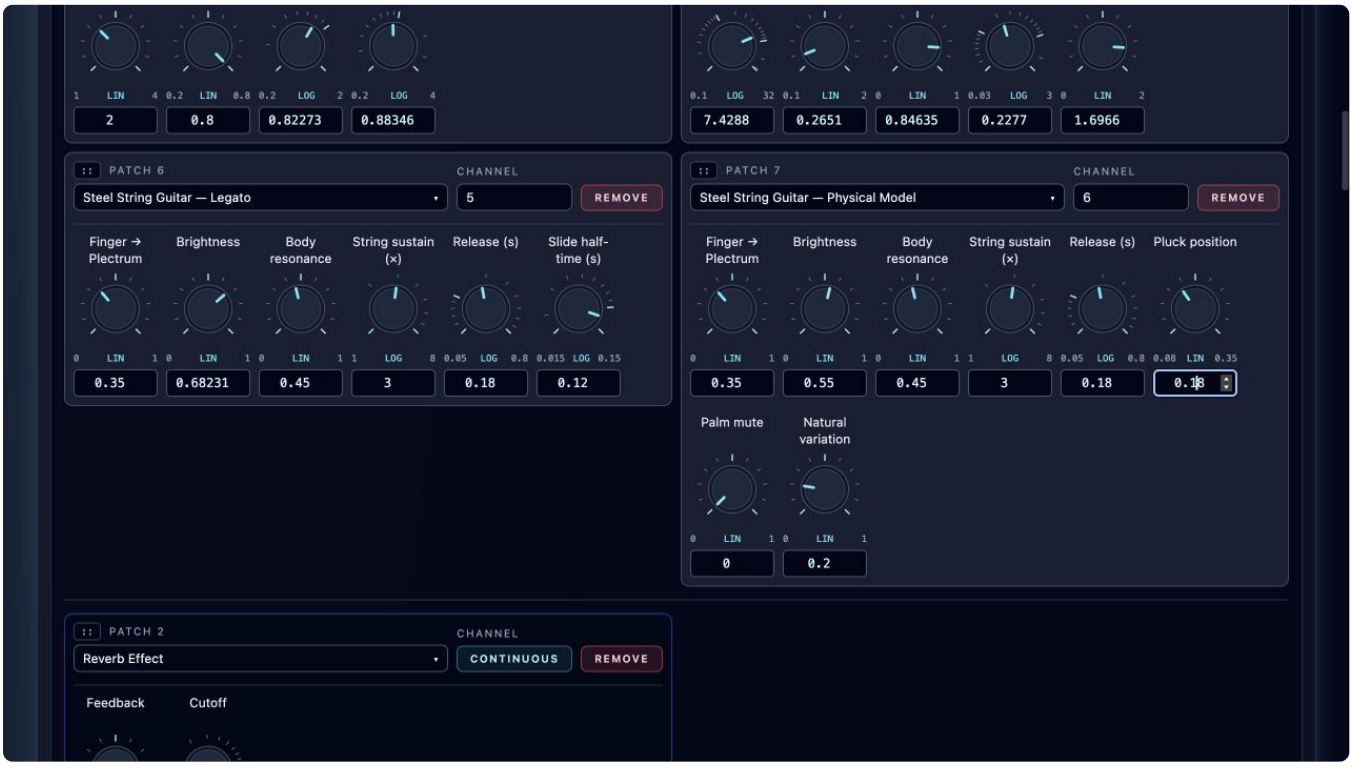
### String interface

| Port | Rate | Meaning |
|---|---|---|
| `aexc` | audio | External pulse/noise excitation. |
| `ifreq` | init | Fundamental, clamped to 20 Hz–sample rate/12. |
| `idecay` | init | Nominal loop T60, clamped to 0.05–30 s; filter loss adds damping. |
| `icutoff` | init | Loss cutoff, clamped to 100 Hz–0.45×sample rate. |
| `ipluck / ipickup` | init | Pluck and observation fractions, clamped to 0.02–0.48. |
| `idispersion` | init | All-pass dispersion, 0–1. |
| `ivariation` | init | Small per-voice contact/position/energy variations, 0–1. |
| `iseed` | init | Nonzero repeats internal variation; zero draws a seed per voice. |
| `kgate` | control | 0–1 damping gate. Falling values damp the feedback and outputs. |
| `astring / abridge` | audio | DC-blocked motion and bridge drive. |

Seeded internal variation is repeatable for identical excitation. External noise has its own random state. Automatic seeds consume the orchestra random generator without resetting it; restarting an orchestra may repeat its default random sequence. All delay/filter state belongs to the individual voice. One helper block is included per orchestra in standalone, mixer, live MIDI and both CSD export modes. Native bundles retain the editable virtual node and require a version of Orchestron with this opcode; CSD exports embed its implementation.

## Graph and controls


DESCRIPTION
Sample-free polyphonic acoustic steel-string guitar. Each note excites two fractional-delay string polarizations with frequency-dependent loss and compensated dispersion. Twelve fixed body modes, subtle per-note
LOAD PATCH
Load Patch
NEW
NEW FROM TEMPLATE
CLONE
SAVE
COMPILE
EXPORT
IMPORT
EXPORT CSD
DELETE
OPCODE CATALOG
Search opcode
Audition
GRAPH EDITOR (44 NODES, 88 CONNECTIONS)
(compiled)
Selected: 0 opcode(s), 0 connection(s)
SHOW RUNTIME
▸ Stereo Input / Stereo Output · Exact channel mapping
Steel String Guitar — Physical Model
8%
-
+
FIT


2
0.8
0.82273
0.88346
7.4288
0.2651
0.84635
0.2277
1.6966
PATCH 6
CHANNEL
Steel String Guitar — Legato
5
REMOVE
Finger → Plectrum
Brightness
Body resonance
String sustain (×)
Release (s)
Slide half-time (s)
0.35
0.68231
0.45
3
0.18
0.12
PATCH 7
CHANNEL
Steel String Guitar — Physical Model
6
REMOVE
Finger → Plectrum
Brightness
Body resonance
String sustain (×)
Release (s)
Pluck position
0.35
0.55
0.45
3
0.18
0.13
Palm mute
Natural variation
0
0.2
PATCH 2
CHANNEL
Reverb Effect
CONTINUOUS
REMOVE
Feedback
Cutoff

## Verification and previews

The example folder contains live host-MIDI, MIDI-file export and SCORE export previews, controller extremes, register and velocity checks, repeated plucks, chords, and authored strums. A matched whole-file RMS comparison plays the previous `wgpluck2` guitar first, then this model. The gain values are recorded alongside the audio; this does not change either instrument's saved gain. The local `revision/` folder also includes an attack-level-matched comparison with the supplied western-guitar recording and the previous Physical Model. The recording informs harmonic balance; its room, microphone and overlapping strings are not replicated by the dry instrument.

Acceptance tests cover every semitone from E2 to E6 at parent block sizes 1, 16, and 64, with 5-cent default and 10-cent extreme tolerances. They also cover frequency-dependent decay, dispersion, velocity, independent releases, separate rack instances, seed behavior, finite out-of-register output, equal stereo, release silence and six-note headroom. Numerical and spectrogram results assess specific properties; they do not establish subjective realism. Listening previews are supplied for a separate musical judgment.

## References

- Csound mode: body resonators and Q.
- Csound deltap3: fractional delay taps.
- Csound setksmps: local sample processing.

# Graph Editor

The graph editor is the central visual patching surface for instrument design.

## Core Graph Editing Features

- Typed input/output ports (audio, control, init, string, function-table)
- Node placement and repositioning (drag nodes on the canvas)
- Connection creation between compatible ports
- Connection and node selection highlighting
- Deletion of selected nodes/connections via explicit delete action
- Zoom controls and fit-to-graph navigation
- Node category color coding
- Inline constant value editing for `const_a`, `const_i`, `const_k`, `const_s`

## Canvas Navigation

The graph editor supports pan/zoom navigation and includes a zoom HUD in the lower-right corner:

- Zoom percentage display
- `−` zoom out
- `+` zoom in
- `Fit` to fit the current graph into view

If the graph is empty, `Fit` resets toward a centered default view.

Horizontal and vertical scrollbars along the canvas edges let you reach every part of a large patch, including expanded If/Switch case frames. They remain available when **Patch controls** is collapsed and adjust when Runtime or the browser window changes the canvas size. Drag a scrollbar, or focus it with **Tab** and use the arrow keys for that direction. Scrolling moves the view without moving nodes or clearing selection. The scrollbars stay synchronized with background panning, zoom, and `Fit`.

## Node Appearance and Visual Cues

### Category Coloring

Nodes are color-coded by category (for example generator, filter, MIDI, constants, etc.) to help visually scan signal flow.

### Optional Inputs

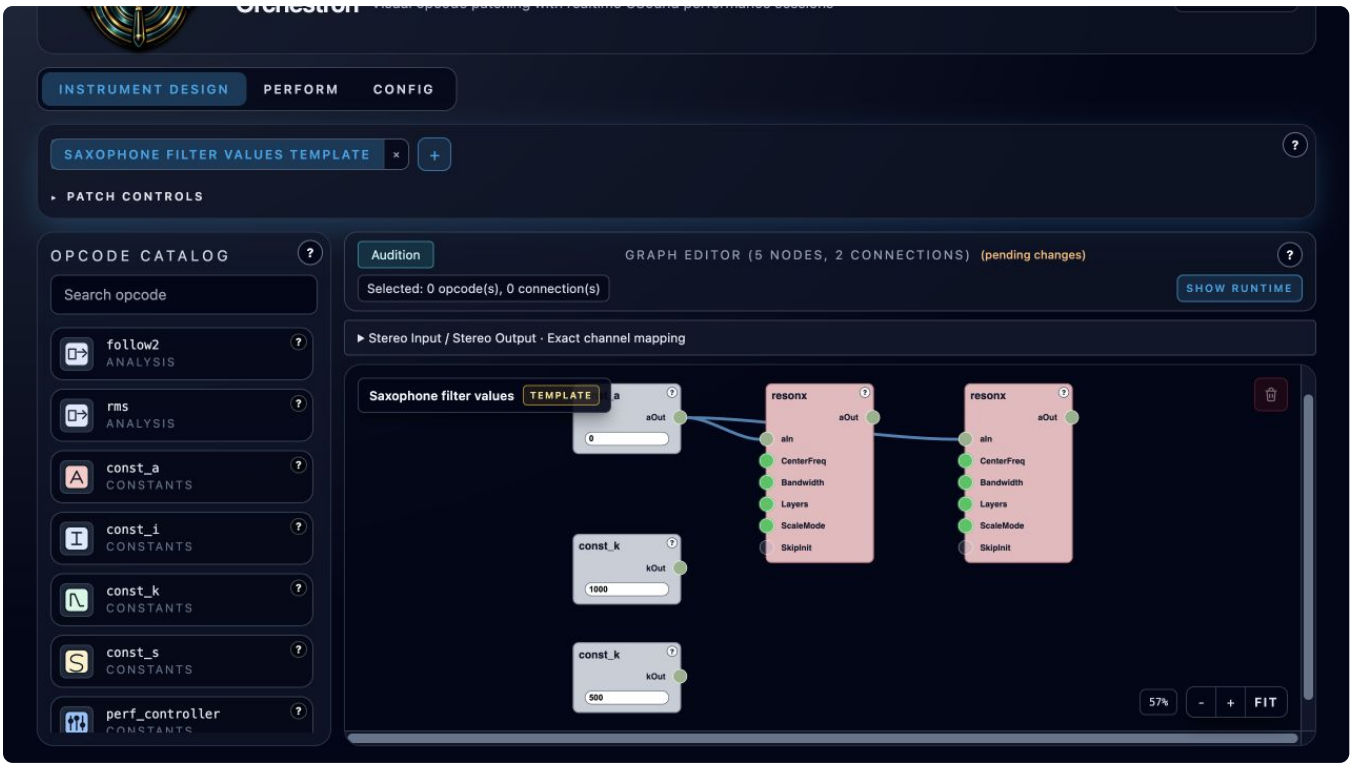
Optional inputs are visually distinct from required inputs.

### Stored Values and Formulas

Bright-green input sockets have an active stored value or saved formula. Zero, false, and explicitly stored values equal to the catalog default count as stored values. A connection overriding a stored value is not highlighted unless it also has a formula. Inherited catalog defaults remain unhighlighted.

Hover over an input to see its source and value. Double-click an input, with or without cables, to inspect or edit its effective value in the Input Formula Assistant. The dialog distinguishes **Stored value**, **Saved formula**, **Connected signals**, **Catalog default**, and **Unset**. Opening or cancelling it leaves the patch unchanged.

**Reset input** removes both the saved formula and the stored value, preserving connections. The dialog previews the resulting connection, default, or unset state before you reset. Required inputs with no remaining source produce the existing compile diagnostic.




The two filters retain their stored CenterFreq, Bandwidth, Layers and ScaleMode values. SkipInit is unhighlighted because its zero comes from the catalog.

## Adding Nodes

You can add nodes in two ways:

- Click from the opcode catalog
- Drag an opcode from the catalog and drop it into the graph canvas

Drag-and-drop placement is preferable for larger patches because it preserves your layout intent.

## Connections

### Creating Connections

- Connect outputs to compatible inputs.
- The backend compiler remains the source of truth for final type/compile validation.

### Selecting Connections

- Connections can be selected directly (not just nodes).
- Selected connections receive a visual highlight.
- Ctrl-based accumulation allows multi-select of connections and nodes.

### Connection Deletion Is Intentional

Orchestron prevents accidental edge removal during socket interaction.

- Connections are not removed by casual socket clicks/drags.
- To remove a connection, select it explicitly and use the delete action.

## Selection Behavior

- Click nodes to select.
- Ctrl-click accumulates node selections.
- Click connections to select them.
- Ctrl-click accumulates/deselects connection selections.
- Background click clears selection (when Ctrl is not held).

# Delete Selected Elements

The graph editor overlay includes a trash/delete button (top-right of the graph canvas area).

- Disabled when nothing is selected
- Enabled when at least one node or connection is selected

Deletion flow:

- If one item is selected, deletion can happen directly.
- If multiple items are selected, Orchestron opens a confirmation dialog listing all affected items (nodes and connections) before deletion.

# Node-Specific Controls

## Constant Nodes

The constant opcodes ( `const_a` , `const_i` , `const_k` , `const_s` ) provide inline editable values directly on the node. `const_s` accepts lowercase letters, digits, and underscores, must begin with a lowercase letter, and is limited to 50 characters.

## Custom IIR Coefficients

The `zfilter2` node has an inline comma-separated coefficient field. Its Coefficients socket also accepts ordered init-rate connections that replace the field. See ZDF and custom IIR filters for coefficient order, counts, and Csound compatibility limits.

## Documentation Button ( `?` )

Nodes with documentation expose a `?` button to open the opcode documentation modal.

## GEN Button (for `GEN` node)

The `GEN` meta-opcode node shows a `GEN` button that opens the specialized GEN table editor.

# Layout Persistence

- Node positions are stored in the patch graph.
- The graph structure (nodes + connections) is part of the patch data.
- The visual layout and related UI metadata (for example input formulas, GEN node configuration) are stored in `graph.ui_layout` .

# Compile Implications

- Compilation order is derived from graph dependencies.
- Invalid wiring, missing required inputs, or invalid formulas surface compile diagnostics.
- Graph compile status is reflected in the instrument page header ( `compiled` , `pending changes` , `errors` ).

# Integrated Help Focus

The graph editor ? help now concentrates on graph-specific behavior: persisted layout, deliberate deletion, constant-node inline editing, and the Input Formula Assistant workflow for combining multiple signals on one input.

## If and Switch at note start

Add **If** or **Switch** from **Control flow**. Drag catalog nodes directly into expanded cases, then patch each case into its managed **Case Result**. Frames measure the complete node bodies, automatically grow to the right and bottom and retain their dimensions, and keep the result in a separate bottom-right row. Drag the **Case Result title or unused body** to resize or shrink the frame; complete member bounds set its minimum. Other selected nodes stay still. This also works for Silence. Escape, cancellation or focus loss restores the previous dimensions and positions. Sizes persist through reload and export/import; bright purple outlines remain about two screen pixels thick at every zoom. The block remains above the first case; top and left movement limits keep members clear of the header.

Ordinary main-graph drags into a case transfer the selection. Ordinary case-member drags rearrange the current case; hold **Alt/Option at drag start** to transfer to another case or the main graph. The cyan/red border shows whether a drop is allowed. Every connection and formula binding must lead to another moved node or an endpoint already in the destination scope. Invalid drops list the connections and restore the whole group; Escape cancels. Existing shared main-graph inputs remain valid and can be wired after placement. A valid drop into Silence activates Synthesis. Dropping onto a collapsed block expands it without choosing a case.

The common output carries only the selected case. Collapse preserves selection, positions, viewport, shared input wiring and formulas. **Configure branches** retains **Move selected nodes** with the same transfer validation, plus case editing, Silence and reviewed mono/stereo changes. The built-in **Drumset** template demonstrates overlapping voices.

See If, Switch and Drumset for the complete workflow, scope rules and version-2 patch/API format.

## Stereo blocks and audio interfaces

Stereo Input and Stereo Output are collapsible views of ordinary paired inleta/outleta nodes. Expand them to edit the underlying opcodes and connections. Exact port names, stored input values and input formulas survive collapsing, expansion, saving and export. Direct Audio Output (outs) stays distinct.

Route an instrument’s Stereo Output through the performance mixer to Master for playback.

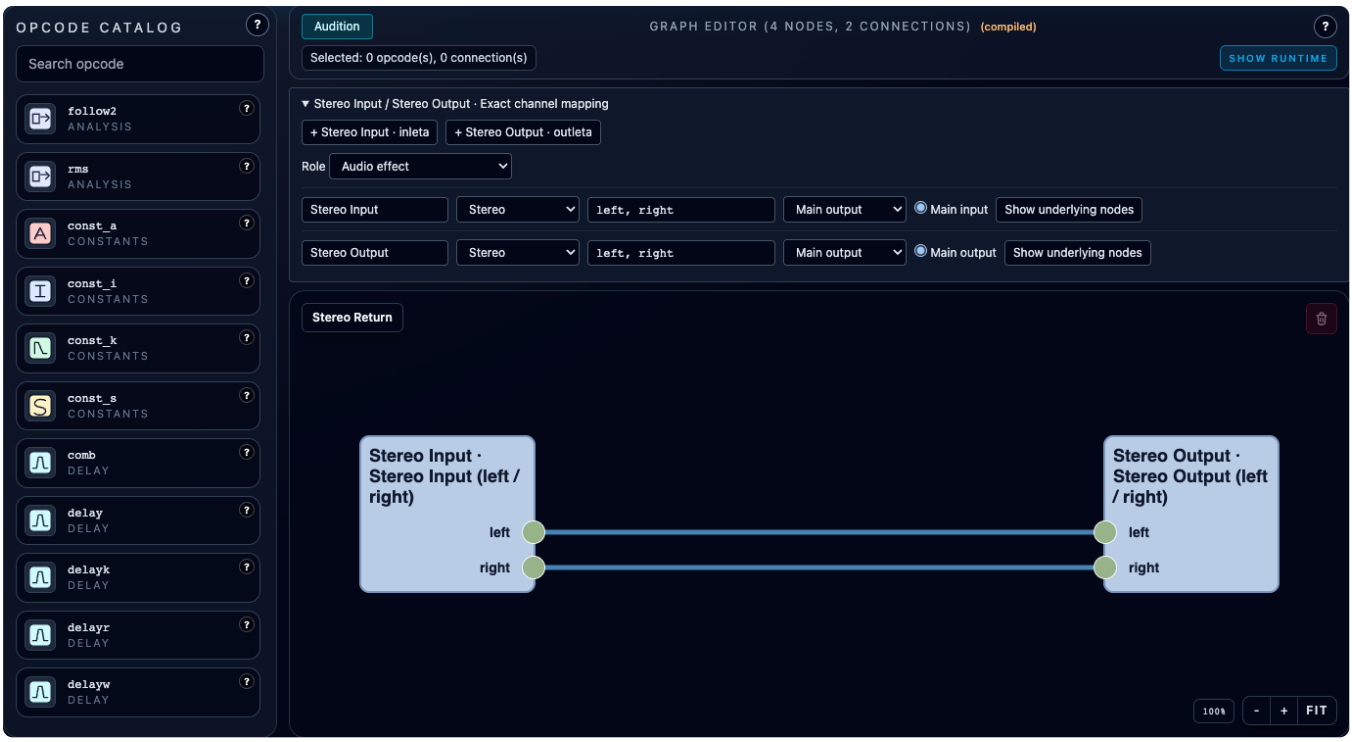
The interface panel describes musical role, stable group IDs, display names, exact member ports, purpose and designated main input/output. Stereo blocks and their mappings share the same creation and deletion operations. Advanced mono/custom groups remain metadata views; changing a stereo group to another layout exposes the ordinary nodes without deleting them.

### Create, rename and remove a stereo interface

1. Click **Stereo Input** or **Stereo Output** in the opcode catalog, drag either onto the canvas, or expand **Stereo Input / Stereo Output · Exact channel mapping** and use its matching add button. Each action creates two channels and their mapping. Connect the new audio ports as needed.
2. Set Role and review the group's display name, purpose and designated Main input/Main output. New pairs use `left` / `right` or an available `busN.left` / `busN.right` pair. Channel names are unique within each direction.
3. Edit **Left** and **Right**, then choose **Apply** to review and confirm the rename. **Cancel** discards the draft. Renaming changes the mapping and underlying opcode names together. A string constant supplying an old name remains available to its other connections.
4. **Show underlying nodes** exposes the pair; **Collapse** returns to its grouped view. Delete a collapsed block or choose **Remove** on its mapping to remove both channels, attached connections and affected input formulas. The confirmation lists all affected elements. Deleting only one expanded channel removes the mapping and leaves the other as an ordinary opcode. Deleting a connection alone retains the mapping.
5. Deleting the designated main group clears its selection without promoting another group. Select a remaining group explicitly when needed.
6. Compile and save before assigning the patch in Perform. Check Audio Mixer and Routing after any external port changes.

## Upgrade or repair an existing instrument

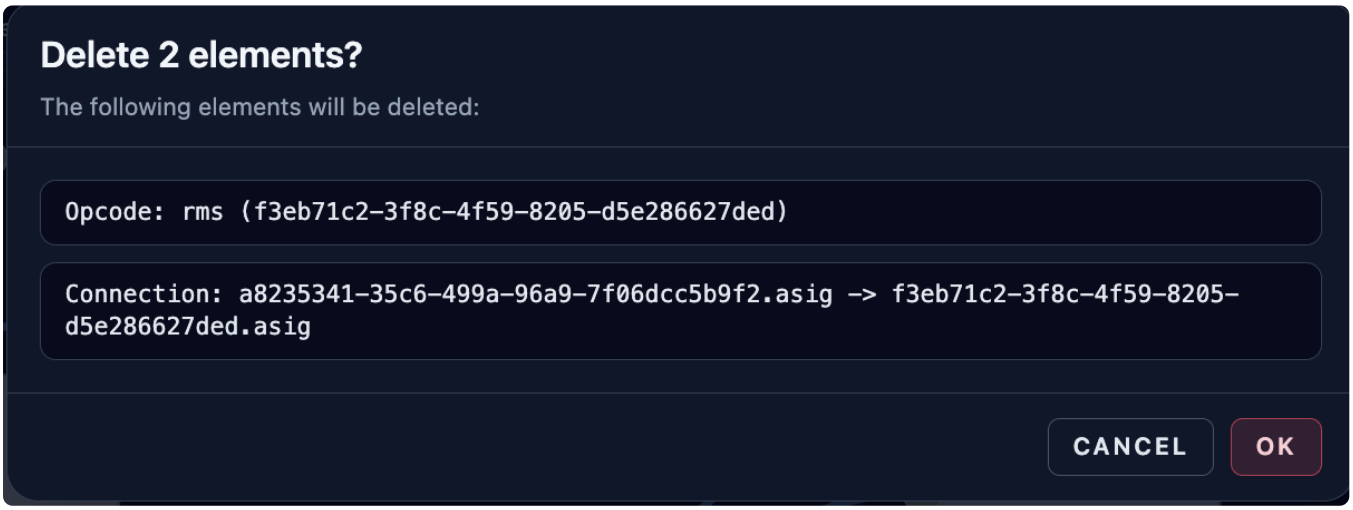
- **Group existing channels:** choose Stereo Input or Stereo Output, select the existing Left/Right nodes and review the change. This preserves the channels' exact names, IDs, positions, audio wiring and formulas. Channel names become part of the stereo block's configuration: values from connected `const_s` nodes are stored directly on the underlying channels, and their naming connections are removed. Dedicated naming constants disappear from the canvas. Constants still used by other nodes or formulas remain available to those consumers; editing or deleting them no longer changes the grouped names. Edit the block's **Left**/**Right** fields to rename its channels. Channels already used by another mapping, duplicate names and unresolved expressions must be repaired first.
- **Convert Direct Audio Output:** select one `outs` node and review its replacement with a named stereo outlet pair. Each side retains its incoming wires, literal value and input formula. Missing inputs remain unconnected. Other direct outputs are left intact. This changes the destination model: connect the named ports in the performance mixer to hear them.
- Ungrouped old patches are not converted automatically. Previously saved valid stereo groups receive the same channel-name cleanup when opened, including groups that are currently expanded. Stale mappings show a reason and **Repair** / **Remove mapping**. Repair can bind a chosen existing pair or explicitly **Create missing channels**. Removing an unresolved mapping removes only metadata, preserving existing nodes.
- Conversion and renaming preserve role and activation. Previews list affected current-performance routes for explicit repair in the mixer after saving; other saved performances remain unchanged.




*A continuous pass-through effect with explicit left/right groups and collapsed Stereo Input and Stereo Output blocks.*

# Screenshots

*Graph editor detail showing selection state, delete button, and zoom controls.*




*Delete-selection confirmation dialog listing selected graph items.*

# Input Formula Assistant

The Input Formula Assistant shows the effective value of an input and lets you edit a constant or define how incoming signals are combined.

## When To Use It

Most commonly, you use it when **multiple outputs are connected to the same input**.

Examples:

- Mix two modulation sources before feeding a filter cutoff input
- Scale one source more than another (`in1 * 0.8 + in2 * 0.2`)
- Apply utility transforms (`abs(in1)`, `dbamp(in1)`) before an input

It is also useful for advanced cases with only one or even zero connected inputs:

- One input: reshape or scale the signal (`ceil(in1)`, `in1 * 0.5`)
- Zero inputs: edit a stored constant (`650`, `-0.25`, `1e-7`) or provide an expression (`ampdb(6)`)

## How To Open It

- Double-click an **input socket** in the graph editor.
- The assistant opens for that exact target node + target input port.

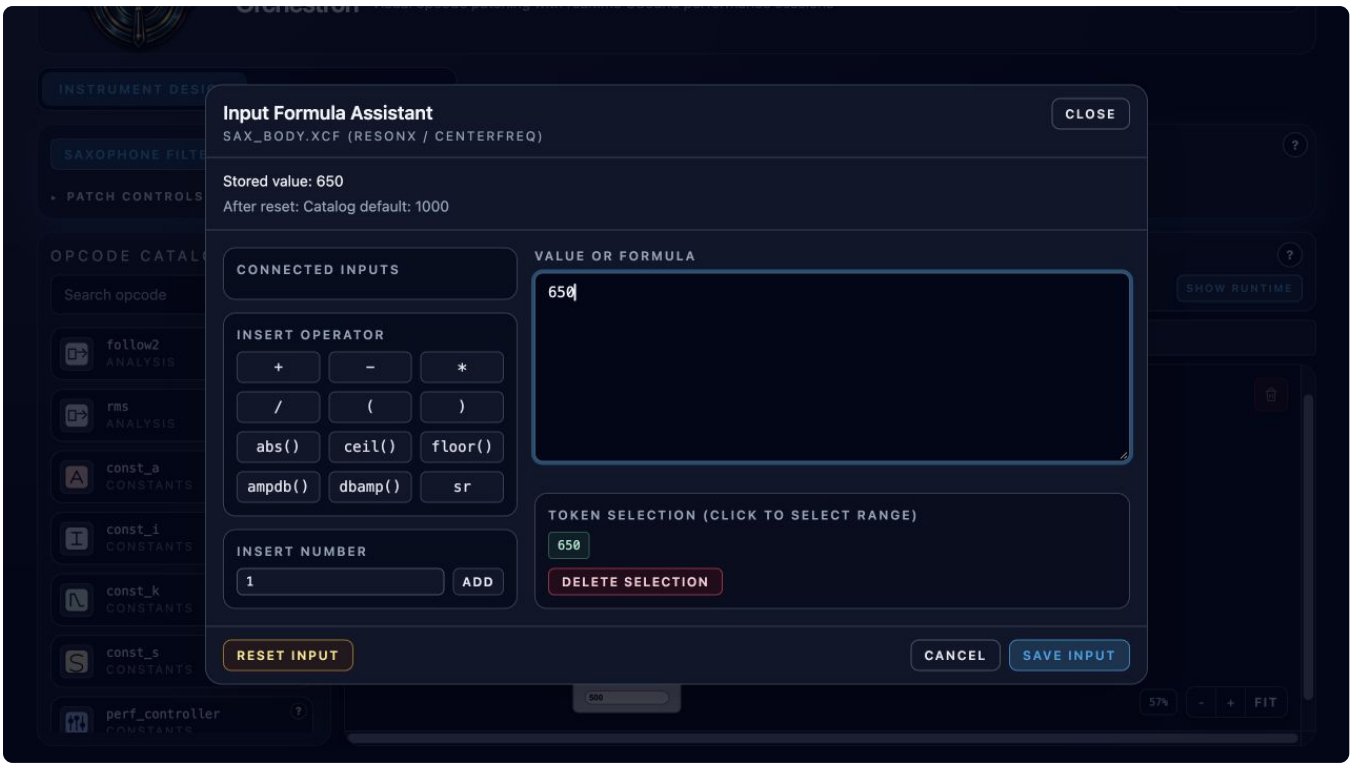
## What The Assistant Shows

### Effective Value and Source

The initial field follows the compiler's precedence:

| Source | Initial contents | Bright-green socket |
|---|---|---|
| Saved formula | Saved expression | Yes |
| Connected signals | Connection token or sum | No |
| Stored value | Stored constant | Yes |
| Catalog default | Inherited default | No |
| Unset | Empty field | No |

A stored value overridden by a connection or formula appears separately as an **Inactive stored fallback**. The **After reset** line previews what will control the input once both saved layers are removed. Boolean constants are editable as their numeric equivalents, `0` and `1`.




### Connected Inputs (Token Mapping)

The left panel lists inbound connections to the target input and assigns tokens such as:

- `in1`
- `in2`
- `in3`

Each token shows source information (node/opcode/port) so you can see what signal you are referencing.

### Operator and Function Buttons

The assistant provides insertion buttons for:

- Operators: `+`, `-`, `*`, `/`
- Grouping: `(`, `)`
- Unary functions: `abs()`, `ceil()`, `floor()`, `ampdb()`, `dbamp()`
- Built-in literals: `sr` (configured audio sample rate)
- Custom numeric literals via the number entry field

### Formula Text Area

You can type directly into the formula field.

### Token Selection / Delete Helper

The assistant tokenizes the current expression and displays clickable tokens.

- Click a token to select that exact range in the text area.
- Use `Delete Selection` to remove the selected token/range quickly.

## Validation Rules

The formula must be syntactically valid before it can be saved.

Validation includes:

- Non-empty expression
- Valid numeric literals
- Supported characters/operators only
- Balanced parentheses
- Valid token ordering
- Known identifiers only ( `inN` tokens, supported built-in literals like `sr` , or supported unary function names)

If validation fails:

- The formula text area border changes to an error style
- A list of validation errors is shown
- `Save input` is disabled

# Save / Reset / Cancel

- **Save input** stores an unconnected finite numeric literal directly as the input's value, removing any previous formula. Scientific notation is accepted for these constants. Arithmetic expressions and values entered on connected inputs use formula metadata and replace the obsolete stored value. The formula language itself is unchanged.
- **Reset input** removes both the stored value and saved formula for this input. Connections stay in place; otherwise the catalog default applies, or the input becomes unset. A required input without a remaining source fails compilation with a missing-input diagnostic.
- **Cancel** / **Close** discards dialog edits. Opening the dialog alone does not modify the patch.

## Specialized Inputs

`maxalloc.icount` accepts only a numeric literal. String inputs use literal text editing. Disconnect a literal-only or string input before editing its literal value. Stereo channel names remain managed by Stereo Input / Stereo Output configuration, and `zfilter2` coefficients retain their dedicated inline list editor. Stored values remain visible in socket tooltips and highlighting for these inputs too.

Collapsed Stereo Output blocks preserve the values and formulas on their underlying left/right audio inputs. Collapsed control-flow captures retain their existing edit restrictions.

# What Happens During Compile

- Inputs resolve in this order: saved formula → connected signals → stored value → catalog default → unset.
- If multiple connections target one input and **no** formula is defined, Orchestron falls back to summing the inbound signals.
- If a formula exists, the compiler resolves the token bindings and compiles the expression.
- Formula validation also happens backend-side, so invalid formulas fail compile with diagnostics.

# Formula Examples

## Weighted Mix

```
in1 * 0.7 + in2 * 0.3
```

## Gain Conversion

```
dbamp(in1)
```

## Rectified Modulation

```
abs(in1)
```

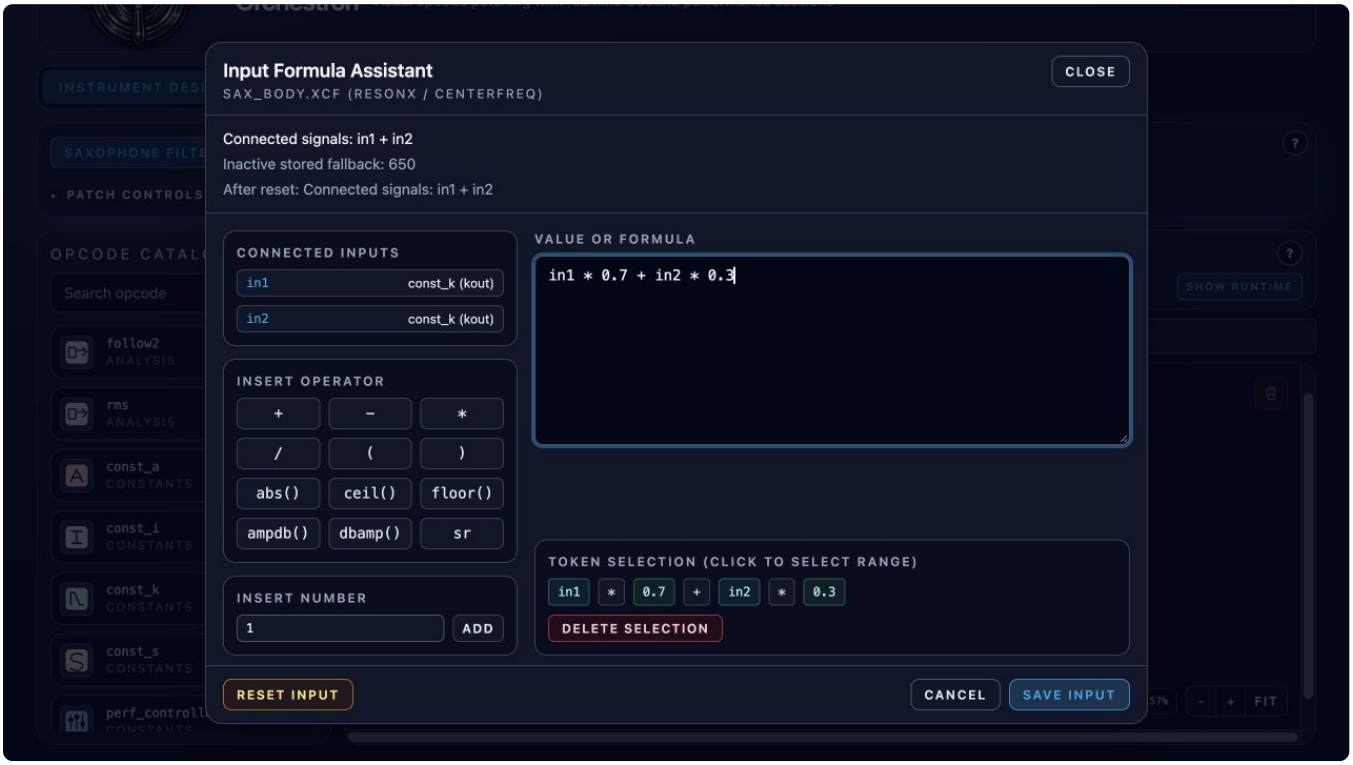
### Constant Override

```
0.5
```

### Sample-Rate-Aware Expression

```
sr * 0.5
```

## Screenshots




*Input Formula Assistant with connected input tokens and editable formula.*

# Standard Effects

These supplied stereo effects can process individual instrument strips or the combined mix as Master inserts. They can also run as separate rack instruments receiving effect sends. Import and save an effect in Instrument Design to make it available in Perform.

## Inserts and Sends

An **insert** sits in the signal path: the strip's entire signal passes through the effect before continuing to its destination. Select the instrument strip or Master, choose the effect under **Inserts…**, and click **Add**. Each insert has its own instance inside that strip's chain. This suits EQ, distortion or other processing applied directly to a sound. Any dry/wet blend comes from the effect's own controls, where available.

An **effect send** creates a parallel path: the instrument keeps its main output route and sends an adjustable copy to a separate effect instrument, called a **return**. The processed return is mixed with the main signal. Several instruments can share one return, each with its own send level; the return's fader controls their combined effect output.

## Add a Shared Send Effect

1. Stop the instruments before changing the rack or routing. In **Perform → Add Instrument**, select the saved effect. It appears as a continuous instrument with its own mixer strip.
2. Connect the effect's output to **Master**. Keep each source instrument's main output connected to Master as well, so its original signal remains audible.
3. In **Mixer → Audio routing**, select the instrument under **Source** and the effect under **Destination**. Leave **Stereo** checked, verify the channel mapping, choose **Send** in the Route selector, and click **Add**.
4. On the source strip, adjust **Send → [effect name]** to control how much signal reaches the effect. New sends start at silence; try **–12 dB** initially. **Post-fader** follows the source's volume and balance; **Pre-fader** is independent of those controls. Repeat for other instruments that should share the effect.
5. Start the instruments, balance the send levels and the return's fader while listening, then **Save Performance**. Existing send levels remain adjustable during playback.

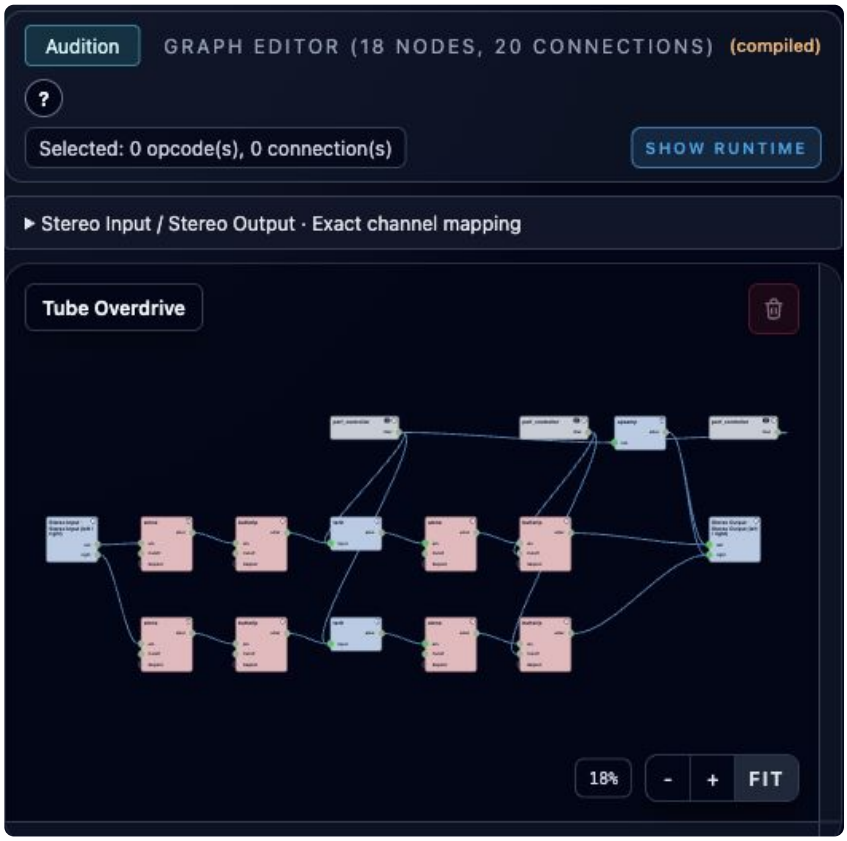
For a parallel return, use **wet-only** output where supported, so the original signal is not added twice. For example, set Stereo Flanger's **Mix to 1**. The supplied Stereo Delay includes dry audio, and Granular Capture passes dry audio while inactive; these are generally better used as inserts. Each guide below explains the effect's routing and mix behavior.

## Effect Guides

- Tube Overdrive
- Stereo Delay
- Stereo Flanger
- Parametric Equalizer
- Stereo Widener, Bit Crusher, and Harmonizer
- Granular Capture
- Stereo Moog Filter

# Tube Overdrive

Download Tube Overdrive and import it using **Instrument Design → Import**. This separate continuous instrument adds warm stereo saturation with standard Csound opcodes. The original **Overdrive** example remains available for comparison.




## Use as an insert

1. Import the instrument and open **Perform** with the instruments stopped.
2. Select the source strip in **Mixer**, choose **Tube Overdrive** in **Inserts…**, and click **Add**. It can also process the combined signal as a Master insert.
3. Set its performance controls, then start the instruments. **Restart the rack after changing a control**: this continuous effect reads settings at startup.
4. Adjust Output to match the untreated level when judging the sound. Save Performance to retain that instance's settings.

The stereo input and output use the exact channel names `left` and `right`. The output is fully processed, with independent left/right signal paths. If used as a return instead, mix the dry and processed signals using the routing and send levels; the patch has no internal dry/wet control.

## Controls

| Controller | Minimum | Maximum | Default | Scale |
|---|---|---|---|---|
| Drive (dB) | 0 | 24 | 12 | Linear |
| Tone (Hz) | 2,000 | 12,000 | 6,500 | Logarithmic |
| Output (dB) | −18 | +6 | −3 | Linear |

**Drive** increases gain before saturation. Start around 3–6 dB for subtle color, 12 dB for moderate breakup, and 18–24 dB for stronger overdrive. Input level also determines how hard the effect is driven. **0 dB Drive is not bypass.**

**Tone** sets the output low-pass cutoff: lower values soften the highs; higher values retain more bite. **Output** trims the processed level. Partial gain compensation reduces level changes as Drive increases, but does not equalize perceived loudness. Positive output gain can exceed full scale; lower Output or the downstream mixer gain if the meter clips.

These are per-instance performance settings, not MIDI CC automation. Their stable IDs are `tube_drive_db`, `tube_tone_hz`, and `tube_output_db`. Native performance bundles and both CSD export modes retain the selected settings.

## How the graph works

Both channels follow the same chain:

```
Stereo Input → atone (20 Hz) → butterlp (12 kHz)
             → biased tanh → atone (10 Hz) → butterlp (Tone)
             → compensated Output gain → Stereo Output
```

The input filters remove subsonic energy and reduce very bright material entering the nonlinear stage. The shaping formula is:

```
aShape = tanh(aPre * ampdb(iDriveDb) + 0.15)
aAC atone aShape - 0.148885033623318, 10
aTone butterlp aAC, iToneHz
aOutput = aTone * ampdb(iOutputDb - iDriveDb / 2)
```

The small bias makes the curve asymmetric, adding even harmonics alongside odd harmonics. Subtracting `tanh(0.15)` keeps silence at zero; the following high-pass removes DC generated by the signal. One shared `upsamp` node converts the computed gain to audio rate for the output formulas. All processing uses existing opcodes and input formulas, so the graph remains editable.

This is a tube-style coloration effect, not a model of a particular amplifier. It has no cabinet simulation, power-supply sag, or oversampling. Filtering does not remove aliasing already generated by the nonlinear stage. High notes and strong Drive can expose this limitation; use the bright-signal comparisons in the example and validation guide to judge the sound for your source.

References: Csound tanh, tube-distortion techniques.

# Stereo Delay

Download Stereo Delay and import it using **Instrument Design → Import**. This continuous stereo effect adds tempo-synced echoes while returning the original left and right channels at unity gain.

## Use as an insert

1. With instruments stopped, open **Perform → Mixer**.
2. On a source strip or Master, open **Inserts…**, select **Stereo Delay**, and click **Add**.
3. Set Tempo to your performance's BPM and Beats to the desired echo interval.
4. Start the instruments. **Restart the rack after changing any of these controls**; the continuous effect reads its settings at startup.
5. Save Performance to retain that instance's settings.

The named stereo input and output channels are `left` and `right`. The effect already includes dry audio; using it as a parallel return also adds that dry signal to the mix. It has no dry/wet control or limiter. Overlapping echoes can exceed full scale, especially with high Feedback; allow headroom in the source or downstream mixer.

## Controls

| Controller | Minimum | Maximum | Default | Scale |
|---|---|---|---|---|
| Tempo (BPM) | 40 | 180 | 120 | Linear |
| Beats | 1 | 16 | 4 | Linear |
| Feedback | 0 | 0.9 | 0.5 | Linear |
| Stereo | 0 | 1 | 0.5 | Linear |

**Tempo and Beats** set the interval to `60 × Beats / Tempo` seconds, rounded to the nearest audio sample. At 120 BPM, Beats 1 means 0.5 seconds and Beats 4 means 2 seconds. Fractional Beats are supported. The full range is approximately 0.333–24 seconds. Beats counts quarter-note beats, independent of the sequencer's meter. Enter Tempo manually; this instrument does not read sequencer tempo.

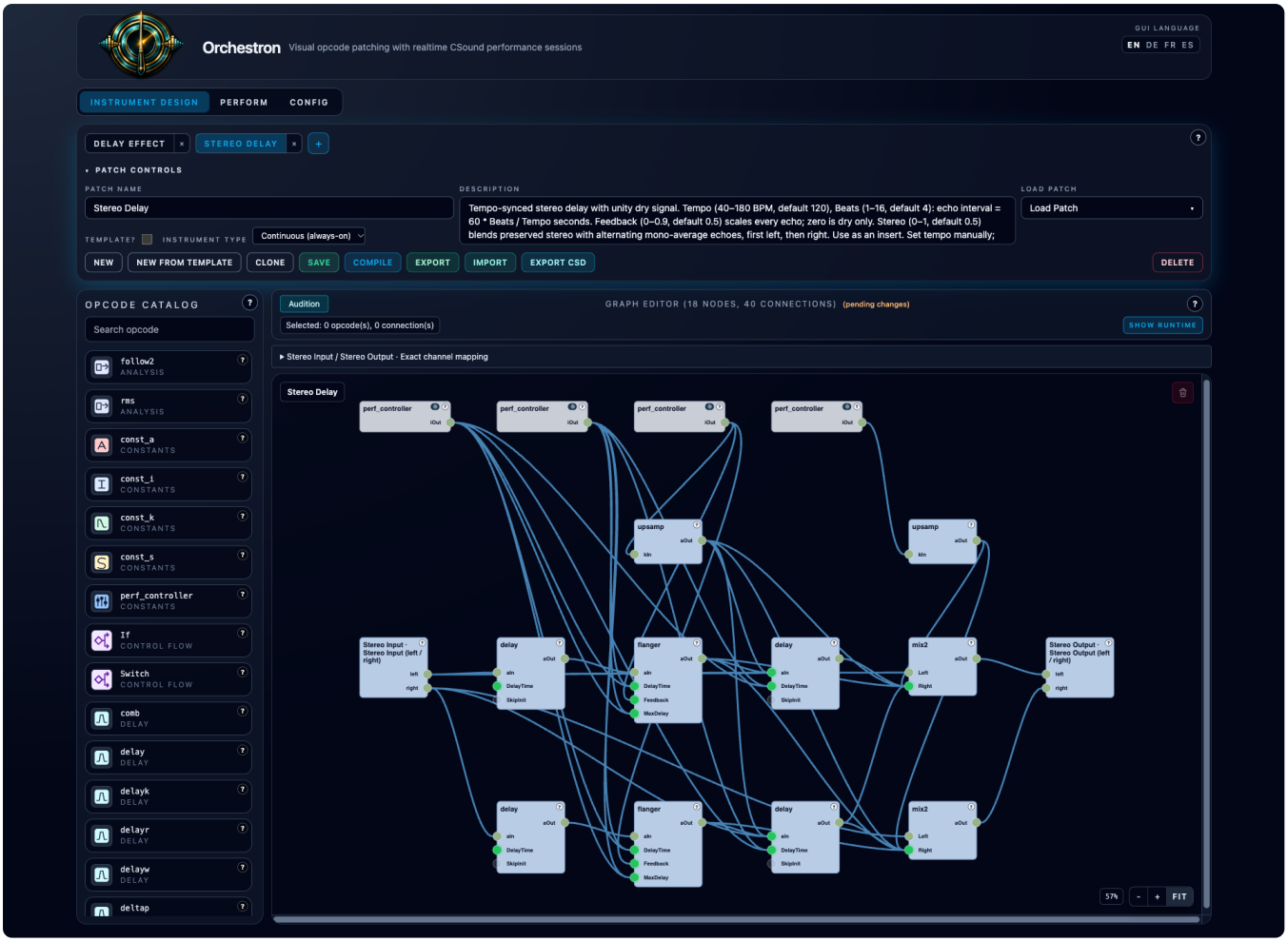
**Feedback** scales every echo. At 0.5, the first echo is multiplied by 0.5, the second by 0.25, the third by 0.125, and so on. Zero produces only the unchanged dry signal. This gain is applied before the Stereo blend.

**Stereo** controls echoes only. At zero, every echo preserves its original left/right channels. At one, each echo uses the average `(left + right) / 2`: the first goes entirely left, the second entirely right, then left again. Intermediate values linearly blend these behaviors. Averaging keeps identical left/right input from doubling in level; a one-sided signal becomes half as large in the fully panned branch, and opposite-phase channels can cancel there.

The four per-instance settings use stable IDs `sd_tempo`, `sd_beats`, `sd_feedback` and `sd_stereo`. Native instrument/performance bundles and both CSD export modes retain the controls and selected instance values.

## Editable graph and validation

Each input channel has separate odd- and even-echo streams. A feedback delay at twice the interval generates the even echoes; a second delay fills the odd positions. A one-sample seed delay and matching feedback-loop compensation keep successive echoes on the selected sample interval. The dry path bypasses these delays and the panning formulas.




The graph uses existing `delay`, `flanger`, `upsamp`, `mix2`, `inleta`, `outleta` and `perf_controller` nodes. See the example and validation guide for reproducible renders and tests. Long intervals with high Feedback need long render tails; the instrument does not extend the export duration automatically.

References: Csound flanger, Csound delay.

# Stereo Flanger

Download Stereo Flanger and import it with **Instrument Design → Import**. This continuous effect mixes each channel with a short, moving delay to produce sweeping comb-filter notches. Left and right retain separate audio paths; Stereo Phase offsets their sweeps.

## Use in a performance

1. With instruments stopped, open **Perform → Mixer**.
2. On an instrument strip or Master, open **Inserts…**, select **Stereo Flanger**, and click **Add**.
3. Start with the defaults, then adjust the six performance controllers.
4. **Restart the rack after changing settings.** Continuous instruments read these values at startup.
5. Save Performance to retain that instance's settings.

For a shared send effect, set Mix to **1** so the return contains only wet audio. Stereo Input and Stereo Output both expose `left` and `right` channels. The effect needs routed audio and does not use MIDI notes.

## Controls

| Controller | Range | Default | Scale |
|---|---|---|---|
| Rate (Hz) | 0.02–5 | 0.25 | Logarithmic |
| Delay (ms) | 0.2–5 | 1 | Linear |
| Depth (ms) | 0–5 | 2 | Linear |
| Feedback | −0.9–0.9 | 0.5 | Linear |
| Stereo Phase (deg) | 0–180 | 90 | Linear |
| Mix | 0–1 | 0.5 | Linear |

**Rate** sets the sweep speed in cycles per second. The default completes one cycle every four seconds; tempo synchronization is manual.

**Delay** is the shortest delay. **Depth** is the sine wave's amplitude, with the same amount added as a positive offset. The delay therefore sweeps from `Delay` to `Delay + 2 × Depth` : **1–5 ms** at the defaults. Depth zero produces a static comb filter. Across all settings, delays stay between **0.2 and 15 ms** inside a 20 ms buffer.

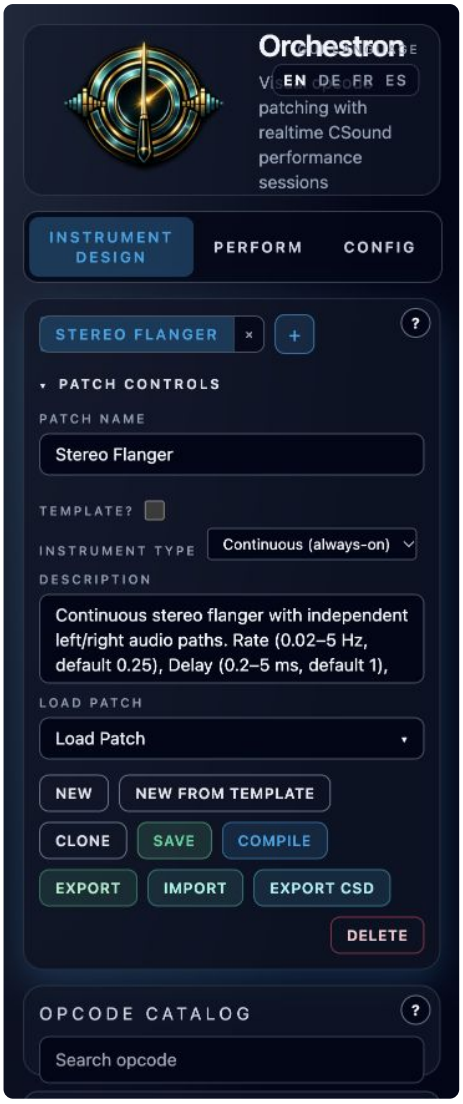
**Feedback** sharpens the resonances. Negative values invert successive repeats, changing the comb's character. Zero removes recirculation but keeps the moving delay. Wet gain is multiplied by `1 − abs(Feedback)` to compensate for feedback buildup; increasing feedback can reduce average loudness while emphasizing the resonant frequencies.

**Stereo Phase** sets the right sweep's phase relative to the left. Zero gives matching sweeps; 90 degrees gives a quarter-cycle offset; 180 degrees makes them sweep in opposite directions. It does not cross-feed audio between channels.

**Mix** crossfades linearly from untouched dry audio at zero to compensated wet audio at one. Intermediate settings produce the familiar flanging interference; wet-only settings emphasize pitch modulation and feedback coloration.

## Editable graph and validation

The graph uses two audio-rate `poscil3` sine oscillators, two `flanger` delay lines, and ordinary stereo input/output blocks. Stable controller IDs are `sf_rate` , `sf_delay` , `sf_depth` , `sf_feedback` , `sf_phase` , and `sf_mix` . Native bundles and both CSD modes preserve the graph and instance settings.




See the example guide for the editable patch, reproducible previews, and regression checks.

References: Csound flanger, Csound poscil3.

# Parametric Equalizer

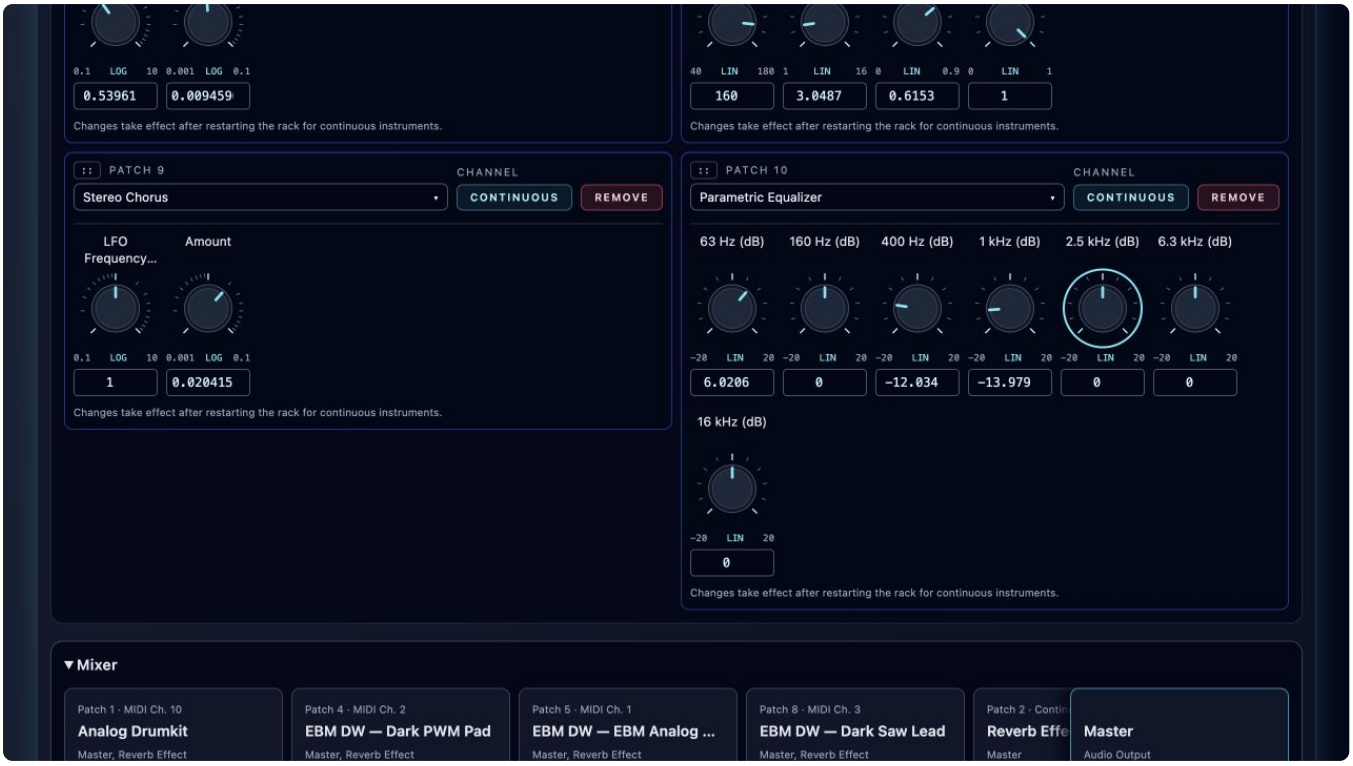
Download Parametric Equalizer and import it using **Instrument Design → Import**. This continuous stereo effect has seven peaking bands, with independent left/right signal paths and linked gain settings. Add it to a source strip or Master using **Perform → Mixer → Inserts…**.

## Controls

| Center | Typical use | Gain range | Default |
|---|---|---|---|
| 63 Hz | Sub/bass weight | –20 to +20 dB | 0 dB |
| 160 Hz | Bass warmth | –20 to +20 dB | 0 dB |
| 400 Hz | Low-mid body | –20 to +20 dB | 0 dB |
| 1 kHz | Midrange | –20 to +20 dB | 0 dB |
| 2.5 kHz | Presence | –20 to +20 dB | 0 dB |
| 6.3 kHz | Definition | –20 to +20 dB | 0 dB |
| 16 kHz | Air | –20 to +20 dB | 0 dB |

All seven controls use a **linear dB scale**. Zero is neutral; –20 dB gives amplitude gain 0.1, and +20 dB gives 10. The graph converts each setting with `ampdb(db)` before feeding `pareq.kv`, as in the Csound EQ example.

**Restart the rack after changing these controls**, then Save Performance to retain the instance settings. Allow headroom when boosting: overlapping bands add gain, and this EQ has no limiter. Its named stereo inputs and outputs are `left` / `right`. Use it as an insert; a parallel dry return changes the resulting EQ response.

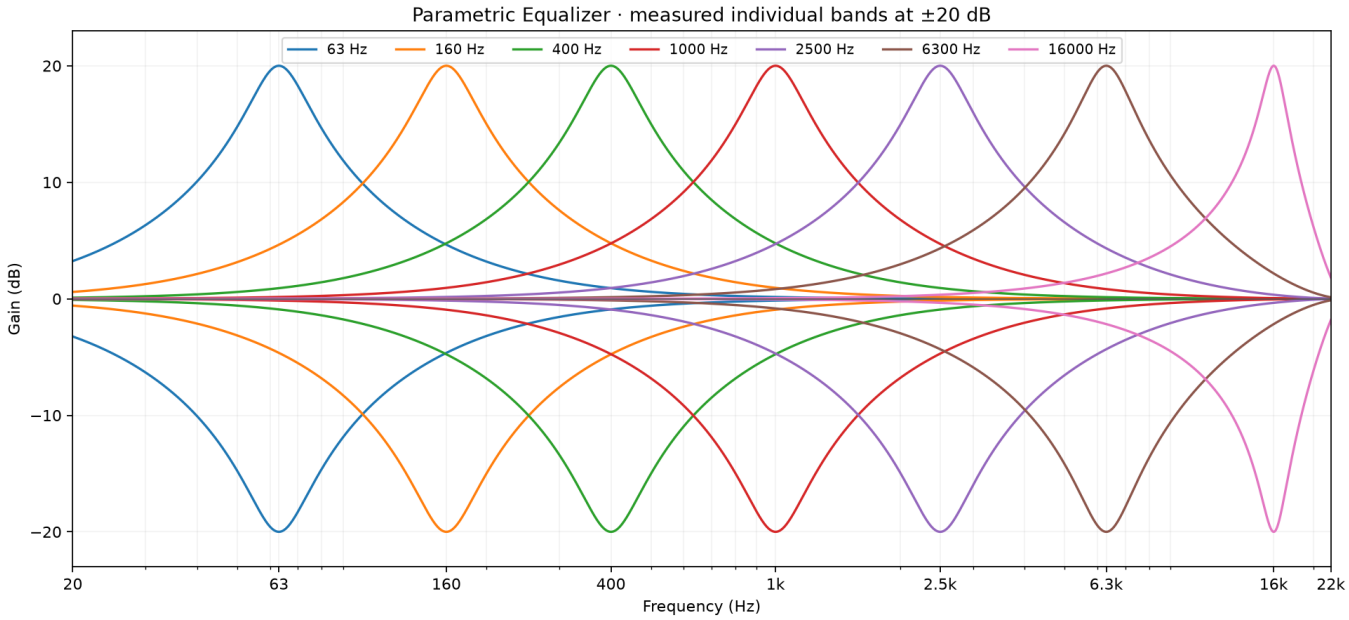


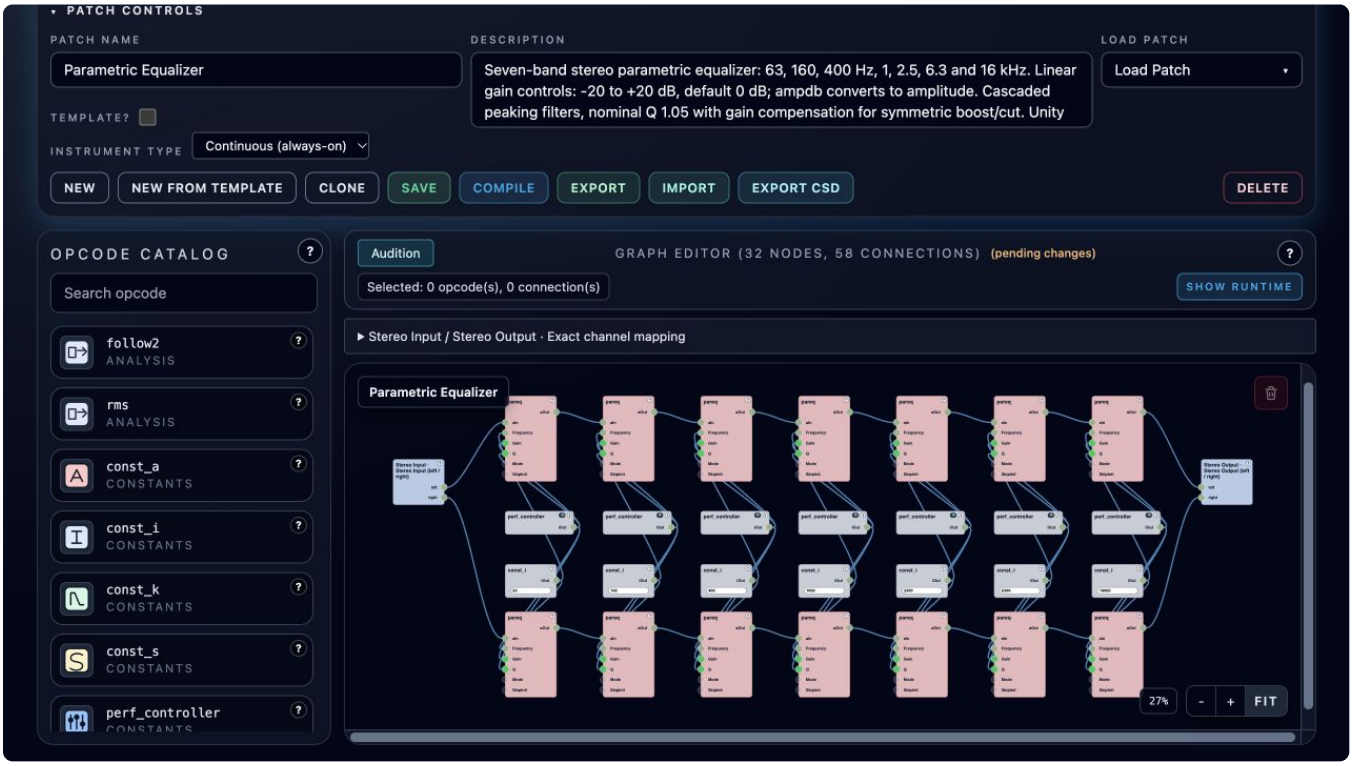

## Frequency spacing and Q

The rounded centers are approximately 1⅓ octaves apart. Nominal **Q = 1.05** gives broad tonal shaping with useful overlap, approximately 1⅓-octave half-gain bandwidth at low frequencies. These are general-purpose defaults; edit the frequency constants and paired Q formulas in Instrument Design for narrower corrections. Digital frequency warping narrows the

uppermost bell near Nyquist. The supplied graph runs at 48 kHz, keeping 16 kHz below Nyquist.

For symmetric boosts/cuts, each `pareq.kq` input uses `1.05 * ampdb(db / 2)`. This adjusts Csound's denominator Q by the square root of linear gain to match the Audio EQ Cookbook's peaking convention. Nominal musical Q stays 1.05; the raw opcode Q varies with gain. The two channels use identical settings. Seven filters run **in series per channel**, giving unity gain when all controls are zero.




## Updating the original three-band instrument

The earlier graph summed three full-range EQ outputs in parallel. At neutral settings this produced amplitude gain 3, or **+9.54 dB**. The new cascade corrects that behavior, so existing mixes can sound quieter and their EQ curves change.

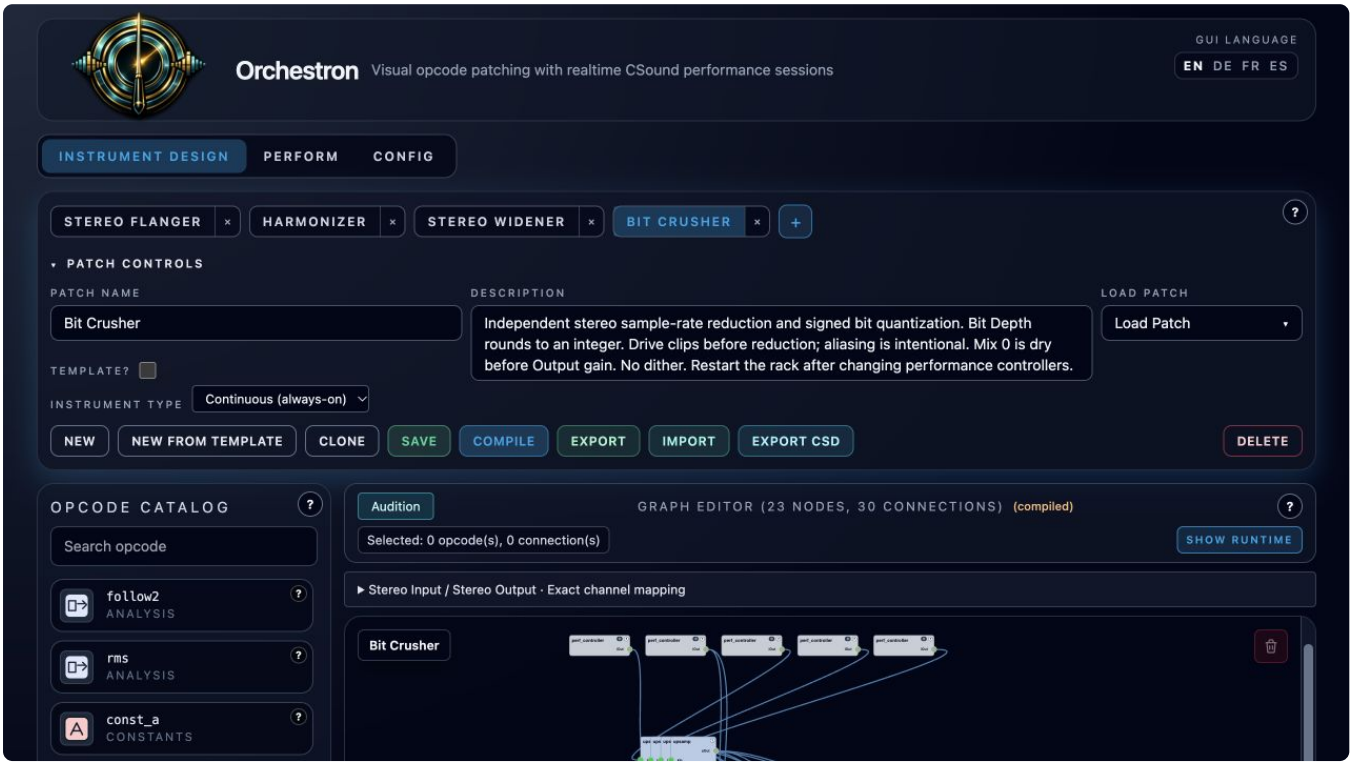
The original three controller IDs are retained, with centers changing from 100 → 63 Hz, 600 → 400 Hz and 1.2 → 1 kHz. Four new bands default to zero. When updating an older library manually, convert any stored amplitude override with `20 * log10(old value)`; for example, 2 becomes +6.0206 dB and 0.2 becomes –13.9794 dB. Range clamping alone does not

convert units. Keep a backup of old performances and reload open browser sessions after migration.

See the example and validation guide for the preserved original graph, rebuild commands and audio checks.

# Stereo Widener, Bit Crusher, and Harmonizer

These three continuous instruments process incoming stereo audio without MIDI notes. Add an instrument to the rack and use its grouped Stereo Input and Stereo Output as a mixer insert. Each instance has its own performance settings; **restart the rack after changing a controller**. The examples include editable graphs and native instrument files.




## Stereo Widener

| Controller | Range | Default |
|---|---|---|
| Width | 0–2 | 1.5 |
| Mono Widening | 0–1 | 0 |
| Delay (ms) | 1–30 | 12 |
| Output (dB) | –24–0 | –6 |

Width scales the side signal while preserving the center. Width 0 makes mono; Width 1 with Mono Widening 0 preserves the original stereo image before Output gain. Width above 1 exaggerates the stereo difference. Mono input stays mono until Mono Widening is raised.

Mono Widening adds the difference between the center signal and a delayed copy to the side signal. Try Mono Widening 0.5–1 with Width 1–2 on a mono synth or pad. Delay controls the coloration and apparent space. The added side cancels when summed to mono; the mono sum remains the original center signal multiplied by Output gain. High width can increase individual channel peaks, so leave headroom.

The graph computes `M=(L+R)/2`, `S=(L−R)/2`, then `S'=Width × (S + MonoWidening × (M−delayedM)/2)`. Its outputs are `M+S'` and `M−S'`, followed by Output gain.

## Bit Crusher

| Controller | Range | Default |
|---|---|---|
| Bit Depth | 2–16 | 8 |

| Controller | Range | Default |
|---|---|---|
| Downsample Factor | 1–64, logarithmic | 4 |
| Drive (dB) | 0–24 | 0 |
| Mix | 0–1 | 1 |
| Output (dB) | –24–0 | –6 |

Each channel is driven and hard-clipped, reduced in sample rate, and quantized. Bit Depth rounds to the nearest integer. Downsample Factor 1 preserves the sample rate; 4 produces an effective 12 kHz rate at a 48 kHz engine rate. Fractional factors are supported. Aliasing and stepped waveforms are intentional parts of the sound.

Quantization uses signed codes from –1 to `1-1/Q`, with `Q=2^(BitDepth-1)`, rounding to the nearest code. Zero remains zero; no dither is added. At 2 bits, quiet inputs below ±0.25 quantize to silence; raise Drive to bring them above that threshold. The sample holder initializes to silence for its first sample. Drive affects the processed path only. Mix 0 gives the original input, and Mix 1 gives the processed signal; Output gain follows the blend. For a send return use Mix 1.

## Harmonizer

| Controller | Range | Default |
|---|---|---|
| Voice 1 Interval (semitones) | –24–24 | +7 |
| Voice 2 Interval (semitones) | –24–24 | +12 |
| Voice 1 Level | 0–1 | 0.7 |
| Voice 2 Level | 0–1 | 0.5 |
| Mix | 0–1 | 0.5 |
| Output (dB) | –24–0 | –6 |

Two spectral pitch shifters add fixed harmony voices. Intervals round with `floor(interval+0.5)` and convert to frequency ratios using `2^(interval/12)`. The defaults add a fifth and an octave. Set either voice level to zero to mute it. If the level sum exceeds 1, both levels are divided by their sum before mixing with the dry signal.

Left and right channels are processed separately. The effect does not detect a key or scale and does not preserve vocal formants. Sustained notes and chords suit this algorithm; sharp transients can smear or ring. There is no feedback tail.

The graph uses a 1024-sample Hann analysis window and FFT, with a 256-sample hop, followed by `pvscale` and `pvsynth`. **Measured processing latency is 1536 samples (32 ms at 48 kHz)** on the validated Csound 6.18 runtime. The dry path uses the same delay, including at Mix 0. Use Mix 1 for a send return; a separate undelayed parallel dry route will still differ in timing. This measured delay supersedes the 1024-sample figure in the manual's illustrative example.

Csound pvscale reference · Csound fold reference

## Files and validation

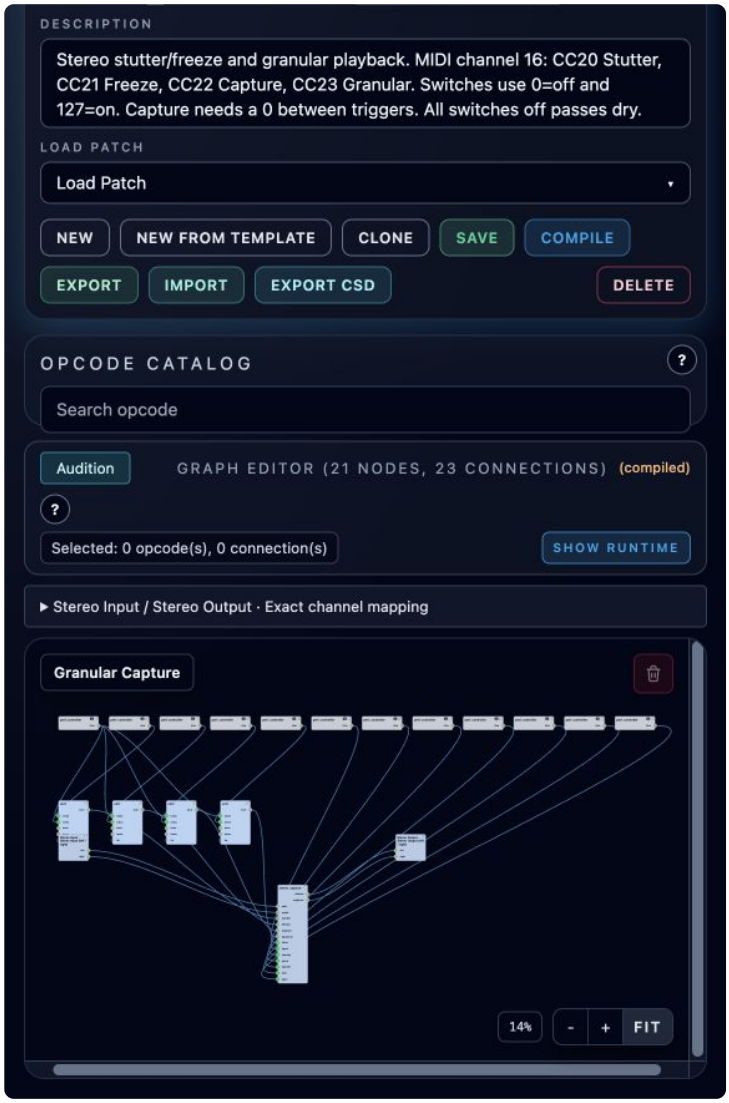
Native exports are in `examples/instruments/`: `Stereo_Widener.orch.instrument.json`, `Bit_Crusher.orch.instrument.json`, and `Harmonizer.orch.instrument.json`.

See the example README for builders and validation commands. Auditions compare mono chords, stereo pitches, and transient noise at defaults and controller endpoints. Tests use independent fixtures. Audio and measurements are written under `output/continuous-effects/`.

See Granular Capture for live MIDI-controlled stutter, freeze and granular playback.

# Granular Capture: Stutter, Freeze and Grains

**Granular Capture** is one continuous stereo effect with Stutter and Granular playback and a shared Freeze control. Add it as a mixer insert, feed its grouped stereo input, and route its grouped output onward. It records incoming audio continuously, even while playing a frozen snapshot. The two channels keep separate samples and share capture timing and grain positions.




## MIDI switches

The default **MIDI Control Channel is 16**. This setting is independent of MIDI note activation: the instrument remains Continuous and needs no note channel. Each switch has its own configurable CC number. Choose distinct CCs and avoid collisions with other receivers. Mapping knobs stop at 95 because Csound handles RPN/NRPN and some higher CCs specially. Send the initial Reset Value 0 before recording; standard CCs such as volume can start above zero. Instances sharing a channel and CC respond together; select another channel or CC set for independent instances.

| Switch | Default CC | 0–63 | 64–127 |
|---|---|---|---|
| Stutter | 20 | Release stutter | Capture and repeat the recent slice; an existing Freeze snapshot is retained |
| Freeze | 21 | Release frozen source | Capture and hold the recent slice |
| Capture | 22 | Arm the next trigger | Rising edge replaces the snapshot, including while frozen |

| Switch | Default CC | 0–63 | 64–127 |
|---|---|---|---|
| Granular | 23 | Loop playback when Stutter/Freeze is on; otherwise dry | Play windowed grains from live history or the held snapshot |

Use **0 and 127** in step cells. Capture is a value-edge trigger: put **0 between successive 127 values**. Repeating 127 sends another MIDI message but does not produce another rising edge. Capture alone updates the snapshot without enabling wet playback. Stutter or Freeze alone produces a loop; Granular on produces grains. Freeze off resumes live grains unless Stutter is still holding the snapshot. All three playback/hold switches off returns to dry audio with Output gain.

## Configure a step pattern

1. Add a Controller Sequencer and select **Step Mode**.
2. Select only MIDI channel **16** and set **Controller # 20**.
3. Set **Reset Value 0**. Enter `127` where stutter should begin and `0` where it should end. Empty cells hold the current state.
4. Add lanes for CC21, CC22 and CC23 when needed, each with Reset Value 0. Enable Freeze before switching to Granular to turn a repeat into a sustained texture.
5. For recapture while frozen, send CC22 `127`, then `0`, then another `127` at the next capture point.

A four-beat pattern at 120 BPM with four steps per beat can use:

| Lane | Populated steps (one-based) |
|---|---|
| Stutter, CC20 | 3: 127; 7: 0 |
| Freeze, CC21 | 9: 127; 15: 0 |
| Capture, CC22 | 13: 127; 14: 0 |
| Granular, CC23 | 11: 127; 16: 0 |

The reproducible example includes the matching API sequencer configuration. Sequencer start/stop and finite endings send Reset Value. Pad changes and loops hold the current value. A controller stopped with a nonzero Reset Value can leave an effect active. Muted controller lanes do not send their reset; release the corresponding CC manually or restart the rack. A separately running Manual pads controller also remains independent of Arranger Stop.

## Performance settings

These are per-instance I-rate settings. Restart the rack after changing them; MIDI switches act during playback.

| Setting | Range | Default |
|---|---|---|
| MIDI Control Channel | 1–16, rounded | 16 |
| Stutter / Freeze / Capture / Granular CC | 0–95, rounded | 20 / 21 / 22 / 23 |
| Slice | 20–1000 ms | 125 ms |
| Grain Size | 5–200 ms | 50 ms |
| Grains / Second | 1–80 | 40 |
| Grain Pitch | –24–24 semitones, fractional allowed | 0 |
| Position Spread | 0–1 | 0.5 |
| Mix | 0–1 | 1 |
| Output | –24–0 dB | –6 dB |

Slice is the **stutter repeat period**, with an extra 5 ms of history captured for its overlapping seam. At 120 BPM, 125 ms is a sixteenth note. This is a millisecond setting, not automatic tempo synchronization: use `60000 / BPM / subdivisions per beat` and restart after adjusting. Grain Pitch affects granular playback only. Position Spread zero starts each grain at the beginning of the slice; higher values scatter starts within it. Sparse short grains sound choppy; overlapping long grains produce a smoother sustained texture.

Mix zero preserves dry audio, with Output gain and no processing delay. Inactive playback also passes dry. Use an insert: a parallel send would duplicate dry audio while inactive. Freeze can intentionally sustain indefinitely, so send a release before the end of an offline render. Capture before enough history exists includes silence. Engine stop/restart clears both history and snapshot buffers.

## Implementation and limits

The editable graph uses four native ctrl7 readers feeding the Orchestron `stereo_capture` processor. The processor is an application-defined Csound UDO, not a stock Csound opcode. Its generated code is included in standalone compilation and both CSD exports. SCORE mode rewrites explicit-channel readers to the score controller array, including channels with no note instrument. No channel assignment or persistence schema extension is needed.

Each instance owns separate two-second live and snapshot stereo tables, freed when the instance ends. Captures copy the live tables once per rising edge; recording continues. Loop seams overlap for 5 ms. State changes crossfade from the preceding output sample over 5 ms. Up to 16 Hann-windowed grains share a deterministic private position sequence; summed gain is bounded by their window weights. Pitch shifting uses interpolated resampling, so high transpositions can alias. This is a time-domain freeze, not a spectral freeze.

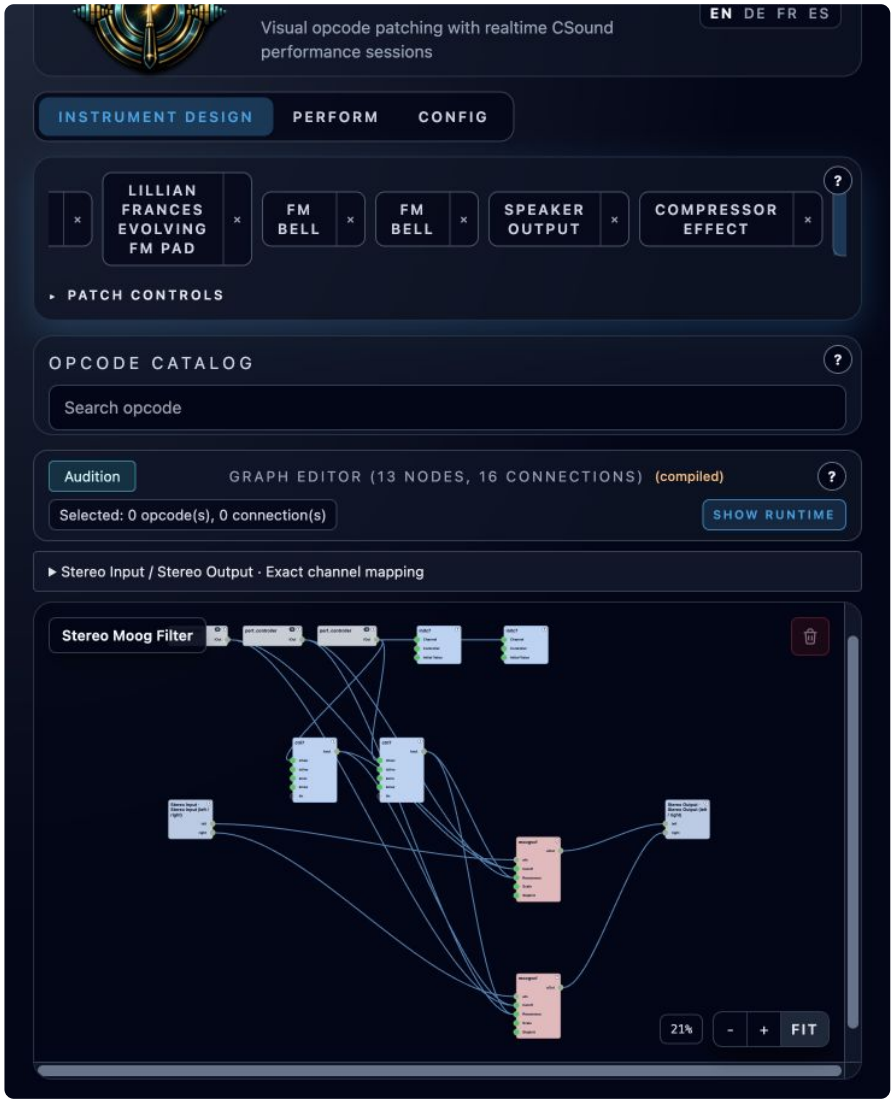
MIDI switch changes are observed on the engine control-block boundary. The internal processor runs sample by sample, independently of the parent block size. MIDI and SCORE paths may differ by one control block because of event ordering. Very short high/low pulses within the same block are not guaranteed to be observed; normal sequencer steps leave ample separation.

## Files and validation

- Native instrument
- Builder, step pattern and auditions
- Independent DSP, API, native round-trip, per-instance settings, controller-step and export tests use committed fixtures.
- Audio validation checks dry identity, held history, recapture, reset tails, stereo isolation, grain pitches, extremes, and finite preview headroom. Mel and log-STFT plots accompany the auditions. Listening is reported separately.

# Stereo Moog Filter

**Stereo Moog Filter** is a Continuous stereo resonant low-pass effect. Import and save the native instrument, then add it as an instrument-strip or Master insert. Its grouped Stereo Input and Stereo Output preserve separate left and right audio; both channels share the same cutoff and resonance settings. No MIDI notes are required.




## Performance settings

| Setting | Range | Default |
|---|---|---|
| Cutoff (Hz) | 20–20,000, logarithmic | 1,000 |
| Resonance | 0–1, linear | 0.3 |
| MIDI Control Channel | 1–16, rounded | 16 |

These settings belong to each rack instance. **Restart the rack after changing them.** The effect is fully processed, with no additional output gain, dry/wet blend or smoothing. Use the mixer strip for output level. Resonance near 1 can self-oscillate, so leave headroom.

## MIDI control

| Controller | Effective value |
|---|---|
| CC20 | Cutoff × CC20 / 127 |
| CC21 | Resonance × CC21 / 127 |

The factors are bounded to 0–1. A value of 127 uses the full performance setting, 64 uses approximately 50.39%, and 0 produces an exact zero factor. The cutoff knob's logarithmic scale does not change the linear MIDI multiplier. For example, Cutoff 4,000 Hz and CC20 64 produce about 2,015.75 Hz.

MIDI changes take effect at the next engine control block for both channels together. A cutoff of zero blocks new input, while the filter's existing state continues to decay nonlinearly; it is not an instant output mute. Raising CC20 reopens the filter.

Starting the instrument initializes CC20 and CC21 to maximum, so the performance settings work immediately without a MIDI controller. Later MIDI values replace that initialization. In Controller Sequencers select the effect's MIDI channel, use CC20 or CC21, and set **Reset Value 127** to return to the selected performance setting when the lane stops. Empty steps hold the preceding value.

**Shared channel behavior:** MIDI state is shared by channel and CC, including startup initialization. Instances on the same channel respond together; starting one resets these two CCs for other receivers. Channel 16/CC20–21 also overlap Granular Capture's default Stutter and Freeze switches. Assign different MIDI Control Channels when using the effects independently.

## Graph and exports

Two native moogvcf nodes process the audio channels. Two ctrl7 nodes supply normalized live factors, and two initc7 nodes initialize them once. The graph bounds the factors because Csound's initializer internally stores `127 × value + 0.5`, slightly above full scale when the normalized initial value is 1.

Native instrument and performance exports preserve the graph and instance settings. CSD MIDI and SCORE exports preserve initialization and automation; SCORE uses the explicit-channel controller array. Their event timing can differ by one control block. Controller state itself is runtime state.

See the example README for reproduction, auditions and validation.

# Configuration

The Configuration chapter covers both UI-visible settings and practical runtime/deployment behaviors that affect how Orchestron is used.

## What Is Covered

- GUI language switching and integrated help/docs behavior
- Config page engine settings ( `sr` , control rate, buffers, derived `ksmps` ) plus browser-clock latency controls when relevant
- MIDI setup and runtime input binding workflow
- Unified browser-clock playback and latency tuning from the Config page
- Internal loopback versus external host-bridge MIDI binding
- App-state persistence and default values

## Chapter Contents

- GUI Language and Integrated Help
- Audio Engine Settings (Config Page)
- MIDI Setup and Inputs
- Browser-Clock Latency
- Persistence and Defaults

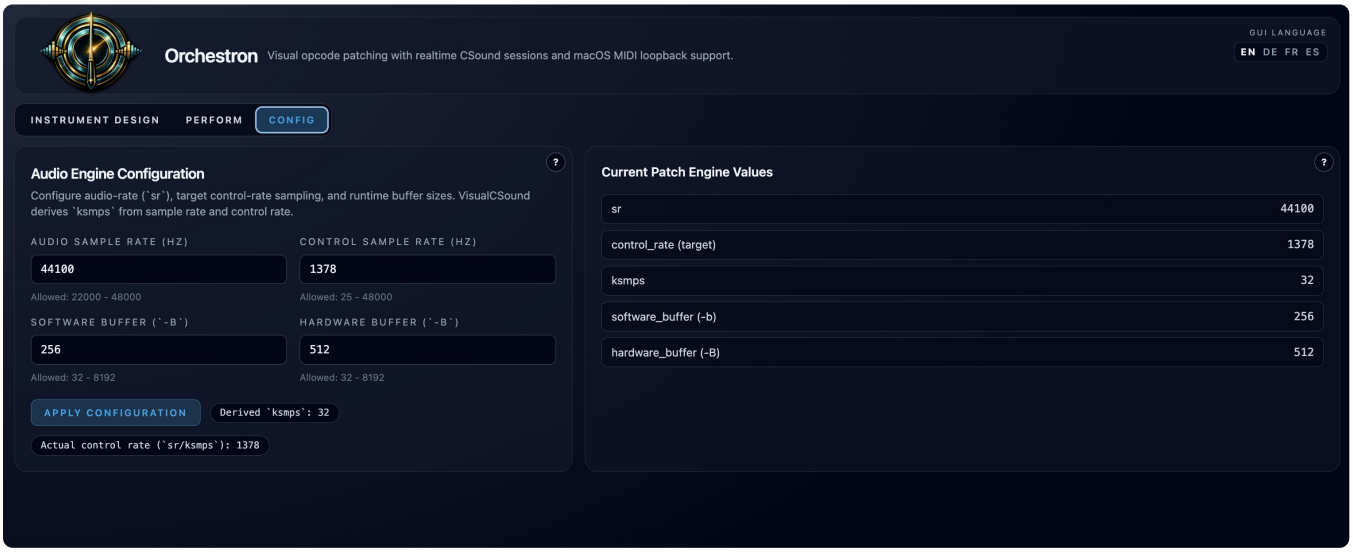
## Configuration Scope Clarification

Orchestron has three kinds of configuration:

- **Patch-level configuration** (stored with a patch, editable in the Config page)
- **Runtime/startup configuration** (browser-clock startup options and optional host MIDI bridge token)
- **Workspace runtime tuning** (browser-clock latency settings stored in app state)

Both are important for successful use.

## Screenshots




*Configuration page overview of engine settings. The dedicated Browser-Clock Latency chapter shows the browser-clock-specific tuning section in detail.*

# Audio Engine Settings (Config Page)

The Config page controls both patch-level Csound engine settings and browser-clock latency settings for the current workspace.

## Where These Settings Are Stored

Config page changes are stored in the current patch's `graph.engine_config`.

That means:

- different patches can use different engine settings
- switching patches can change the visible Config page values

Browser-clock latency settings are different:

- they apply to the active browser-clock session runtime
- they are stored in app state, not in the patch
- applying them updates the active browser-clock controller without changing patch data

## Editable Settings (Config Page)

### Patch Engine Section

### Audio Sample Rate (`sr`)

- Field label: `Audio Sample Rate (Hz)`
- Allowed range: **22000 .. 48000**
- Integer only

### Control Sample Rate (`control_rate` target)

- Field label: `Control Sample Rate (Hz)`
- Allowed range: **25 .. 48000**
- Integer only

This is the target control-rate value used to derive `ksmps`.

### Software Buffer (`-b`)

- Field label: `Software Buffer (-b)`
- Allowed range: **32 .. 8192**
- Integer only

### Hardware Buffer (`-B`)

- Field label: `Hardware Buffer (-B)`
- Allowed range: **32 .. 8192**
- Integer only

## Derived Values Shown In The UI

The Config page previews derived values before applying:

- Derived `ksmps`
- Actual control rate (`sr / ksmps`)

This helps you see the real runtime effect of your target control-rate choice.

## Current Patch Engine Values (Read-Only Panel)

The right-side panel shows the normalized engine values stored in the patch:

- `sr`
- `control_rate`
- `ksmps`
- `software_buffer`
- `hardware_buffer`

Use this panel to verify what will be used at compile/start time.

## Browser-Clock Latency Section

The Config page shows an additional section for browser-owned audio latency tuning whenever a browser-clock session is active.

See Browser-Clock Latency for the unified browser-clock workflow explanation and the full UI screenshot of this section.

Editable fields:

- steady queue low water / high water ( `ms` )
- startup queue low water / high water ( `ms` )
- underrun recovery boost / maximum underrun boost ( `ms` )
- maximum blocks per render request
- steady / startup / recovery parallel request limits
- immediate note render blocks
- immediate note render cooldown ( `ms` )

These values control the browser PCM queue depth and render request behavior, which are the dominant latency/stability knobs on the unified browser-clock path.

The patch engine section still matters because `sr` , `ksmps` , and the configured buffers are used by the backend render worker when it compiles and starts the headless Csound session.

### Field-By-Field Meaning

#### Steady Low Water / Steady High Water

- These define the normal browser PCM queue target after startup is complete.
- Lower values reduce live-play latency.
- Higher values make clicks and underruns less likely.

#### Startup Low Water / Startup High Water

- These define the larger queue target used while the stream is still priming after connect or after an underrun recovery phase.
- They are usually kept above the steady values so the stream can stabilize before dropping into lower-latency playback.

#### Underrun Recovery Boost / Max Underrun Boost

- When the browser detects underruns, VisualCSound temporarily adds extra queue headroom.
- `Underrun Recovery Boost` controls how much extra buffer is added per underrun event.
- `Max Underrun Boost` limits how far that temporary recovery buffer can grow.

#### Max Blocks Per Request

- Caps the maximum backend render chunk size requested in one browser-clock render request.
- Smaller values reduce head-of-line delay for live notes.
- Larger values reduce request overhead but can make live interaction feel less immediate.

#### Steady / Startup / Recovery Parallel Requests

- These limits control how many render requests may remain in flight at once.
- More parallelism can keep the queue full more aggressively.
- Too much parallelism can also create larger bursts and less predictable latency.

#### Immediate Note Render Blocks / Immediate Note Render Cooldown

- A live `note_on` sends a small urgent render request in addition to the normal refill logic.
- `Immediate Note Render Blocks` sets the size of that urgent render burst.
- `Immediate Note Render Cooldown` throttles how often these urgent requests can be sent.
- These controls help manual piano playing feel more responsive, but values that are too aggressive can reintroduce clicks.

### Practical Adjustment Order

1. Start with `steady low/high water`.
2. If clicks remain, raise the steady or startup watermarks slightly.
3. If latency still feels too high, reduce `max blocks per request` carefully.
4. Only then tune immediate note render and parallel request limits.

## Validation Behavior

The page validates:

- integer format
- allowed range
- high-water fields must be greater than their matching low-water fields

`Apply Configuration` is disabled until all visible values are valid.

## Apply Configuration

`Apply Configuration` writes the new engine settings into the current patch state.

Practical effect:

- future compile/start operations for the patch use the updated engine config
- the patch must still be saved if you want the change persisted in the patch library

`Apply Browser-Clock Settings` writes the browser-clock latency settings into app state and refreshes the active browser-clock controller if one is connected.

## Other Engine Fields Present But Not Edited In UI

The patch engine config also contains values such as:

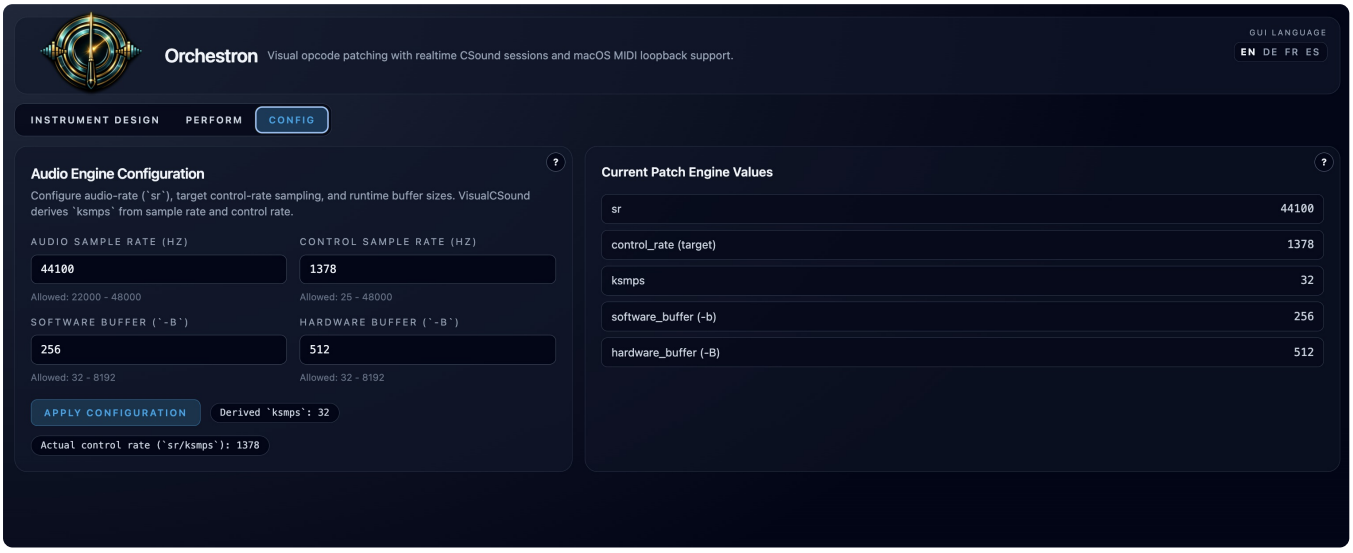
- `nchnls` (default `2`)
- `0dbfs` (default `1`)

These are part of the patch model, but the current Config page focuses on the user-facing timing and buffer controls listed above.

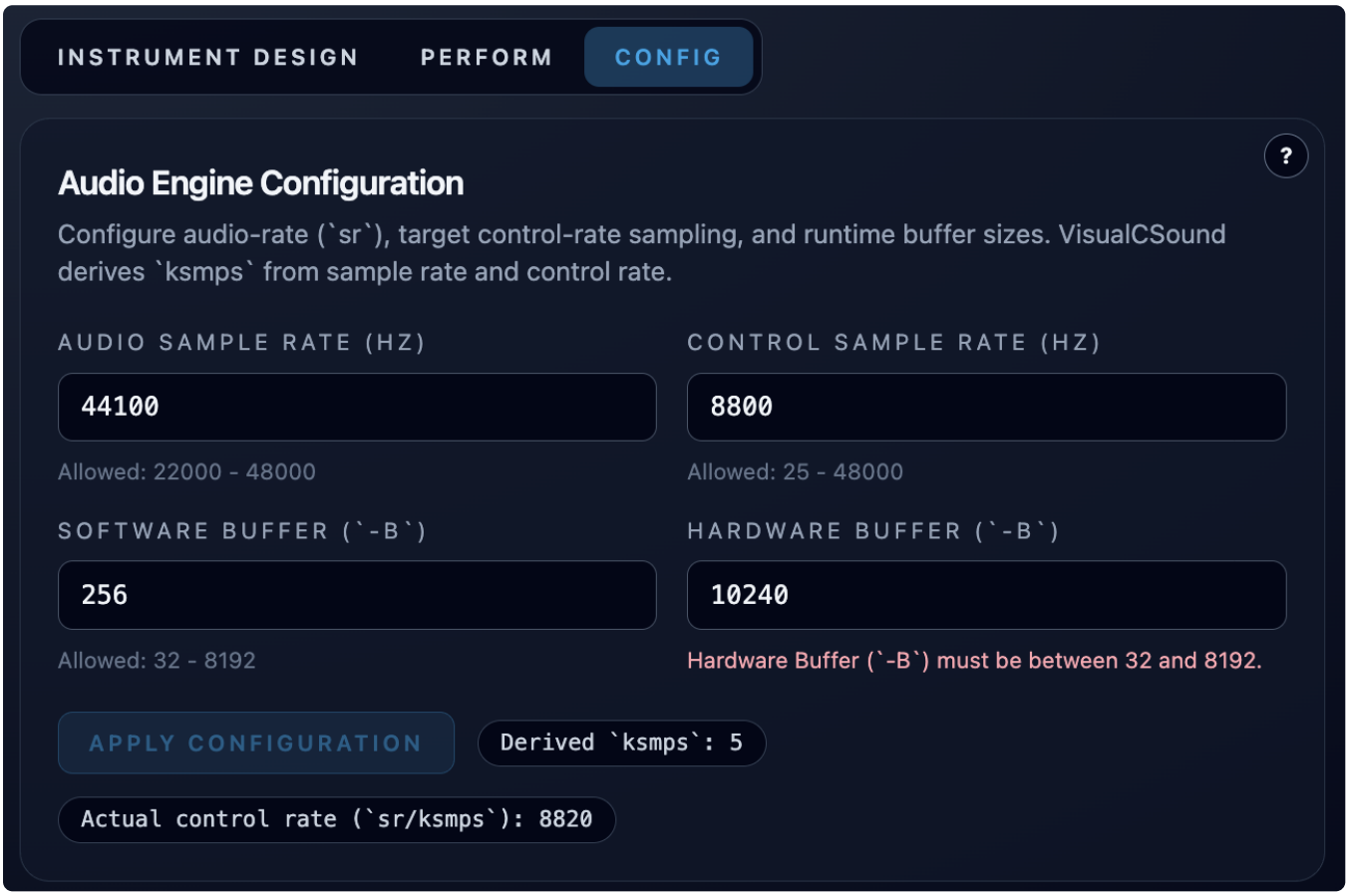
## Tuning Tips

- Lower buffers reduce latency but increase glitch risk.
- Higher buffers improve stability but increase latency.
- Use the Runtime panel and live testing to find a stable setting for your machine and patch complexity.
- On the browser-clock path, start by adjusting browser-clock low/high water, render blocks, and parallel request limits before changing patch `-b/-B`.
- For live MIDI sessions, prefer `ksmps <= 32`. The runtime warns when a live session starts with a larger `ksmps` because MIDI execution is still quantized to the k-period.

## Screenshots




*Config page audio engine settings in a valid state.*




*Config page validation messages with invalid values entered.*

The dedicated Browser-Clock Latency chapter shows the browser-clock tuning section in detail.

# Persistence and Defaults

Orchestron persists both library data (patches/performances) and a separate app-state snapshot (workspace state).

Persistent JSON documents are bounded before storage so direct API clients cannot grow request memory or the SQLite database without limit. Defaults are 8 MiB for app state ( `VISUALCSOUND_APP_STATE_MAX_BYTES` ), 4 MiB for saved patch graphs ( `VISUALCSOUND_PATCH_GRAPH_MAX_BYTES` ), 1 MiB for patch UI layout metadata ( `VISUALCSOUND_PATCH_UI_LAYOUT_MAX_BYTES` ), 8 MiB for performance configs ( `VISUALCSOUND_PERFORMANCE_CONFIG_MAX_BYTES` ), and 64 KiB for any single nested JSON string ( `VISUALCSOUND_PERSISTED_JSON_STRING_MAX_BYTES` ).

## Two Persistence Layers (Important)

### 1. Library Data (Explicit Save)

These are stored when you click save actions:

- Patches (Instrument Design `Save` )
- Performances (Perform page `Save Performance` )

### 2. App-State Snapshot (Automatic)

Orchestron also persists a workspace/app-state snapshot in the backend to restore your working context after reload.

Persisted app state includes (current implementation):

- active page ( `Instrument Design / Perform / Config` )
- GUI language
- instrument tabs (including editable patch snapshots)
- active instrument tab
- sequencer state (tracks, controller sequencers, arpeggiators, piano rolls, MIDI controllers)
- performance workspace metadata ( `currentPerformanceId` , name, description)
- sequencer instrument bindings (stable rack instance IDs)
- explicit audio routes, Master selection, insert ownership and mixer strip/send values (app-state version 2)
- active MIDI input selection reference
- browser-clock latency settings for the current workspace/runtime path

New sessions default that MIDI input reference to `internal:loopback` unless a different external input is explicitly bound.

## What This Means In Practice

- You can reload the app and continue from a similar workspace state.
- Unsaved edits may reappear because they exist in the app-state snapshot.
- Unsaved edits are **not** the same as saved library data.

Always use explicit save actions when you want stable, shareable library entries.

## Default New Patch Values

New opens the built-in/template chooser. The selected starter supplies its name, graph, Activation and audio interface. Choose Empty patch for an empty graph ( `nodes = []` , `connections = []` ). All starters use these default engine settings:

- `sr = 48000`
- `control_rate = 1500`
- `ksmps = 32`
- `nchnls = 2`
- `software_buffer = 128`

- `hardware_buffer = 512`
- `0dbfs = 1`

## Saved mixes and older performances

Save Performance, Load Performance, Clone and native performance bundles retain version 11 audio routing and mixer settings, including Master, dedicated inserts, sends and pre/post selection. App-state restore also retains the current mix. These are snapshots; recording mixer automation is not included.

Opening versions 1–10 converts the old Level values to audio gain in dB and resolves legacy connections into explicit routes. Missing Level means 0 dB. The migration notice explains that Level no longer scales MIDI velocity, so velocity-sensitive instruments may change timbre. Check the mix and save the migrated performance. See Mixer persistence and compatibility.

## Compile Status vs Persistence

The compile status badge ( `compiled` , `pending changes` , `errors` ) indicates compile state for the current patch snapshot. It does **not** indicate whether the patch has been saved to the patch library.

## Runtime Defaults

- runtime audio mode is fixed to `browser_clock`
- the default session MIDI binding is `internal:loopback`
- external host MIDI is disabled unless `VISUALCSOUND_HOST_MIDI_TOKEN` is configured and a helper connects
- runtime session creation is bounded by `VISUALCSOUND_SESSION_MAX_ACTIVE` , `VISUALCSOUND_SESSION_MAX_ACTIVE_PER_CLIENT` , `VISUALCSOUND_SESSION_CREATE_RATE_PER_MINUTE` , and `VISUALCSOUND_SESSION_CREATE_RATE_BURST` ; stopped idle sessions are deleted after `VISUALCSOUND_SESSION_IDLE_TIMEOUT_SECONDS`
- session event WebSocket observers are bounded by `VISUALCSOUND_SESSION_EVENT_WS_MAX_SUBSCRIPTIONS_TOTAL` , `VISUALCSOUND_SESSION_EVENT_WS_MAX_SUBSCRIPTIONS_PER_SESSION` , `VISUALCSOUND_SESSION_EVENT_WS_CONNECT_RATE_PER_MINUTE` , and `VISUALCSOUND_SESSION_EVENT_WS_CONNECT_RATE_BURST`

## Recommended Habits

1. Use app-state restore for convenience.
2. Use `Save` / `Save Performance` for intentional versions.
3. Use `Export` bundles for backups or sharing across machines.

# Appendix

These optional tools let a coding agent create instruments and performances from a musical brief. You can use Orchestron entirely through its graphical interface; the tools are included here for readers who want to automate authoring or inspect the underlying workflow.

- Agent CLI introduces the command-line utilities.
- Orchestron Patch Creator Skill describes instrument and effect authoring.
- Orchestron Performance Creator Skill describes arranging and mixing with existing instruments.
- Orchestron Composition Creator Skill develops playable melodic candidates through listening, selection and refinement.
- Sung Melody Transcription Skill independently converts singing and optional claps to reviewable MIDI, note JSON and a piano preview.

Detailed skill links open the repository sources so they are usable from the PDF as well. The example prompts refer to your local checkout.

## Use a Skill in a Coding Agent

A skill is a directory containing a `SKILL.md` instruction file, supporting references, and executable tooling. It guides the agent through the task; the command-line utility performs the operations.

1. Open the Orchestron repository in a coding agent with access to its files and a terminal.
2. Ask the agent to read the chosen `SKILL.md` under `integrations/skills/` and follow its linked references. Keep the whole skill directory available, including its `src`, `references`, and project files.
3. Give a musical brief, the desired instrument or performance name, and the backend URL. State whether you want a new item or an update to an existing one, and whether playback is wanted.
4. Ask the agent to validate the result and report what it saved, compiled, rendered, and actually listened to. Review the resulting instrument or performance in Orchestron.

Explicitly asking an agent to read the file does not require a product-specific skill installation. Automatic skill discovery and shortcut invocation depend on the agent; the prompts in the following chapters use ordinary language and repository paths.

For Codex plugin installation, run `make -C integrations` from the repository root to package all four skills. The versioned ZIP appears in `integrations/plugin-dist/`. It includes skill sources, preview assets, and dependency lockfiles; `uv` installs the Python dependencies on first use. See the plugin build and installation guide. Installed skills resolve commands from their own directory; keep musical work and edit sessions outside the plugin. After verifying the plugin, remove duplicate standalone skill installations while retaining the repository sources.

The utilities use `uv` and require Python 3.13. Library operations, compilation, performance validation and live sessions need a reachable Orchestron backend, normally `http://localhost:8000/api`. Run the backend using the normal setup procedure (`make run` from the repository root), or provide an existing server's URL. Local patch-spec validation and graph generation can run without the backend. Composition generation, audition and score export also run locally without the backend. The utilities access the library through the API, not by editing SQLite directly.

# Agent CLI

Two utilities under `integrations/skills/` support repeatable authoring from YAML/JSON specs.

| Utility | Purpose |
|---|---|
| `orchestron_patch_cli` | Generate instrument graphs, validate patch specs, and create, update or compile library patches. |
| `orchestron_cli` | Discover instruments, stage and save performances, configure routing and mixers, import/export, and manage optional live sessions. |

## Connect and Discover

Run these read-only checks from the repository root:

```
uv run --project integrations/skills/orchestron-patch-creator \
  orchestron_patch_cli --json health
uv run --project integrations/skills/orchestron-performance-creator \
  orchestron_cli --json patches list
```

Both default to `http://localhost:8000/api`; use `--api-url http://HOST:PORT/api` for another backend. `--json` returns structured results and retry hints; `--help` lists commands. Use `performances list` to discover saved performances, and inspect actual names, IDs and ports before editing.

## Validate, Save and Optionally Play

For patches, `spec validate` checks a spec locally; `graph render` produces inspectable JSON. `patch create` and `patch update` write through the backend. Add `--compile` for compile preflight before the final write, then follow the patch skill's audio checks.

For performances, begin a staged edit, add instruments, routes and music, then run `edit validate` before `edit commit`. Validation checks references and routes with the backend; commit saves without starting playback.

For live testing, `edit create-runtime --start` creates and starts a CLI-owned session. `edit push-runtime` updates sequencer, arpeggiator, mixer and controller values with a matching compiled rack; `edit rebuild-runtime` handles instrument, patch or route changes. Continuous instruments need a restart for I-rate controller changes. Audible browser-clock playback requires a connected browser audio client.

## Capabilities and Detailed References

The performance utility writes configuration version 18 and accepts versions 1–18. It preserves routing, mixer state, instance overrides, arrangement metadata and existing insert chains. Use the app to create or reorder inserts.

- **Sound settings:** discover and set per-instance values with `edit performance-controllers list/set/reset`. Patch specs define the controls; performance edits set their overrides. See controller authoring and instance settings.
- **Routing and mixing:** `edit instruments list`, `edit routes` and `edit mixer` expose ports, main/send paths and strip controls. Quote shell identifiers such as `'$master'`. The optional standard-effects preset requires suitable effect patches. See routing, mixer controls and the preset.
- **Patterns and expression:** discover tracks with `edit sequencers list`; use `edit step-timing` for early/late notes and `edit ratchets` for drum rolls and velocity ramps. Pads are numbered 1–8; steps start at 0. Seetiming, ratchets and score specs.

The backend launch option `--audio-output-mode browser_clock` belongs to the application server, not to either authoring utility. Browser-clock mode is the normal audio path.

# Orchestron Patch Creator Skill

Use Orchestron Patch Creator to turn a sound description into an Instrument Design patch, or to revise an existing instrument or effect. The skill combines synthesis guidance with the `orchestron_patch_cli` utility.

## What It Does

The agent chooses a suitable template family, writes a structured patch spec, validates it, and creates or updates the graph through the backend. Specs describe sources, envelopes, processing, output, input formulas and optional `perf_controller` settings for later adjustment in the rack. Generated instruments expose a named Stereo Output; route it through the performance mixer to Master when using the instrument in a performance.

For an existing graph, preserve stable node IDs and its control-flow structure. The high-level layer syntax does not create If/Switch branches automatically. See patch specs and graph compatibility for these details.

## Example Agent Prompt

> Read `integrations/skills/orchestron-patch-creator/SKILL.md` and follow its references. Use the backend at `http://localhost:8000/api`. Create a new MIDI instrument named “Glass Pad” with a soft attack, a clear bell-like tone, gentle stereo movement and a long release. Expose attack and release as rack controls. Preserve existing instruments. Validate and compile the patch, render representative notes and release tails, and inspect both Mel and log-frequency STFT spectrograms. Report the saved name and which audio checks you performed. Do not start live playback.

The agent should check finite samples, clipping, both channels, velocity response and relevant controller settings. Spectrogram inspection supports verification; it does not replace listening or justify claiming that listening occurred.

## Local Validation and Library Creation

After the agent has written `patch.yaml`, these commands illustrate the boundary between local validation and a backend write. Run them from the repository root:

```
uv run --project integrations/skills/orchestron-patch-creator \
  orchestron_patch_cli --json spec validate patch.yaml
uv run --project integrations/skills/orchestron-patch-creator \
  orchestron_patch_cli --json patch create patch.yaml --compile
```

Use `patch update PATCH_ID patch.yaml --compile` for an explicitly selected existing patch. A failed validation or compile preflight must be corrected before updating it. See the authoring workflow and audio validation procedure for the full process.

# Orchestron Performance Creator Skill

Use Orchestron Performance Creator to compose, edit, import, validate and save performances using instruments in the library. It guides an agent through the `orchestron_cli` utility, with references for musical form, harmony, rhythm, routing and mixing.

## What It Does

The agent discovers available patches, starts a staged edit, assigns rack instruments, connects their audio paths and adds melodic, drummer, controller or arpeggiator material. It can write a YAML/JSON score for longer arrangements, adjust per-instance sound settings, program note timing and drum rolls, and configure mixer levels and sends. The built-in Master needs no extra speaker instrument.

Before saving, the agent validates the staged performance against the backend. Commit saves the result; creating or updating a live session is a separate, optional step. Existing insert chains are retained, while insert creation and reordering use the app.

## Example Agent Prompt

> Read `integrations/skills/orchestron-performance-creator/SKILL.md` and follow its references. Use the backend at `http://localhost:8000/api`. First list available instruments and choose a drum kit, bass and pad from the library. Create a new 32-bar performance named “Night Drive” at 120 BPM in 4/4 and C minor: four bars of drums, an evolving bass section, a pad-led middle, and a short ending. Keep the bass and pad harmonically compatible, route the instruments to Master, and set balanced levels. Validate and save it without changing existing performances or starting playback. Report the instrument choices and arrangement.

If suitable instruments are missing, the agent should report that and use the Patch Creator skill when creating the missing sounds is part of the request. Avoid assuming that example patch names exist in every library.

## Stage and Save

From the repository root, the agent starts a new edit with:

```
uv run --project integrations/skills/orchestron-performance-creator \
  orchestron_cli --json edit begin --new --name "Night Drive"
```

After populating the staged rack, routes and music, validate and save:

```
uv run --project integrations/skills/orchestron-performance-creator \
  orchestron_cli --json edit validate
uv run --project integrations/skills/orchestron-performance-creator \
  orchestron_cli --json edit commit
```

See composition workflow and score specs for detailed authoring instructions.

## Use an Accepted Composition

Composition Creator develops melodic candidates through listening and selection, then exports `orchestron-score.json`. Performance Creator imports score versions 1 and 2, including per-note velocity and local meter/subdivision/beat-rate timing. Version 2 checks explicit event overlap and pad boundaries. Polyphonic parts become coordinated voice tracks on the instrument's MIDI channel, preserving normal editable note/chord cells.

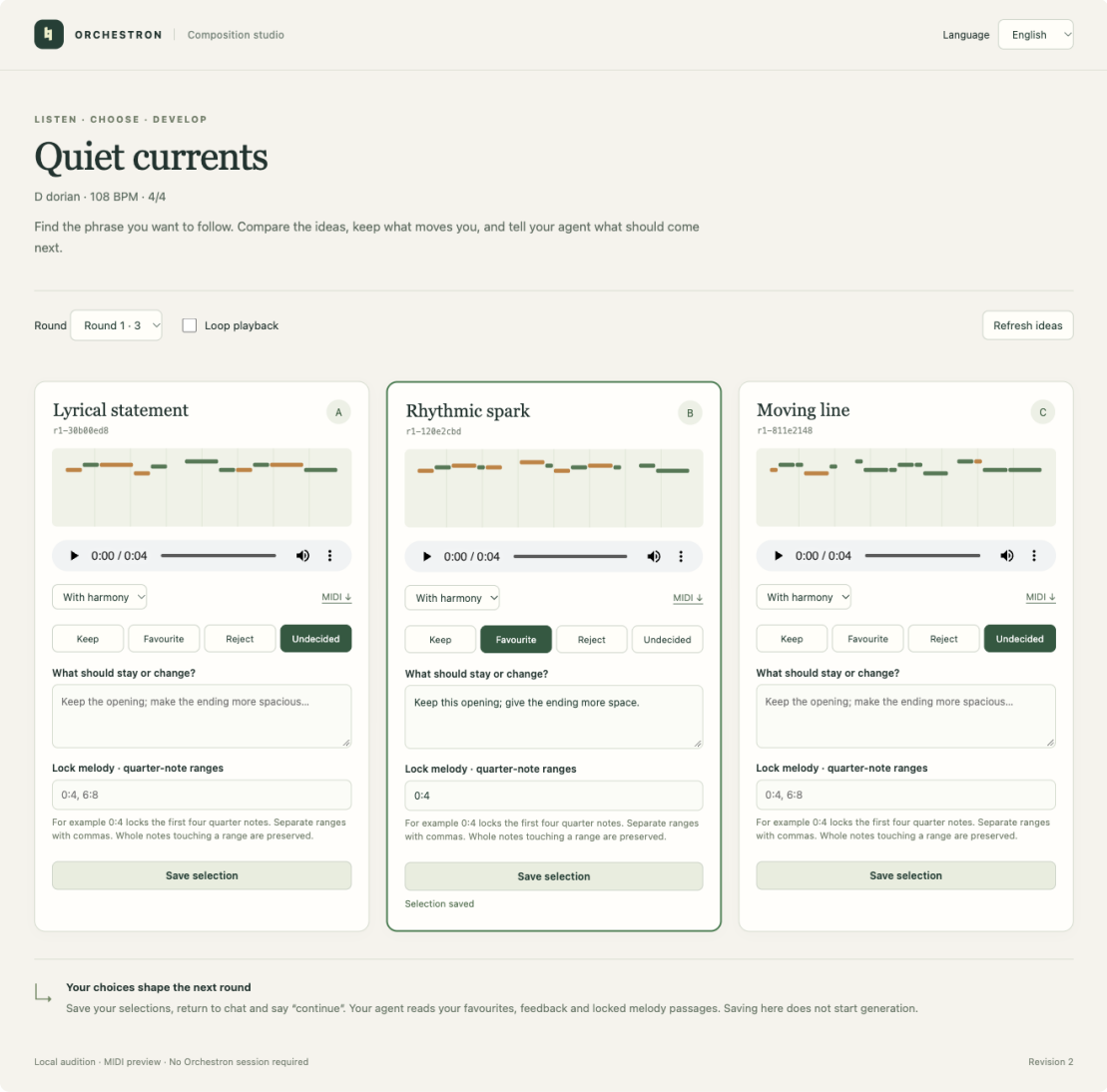
Start a new staged performance, bind the score's channels to suitable instruments, connect their outputs, then use `edit apply-score /absolute/path/orchestron-score.json`. Apply once: the command appends tracks. Validate and save using the commands above. The score is separate from a native `.orch.json` performance bundle.

## Combine Both Skills

Ask the agent to create and validate a new sound with Patch Creator, then read Performance Creator and discover the saved patch in the backend library. It can add that patch to a new performance, connect its Stereo Output to Master, write complementary parts, validate and save. Include the desired names, musical constraints and playback preference in the same request. Creating the patch, saving the performance and starting playback are separate operations.

# Orchestron Composition Creator Skill

Use Orchestron Composition Creator to explore melodies by listening, choosing and developing related ideas. It uses music21 and an arvo compatibility copy, runs independently of the Orchestron backend, and exports a machine-readable score for Performance Creator.




## Try It With an Agent

> Read `integrations/skills/orchestron-composition-creator/SKILL.md`. Propose three two-bar melodies in D dorian at 108 BPM, with a calm but slightly syncopated character. Include quiet harmony for comparison. Open the audition page and wait for my choice. Develop the melody I select, preserving any passages I lock. When I accept the phrases, export a score for Orchestron Performance Creator.

The agent creates a session and generates a small shortlist. Each candidate has a contour display and a player. Compare **With harmony** and **Melody only**, loop the phrase, or download its MIDI. All candidates use consistent preview sound and gain. The default simple MIDI synthesizer needs no extra sound installation; FluidSynth with a local SoundFont is optional. Preview timbre differs from the instrument eventually chosen in Orchestron.

## Choose and Continue

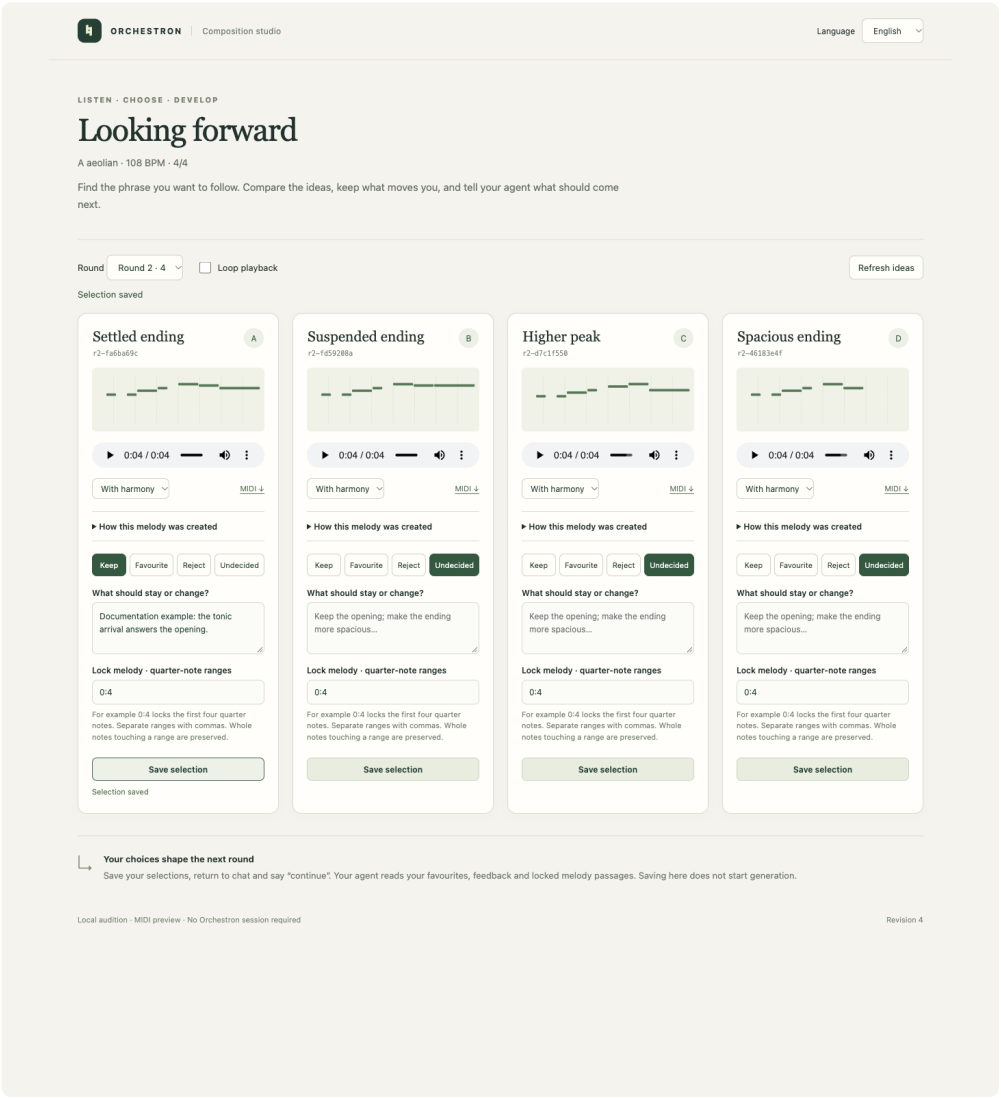
Mark **Keep**, **Favourite**, **Reject** or **Undecided**, add feedback, and click **Save selection**. To retain an opening, enter a melody lock such as `0:4` for the first four quarter notes. Multiple ranges use commas. Complete notes touching a locked range remain unchanged. Locks protect the melody; ordinary development also leaves its existing harmony intact.

Return to chat and say **“continue”**. The agent reads the saved selections and creates related ideas. It can transpose or invert a motif, develop cyclic or additive material, combine one accepted idea's pitches with another's rhythm, or append a continuation. Feedback guides the agent; the command-line generator does not interpret prose by itself. Saving in the page does not start generation.

Use **Refresh ideas** to see the next round. Earlier rounds, comments and choices remain available. The page works in English, German, French and Spanish. Unsaved changes are marked. A conflicting newer revision must be refreshed before saving. The session is stored locally and can be resumed after closing the page.

## Compare Endings and Harmonic Settings

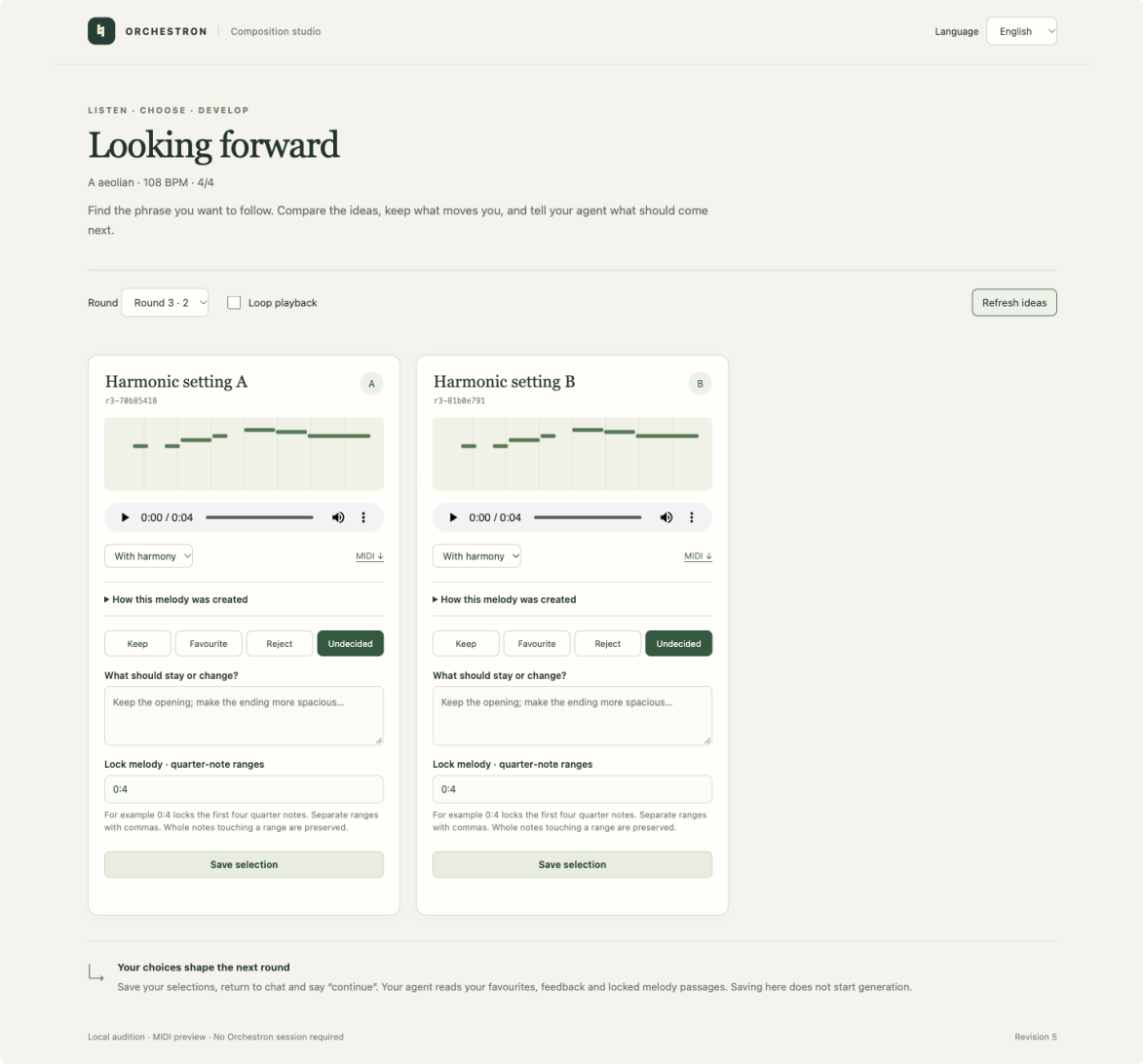
When you like an opening but are unsure of its ending, ask the agent to preserve that opening and compare four alternatives: **Settled ending**, **Suspended ending**, **Higher peak** and **Spacious ending**. The first descends to a held tonic; the second holds scale degree two; the third reaches a new high point; the fourth arrives early and leaves silence. These are intentions to test by listening.




The opening's notes, dynamics and rests stay fixed. A sustained note crossing the chosen boundary is retained whole. Existing locks also apply to the tail; a conflict stops the complete comparison without removing those locks. All four

candidates are saved in one undoable round and wait for your selection. Compare melody alone and with harmony, then record what works and what you want to change.

After you choose an ending, ask to compare two harmonic settings of it. The complete melody remains identical, while the selected accompaniment part changes. The default compares tonic and mediant triads with their bass notes; in the included A-minor study these are Am and C. Solo previews are identical. Use the mix previews to hear the difference, and keep listening levels consistent.




For an original two-bar starting motif, run the study helper from the repository root:

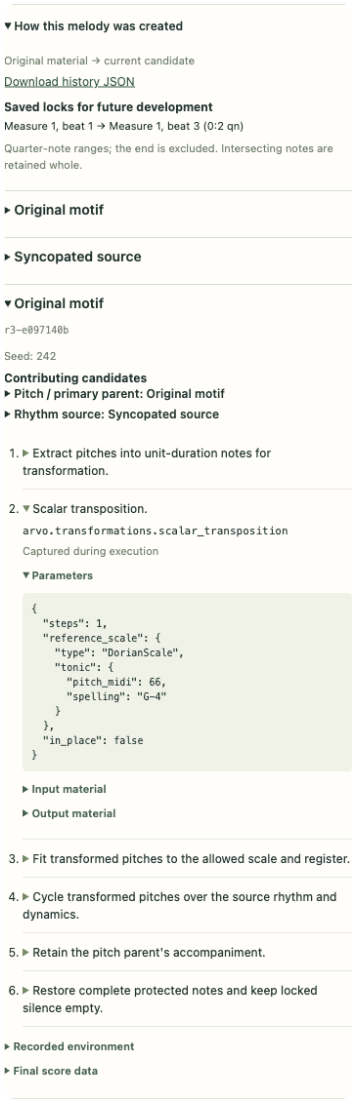
```
uv run --locked --project integrations/skills/orchestron-composition-creator \
  python integrations/skills/orchestron-composition-creator/scripts/start_motif_study.py \
  /absolute/path/to/my-study
```

It imports and renders one pending seed, then waits for your choice. The agent uses `compare-endings SESSION --from ACCEPTED_ID --start 4` , followed after selection by `compare-harmony SESSION --from ACCEPTED_ENDING_ID` . Read the comparison reference for custom harmonic settings, exact timing limits and preservation guarantees. Export only the chosen setting; the alternatives are comparisons, not song sections.

## Follow a Melody's Creation

Open **How this melody was created** beneath a candidate's player. The panel shows its original material, earlier versions and contributing candidates in creation order. Expand an ancestor or a step to inspect arvo function names, resolved arguments

and input/output material. It also records pitch fitting, rhythm combinations, accompaniment and continuation, so the history explains the final melody rather than just the first transformation.




Parent roles identify where pitches and rhythm came from. Saved feedback and reviews are captured when a child is created. **Locks applied during creation** show the ranges used for that operation, with measure/beat positions and exact quarter-note values. Retained notes, protected silence and discarded generated notes are available in the details. **Saved locks for future development** are the current candidate's review and can differ from those historical locks.

The panel loads on demand without interrupting playback or unsaved reviews. **Download history JSON** saves the complete record. CLI users can read the same record with `orchestron_compose trace SESSION CANDIDATE_ID .`

Built-in generation records its operations automatically. Custom authoring scripts use the trace recorder and import their records with `import-score --trace FILE .` Authored descriptions are distinguished from captured function calls. Earlier candidates and unrecorded imports show gaps explicitly; the tool does not reconstruct their missing history.

## Local Commands

From the repository root:

```
uv run --locked --project integrations/skills/orchestron-composition-creator \
  orchestron_compose init output/composition/quiet-currents \
  --title "Quiet currents" --key D --mode dorian --tempo 108 --bars 2
uv run --locked --project integrations/skills/orchestron-composition-creator \
  orchestron_compose generate output/composition/quiet-currents
uv run --locked --project integrations/skills/orchestron-composition-creator \
  orchestron_compose serve output/composition/quiet-currents
```

Open the printed loopback URL. Stop the server with Ctrl-C when finished; saved choices remain on disk. The tools require Python 3.13 and `uv` on macOS or Linux. Dependencies are downloaded on first use and locked for repeatability.

## Export and Realise the Song

Ask the agent to arrange accepted phrases in a specified order. Export includes `composition.json`, `orchestron-score.json`, MIDI, MusicXML and playable WAV files. The canonical score retains exact rational timing, note dynamics, phrase sections and accepted-candidate provenance. The exporter finds exact supported grids and creates editable voice tracks and pads; it reports unsupported timing or capacity instead of silently changing the music. Initial scope is fixed tempo/meter and standard MIDI pitches, with straight/triplet grids and supported subdivisions. The canonical score also retains full recorded creation history and phrase placements, including repetitions, when imported into another composition session. Performance Creator's score format and native performance bundles are unchanged.

Performance Creator then discovers instruments, maps the score's channels, sets routing and mixer levels, applies the score, validates and saves the performance. These score files are not native Orchestron bundles and should not be passed to the application's Import dialog.

# Sung Melody Transcription Skill

This plugin skill transcribes a sung or hummed monophonic melody into MIDI, note JSON and a playable piano WAV. Rhythmic claps can establish pulse timing; they are treated separately from vocal attacks. You can supply tempo, meter, bar count and an ending preference. The default meter is 4/4 with a sixteenth-note grid, and the recorded ending is preserved.

The skill searches all major and natural-minor pitch collections while fitting one tuning offset within ±50 cents. It reports tied collections, including relative major/minor, rather than claiming a unique key. Conservative cleanup merges false splits and removes brief unstable onset scoops and release dips; the original pitch track and every editorial change remain available for review.

## Run and review

Ask a coding agent to read the skill source. For example:

```
Transcribe this sung melody as four bars in 4/4. Use the claps for tempo and

leave a short rest at the end. Return a piano preview, MIDI, notes and an edit

summary so I can check the repeated notes and phrasing.
```

The skill requires Python 3.11–3.13, `uv` , the RVC RMVPE checkpoint, FluidSynth and a piano SoundFont. The checkpoint downloader verifies its hash. The setup and command reference explains asset configuration and optional inputs. Python dependencies are locked within the skill; transcription runs locally.

Review `preview.wav` against the recording, particularly repeated attacks, short gestures, rests and the ending. `melody.mid` and `notes.json` contain the quantized result; `pitch.json` retains continuous pitch and voicing evidence; `edits.txt` summarizes the changes. Detailed before/after edits are in `notes.json` . Requested bar counts define the exact MIDI phrase length, including silence. Clipped or excluded notes are reported. FluidSynth's acoustic release tail may continue after the final MIDI note-off.

Claps do not identify a musical downbeat, meter or count-in by themselves. Supply beat zero when the phrase begins after a count-in. Without supplied tempo or a reliable clap pulse, 120 BPM is explicitly marked as a review default. Dense or ambiguous material may need a finer grid or manual review. The output is a draft transcription, not an automatic confirmation of the intended melody.

## Plugin packaging and independent previews

The source lives under `integrations/skills/sung-melody-transcription/` and is included in the Orchestron plugin ZIP. Install the plugin or ask the agent to read the skill directly. Transcription and FluidSynth piano previews need no Orchestron backend and do not create instruments, performances or native bundles. Any later Orchestron import or instrument assignment is a separate workflow after reviewing the transcription.